ORACLE UNIVERSITY



Oracle SOA Suite 12*c*: Build Composite Applications

Activity Guide
D84361GC10
Edition 1.0 | November 2014 | D89151

Oracle International Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. You may copy and print this document solely for your own use in an Oracle training course. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice. If you find any problems in the document, please report them in writing to: Oracle University, 500 Oracle Parkway, Redwood Shores, California 94065 USA. This document is not warranted to be error-free.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT RIGHTS
The U.S. Government's rights to use, modify, reproduce, release, perform, display, or disclose these training materials are restricted by the terms of the applicable Oracle license agreement and/or the applicable U.S. Government contract.

Trademark Notice

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Author

Ron Pinkerton

Technical Contributors and Reviewers

Simone Geib, Jay Kasi, David Mills, Ted Witiuk, Pete Laseau and Tom Barrett

This book was published using: Oracle Tutor

Oracle Internal & Oracle Academy Use Only

Table of Contents

Practices for Lesson 1: Introduction to Service-Oriented Architecture 1-1
Practices for Lesson 1: Overview 1-2
Practice 1-1: Installing Oracle SOA Suite 1-4

Practices for Lesson 2: Getting Started with Composite Applications 2-1
Practices for Lesson 2: Overview 2-2
Practice 2-1: Creating the Composite Application 2-3

Practices for Lesson 3: Managing and Monitoring 3-1
Practices for Lesson 3: Overview 3-2
Practice 3-1: Deploying and Testing the HelloWorld Composite Application 3-3
Practice 3-2: Modifying the HelloWorld File Adapter to Use a Logical Name 3-9
Practice 3-3: Generating and Validating a Configuration Plan 3-13
Practice 3-4: Testing a Composite That Is Deployed with a Configuration Plan 3-16

Practices for Lesson 4: Working with Data 4-1
Practices for Lesson 4: Overview 4-2
Practice 4-1: Creating the Composite Application 4-3
Practice 4-2: Defining the Native Data Format 4-5
Practice 4-3: Creating the Service Components and External References 4-11
Practice 4-4: Adding a Routing Rule to the Mediator 4-13
Practice 4-5: Deploying and Running the Project 4-15
Practice 4-6: Optional Challenge: Creating a Fixed-Length Output File 4-17

Practices for Lesson 5: Using JMS and JDBC Adapters 5-1
Practices for Lesson 5: Overview 5-2
Practice 5-1: Configuring the Database 5-3
Practice 5-2: Creating the CCValidate SOA Project 5-5
Practice 5-3: Configuring the Mediator Component from a WSDL Definition 5-7
Practice 5-4: Deploying and Testing the CCValidate Composite 5-13
Practice 5-5: Creating and Configuring a JMS Queue 5-15
Practice 5-6: Creating a Composite Application and JMS Adapter 5-26
Practice 5-7: Deploying and Testing the Application 5-29
Practice 5-8: Optional: Creating a JMS Consumer Application 5-31
Practice 5-9: Optional: Setting Message Properties and Using Message Selectors 5-33

Practices for Lesson 6: Introduction to BPEL 6-1
Practices for Lesson 6: Overview 6-2
Practice 6-1: Creating a New Application 6-3
Practice 6-2: Deploying and Testing the Application 6-11

Practices for Lesson 7: More BPEL Activities 7-1
Practices for Lesson 7: Overview 7-2
Practice 7-1: Creating a New Project 7-3
Practice 7-2: Configuring the Asynchronous Service 7-5
Practice 7-3: Creating a BPEL Process to Make an Async Call 7-11
Practice 7-4 Deploying and Testing Your Work 7-19

Practices for Lesson 8: Handling Faults in Composite Applications 8-1
Practices for Lesson 8: Overview 8-2

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 8-1: Opening and Deploying the BookingSystem Project ... 8-3
Practice 8-2: Creating and Configuring the Enroll Project ... 8-4
Practice 8-3: Configuring the EnrollmentProcess Process ... 8-9
Practice 8-4: Deploying and Testing the Application ... 8-21
Practice 8-5: Using the Fault Policy Wizard ... 8-24

Practices for Lesson 9: Implementing Human Workflow ... 9-1
Practices for Lesson 9: Overview ... 9-2
Practice 9-1: Seeding the Demo User Community ... 9-3
Practice 9-2: Configuring the Email Driver ... 9-5
Practice 9-3: Creating a Composite Application ... 9-9
Practice 9-4: Implementing Manual Approval in the BPEL Process ... 9-14
Practice 9-5: Generating the ADF Task Form for the Human Task ... 9-19
Practice 9-6: Deploying and Testing the Application ... 9-21
Practice 9-7: Sending Order Status Email Notification to a Customer [Optional] ... 9-29
Practice 9-8: Redeploying and Testing the Application ... 9-34

Practices for Lesson 10: Sharing Functionality in Oracle SOA Suite ... 10-1
Practices for Lesson 10: Overview ... 10-2
Practice 10-1 Using a Component Template ... 10-3
Practice 10-2: Deploying and Testing the OrderFlow Application ... 10-14

Practices for Lesson 11: Using the REST Adapter ... 11-1
Practices for Lesson 11: Overview ... 11-2
Practice 11-1: Exposing a REST Service ... 11-3
Practice 11-2: Creating and Configuring a REST Interface ... 11-10
Practice 11-3: Deploying and Testing the Project ... 11-16

Practices for Lesson 12: Developing Business Rules ... 12-1
Practices for Lesson 12: Overview ... 12-2
Practice 12-1: Creating and Configuring a Composite Application ... 12-3
Practice 12-2: Implementing Business Rules in a BPEL Process ... 12-13
Practice 12-3: Deploying and Testing the Application ... 12-22
Practice 12-4: Adding a Shipping Rules Business Rule [Optional] ... 12-26
Practice 12-5: Modifying the BPEL Process [Optional] ... 12-33
Practice 12-6: Deploying and Testing the OrderProcessing Application [Optional] ... 12-37

Practices for Lesson 13: Designing and Managing Business Events ... 13-1
Practices for Lesson 13: Overview ... 13-2
Practice 13-1: Publishing an Event ... 13-3
Practice 13-2: Subscribing to an Event ... 13-9
Practice 13-3: Testing Event Publication and Subscription ... 13-12

Practices for Lesson 14: Testing and Debugging ... 14-1
Practices for Lesson 14: Overview ... 14-2
Practice 14-1: Creating a Test Suite for the CCValidate Composite ... 14-3
Practice 14-2: Creating Test Cases in the CCValidate Test Suite ... 14-7
Practice 14-3: Deploying and Executing the CCValidate Test Suite ... 14-11
Practice 14-4: Debugging a Composite Application in JDeveloper ... 14-15

Practices for Lesson 15: Securing Composite Applications and Invoking Secured Services ... 15-1
Practices for Lesson 15: Overview ... 15-2
Practice 15-1: Applying Security Policies Post Deployment ... 15-4

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Practice 15-2: Applying Security Policies at Design Time..... 15-13

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Practices for Lesson 1: Introduction to Service-Oriented Architecture

Chapter 1

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 1: Overview

Job Title*

The Job Title [list@YourCompany.com?Subject=Practices for Lesson 1: Introduction to Service-Oriented Architecture: EDAG0001.doc] is responsible for ensuring this document is necessary, reflects actual practice, and supports corporate policy.

Practices Overview

The Oracle SOA Suite Quick Start distribution allows you to quickly install a development or an evaluation environment on a single-host computer. Quick Starts are installations for development or evaluation only. If you want a production environment, start with "Planning the Oracle SOA Suite and Business Process Management Installation" in *Installing and Configuring Oracle SOA Suite and Business Process Management* to install a full-scale development environment that can be scaled to a production environment.

In this practice, you are guided through the Developer 'Quick Start' installation process and should be able to use Oracle SOA Suite in about 30 minutes. The installer:

- Installs all components that are necessary for development with the core of Oracle SOA Suite (BPEL, Business Rules, Mediator, and Human Work Flow)
- Launches JDeveloper after the installation, automatically registering the IDE plug-ins for Oracle SOA Suite

The SOA Suite 12*c* Developer installation contains the following:

- JDeveloper
- WebLogic Server
- SOA Suite runtime components
  - BPEL, Human Workflow, Rules, and Mediator
  - Service Bus
- JDeveloper IDE plug-ins for the preceding components

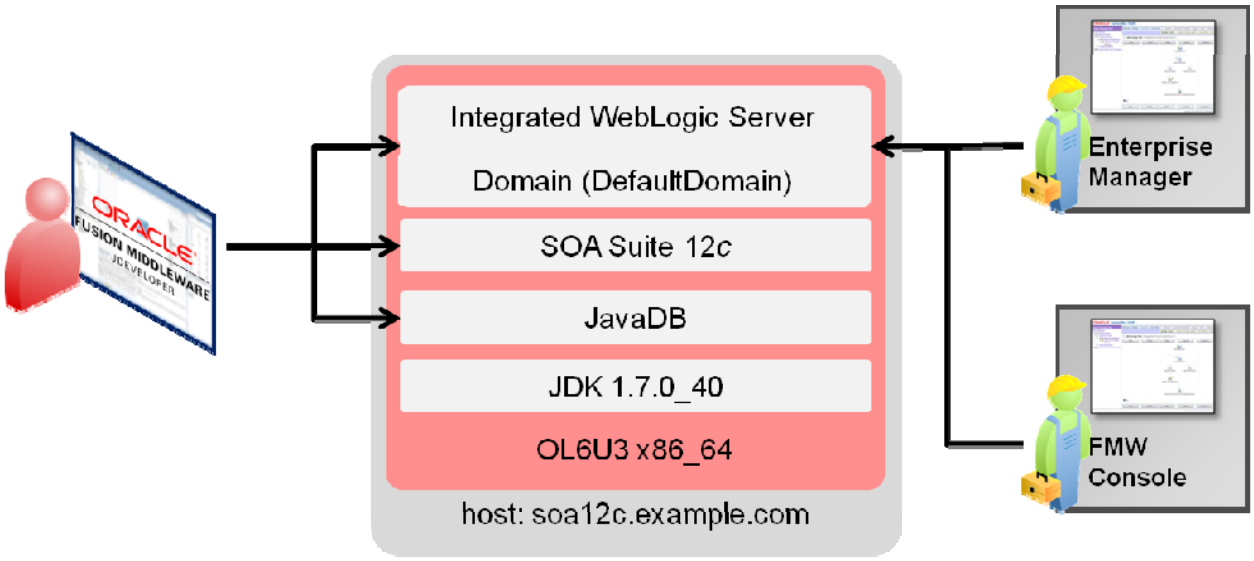
All software is installed in a single middleware home directory, which you specify.

In JDeveloper, the WebLogic Integrated Server is pre-configured with the preceding SOA Suite runtime components and JavaDB. JavaDB is a development database that allows you to start development with SOA Suite without the need to run the repository creation utility (RCU).

Your environment looks like the following on the hosted server when you have finished:

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 1-1: Installing Oracle SOA Suite

Tasks

1. Right-click the Terminal icon on your desktop and select Open.

   A terminal window opens.

2. In the terminal window, initiate the installation script by issuing the following commands:

   ```
   cd /home/oracle/labs/scripts/util
   sh startInstall.sh
   ```

   The package is extracted and the Installation Inventory Setup dialog box appears.



   Note: On UNIX and Linux systems, the wizard begins by prompting for the location of the Installation Inventory file. (This screen does not appear on Windows operating systems.) The Central Inventory is a master list of all Oracle products installed on this host. Provide the location of an existing Central Inventory file or specify where you want to create your Central Inventory. Make sure that the operating system group name selected on this screen has write permissions to the Central Inventory location.

   There is more to know about Central Inventory files and directories but for the purpose of this course, we accept the default `/home/oracle/oraInventory` as the directory name for our installation inventory. For more information about the Central Inventory, see "Understanding the Oracle Central Inventory" in *Installing Software with the Oracle Universal Installer*.

3. Click OK to accept the default installation inventory directory.

   The package is extracted and the installation wizard appears.

4. Use the instructions in the following table to complete the installation wizard.

1	Welcome	Click **Next**.
2	Installation Location	Provide the installation directory: `/u01/app/fmw12c/Oracle_Home` Click **Next**.
3	Prerequisite Checks	Verify success and click **Next**.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

4	Installation Summary	Review all the components that are installed in one step. Click **Install**.
5	Installation Progress	This step installs, links, and configures all software. This step takes 6–7 minutes. When 100% is complete, click **Next**.
6	Installation Complete	Deselect the "Start JDeveloper" check box. Click **Finish**.

Note: For more information about the Oracle Fusion Middleware directory structure, see "Selecting Directories for Installation and Configuration" in *Planning an Installation of Oracle Fusion Middleware.*

5. Start JDeveloper.
   a. In the open terminal window, issue the following commands:

      ```
      cd
      cp /home/oracle/labs/scripts/util/startJDev.sh .
      ```

      The script to start JDeveloper is copied to a more convenient location.
   b. In the terminal window, issue the following command to start JDeveloper:

      ```
      sh startJDev.sh
      ```

      JDeveloper is started. The Select Role dialog box opens.
6. Complete the following steps to configure JDeveloper defaults:
   a. Deselect the check box next to "Always prompt for role selection on startup."
   b. To accept the Studio Developer (default) role, click **OK**.

      The Confirm Import Preferences dialog box is displayed.
   c. Click No.

      The Oracle Usage Tracking dialog box is displayed.
   d. Deselect the check box next to "Allow automated usage reporting to Oracle."
   e. Click OK.

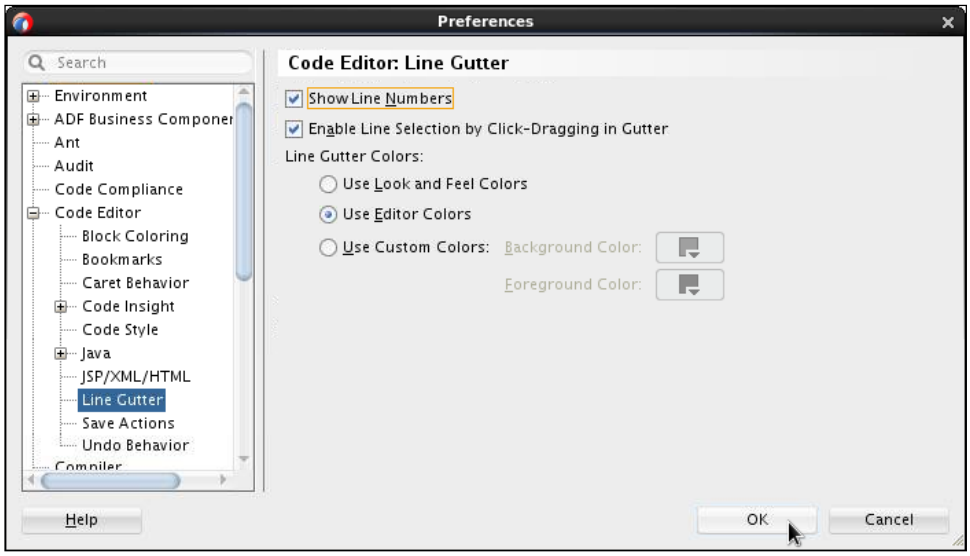
      JDeveloper opens and displays the Start Page tab.
   f. Deselect the check box next to "Show on Startup" and close the Start page.

   Note: In 12*c,* JDeveloper automatically scans the middleware directory and registers all the IDE plug-ins. Unlike earlier releases, you do not have to download them separately. When JDeveloper comes up, it is ready for SOA Suite development.
7. Set JDeveloper preferences.
   a. In the JDeveloper menu, select Tools > Preferences.

      The Preferences dialog box is displayed.
   b. Navigate to Code Editor > Line Gutter.
   c. Select the Show Line Numbers check box.
   d. Click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



8. Increase JDeveloper PermMemory size.

   a. Use JDeveloper to open the file `/u01/app/fmw12c/Oracle_Home/jdeveloper/jdev/bin/jdev.conf`.

   b. Locate the following entry (at line 83):

   ```
   AddVMOptionHotspot -XX:MaxPermSize=320M
   ```

   c. Modify the MaxPermSize value to 784M.

   ```
   AddVMOptionHotspot -XX:MaxPermSize=784M
   ```

   d. Save your edit and close the file.

   e. Close JDeveloper.

9. In the open terminal window, restart JDeveloper by issuing the following commands:

   ```
   cd /home/oracle
   sh startJDev.sh
   ```

   Tip: Each time you are asked to start JDeveloper in this course, use this command.

10. Start the Integrated Server and configure a domain.

    Note: Launching the Integrated WebLogic Server from JDeveloper generates a domain, which is designated as the default domain. However, you also have the option of launching a JDeveloper-independent version of the WebLogic Server to configure a stand-alone domain. For more information, see “Configuring a Domain” in *Installing SOA Suite and Business Process Management Suite Quick Start for Developers.*

    a. In the JDeveloper main menu, select Run > Start Server Instance.

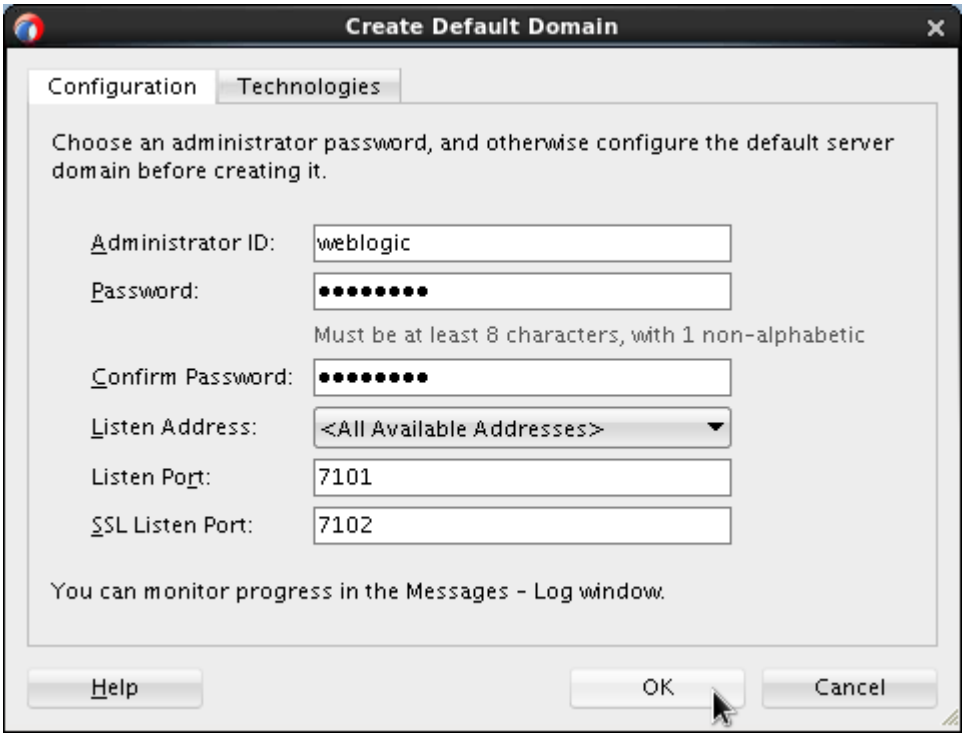
    The Create Default Domain dialog box opens.

    b. Provide and confirm the password **`welcome1`**.

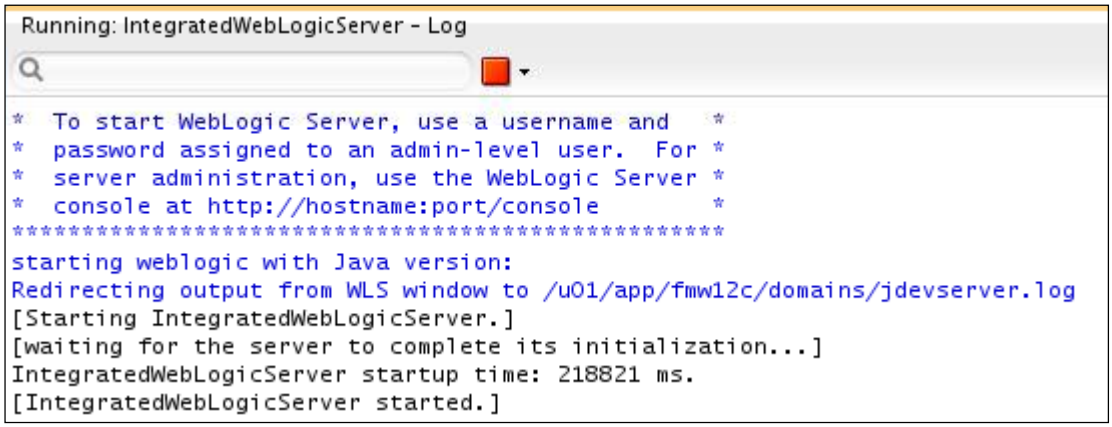
    c. Click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Domain creation begins. This takes several (about 10) minutes. You can monitor progress in the Running: IntegratedWebLogicServer pane at the bottom center of JDeveloper screen.



11. Verify your domain.

    a. Open a web browser and access Enterprise Manager Fusion Middleware Control, which is located at http://localhost:7101/em.

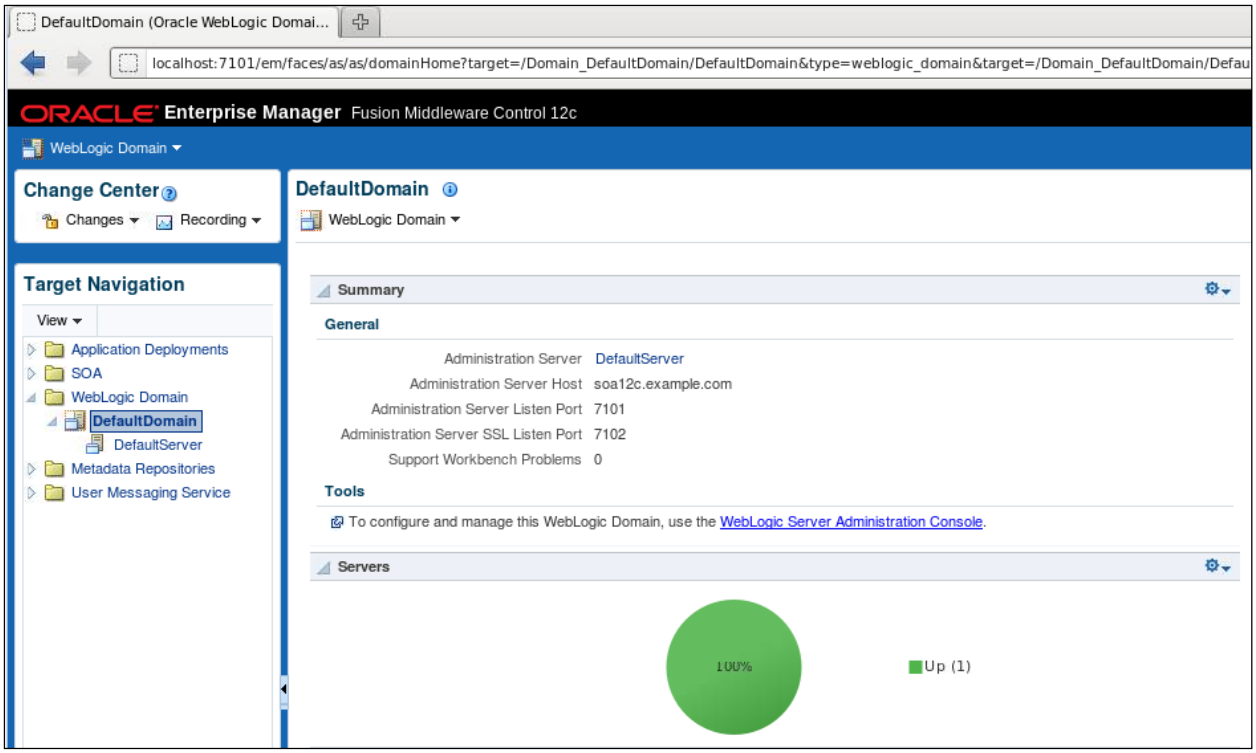
       Tip: You may wish to bookmark this URL. You will use it frequently in this course.

    b. Log in by using the username `weblogic` and password `welcome1`.

       You are logged in to DefaultDomain. The server status is shown as Up.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



c. Log out of Enterprise Manager.

12. Disable Secure Sockets Layer (SSL).

    SSL is enabled by default in the Integrated WebLogic Server. We do not have stringent design time requirements, and some later practices are simplified by not using it, so we disable SSL by completing the following steps:

    a. Go to the Administrator Console at http://localhost:7101/console.

       Tip: You may wish to bookmark this URL. You will use it again in this course.

    b. Log in by using the username `weblogic` and password `welcome1`.

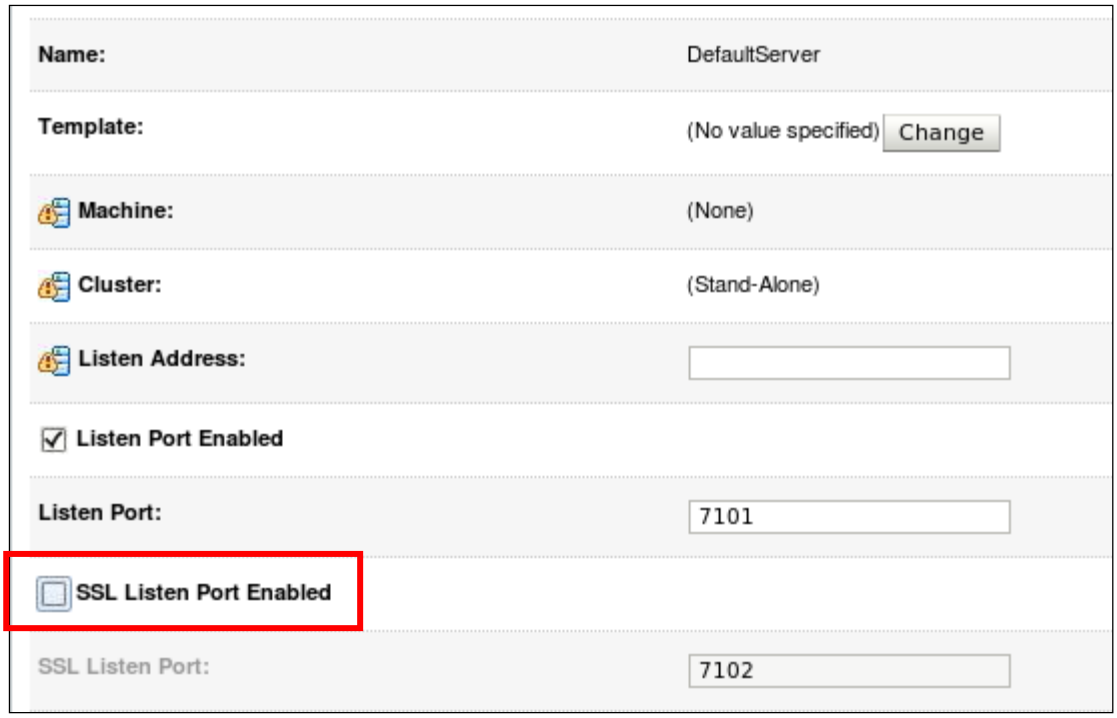
    c. In the Domain Structure pane, click Environment > Servers.

    d. In the main pane, click DefaultServer(admin).

    e. Deselect **SSL Listen Port Enabled**.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



f. Click Save.

g. Log out of Fusion Middleware Console.

h. From the JDeveloper main menu, stop and restart the Integrated WebLogic Server.

- Select Run > Terminate > IntegratedWebLogicServer.
- Select Run > Start Server Instance (IntegratedWebLogicServer).

Summary

In this practice, you have completed the one-time task of installing Oracle SOA Suite. You have also been introduced to several tasks that you execute repeatedly in this course, including:

- Accessing Enterprise Manager
- Accessing Fusion Middleware Console
- Starting and stopping the Integrated WebLogic Server

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 2: Getting Started with Composite Applications

Chapter 2

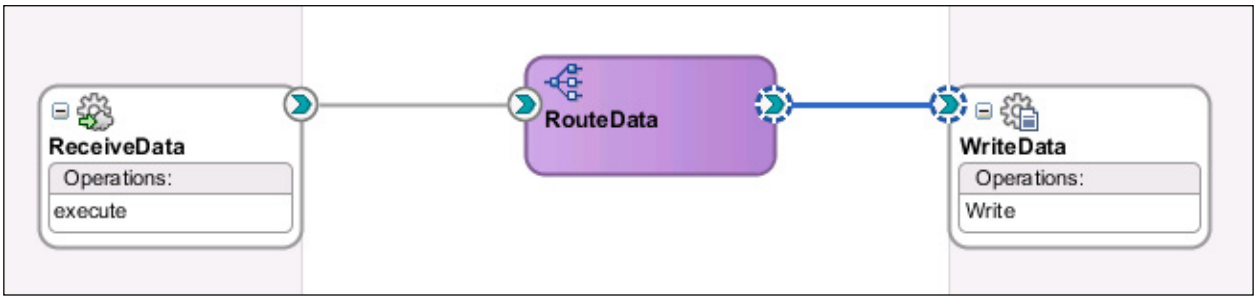
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 2: Overview

Practices Overview

In this practice, you create, deploy, and run a simple SOA composite application. The goal of this practice is to become acquainted with the development environment and to begin forming an understanding of composite application components.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 2-1: Creating the Composite Application

Overview

In this practice, you create a composite application that includes a Mediator component that routes incoming data to a File adapter service reference. The File adapter writes the data to a file.

Assumptions

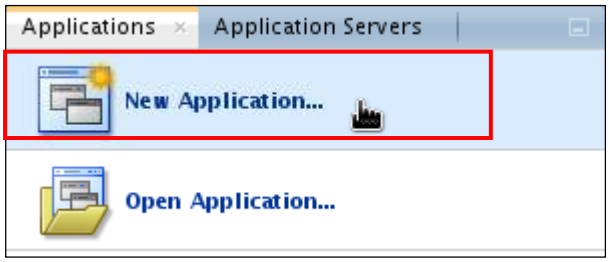
This practice assumes that you have started JDeveloper.

Tasks

1. Create a new application.

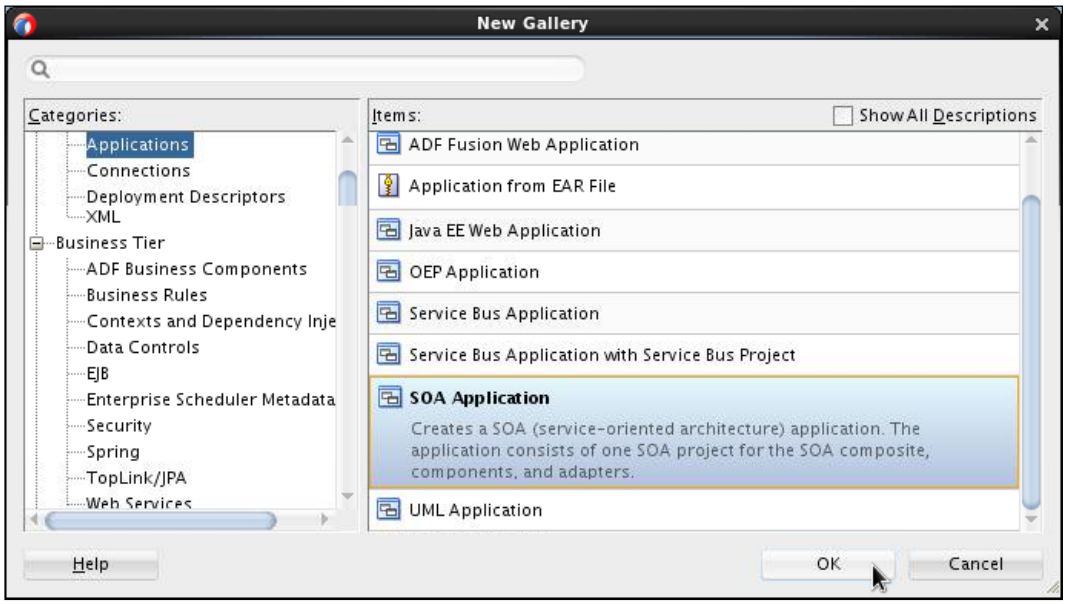
   Note: When you create a SOA application, Oracle JDeveloper creates a project that contains all the source files related to your application. You can then use Oracle JDeveloper to create additional projects needed for your application.

   a. In the JDeveloper Application Navigator, select New Application.



   The New Gallery dialog box is displayed. The Applications Category is selected by default.

   b. Select SOA Application from the Items list and click OK.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Create SOA Application wizard is displayed. The first steps in building a new application are to assign it a name and to specify the directory in which to save the source files.

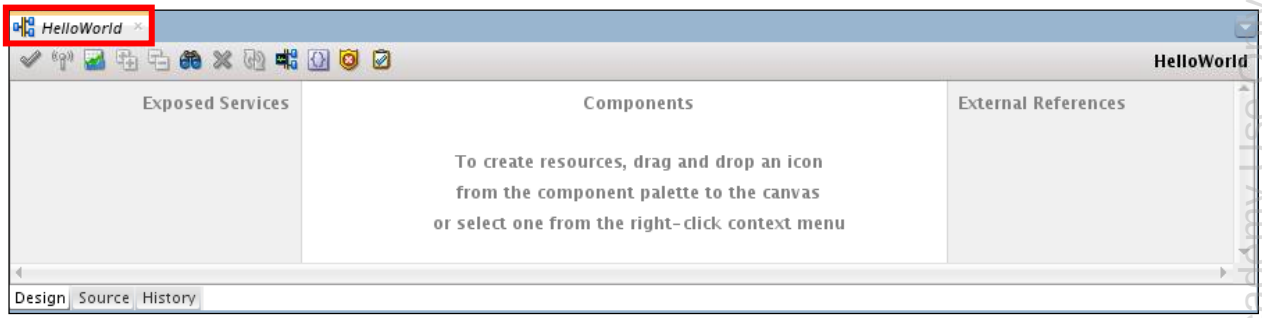
c. Supply `Basics` as the application name and click Next.

d. Supply `HelloWorld` as the project name and click Next.

The Project SOA Settings are displayed.

e. Click Finish. (A minute or more may elapse as the new project is created.)

Note: The *composite_name* file (in this case, *HelloWorld*) appears as a tab in the designer and as a file in the Applications window. This file (also known as the `composite.xml` file) is automatically created when you create a new SOA project. This file describes the entire composite assembly of services, service components, and references. There is one `composite.xml` file for each SOA project.



Note: The left swimlane is for services (such as web services, REST adapters, or JCA adapters) that provide an entry point to the SOA composite application. The right swimlane is for references that send messages to external services in the outside world, such as web services or JCA adapters. You drag and drop service components such as BPEL processes, business rules, human tasks, Oracle Mediators, and spring components onto the designer in the Components swimlane.

f. Select File > Save All (or click the Save All icon on the toolbar) to save the changes to your application and project.

Creating and Configuring the Exposed Service Interface

This interface describes how a client can call the composite application.

2. In the overview window, drag a SOAP technology component from the Technology section of the Component Palette into the Exposed Services column.

   The Create Web Service dialog box opens.

3. To configure the interface for the web service interface, perform the steps in the following table:

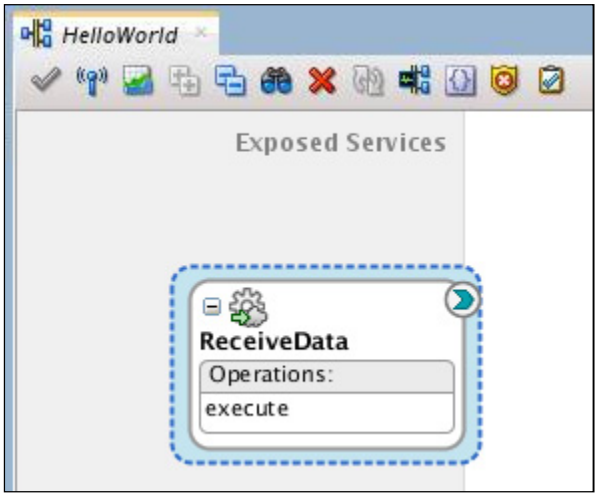
Step	Window/Page Description	Choices or Values
a.	Create Web Service	Name: ReceiveData Next to the WSDL URL field, click the “Generate WSDL from schema(s)” icon ().
b.	Create WSDL	Click the “Add a new message part” icon ().

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Step	Window/Page Description	Choices or Values
c.	Add Message Part	Click the “browse for schema file” icon ( ).
d.	Type Chooser	Click the Import Schema ( ) icon.
e.	Import Schema File	With FileSystem selected, locate the `/home/oracle/labs/files/xsd` folder and select `po.xsd`. Click OK.
f.	Localize Files	Copy Options: Deselect the “Maintain original directory structure for imported files” option. Click OK.
g.	Type Chooser	Expand the Project Schema Files > `po.xsd` entry (if needed), and select the PurchaseOrder entry. Click OK.
h.	Add Message Part	Click OK.
i.	Create WSDL	Click OK.
j.	Create Web Service	Click OK.

4. In the overview window, verify that the ReceiveData icon appears in the Exposed Services column. Example:



5. Select File > Save All (or click the Save All icon on the main toolbar) to save the changes to the project.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Creating and Wiring the Service Mediator Component

In this step, you add a Mediator component, which receives data from the application's client, and then routes that data to a File adapter (which you build in the next step).

6. Create a Mediator component and wire the ReceiveData service entry point to the Mediator component, by performing the following steps:

   a. Drag a Mediator component from the Component Palette into the Components column of the overview window.

      The Create Mediator dialog box opens.

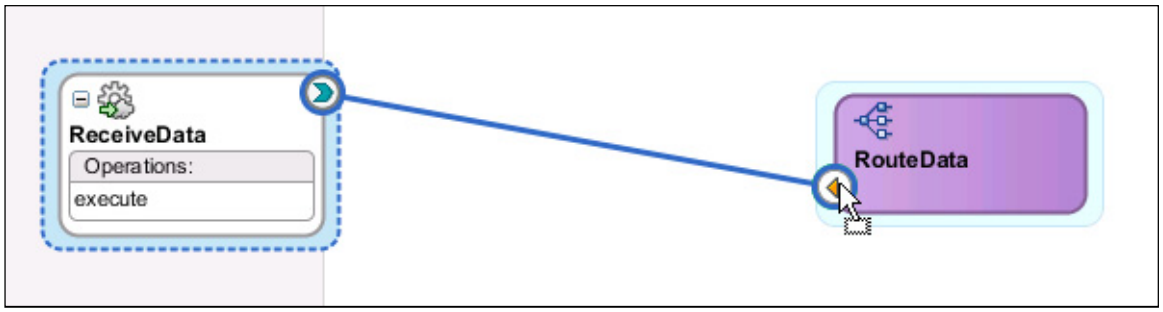
   b. Set the name to RouteData.

   c. Accept the [default] Define Interface Later template.

   d. Click OK.

      Note: The Mediator interface is defined in the next step by wiring the SOAP service to the Mediator component. You can drag a defined interface to an undefined interface in either direction (reference to service or service to reference). The undefined interface then inherits the defined interface.

   e. Create a wire from the ReceiveData exposed service icon to the RouteData Mediator icon, by dragging the right-edge arrow icon from the ReceiveData exposed service to the left-edge arrow on the Mediator component. Use the following image as a guide:



7. Select File > Save, or click the Save All icon to save the changes to your application.

Creating a File Adapter as an External Reference

You now create an external reference for the File adapter to write the input message structure to a file in the file system.

8. In the overview window, drag a File adapter component from the Technology section of the Component Palette onto the External References column.

   The FILE Adapter Configuration wizard opens.

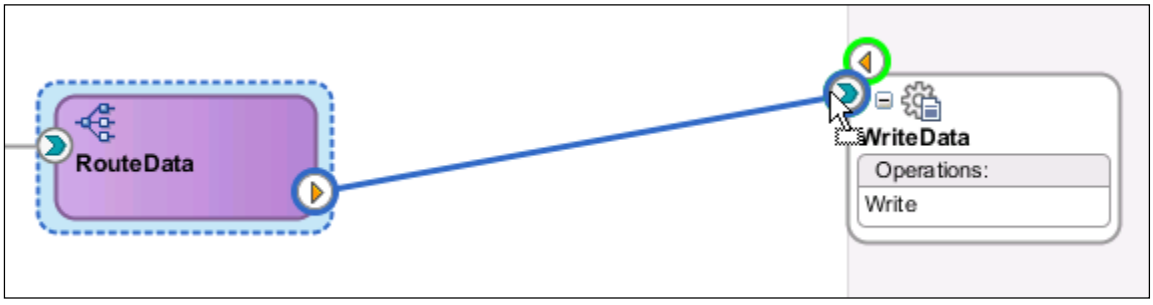
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

9. On the Adapter Configuration wizard pages, use the instructions in the following table to configure the File Adapter:

Step	Window/Page Description	Choices or Values
a.	File Adapter Reference	Service Name: WriteData Click Next.
b.	Adapter Interface	Accept the [default] "Define from operation and schema (specified later)" option. Click Next.
c.	File Server Connection	File Server JNDI Name: `eis/FileAdapter` Click Next.
d.	Operation	Select the Write File option. Click Next.
e.	File Configuration	Directory for Outgoing Files: `/home/oracle/labs/output/podata` File Naming Convention: `order_%SEQ%.xml` Click Next.
f.	Messages	Click the "browse for schema file" icon.
g.	Type Chooser	Expand the Project Schema Files > `po.xsd` entry and select `PurchaseOrder`. Click OK.
h.	Messages	Click Next.
i.	Finish	Click Finish.

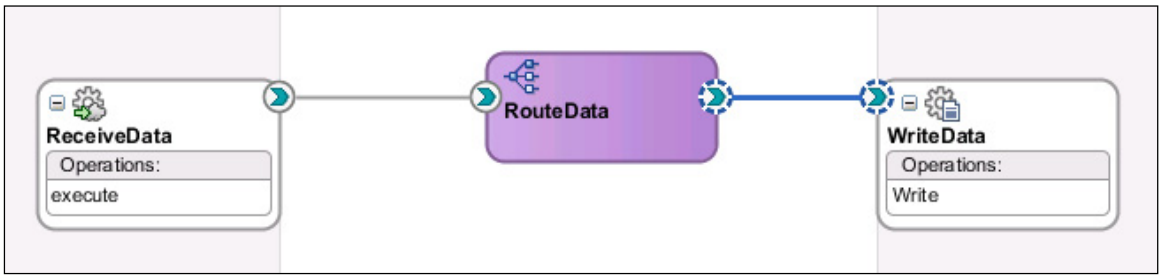
10. Create a wire from the Mediator component to the File adapter by dragging the right-edge arrow of the Mediator component to the left-edge arrow on the File adapter icon. Use the following image as a guide:



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

11. Verify and save your work.



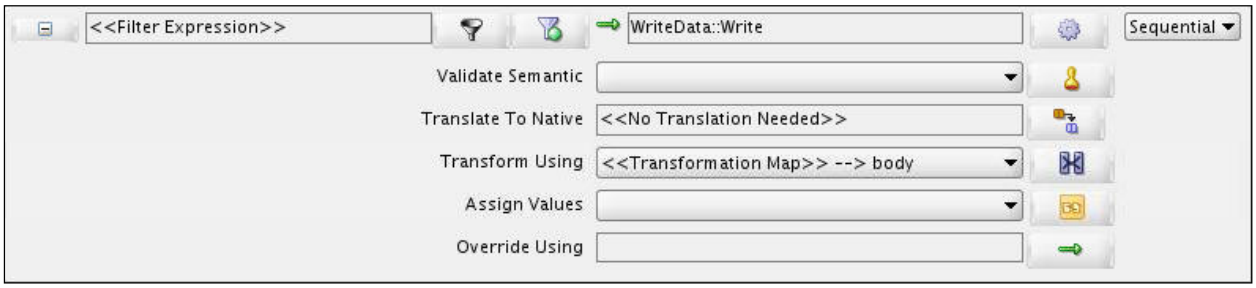
Adding a Transformation to the Mediator

This transformation reformats the incoming data before passing it to the File adapter.

12. In the overview window, right-click Mediator RouteData and select Edit.

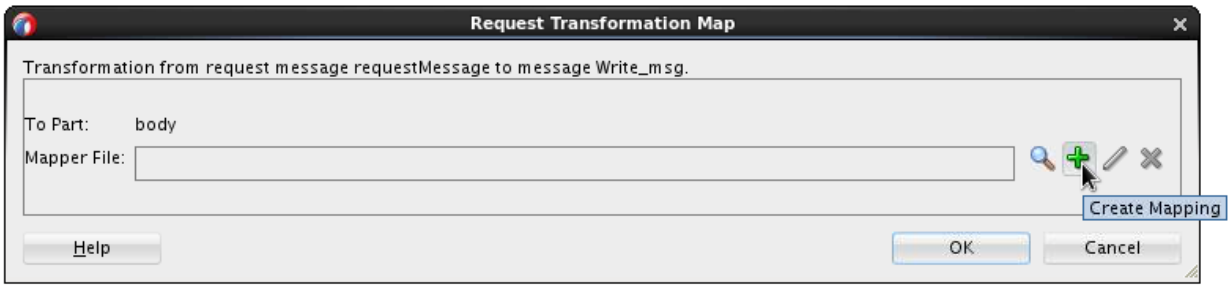
    The Mediator configuration editor (`RouteData.mplan`) opens.

13. Click the "Select an existing mapper file or create a new one" icon next to the Transform Using field.



    The Request Transformation Map dialog box is displayed.

14. Click the Create Mapping icon.

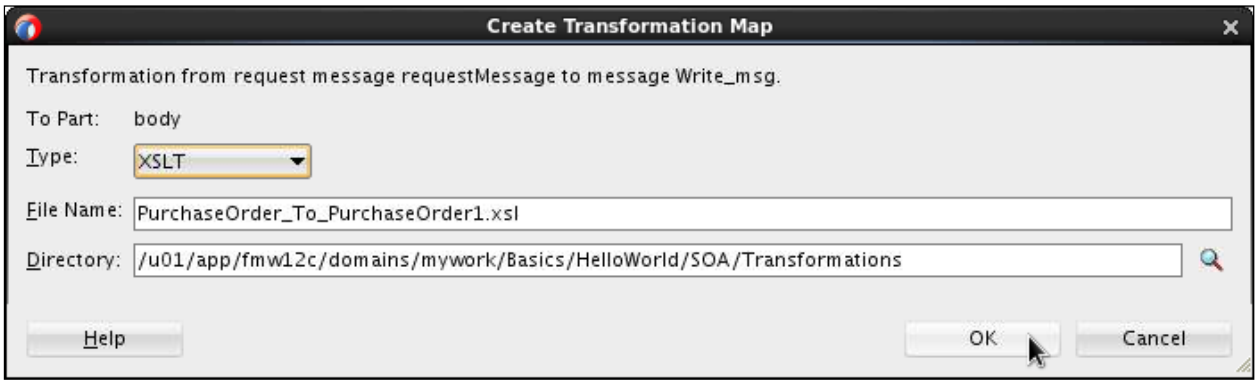


    The Create Transformation Map dialog box opens.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

15. To accept the default values, click OK.



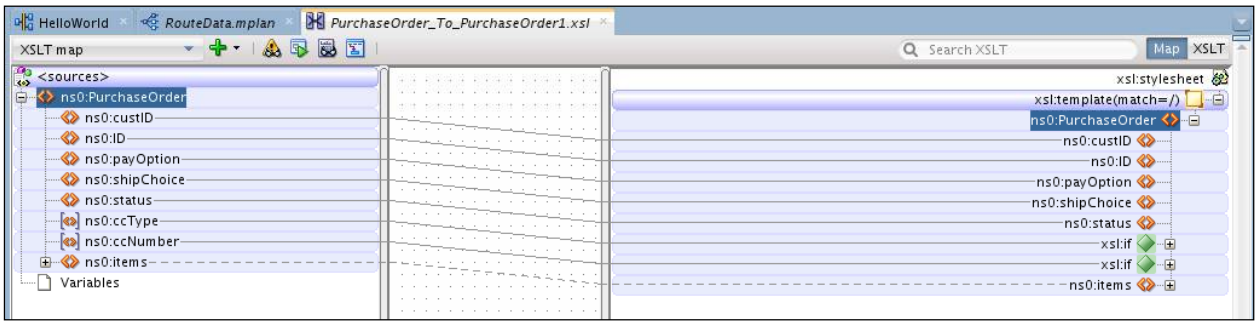
The XSLT Mapper (`PurchaseOrder_To_PurchaseOrder1.xsl`) window opens.

16. Click OK to close the Request Transformation Map dialog box.
17. Drag the PurchaseOrder element in the source column and drop it onto the PurchaseOrder element in the target column.

    The AutoMap Preferences window opens.

18. Click OK to accept the defaults.

    The AutoMap feature generates an XSLT mapping of the source nodes to the destination nodes.



Note: Lots of interesting things are happening in this window! We will discuss data transformation in more detail a little later. For now, we have generated the minimum possible transformation to allow us to focus our attention elsewhere for this first composite application.

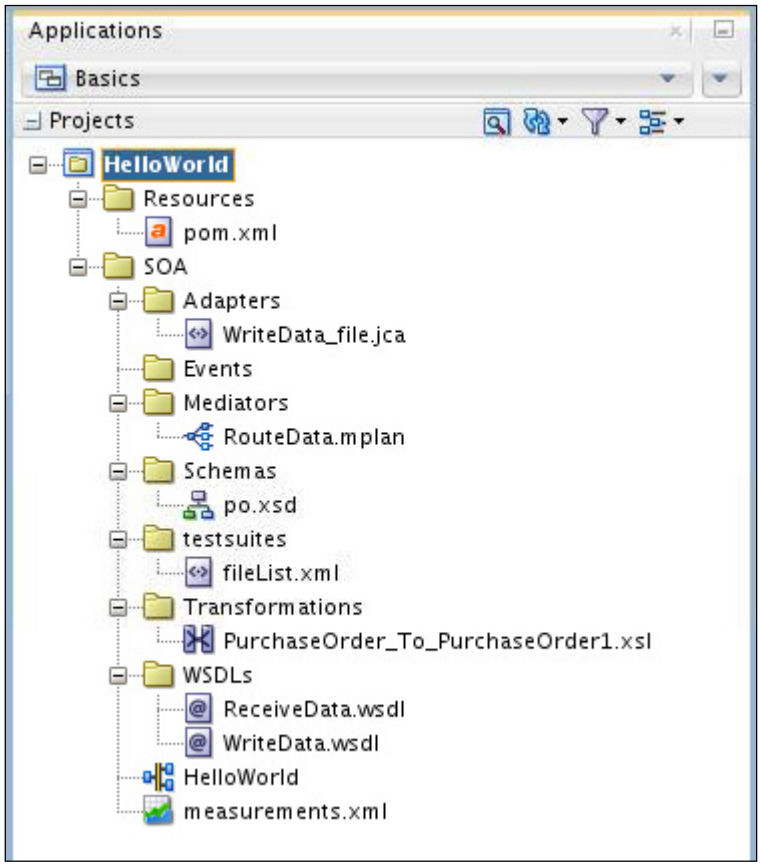
19. Save your work. Close the `.xsl` and `.mplan` file editors.

    Note: The next lesson titled “Managing and Monitoring a Composite Application” introduces the SOA Suite runtime environment, and the tools for managing, monitoring, and testing composite applications. In the practice for that lesson, you will deploy and test this application.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

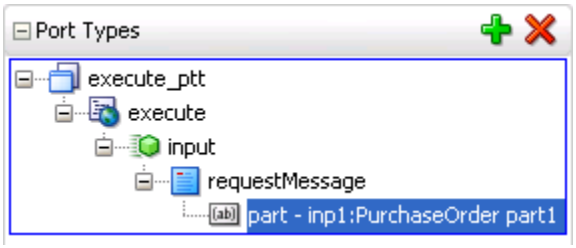
Examining the Generated Configuration Files



20. Open the `ReceiveData.wsdl` file in JDeveloper and consider the following questions:

    a. In the following diagram, what does `inp1:PurchaseOrder` reference?

    b. To learn more about `PurchaseOrder`, which file would you open?

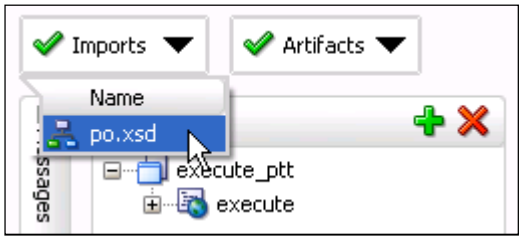


    c. The reference `inp1:PurchaseOrder` names the element of the message that describes the structure of the request message.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

d. Because the message formats are described in an imported XSD file, to learn more about PurchaseOrder, you would open the file `po.xsd`.



21. Close the `ReceiveData.wsdl` file and the Component—RouteData—Properties window.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 3: Managing and Monitoring

Chapter 3

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 3: Overview

Practices Overview

In this practice, you first deploy the HelloWorld composite application project to the SOA server and use the Enterprise Manager web application to test the service with a sample input. You then modify the HelloWorld File adapter to use a logical name and rerun your test. Finally, you create a configuration plan for the HelloWorld composite application to replace the destination directory for the `orderfiles` logical name reference property.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 3-1: Deploying and Testing the HelloWorld Composite Application

Overview

In this practice, you deploy the HelloWorld composite application project to the SOA server and use the Enterprise Manager web application to test the service with a sample input. To complete this task, perform the following steps:

Assumptions

This practice assumes that you have completed Practice 2 successfully.

Tasks

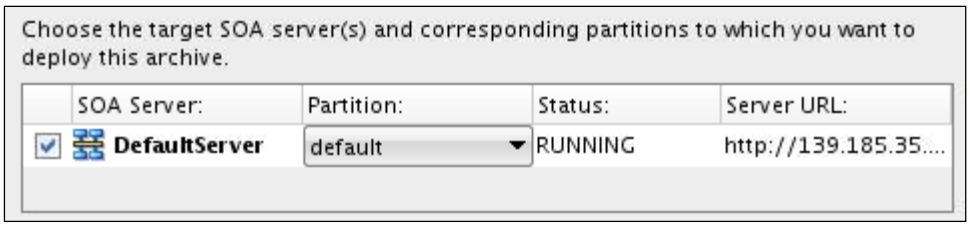
Deploying the HelloWorld Composite Application

1. In the JDeveloper Application Navigator, right-click the HelloWorld project and select Deploy > HelloWorld.

   The Deploy HelloWorld wizard opens.

2. Use the instructions in the following table to complete the deployment:

Step	Window/Page Description	Choices or Values
a.	Deployment Action	Deploy to Application Server. Click Next.
b.	Deploy Configuration	Select the “Overwrite any existing composites with the same revision ID” check box. Click Next.
c.	Select Server	IntegratedWebLogicServer Click Next.
d.	SOA Servers	Verify that the DefaultServer status is RUNNING as shown in the following screenshot. Click Next.
e.	Summary	Click Finish.

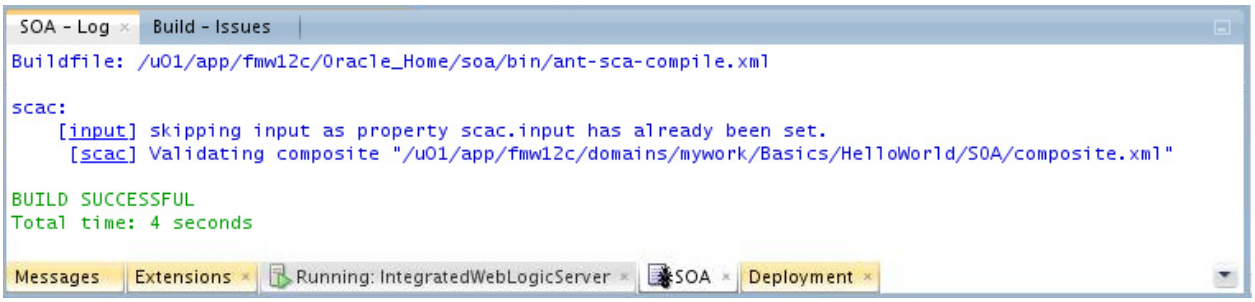


Deployment processing starts.

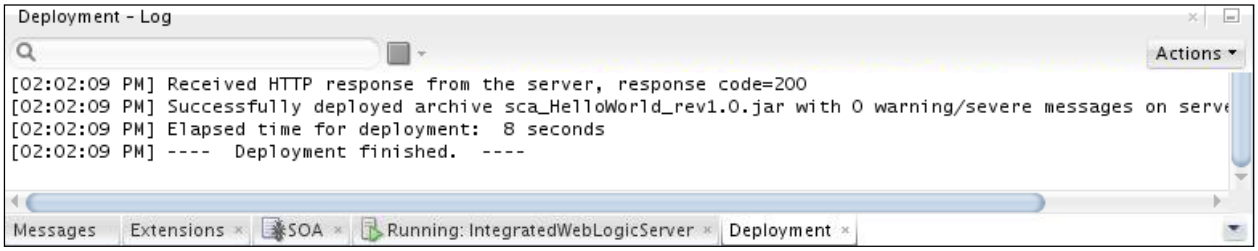
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

3. Monitor deployment progress and check for successful compilation in the SOA – Log window.



4. Verify that deployment is successful in the Deployment – Log window, similar to what is shown in the following screenshot:



Testing HelloWorld by Using Enterprise Manager Web Interface

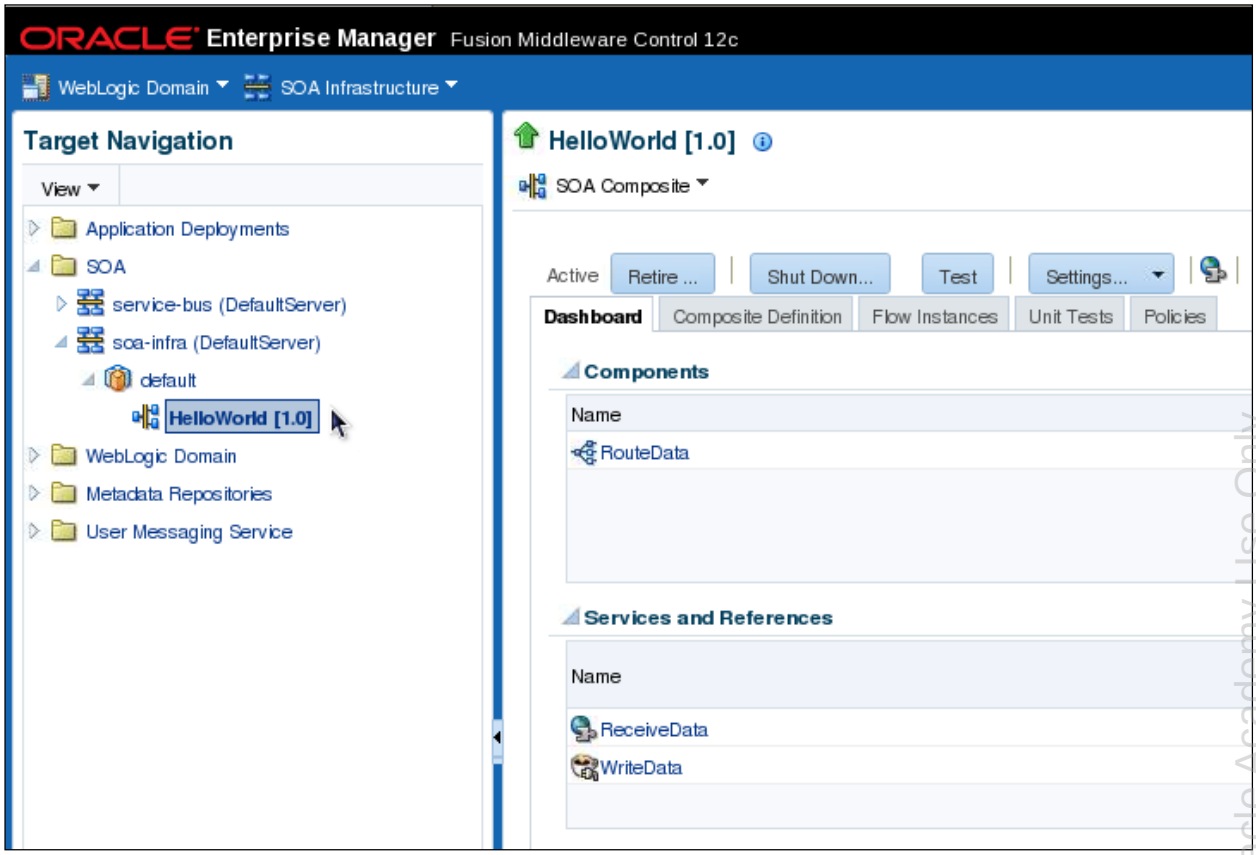
In this section, you use a web browser to access Oracle Enterprise Manager 12*c* Fusion Middleware Control and initiate a test of the deployed HelloWorld application.

5. Log in to the Oracle Enterprise Manager Fusion Middleware Control 12*c* application by performing the following steps:
   a. Open a web browser and provide the URL http://localhost:7101/em.
   b. Log in by using the username `weblogic` and the password `welcome1`.

Oracle Internal & Oracle Academy Use Only

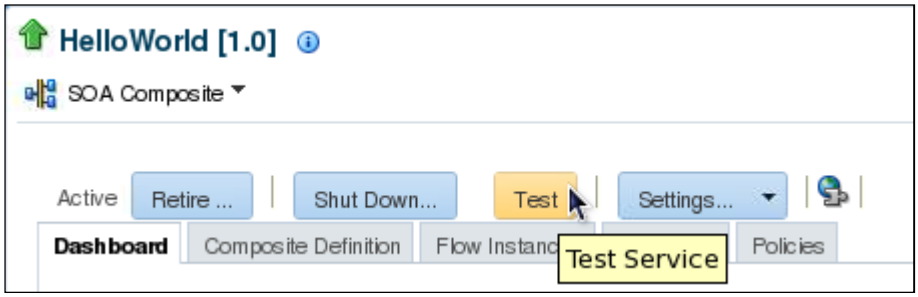
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

6. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree and click the “HelloWorld [1.0]” link.



7. To initiate a test of the HelloWorld composite, perform the following steps:

   a. On the “HelloWorld [1.0]” home page, click Test.

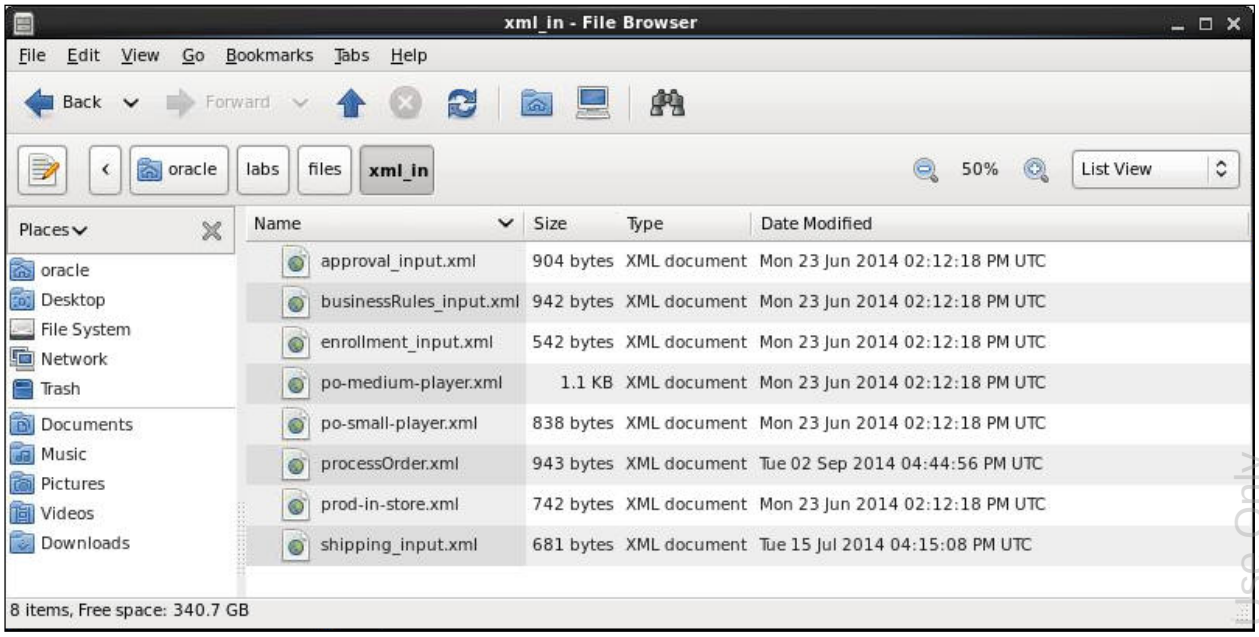


   b. On the Request tab, click the Browse button and navigate to the `/home/oracle/labs/files/xml_in` folder.

Oracle Internal & Oracle Academy Use Only

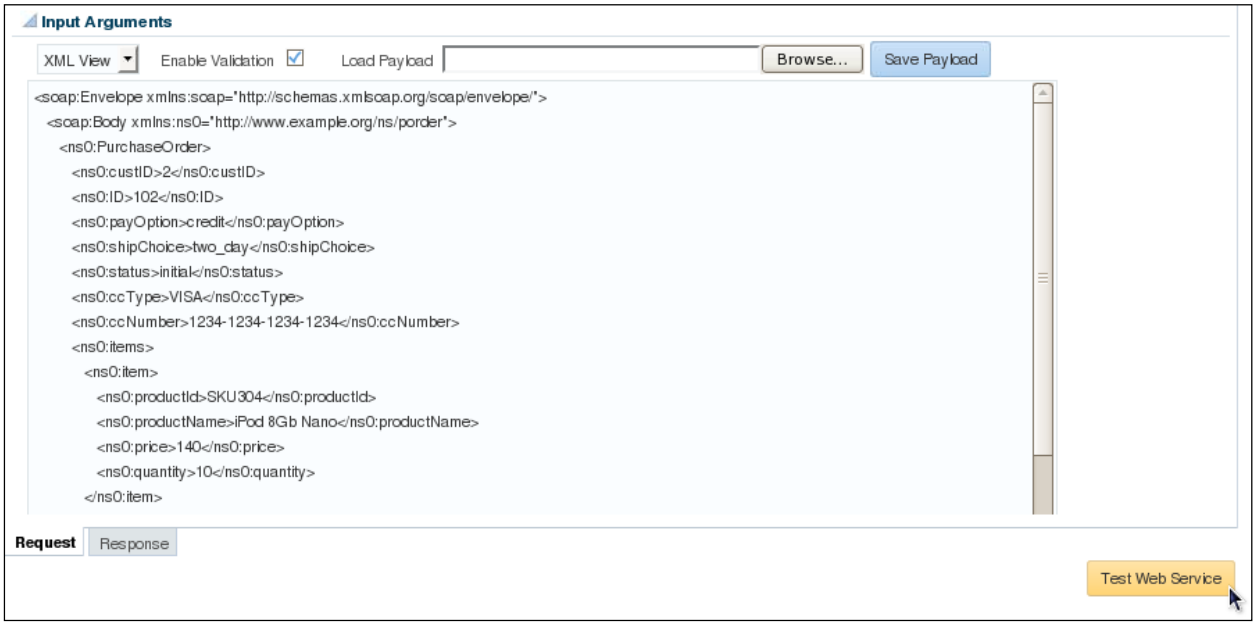
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

c. Select `po-medium-player.xml`. Click Open.



The initially supplied XML text is replaced by the contents of the file.

8. Click Test Web Service.

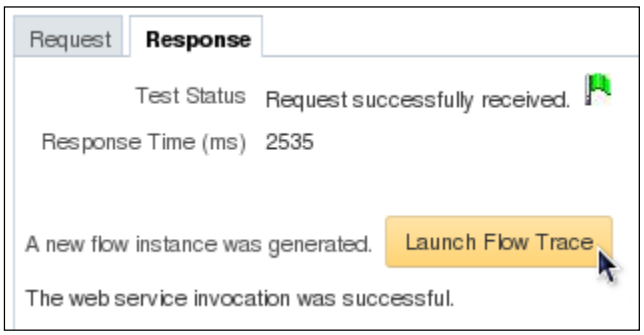


Note: This action sends the XML data as an input message to the HelloWorld composite application. It might take several seconds for the page to refresh with the response that is displayed on the Response tab. Wait until the page is refreshed.

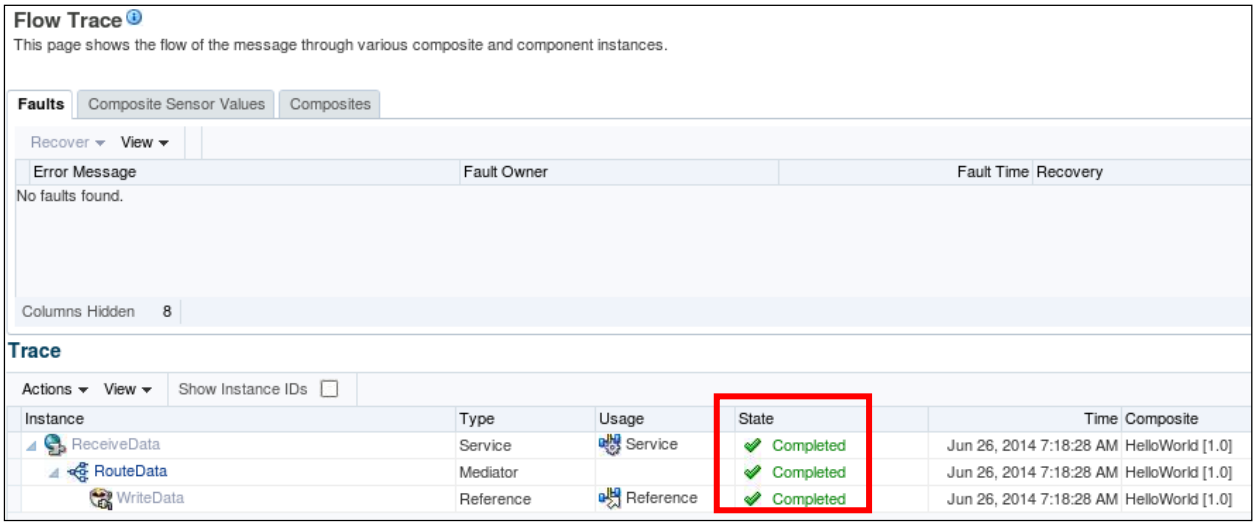
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

9. On the “HelloWorld [1.0]” Response tab page, click the Launch Flow Trace button to view the results of the asynchronous composite application.



10. On the Flow Trace page, verify that the ReceiveData, RouteData, and WriteData components have a Completed state, indicating that the application executed successfully.

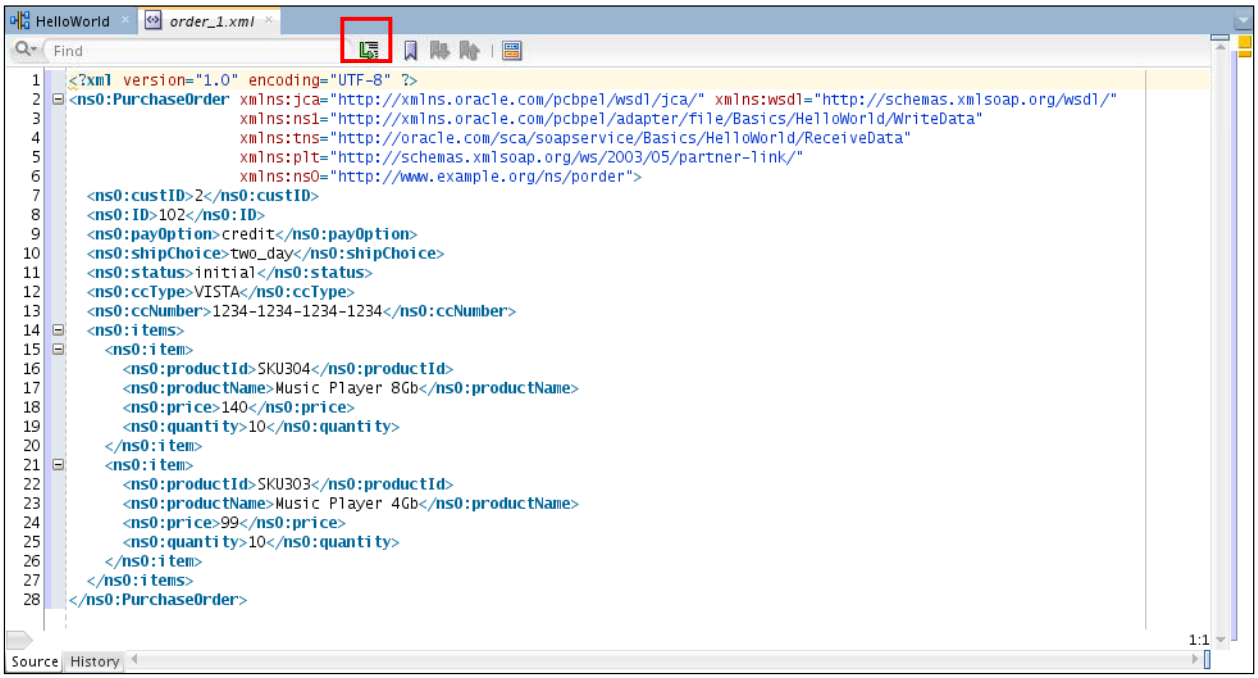


11. Close the Flow Trace window and minimize your browser.
12. Using JDeveloper, open the output file `/home/oracle/labs/output/podata/order_1.xml` to view its contents.
13. Click the Reformat icon.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

14. Verify that the XML file contains data similar (or identical) to the following screenshot:



15. In the JDeveloper window, close the `order_1.xml` window.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 3-2: Modifying the HelloWorld File Adapter to Use a Logical Name

Overview

In this practice, you modify the HelloWorld composite application and alter the File adapter to use a logical name instead of an explicit directory for orders written to the file system. You then create a binding property that is to set the directory to the `/home/oracle/labs` directory, deploy the application by using JDeveloper, and test that the order is written to a file in the `/home/oracle/labs` directory.

Assumptions

This practice assumes that you have completed Practice 3-1 successfully.

Tasks

Modifying the File Adapter to Use a Logical Name

1. If the HelloWorld overview window is not already open, in the JDeveloper Application Navigator, expand the HelloWorld project and open the `HelloWorld` file.
2. Right-click the WriteData external reference icon and select Edit.
3. Edit the File adapter external reference to use a logical name instead of a physical path. Use the following table of instructions as a guide:

Step	Window Description	Choices or Values
a.	File Adapter Reference	Click Next.
b.	Adapter Interface	Click Next.
c.	File Server Connection	Click Next.
d.	Operation	Click Next.
e.	File Configuration	Directory Specified as: Logical Name Directory for outgoing files (logical name): `orderfiles` Click Next.
f.	Messages	Click Next.
g.	Finish	Click Finish.

4. To set the path value for the `orderfiles` logical name, perform the following steps:
   a. To display the Properties pane, in the JDeveloper main menu, select Window > Properties.

      The Property Inspector for the Reference, WriteData, is displayed in a tabbed pane.

   b. Expand the Adapter and Composite properties.

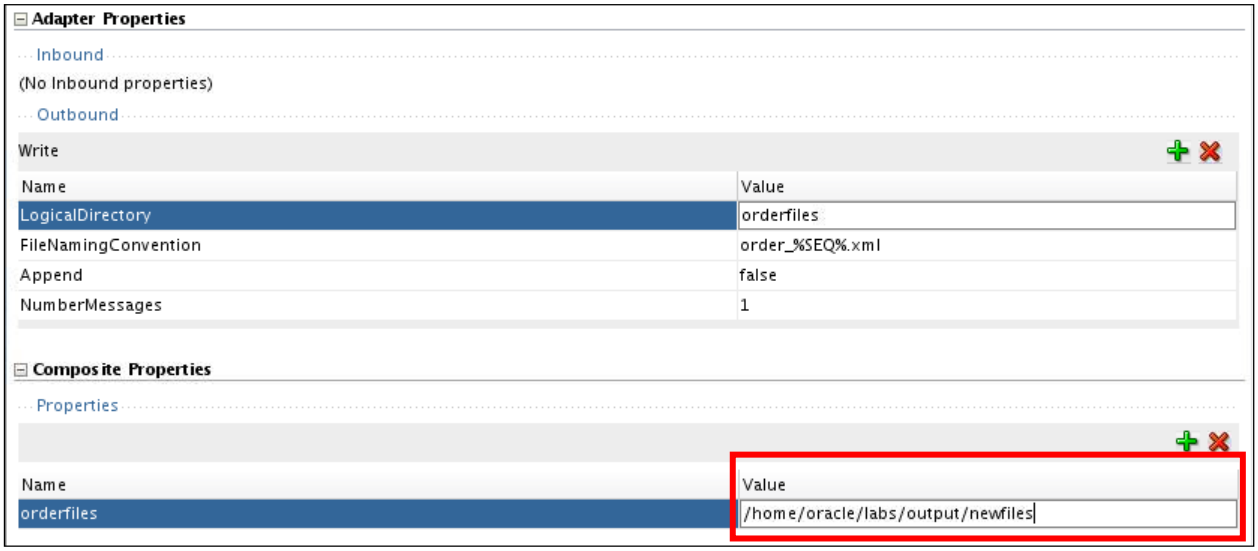
      The adapter includes a reference property called LogicalDirectory with the value `orderfiles`.

   c. Set the `orderfiles` reference property. In the Composite properties, click in the Value cell and enter the string value `/home/oracle/labs/output/newfiles`.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

d. Press Enter.



5. Select File > Save All to save the changes to the application.

Deploying the Modified HelloWorld Composite Application

6. In the JDeveloper window, redeploy the modified HelloWorld project by performing the following steps:

   a. In the Application Navigator, right-click the HelloWorld project name and select Deploy > HelloWorld to IntegratedWebLogicServer.

   b. In the Deployment – Log window, observe the log messages to ensure that deployment was successful.

Testing the Modified HelloWorld Composite Application

7. Restore or re-open your browser and access Enterprise Manager. (The URL is http://localhost:7101/em. Log in as `weblogic` with the password `welcome1`.)

   Enterprise Manager opens.

8. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree and click the “HelloWorld [1.0]” link.

9. Supply test data.

   a. On the “HelloWorld [1.0]” page, click Test.

   b. In the Input Arguments section, in Tree View mode, enter the following values:

custID:	`1`
ID:	`2`
payOption:	`credit`
shipChoice:	`two_day`
status:	`Initial`
ccType:	`AMCD`

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

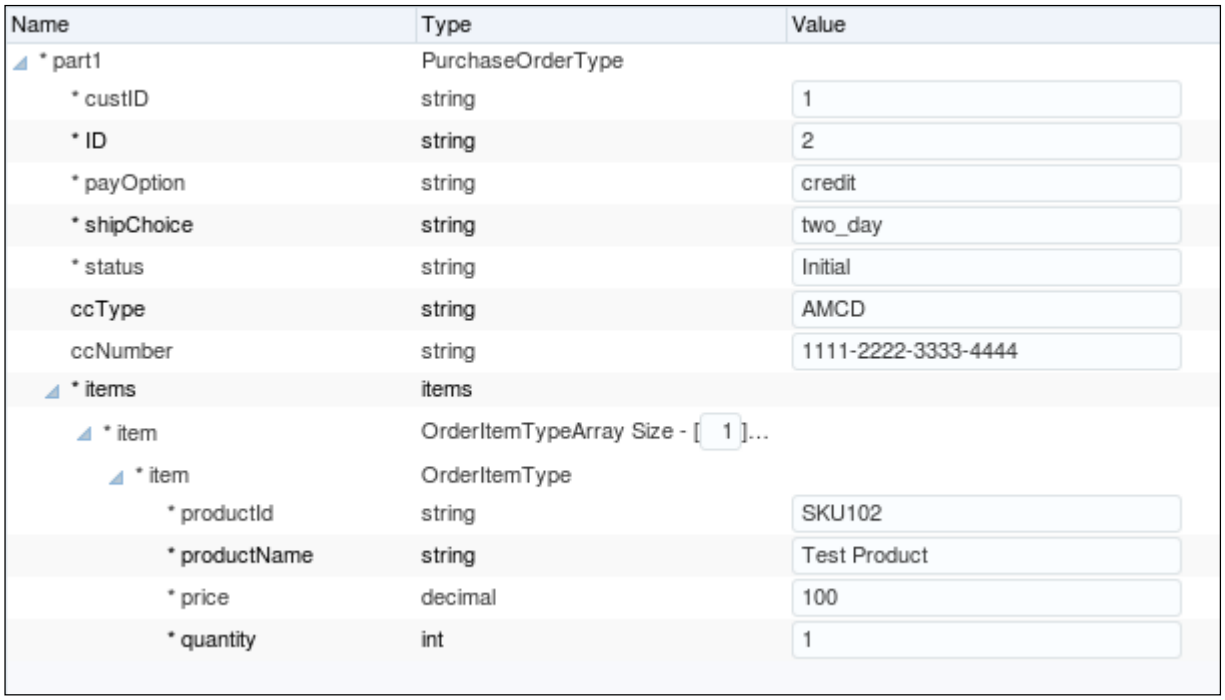
ccNumber:	`1111-2222-3333-4444`

c. While still in Tree View mode, expand the “items” field, and in the item field, enter the value `1` in OrderItemTypeArray Size. Click the Refresh icon.

d. Expand the OrderItemTypeArray item and the OrderItemType item, and enter the following field values:

productId:	`SKU102`
productName:	`Test Product`
price:	`100`
quantity:	`1`

e. Verify your input.



10. Click Test Web Service.

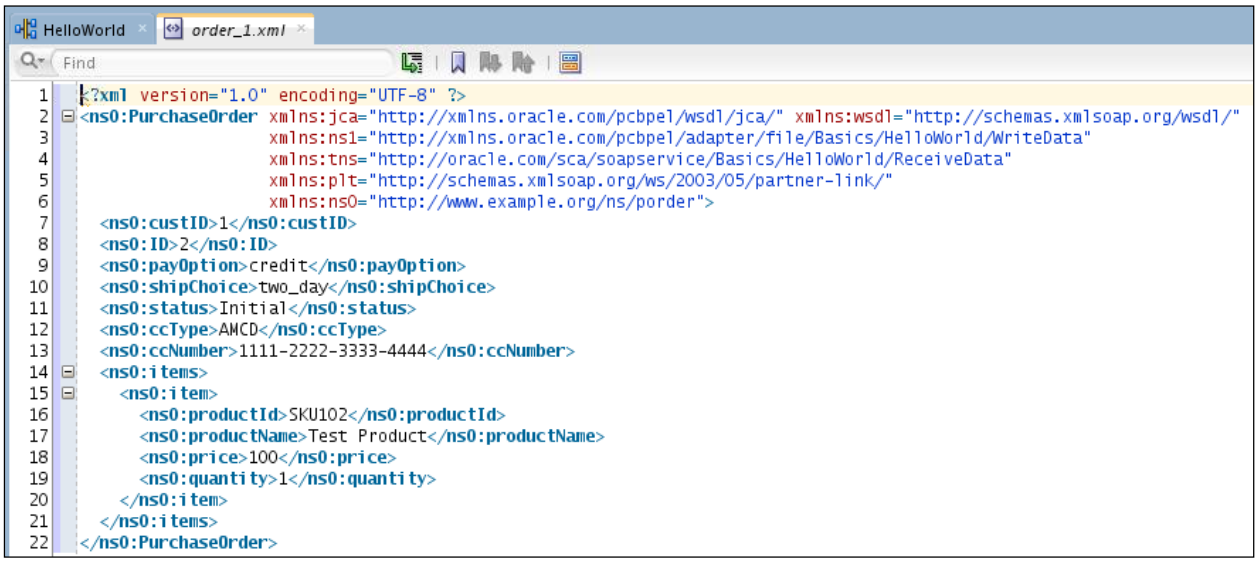
    The web service is tested. The “HelloWorld [1.0]” Response tab page is displayed. The message “Request successfully received.” is displayed.

11. In JDeveloper, open the file `/home/oracle/labs/output/newfiles/order_1.xml`.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

12. Verify that the file contains the values entered in the test page form fields.



13. In JDeveloper, close the `order_1.xml` file.
14. Close or minimize the open browser windows.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 3-3: Generating and Validating a Configuration Plan

Overview

As you move projects from one environment to another (for example, from testing to production), you typically must modify several environment-specific values, such as JDBC connection strings, host names of various servers, and so on. Configuration plans enable you to modify these values by using a single text (XML) file. The configuration plan is created in either Oracle JDeveloper or from the command line. During process deployment, the configuration plan is used to search the SOA project for values that must be replaced to adapt the project to the next target environment.

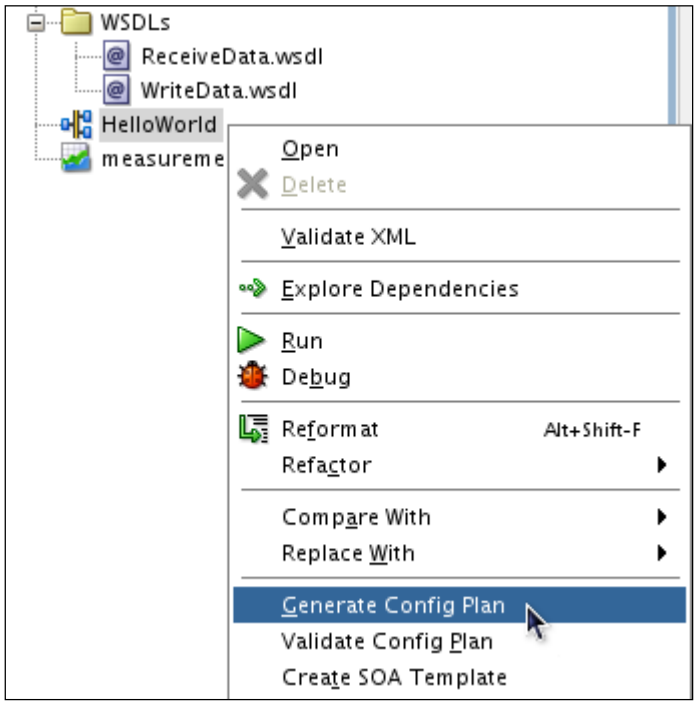
In this practice, you create a configuration plan for the HelloWorld composite application to replace the destination directory for the `orderfiles` logical name reference property. After you verify the configuration plan, you redeploy the composite application by using the Enterprise Manager web interface and applying the configuration plan, and then verify and test the changes.

Assumptions

This practice assumes that you have completed Practice 3-2 successfully.

Tasks

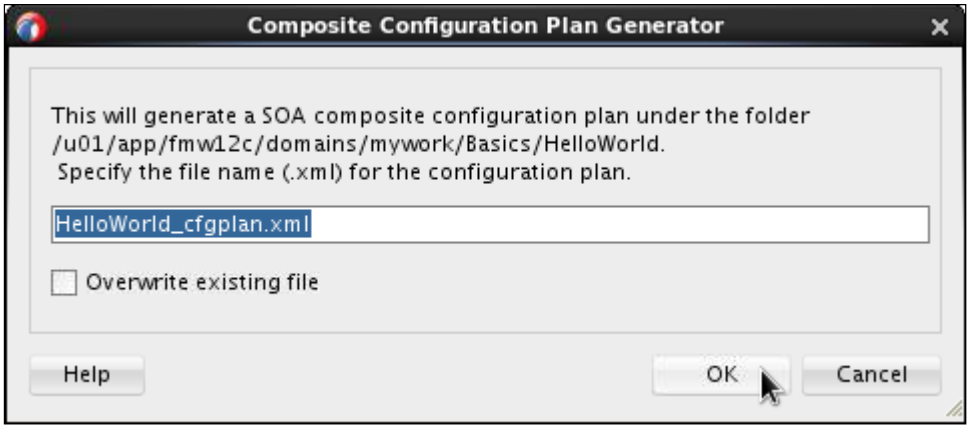
1. Generate a configuration plan.
   a. In the Application Navigator, expand the HelloWorld project.
   b. Right-click HelloWorld (the `composite.xml` file) and select Generate Config Plan.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

c. In the Composite Configuration Plan Generator dialog box, accept the default plan file name. Click OK.



JDeveloper opens the `HelloWorld_cfgplan.xml` file in its own tabbed window.

2. Modify the configuration plan.

   a. In the `HelloWorld_cfgplan.xml` window, scroll down until you locate the `<reference name="WriteData">` element.

   b. Change the value `/home/oracle/labs/output/newfiles` to `/home/oracle/labs/output/podata`.

   c. Compare your file to the following screenshot:

```
45      <reference name="WriteData">
46        <property name="orderfiles">
47          <replace>/home/oracle/labs/output/podata</replace>
48        </property>
```

   d. Save your work.

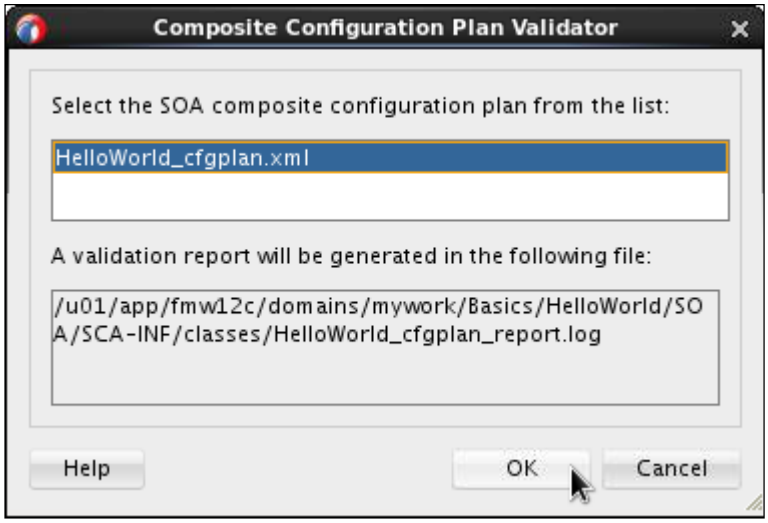
3. Validate the configuration plan.

   a. In the Application Navigator, right-click HelloWorld (the `composite.xml` file) and select Validate Config Plan.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

b. In the Composite Configuration Plan Validator dialog box, accept the default selection and click OK.



c. In the `HelloWorld_cfgplan_report.log` window, verify that the `orderfiles` Reference Property is replaced with the new value of `/home/oracle/labs/output/podata` that is specified in the configuration plan that is being validated.

```
19      Reference   [ WriteData ]
20        Property [ orderfiles ]
21            Old [ /home/oracle/labs/output/newfiles ]
22            New [ /home/oracle/labs/output/podata ]
```

In JDeveloper, close the configuration plan `.xml` and `.log` files.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 3-4: Testing a Composite That Is Deployed with a Configuration Plan

Overview

In this practice, you redeploy the HelloWorld composite application by using the configuration plan file that was generated and validated in Practice 3-3. You also test the deployed application to confirm whether the configuration plan changes are applied.

Assumptions

This practice assumes that you have completed Practice 3-3 successfully.

Tasks

1. Return to your web browser with the Enterprise Manager page and redeploy the HelloWorld composite application by using the deployment plan file that was modified in the previous practice. To complete this task, perform the instructions in the following table:

Step	Screen/Page Description	Choices or Values
a.	Target Navigation Pane	Expand SOA > soa-infra > default. Right-click “HelloWorld [1.0]”. Select SOA Deployment > Redeploy.
b.	“HelloWorld [1.0]” Redeploy SOA Composite	In the Archive or Exploded Directory section, accept the default “Archive is on the machine where this Web browser is running” option. Click Browse.
c.	File Upload	Navigate to the `/u01/app/fmw12c/domains/mywork/Basics/HelloWorld/deploy` folder. Select the `sca_HelloWorld_rev1.0.jar` file. Click Open.
d.	“HelloWorld [1.0]” Redeploy SOA Composite	In the Configuration Plan section, select the “Configuration plan is on the machine where this Web browser is running” option. Click Browse.
e.	File Upload	Navigate to the `/u01/apps/fmw12c/domains/mywork/Basics/HelloWorld` folder. Select the `HelloWorld_cfgplan.xml` file. Click Open.
f.	“HelloWorld [1.0]” Redeploy SOA	Scroll back to the top (if needed) and click Next.

Oracle Internal & Oracle Academy Use Only

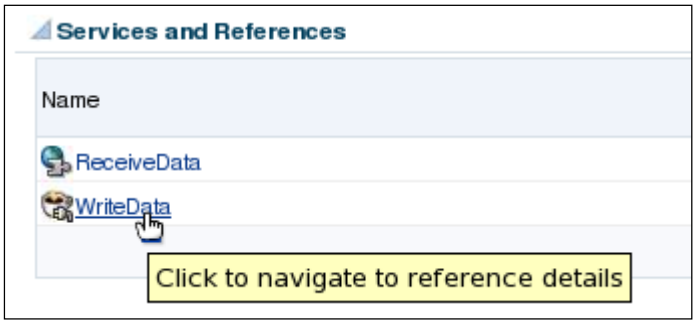
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Step	Screen/Page Description	Choices or Values
	Composite	
g.	"HelloWorld [1.0]" Redeploy SOA Composite	Click Redeploy.

2. The Redeployment Succeeded dialog box is displayed when the deployment successfully completes.



3. Click Close.
4. Click the "HelloWorld [1.0]" link in the Target Navigation pane to view the home page (if needed).
5. In the Services and References pane, click WriteData.



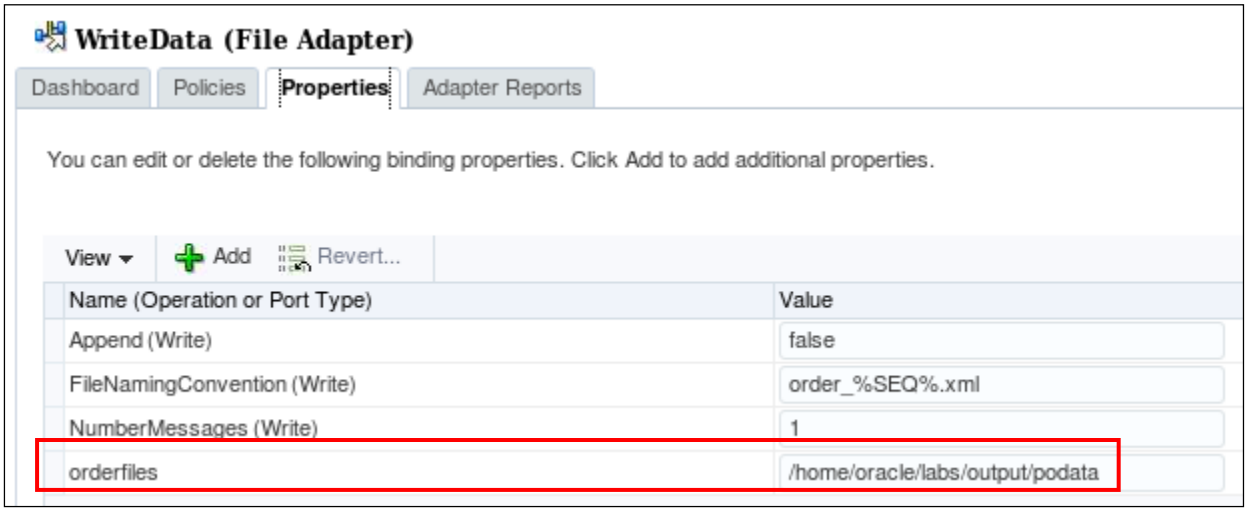
6. Click the Properties tab.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

7. Verify that the value is `/home/oracle/labs/output/podata`, as expected due to redeployment with the configuration plan.



Note: We have not modified the HelloWorld composite application source code and have not even re-created the SOA Archive file during the redeployment process.

8. Test that the HelloWorld composite application now writes order files in the `/home/oracle/labs/output/podata` folder, as defined by the configuration plan.
   a. In Enterprise Manager, on the HelloWorld home page, click Test.
   b. On the Input Arguments Request tab, select the XML View option.
   c. Use the Browse button to replace the supplied XML data with the contents of the `/home/oracle/labs/files/xml_in/po-small-player.xml` file.
   d. Click Test Web Service.
   e. Verify that the new HelloWorld composite application instance completed successfully on the Flow Trace page.
   f. Verify that the `order_2.xml` file has been created with the supplied data in the `/home/oracle/labs/output/podata` folder.

      Note: If more than two XML files exist in the `podata` subfolder, the file with the highest sequence number contains the latest order information.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 4: Working with Data

Chapter 4

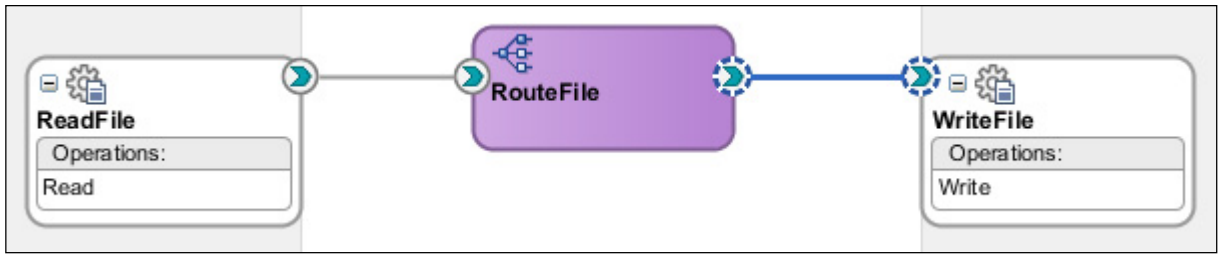
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 4: Overview

Practices Overview

In this practice, you create, deploy, and run a composite application that receives delimited data from a File adapter. This data is transformed to an XML format and passed to a Mediator. The Mediator transforms the data to a second XML format, and routes the file to an outbound File adapter, which passes the data from the composite application to the file system.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 4-1: Creating the Composite Application

Overview

In this section of the practice, you create a new composite application and add an external service interface.

Assumptions

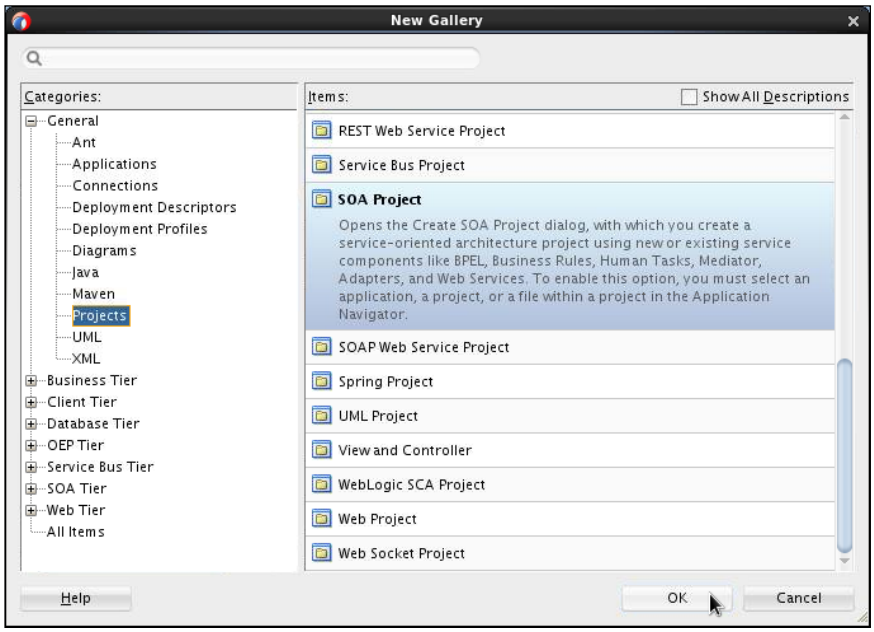
This practice assumes that you have started JDeveloper.

Tasks

1. Create a new SOA project named `NativeData`.
   a. In the JDeveloper Application Navigator menu, select New > Project.

      The New Gallery window appears.

   b. Select SOA Project and click OK.



      The Create SOA Project wizard appears.

   c. Name the project `NativeData` and click Next.

   d. Verify that the Empty Composite template is selected and click Finish.

      The composite overview window is opened.

      Note: Remember that it may take a minute or more to generate the new project.

   e. Save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Creating and Configuring the Exposed Service Interface

This interface describes how a client can call the composite application.

2. Drag a File adapter from the Component Palette into the Exposed Services column.

   The FILE adapter configuration wizard opens.

3. To configure the File adapter, use the instructions listed in the following table:

Step	Window Description	Choices or Values
a.	File Adapter Service	Service Name: `ReadFile` Click Next.
b.	Adapter Interface	Accept the default values. Click Next.
c.	File Server Connection	Accept the default values. Click Next.
d.	Operation Type	Select the Read File radio button. Click Next.
e.	File Directories	Use the Browse button to specify the Directory for Incoming Files: `/home/oracle/labs/input` Deselect Process Files Recursively. Select Archive Processed Files. Use the Browse button to specify the Archive Directory: `/home/oracle/labs/archive` Click Next.
f.	File Filtering	Include Files with Name Pattern: `po*.dat` Click Next.
g.	File Polling	Polling Frequency: 10 seconds Click Next.
h.	Messages	Click the “Define Schema for Native Format” (gear) icon.

   The Native Format Builder opens.

   Note: So far, we have specified that the File adapter is to poll the directory `/home/oracle/labs/input` every 10 seconds. If it finds any files that match the specification `po*.dat`, it moves them to the `/home/oracle/labs/archive` directory. It also passes a copy of the file to the composite application that we are about to build.

   Continue with Practice 4-2 to define the native data format of the incoming data.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 4-2: Defining the Native Data Format

Overview

In this section of the practice, you define the incoming native data format within an XSD so that the incoming adapter can reformat the incoming message as XML. The incoming message contains end-delimited purchase order data. The first line of the file includes customer and payment data associated with the purchase itself. That line of data is followed by a variable number of lines of data that describe each of the items in the order. The following example includes two item records:

```
2,100,credit,two_day,initial,VISTA,1234-1234-1234-1234
SKU301|Music Player 1Gb|45|3
SKU305|Music Player 120Gb|250|20
```

Assumptions

This practice assumes that you have completed Practice 4-1, and that the Native Format Builder is still open.

Tasks

1. To configure the handling of native data, perform the steps in the following table:

Step	Window	Choices or Values
a.	Welcome	Click Next.
b.	File Name and Directory	File Name: `nxsd_po_schema.xsd` Click Next.
c.	Choose Type	Select Complex Type. Click Next.
d.	File Description	File Name: `/home/oracle/labs/files/xsd/purchaseOrder.dat` Number of rows to sample: 2 Root element: `purchaseOrder` Click Next.

   The Design Schema pane opens.

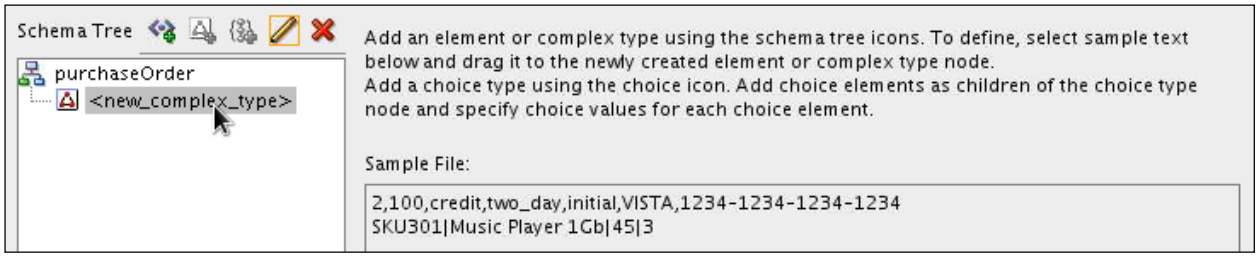
2. Create and configure a complex type to describe customer data.
   a. Add a complex type to the schema.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

b. Highlight the sample data that corresponds to the complex type (the first line of the sample file). Drag and drop the sample data onto the <new_complex_type> component of the schema tree.



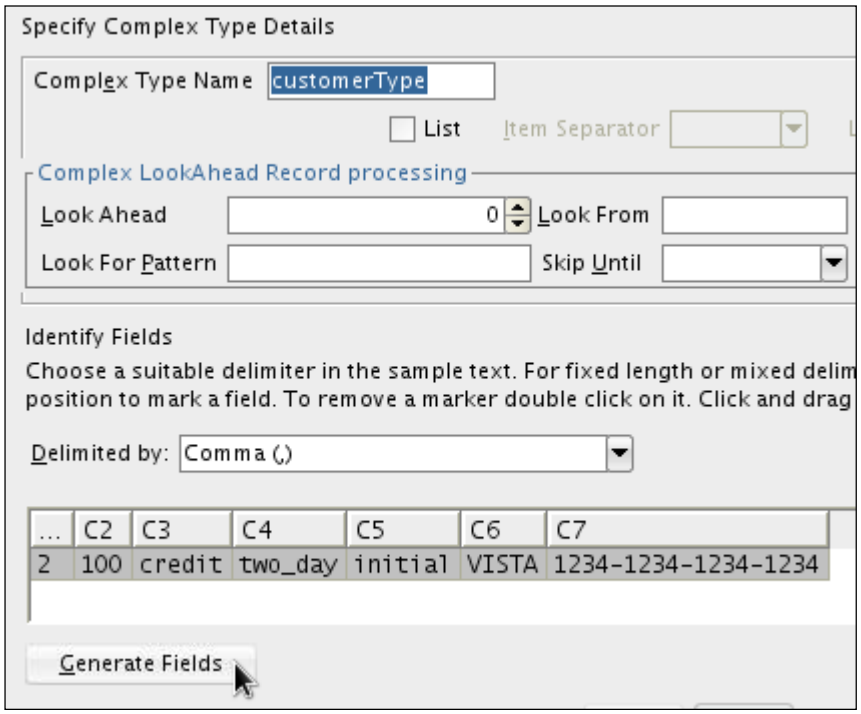
The Complex Type Details pane opens.

c. Name the complex type `customerType`.

d. Specify that the data is delimited by Comma (,).

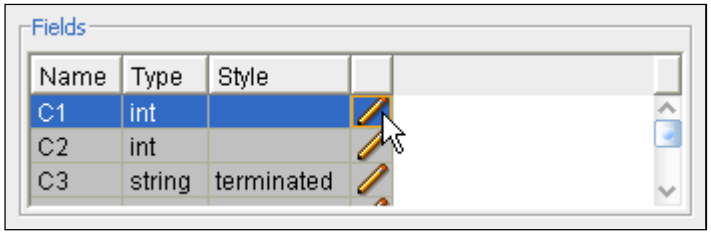
The sample data is parsed.

e. Click Generate Fields.



The wizard generates a series of fields based on the sample data and the parameters provided thus far. The properties of each of the fields must be edited manually.

f. Click the Pencil icon to edit the first field.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The field properties editor is displayed.



g. Use the data in the following table to update the Field Name, Type, Style, and Terminated By fields as needed for the complex type `customerType`.

Field Name	Type	Style	Terminated by
`custID`	string	terminated	,
`ID`	string	terminated	,
`payOption`	string	terminated	,
`shipChoice`	string	terminated	,
`status`	string	terminated	,
`ccType`	string	terminated	,
`ccNumber`	string	terminated	${eol}

h. Click OK to close the Complex Type Details window.

3. Create and configure a complex type to describe item data.

a. Highlight the schema root node (`purchaseOrder`).

b. Add another complex type to the schema.

c. Highlight the sample item data that corresponds to the complex type (the second line of the file). Drag and drop it onto the <`new_complex_type`> component of the schema tree.

d. Name the complex type `itemType`.

e. Specify that the data is delimited by the pipe character |. (Enter the pipe character into the “Delimited by” field.)

The sample data is parsed.

f. Click Generate Fields.

g. Use the information in the following table to edit the individual field properties:

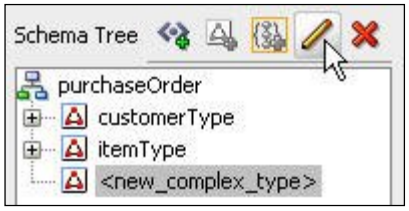
Field Name	Type	Style	Terminated by
`prodID`	string	terminated	|
`prodName`	string	terminated	|
`price`	decimal	terminated	|
`quantity`	int	terminated	${eol}

h. Click OK to close the Complex Type Details window.

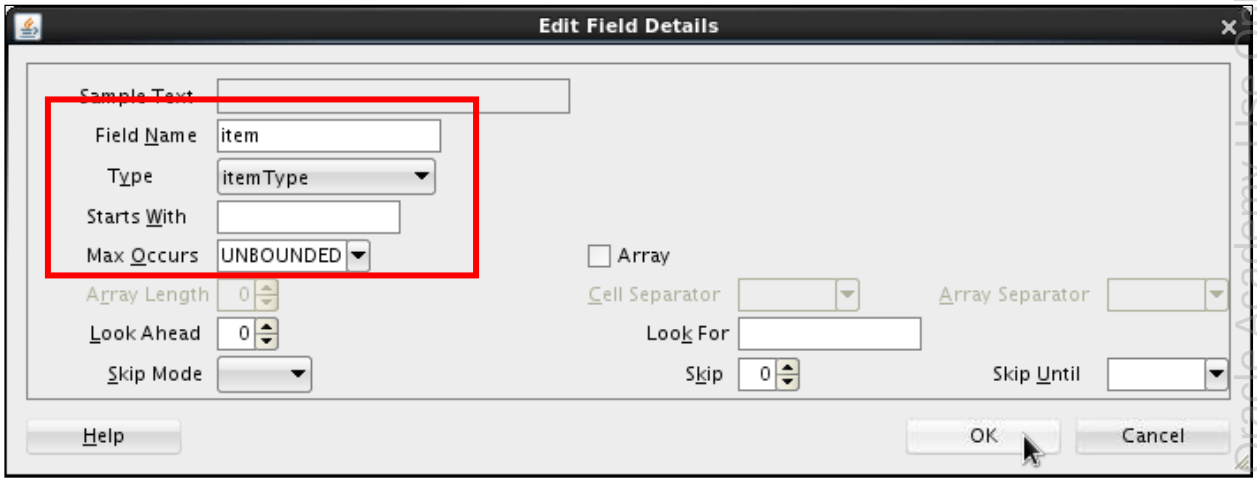
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4. Create and configure a complex type to describe the list of item data.
   a. Add a complex type to the schema tree.
   b. Click the Pencil icon to edit the new complex type properties.



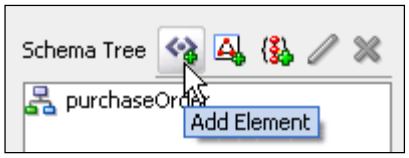
   c. Name the new type `itemListType`.
   d. Edit the field C1 to define an element named `item` of type `itemType`.
   e. Set the `maxOccurs` property of `item` to UNBOUNDED.



Note: The `UNBOUNDED` value specifies that an unlimited number of `item` records can be included in the data.

   f. Click OK (twice).

5. Add the following two *elements* (not complex types) to `purchaseOrder` and configure them:

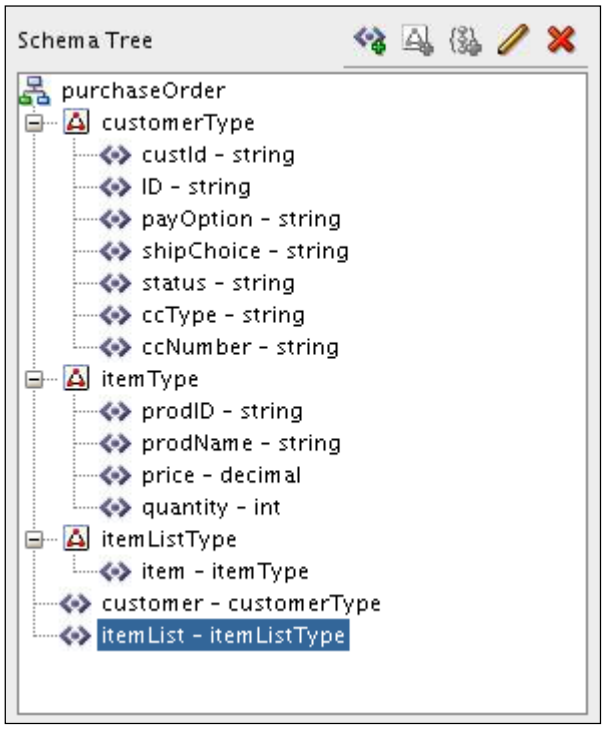


Name	type	maxOccurs
`customer`	customerType	1
`itemlist`	itemListType	1

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

6. Verify your work.

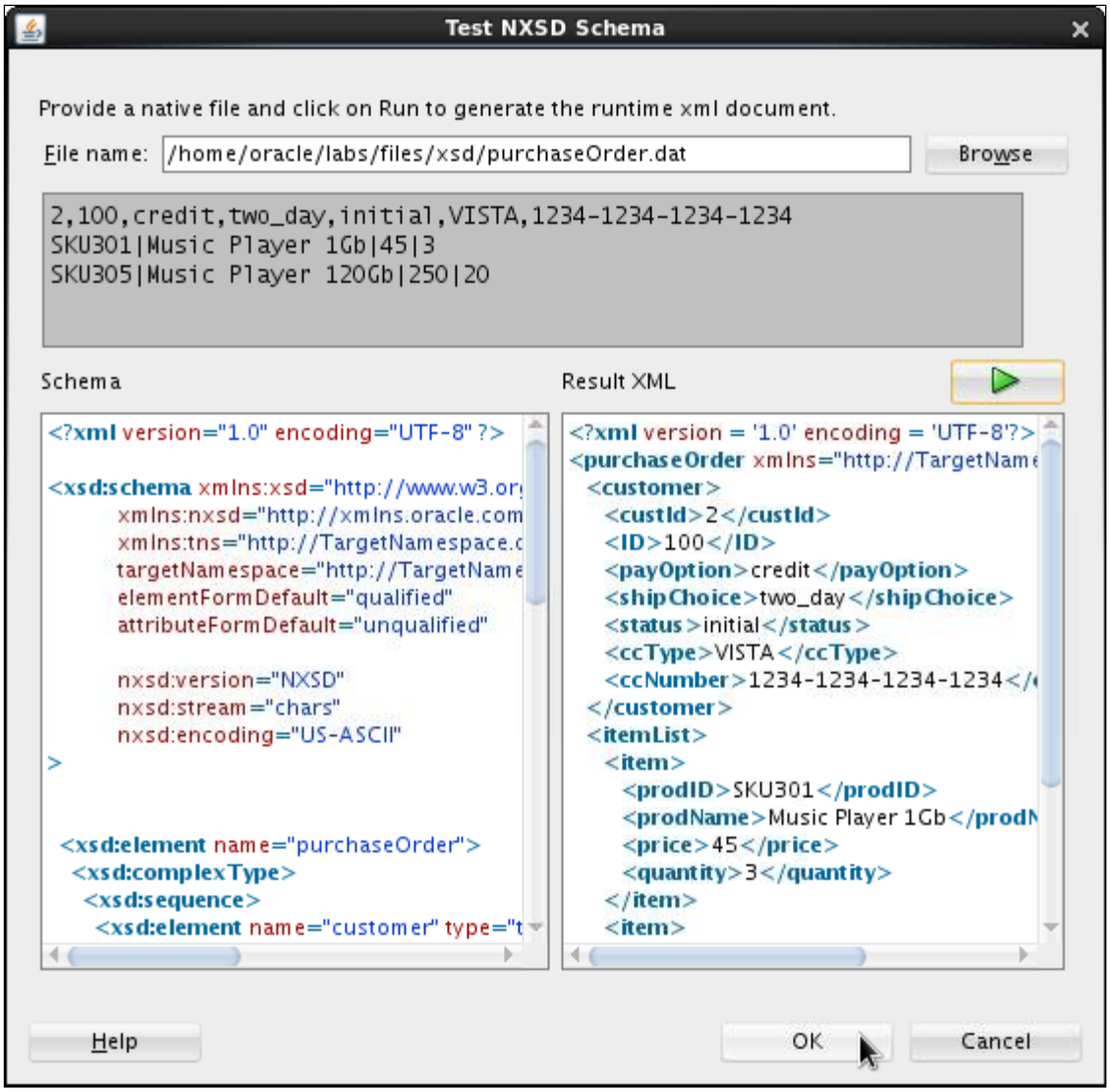


7. Click Next.
8. Click Test.
9. Provide the native file name `/home/oracle/labs/files/xsd/purchaseOrder.dat`.
10. Click the test (green right arrow) button.

    The sample data is mapped to an XML format.
11. Verify your results.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



12. Click OK.

    The Test NXSD Schema window closes.

13. Click Next, and then Finish.

    The Native Format Builder closes.

14. Click Next, and then Finish.

    The File Adapter Configuration wizard closes.

15. Save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 4-3: Creating the Service Components and External References

Overview

In this section of the practice, you add a Mediator service component, and another File adapter as an external reference (to write the outbound message).

Assumptions

This section of the practice assumes that you have completed Practice 4-2 successfully.

Tasks

1. Create a Mediator component.
   a. In the Composite Editor, drag a Mediator component from the Component Palette into the Components column.

      The Create Mediator dialog box opens.

   b. Set the name to `RouteFile`.

   c. Accept the Define Interface Later template. Click OK.

      Note: The interface is automatically defined in the next step, when the wire is created from the exposed service to the Mediator.

   d. Create a wire from the ReadFile exposed service icon to the RouteFile Mediator icon.

   e. Save your work.
2. Create and configure a File adapter as an external reference.
   a. Add a File adapter to the External References column.

      The FILE Adapter Configuration wizard opens.

   b. Use the instructions in the following table to configure the File adapter:

Step	Window	Choices or Values
a.	File Adapter Reference	Name: `WriteFile` Click Next.
b.	Adapter Interface	Accept “Define from operation and schema (specified later)”. Click Next.
c.	File Server Connection	Accept the File Server JNDI name: `eis/FileAdapter` Click Next.
d.	Operation Type	Select the Write File option. Click Next.
e.	File Configuration	Directory for Outgoing Files: `/home/oracle/labs/output/podata` File Naming Convention: `po_%yyMMddHHmmss%.xml`

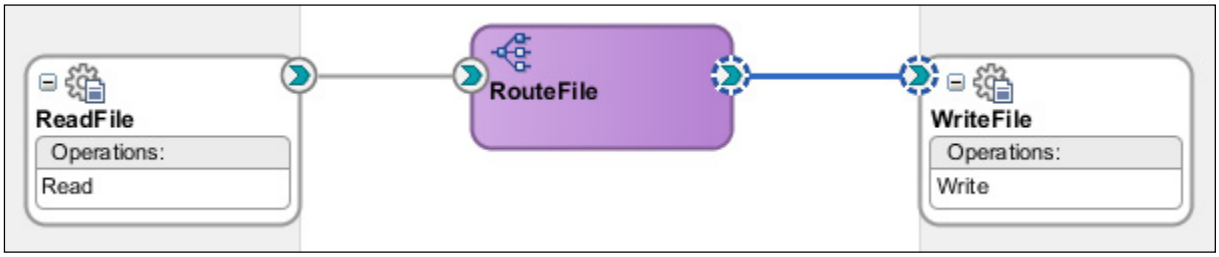
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Step	Window	Choices or Values
		Click Next.
f.	Messages	Click the “Browse for schema file” icon.
g.	Type Chooser	Click the Import Schema icon.
h.	SOA Resource Browser	With File System selected, navigate to the `/home/oracle/labs/files/xsd` folder and select `internalorder.xsd`. Click OK.
i.	Localize Files	Deselect the “Maintain original directory structure for imported files” check box. Click OK.
j.	Type Chooser	Expand the Project Schema Files > `internalorder.xsd` entry (if needed), and select `order`. Click OK.
k.	Messages	Click Next.
l.	Finish	Click Finish.

The Adapter Configuration wizard closes.

3. Create a wire from the Mediator component to the File adapter.
4. Verify and save your work.



Note: JDeveloper may list a number of error and warning messages during the creation of your project. It performs validation checks each time you save, and these errors and warnings simply indicate that the project is not fully configured. Error messages should be taken more seriously if they are seen when you attempt to deploy your project.

5. **Things to Consider:** What files were created when you added the Mediator? The File adapter? What does each of the files contain?

   Note: If you are not sure, refer to the slides titled “Creating a Mediator” and “Creating a File Adapter” in the lesson titled “Getting Started with Composite Applications.”

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 4-4: Adding a Routing Rule to the Mediator

Overview

In this section of the practice, you configure the Mediator by adding a routing rule. This rule includes a transformation to map the incoming message format to the outgoing message format. You use the Auto Map feature, use XPath functions to perform type conversions, and create a data dictionary.

Assumptions

This practice assumes that you have completed Practice 4-3 successfully.

Tasks

1. Right-click the `RouteFile` Mediator and select Edit.

   The Mediator configuration editor opens.

2. Click the “Select an existing mapper file or create a new one” icon next to the Transform Using field.

   The Request Transformation Map dialog box appears.

3. Click the Create Mapping icon.

   The Create Transformation Map dialog box appears.

4. Click OK.

5. Click OK to close the Request Transformation Map dialog box.

   The XSLT Mapper (`purchaseOrder_To_order1.xsl`) window opens.

6. Drag the PurchaseOrder element in the source column and drop it onto the Order element in the target column.

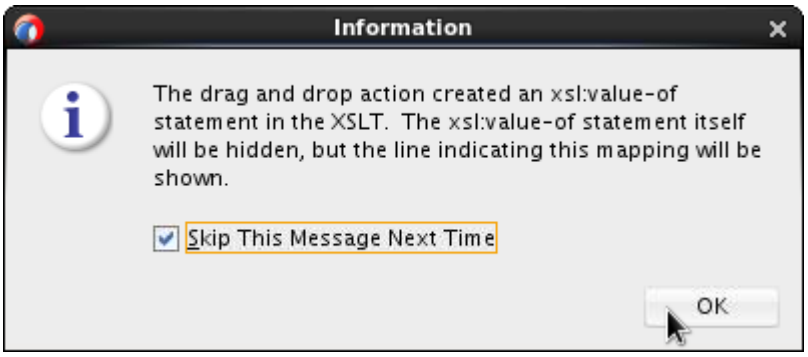
   The Auto Map Preferences window opens.

7. Click OK to accept the defaults.

   The Auto Map feature finds no matches. (Points to Consider: Why was the source node `status` not mapped to the target node `status`?)

8. Manually map the leaf nodes. Use the table that follows if needed. (Note that there is no mapping to the target node `orderTotal`.)

   The first time you map a field, the following information dialog box appears. Select the Skip This Message Next Time check box to avoid repeated display of this message.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Source	Target
custID	customerId
ID	orderId
payOption	payMethod
shipChoice	shipMethod
status	status
ccType	cardType
ccNumber	cardNumber

9. Map the node `itemlist` to the node `items`.

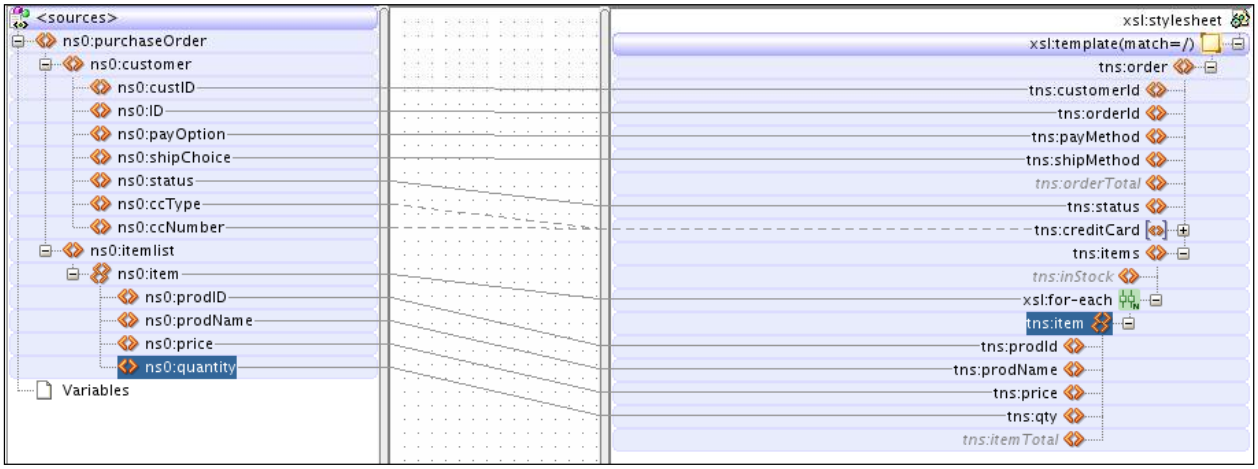
   The Auto Map Preferences window appears again.

10. Click OK to accept the defaults.

    Note: Something very different happens this time. First, Auto Map finds several matches. (Point to Consider: *What is different this time*?) Secondly, a `for-each` node is added to the target message structure, and the repeating element `item` is mapped to it. The `for-each` node is a graphic representation of an XSL construct that allows processing of repeating message elements in a loop.

11. Manually map the node `quantity` to `qty`.

12. Verify and save your work.



13. Close all editors, except the NativeData overview window.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 4-5: Deploying and Running the Project

Overview

In this section of the practice, you deploy and test the project.

Assumptions

This practice assumes that you have completed Practice 4-4 successfully.

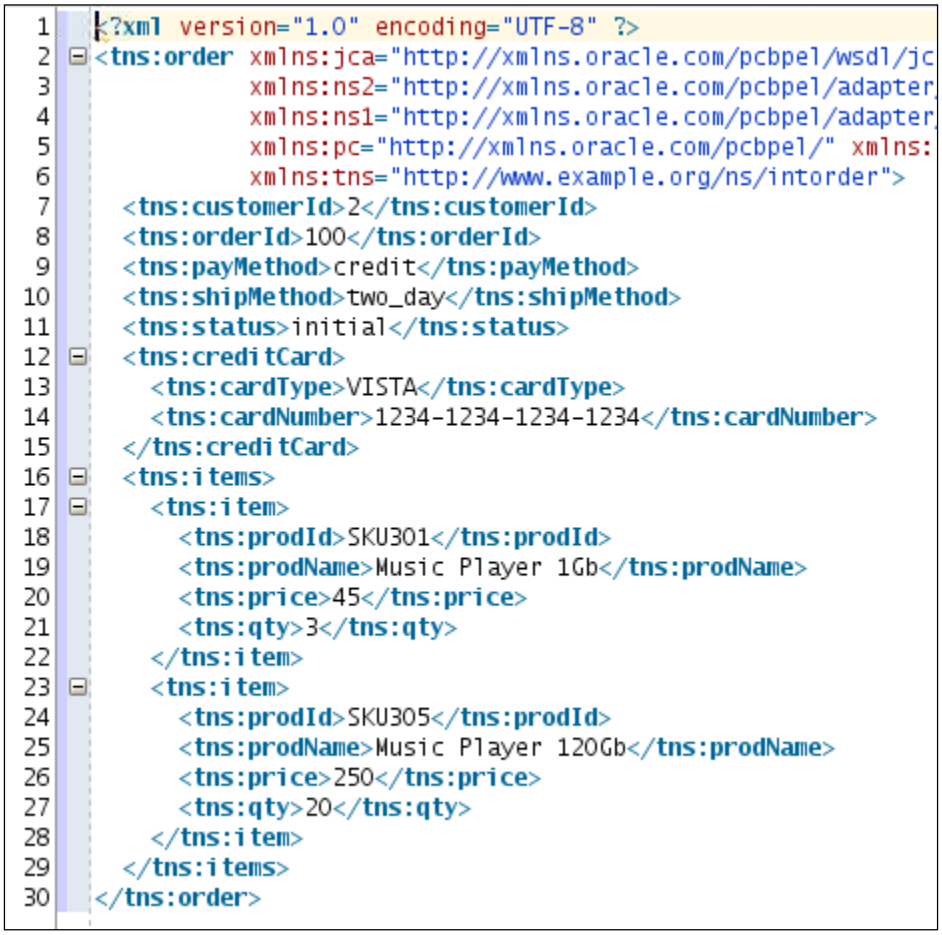
Tasks

1. Deploy the NativeData project to the `IntegratedWebLogicServer` Application Server.
   a. Remember to select the check box to overwrite any existing composites with the same revision ID.
   b. Verify successful deployment in the Message pane.
2. Copy the file `/home/oracle/labs/files/xsd/purchaseOrder.dat` to `/home/oracle/labs/input/po.dat`.
3. Confirm that within 10 seconds, the following occur:
   - The file is removed from the input directory.
   - The original file is moved to `/home/oracle/labs/archive/po.dat_<`*`UUID`*`>`.
   - A copy of the file is passed to the composite application, and then, after transformation, it is written to `/home/oracle/labs/output/podata/po_<`*`dateTimeStamp`*`>.xml` in the application.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4. Use JDeveloper or a text editor to open the output file and verify that the data is correctly formatted.



```
<?xml version="1.0" encoding="UTF-8" ?>
<tns:order xmlns:jca="http://xmlns.oracle.com/pcbpel/wsdl/jc
           xmlns:ns2="http://xmlns.oracle.com/pcbpel/adapter
           xmlns:ns1="http://xmlns.oracle.com/pcbpel/adapter
           xmlns:pc="http://xmlns.oracle.com/pcbpel/" xmlns:
           xmlns:tns="http://www.example.org/ns/intorder">
  <tns:customerId>2</tns:customerId>
  <tns:orderId>100</tns:orderId>
  <tns:payMethod>credit</tns:payMethod>
  <tns:shipMethod>two_day</tns:shipMethod>
  <tns:status>initial</tns:status>
  <tns:creditCard>
    <tns:cardType>VISTA</tns:cardType>
    <tns:cardNumber>1234-1234-1234-1234</tns:cardNumber>
  </tns:creditCard>
  <tns:items>
    <tns:item>
      <tns:prodId>SKU301</tns:prodId>
      <tns:prodName>Music Player 1Gb</tns:prodName>
      <tns:price>45</tns:price>
      <tns:qty>3</tns:qty>
    </tns:item>
    <tns:item>
      <tns:prodId>SKU305</tns:prodId>
      <tns:prodName>Music Player 120Gb</tns:prodName>
      <tns:price>250</tns:price>
      <tns:qty>20</tns:qty>
    </tns:item>
  </tns:items>
</tns:order>
```

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 4-6: Optional Challenge: Creating a Fixed-Length Output File

Overview

In this unstructured practice, you add an additional File adapter as an external reference that writes the purchase order header information to a fixed-length format file. This section of the practice builds on the work that you have already done. For example, passing the same file to the application three times should produce files similar to the following:

```
2 100 credit two-day       initial MCRD  4321-4321-4321-4321
2 100 credit two-day       valid   VISTA 1234-1234-1234-1234
2 100 credit international initial VISTA 1234-1234-1234-1234
```

Note: This data sample has blank spaces added between fields for visual clarity on this page. The fixed-length data in your implementation is not likely to include spaces between fields (although spaces within fields may be common). For the purpose of this exercise, you may configure your adapter to preserve these extra spaces or not, as you see fit.

A sample file to assist the definition of the native format is available at `/home/oracle/labs/files/xml/po_header.txt`. Some additional basic guidance is available in the Tasks section of this practice, but *the specifics are up to you*. After you have modified the application, redeploy and test it.

Assumptions

This practice assumes that you have completed Practice 4-5 successfully, and that you want to explore a less structured approach to modifying your practice exercise to test and reinforce your understanding of how adapters modify native format data.

Tasks

1. Add an additional File adapter as an external reference that writes the purchase order header information to a fixed-length file.
2. Define the fixed-length native format of the output file. When prompted, use the `/home/oracle/labs/files/xml/po_header.txt` file as a sample. Test your work before proceeding.
3. Add an additional routing rule to the Mediator with an appropriate transformation.
4. Create a wire between the Mediator and the File adapter.
5. Redeploy the application.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

6. Test the application:
   a. Copy the `/home/oracle/labs/files/xsd/purchaseOrder.dat` file to `/home/oracle/labs/input/po_3.dat`.
   b. Repeat the preceding step “a” three or more times. For at least one execution of this step, open and modify the file `po_3.dat`. Change values in the data, similar to the sample data in the Overview section of this practice. For instance, replace the word `two_day` with the word `international`.
   c. Verify that both the original and new File adapters produce the desired files.
      - Does your fixed-length output appear similar to the sample data in the Overview section of this practice?
      - Were the various data variations (such as the word `international`) written correctly?
      - If not, what changes would you need to make to your native format definition?

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 5: Using JMS and JDBC Adapters

Chapter 5

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 5: Overview

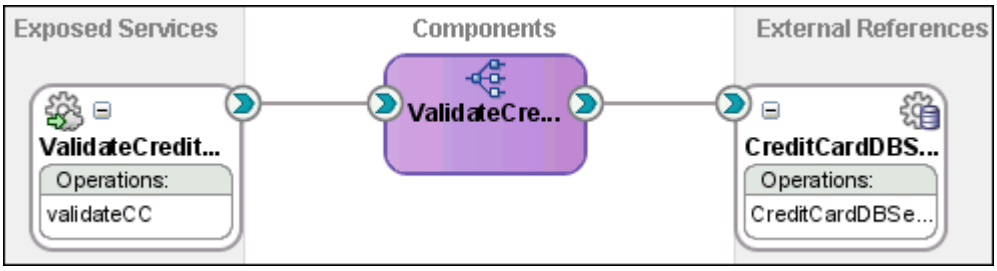
Practices Overview

In this practice, you create a composite application that receives a credit card number and a purchase order total. The application uses a Database adapter to query credit card information in an Oracle Database. Based on the results of that query, a status of `VALID` or `INVALID` is returned.

To create this application, perform the following steps:

- You create a new SOA project with a Database adapter that is configured to perform a `select` operation on the database.
- You then add a synchronous Mediator, defining a WSDL interface based on the elements of the `creditcheck.xsd` file.
- You deploy the composite application and test it with both valid and invalid credit card numbers.

The following image represents the composite application design for your CCValidate project:



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-1: Configuring the Database

Overview

The SOA Quickstart installation includes a JavaDB (Derby) database. In this section, you create and populate tables for use in this and subsequent practices.

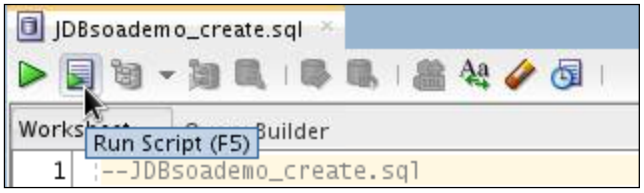
Assumptions

This practice assumes that JDeveloper is open and that WebLogic Server is started.

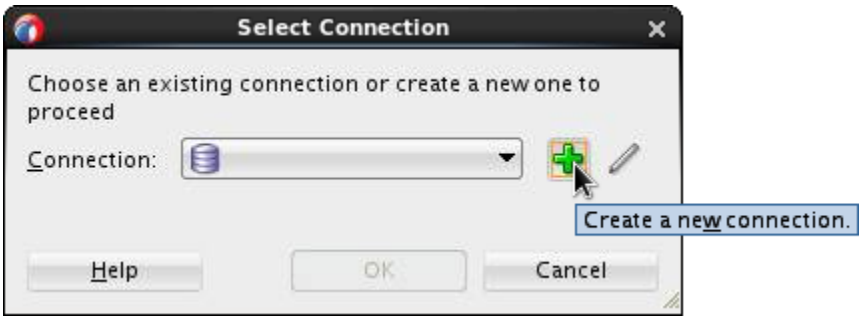
Note: Use *exact* names (both spelling and case are important) as specified for your project and components. The practice for the lesson titled “Sharing Functionality in Oracle SOA Suite” will expect to find these names in use.

Tasks

1. From the JDeveloper menu, select File > Open.
2. Navigate to `/home/oracle/labs/scripts/SQL`.
3. Select `JDBsoademo_create.sql`.
   The file is opened in a SQL worksheet.
4. Click the Run Script icon.



   The Select Connection dialog box is displayed.
5. Click the “Create a new connection” icon.



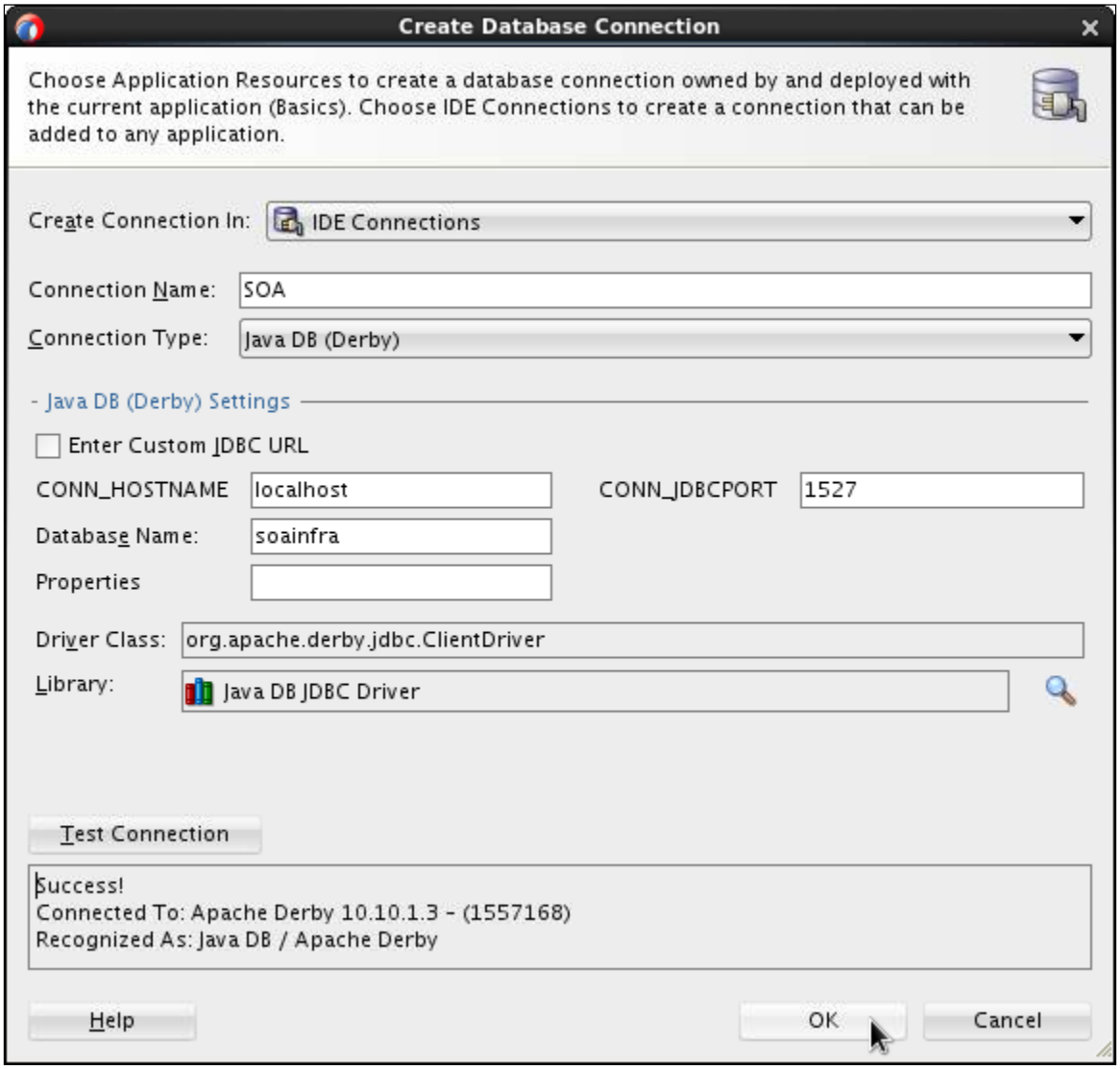
6. Enter the following details:
   - Connection Name: **soa**
   - Connection Type: **Java DB (Derby)**
7. Ensure that the default Java DB (Derby) settings match the following:
   - CONN_HOSTNAME: `localhost`
   - CONN_JDBCPORT: `1527`
   - Database Name: `soainfra`
   - Library: Java DB JDBC Driver

   Note: You may need to select the library from the drop-down list.
8. Click the **Test Connection** button to verify that the connection works.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The following message is displayed: Success!



9. Click OK.
10. Click OK to close the Select Connection dialog box.
    The script is run.
11. Open and run the `/home/oracle/labs/scripts/SQL/JDBsoademo_data.sql` file.
    When prompted, select the SOA connection that you defined a moment ago.
12. Open and run the `/home/oracle/labs/scripts/SQL/JDBcreate_waitingorders.sql` file.
13. Close each of the `.sql` file tabs that you opened.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-2: Creating the CCValidate SOA Project

Overview

In this practice, you create a new SOA project in the Basics workspace, which contains a Database adapter.

Assumptions

This practice assumes that JDeveloper is open and that WebLogic Server is started.

Note: Use exact names (both spelling and case are important) as specified for your project and components. The practice for the lesson titled “Sharing Functionality in Oracle SOA Suite” will expect to find these names in use.

Tasks

1. In JDeveloper, create a new empty SOA project named `CCValidate` in the Basics application.
   The composite overview window opens.
2. Add a Database adapter to the External References swimlane.
   The Adapter Configuration wizard appears.
3. To configure the Database adapter, perform the following steps:

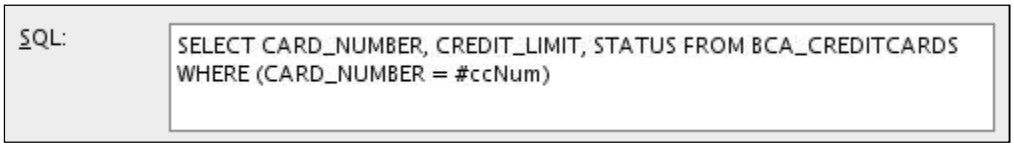
Step	Screen	Choices or Values
a.	Database Adapter Reference	Name: `CreditCardDBService` Click Next.
b.	Service Connection	Use the Browse option to copy the SOA service connection. Verify that the JNDI Name has the value `eis/DB/SOA`. Click Next.
c.	Operation Type	Select the “Perform an operation on a Table” option. Ensure that the Select check box is selected. Deselect the other operations. Click Next.
d.	Select Table	Click Import Tables.
e.	Import Tables	Set the filter to BCA_%. Click Query.
f.	Import Tables	Move the `BCA_CREDITCARDS` table from the Available list to the Selected list. Click OK.
g.	Select Table	Verify that the `BCA_CREDITCARDS` table is listed. Click Next.
h.	Relationships	Click Next.
i.	Attribute Filtering	Select the `creditLimit` and `status` attributes. Deselect the remaining options.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Step	Screen	Choices or Values
		Note: The cardNumber (key attribute) cannot be deselected. Click Next.
j.	Define Selection Criteria	Next to the Parameters section, click Add.
k.	Parameter Name	Enter `ccNum`. Click OK.
l.	Define Selection Criteria	Next to the SQL field, click Edit.
m.	Expression Builder	Click Add.
n.	Expression Builder	Form the condition: `cardNumber` EQUAL `ccNum`, where: First Argument Query Key: `cardNumber` Operator: `EQUAL` Second Argument: Select the Parameter option, and select `ccNum`. Click OK.
o.	Define Selection Criteria	Verify that your SQL statement is correct by comparing it to the screenshot that follows this table. Click Next.
p.	Advanced Options	Click Next.
q.	JCA Endpoint Properties	Click Next.
r.	Finish	Click Finish.

The Adapter wizard closes.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-3: Configuring the Mediator Component from a WSDL Definition

Overview

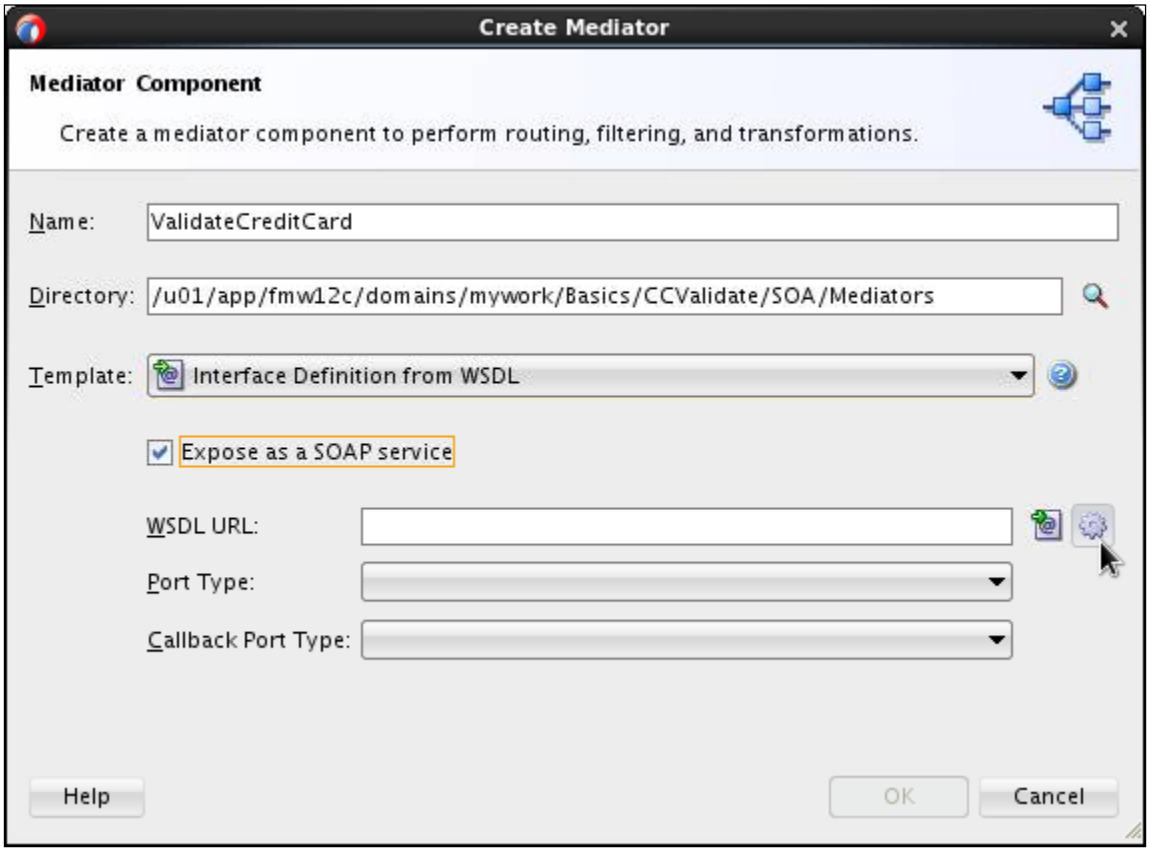
In this practice, you create a synchronous Mediator. You define its WSDL interface based on the elements of the `creditcheck.xsd` file. The `.xsd` file describes both the message that the Mediator will receive from the client that calls the application and the message that it will return to that caller.

Assumptions

This practice assumes that you have successfully completed Practice 5-2.

Tasks

1. Create and configure a Mediator component.
   a. Add a Mediator to the Component column of the composite application editor.
      The Create Mediator window opens.
   b. Set the Name to `ValidateCreditCard`.
   c. Select “Interface Definition from WSDL” from the Template drop-down list.
   d. Ensure that the “Expose as a SOAP service” check box is selected.
   e. Click the “Generate WSDL from Schema(s).” icon (next to the WSDL URL field).



The Create WSDL window opens.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

2. Define the Mediator WSDL Interface by modifying the following field values:

Port Type:	`validateCC_ptt`
Operation:	`validateCC`
Interface Type:	Synchronous

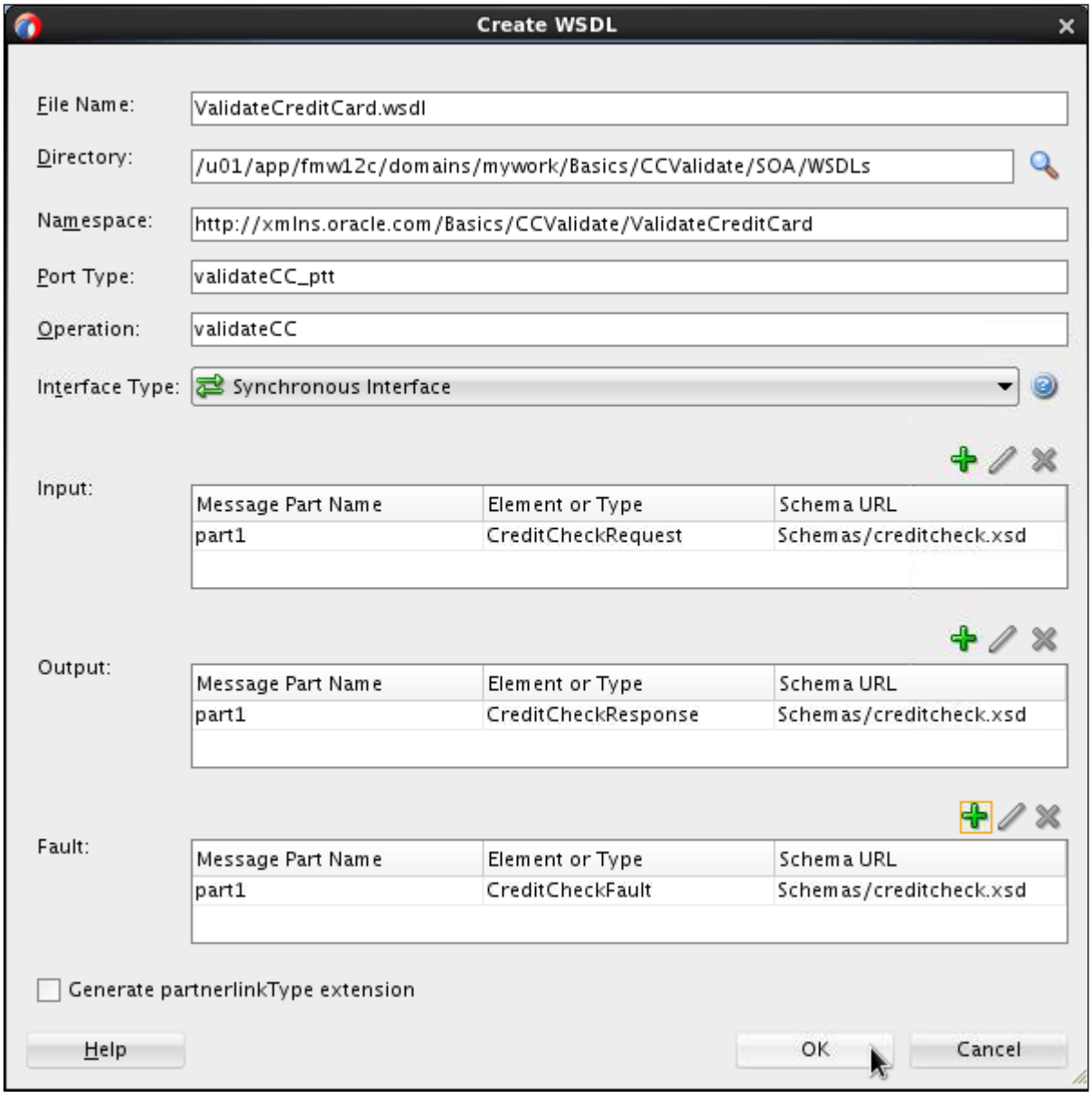
3. Define the request, response, and fault messages for the Mediator component.

Step	Screen	Choices or Values
a.	Create WSDL	Click the **Input** Add a new Message Part icon.
b.	Add a new Message Part	Click the Browse icon next to the URL field.
c.	Type Chooser	Click the Import Schema icon.
d.	SOA Resource Browser	Using the File System option, locate and select the `creditcheck.xsd` file in the `/home/oracle/labs/files/xsd` folder. Click OK.
e.	Localize Files	Verify that `creditcheck.xsd` is selected. Deselect the "Maintain original directory structure for imported files" check box. Click OK.
f.	Type Chooser	Expand the Project Schema Files > `creditcheck.xsd` nodes (if needed). Select the CreditCheckRequest element. Click OK.
g.	Add a new Message Part	Click OK.
h.	Create WSDL	Click the **Output** Add a new Message Part icon.
i.	Add Message Part	Click the Browse icon next to the URL field.
j.	Type Chooser	Expand the Project Schema Files > `creditcheck.xsd` nodes (if needed) and select the CreditCheckResponse element. Click OK.
k.	Add a new Message Part	Click OK.
l.	Create WSDL	Click the **Fault** Add a new Message Part icon. **Note:** The subject of faults is covered in the lesson titled "Handling Faults in Composite Applications."
m.	Add Message Part	Click the Browse icon next to the URL field.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Step	Screen	Choices or Values
n.	Type Chooser	Expand the Project Schema Files > `creditcheck.xsd` nodes (if needed) and select the `CreditCheckFault` element. Click OK.
o.	Add Message Part	Click OK.
p.	Create WSDL	Compare your work to the screenshot that follows this table. Click OK.

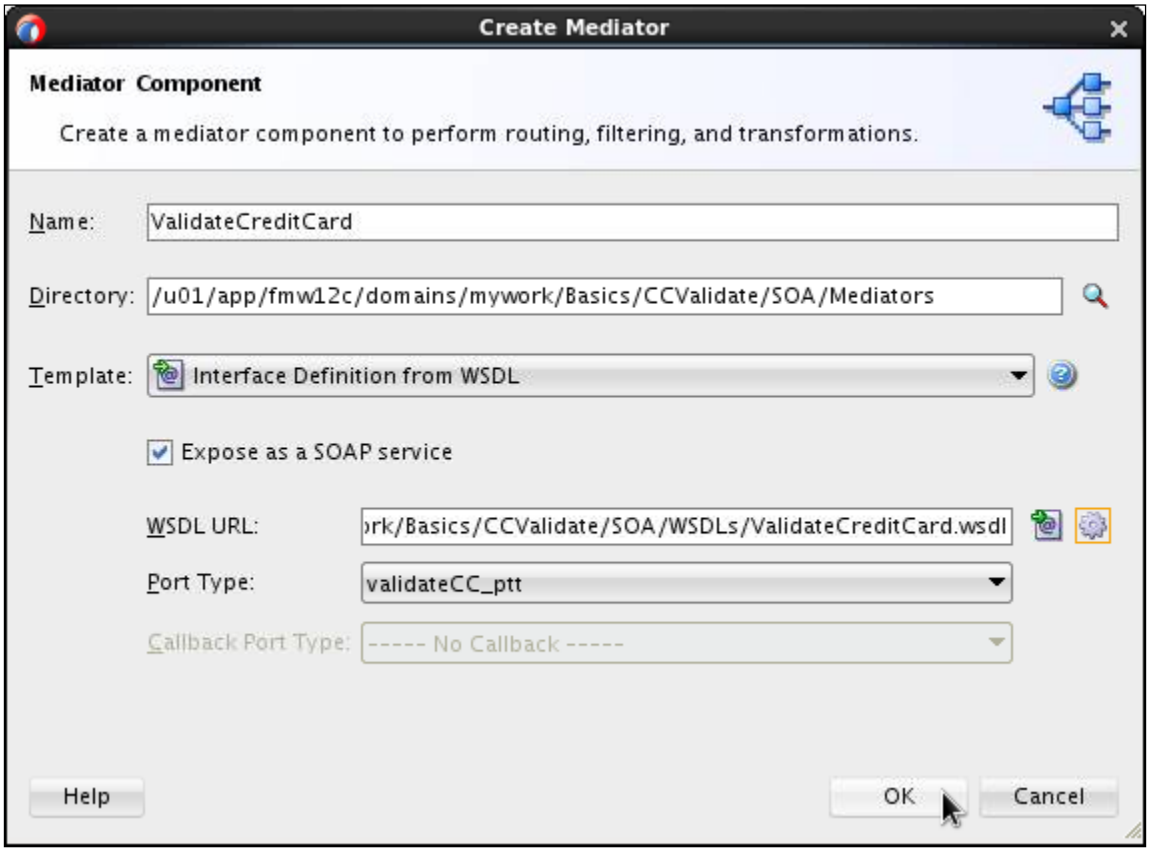


You are returned to the Create Mediator window.

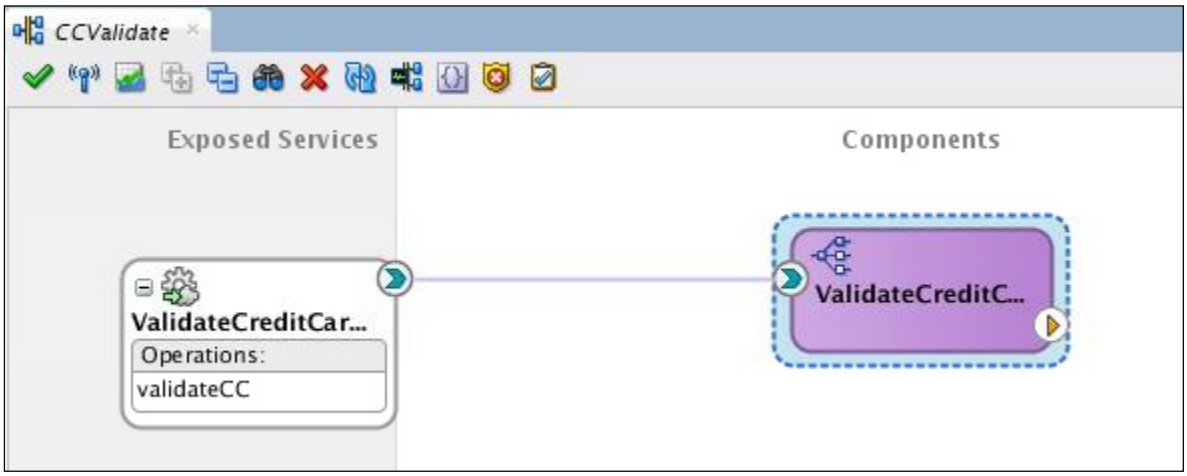
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4. Verify your work and click OK.



5. Verify and save your work.



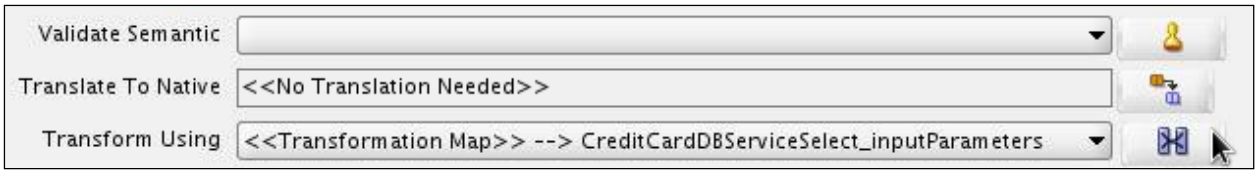
Wiring the Mediator to the DB Adapter and Creating Transformations

6. Create a wire from the Mediator component to the Database adapter.
7. Edit the Mediator component to add transformations, by right-clicking the Mediator icon and selecting Edit.

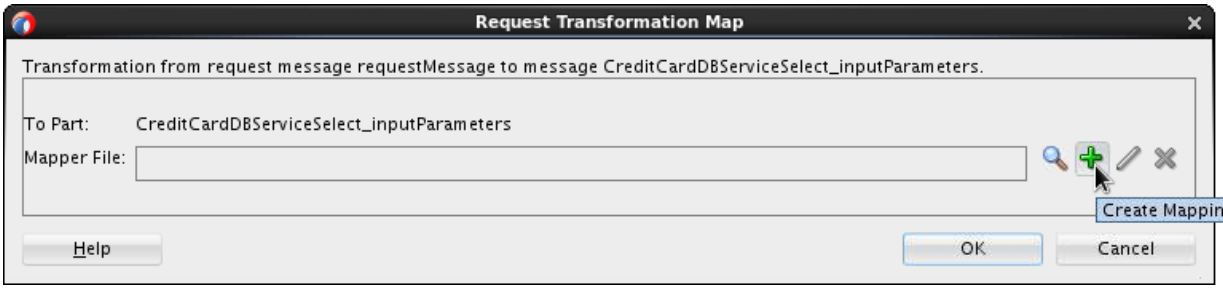
   The `ValidateCreditCard.mplan` window opens.
8. Expand the Static Routing rule that was created by the wire.
9. In the upper (request) section of the Static Routing rule, click the “Select an existing mapper file or create a new one” icon.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



a. In the Request Transformation Map dialog box, click the Create Mapping icon.

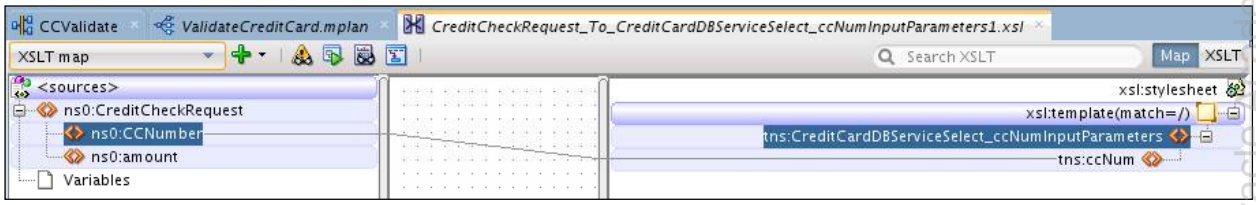


The Create Transformation Map dialog box is displayed.

b. Accept the default file name supplied. Click OK.

c. Click OK to close the Request Transformation Map dialog box.
The XSLT Mapper window opens.

d. Map the `CCNumber` element in the Source column to the `ccNum` element in the target column.



e. Save your work.

f. Close the XSLT editor and return to the `ValidateCreditCard.mplan` window.

10. Configure the Reply transformation.

a. In the Synchronous Reply section of the routing rule, click the “Select an existing mapper file or create a new one” icon.



b. In the Reply Transformation Map dialog box, click the Create Mapping icon.
The Create Transformation Map dialog box is displayed.

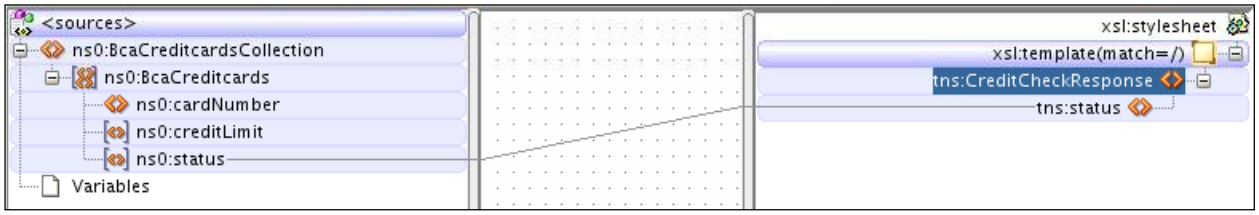
c. Accept the default file name supplied. Click OK.

d. Click OK to close the Reply Transformation Map dialog box.
The XSLT Mapper window opens.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

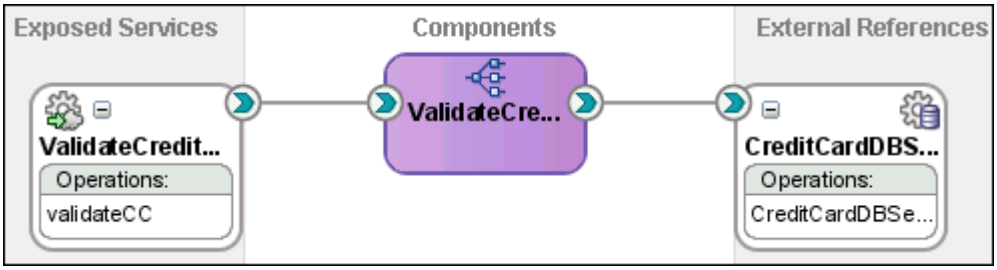
e. Map the status element in the Source column to the status element in the target column.



f. Save your work.

g. In the JDeveloper window, close the XSLT Mapper and the `ValidateCreditCard.mplan` windows.

11. Verify that the composite application assembly model resembles the following image:



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-4: Deploying and Testing the CCValidate Composite

Overview

In this practice, you deploy and test the CCValidate composite application.

Assumptions

This practice assumes that you have completed all work through Practice 5-3 successfully.

Tasks

1. In the JDeveloper Application Navigator, right-click the CCValidate project and select Deploy > CCValidate.

   The Deploy CCValidate wizard appears.

2. Use the instructions in the following table to complete the deployment:

Step	Window	Choices or Values
a.	Deployment Action	Deploy to Application Server. Click Next.
b.	Deploy Configuration	Select the “Overwrite any existing composites with the same revision ID” check box. Click Next.
c.	Select Server	`IntegratedWebLogicServer` Click Next.
d.	SOA Servers	Click Next.
e.	Summary	Click Finish.

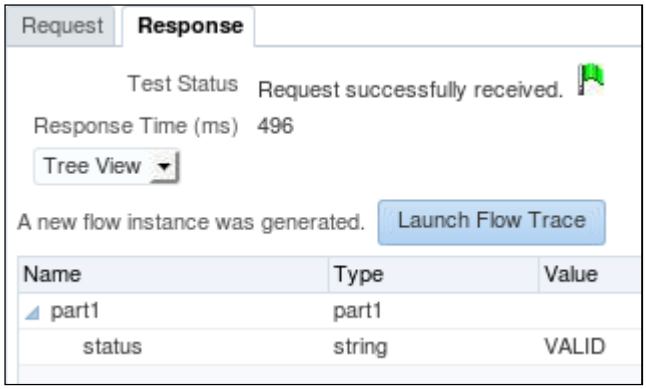
3. Deployment processing starts. Monitor deployment progress and check for successful compilation in the SOA – Log window. Verify that deployment is successful in the Deployment – Log window.
4. In a web browser window, access Enterprise Manager at http://localhost:7101/em.
   a. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “CCValidate [1.0]” link.
   b. On the “CCValidate [1.0]” home page, click Test.
   c. On the Test Web Service page, scroll down to the Input Arguments section of the Request tab. Expand **part1** and enter the following values:
      - CCNumber: 1234-1234-1234-1234
      - Amount: 1000
   d. Click Test Web Service (on the lower-right corner of the screen).

      The Response tab is displayed.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

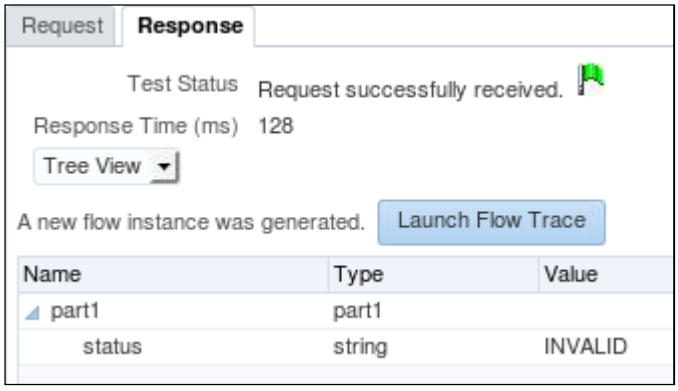
e. Verify that the status value returned is VALID, as shown in the following screenshot:



Note: The credit card number was found in the `BCA_CREDITCARD` table. The status field of the record is VALID, which was returned in the preceding test.

	CARD_NUMBER	CARD_TYPE	CUST_ID	CREDIT_LIMIT	STATUS	RATING
1	5678-5678-5678-5678	AMCD	1	20000.00	VALID	4
2	1234-1234-1234-1234	VISTA	2	20000.00	VALID	3
3	4321-4321-4321-4321	MCRD	3	20000.00	INVALID	3
4	8765-8765-8765-8765	AMCD	4	30000.00	VALID	5

5. To perform a second test, enter another request and click Test Web Service on the Request tab.
   a. On the Request tab page, modify the CCNumber value to be `4321-4321-4321-4321`. Leave the amount at `1000`. Click Test Web Service.
   b. On the Response tab page, verify that the status string value that is returned is INVALID, as shown in the following screenshot:



6. In JDeveloper, close the CCValidate overview tab.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-5: Creating and Configuring a JMS Queue

Overview

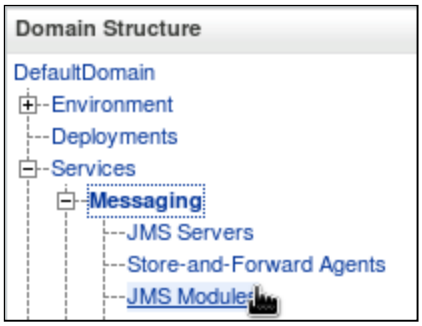
In this section, you create a JMS Queue named soaDemoQueue.

Assumptions

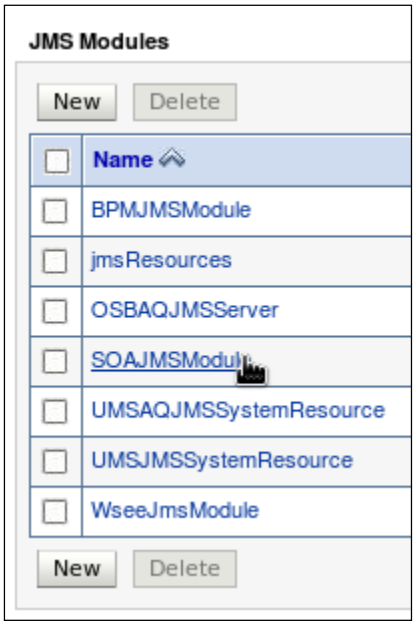
This practice makes no assumptions.

Tasks

1. Log in to the WebLogic Administration Server Console (http://localhost:7101/console) with the username `weblogic` and the password `welcome1`.
2. In the Domain Structure section, expand Services > Messaging and click JMS Modules.



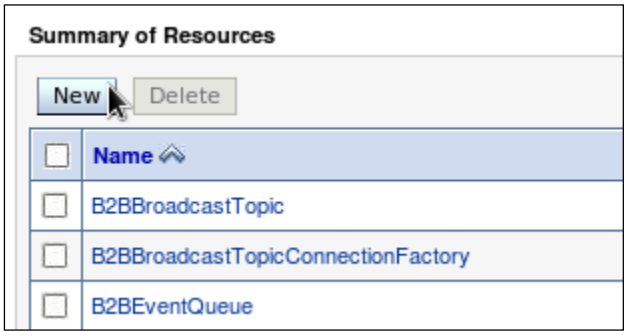
3. On the JMS Modules page, click the SOAJMSModule link.



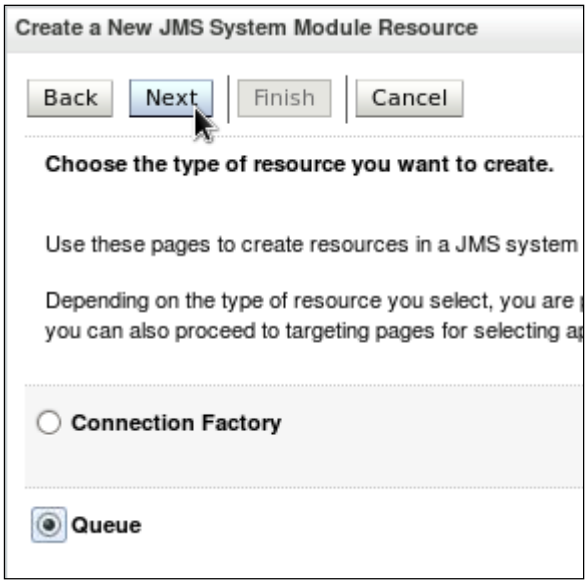
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4. On the Settings for SOAJMSModules Configuration page, in the Summary of Resources table, click New.



5. On the Create a New JMS System Module Resource page, select the Queue (resource type) option and click Next.

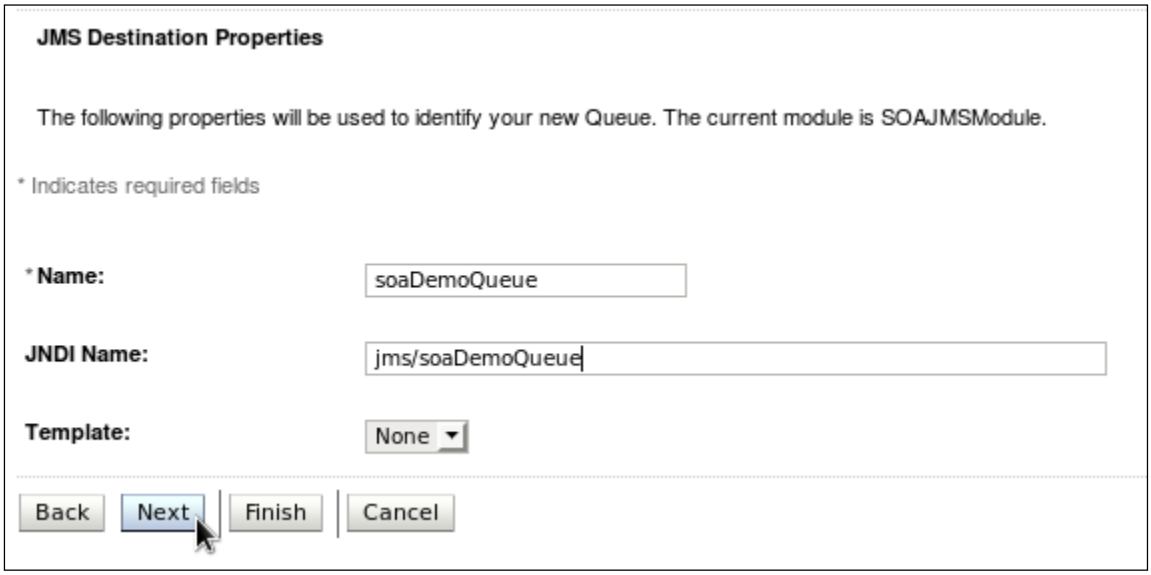


6. On the Create a New JMS System Module Resource page, in the JMS Destination Properties section, enter:
   - Name: `soaDemoQueue`
   - JNDI name: `jms/soaDemoQueue`

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

7. Verify your entries and click Next.

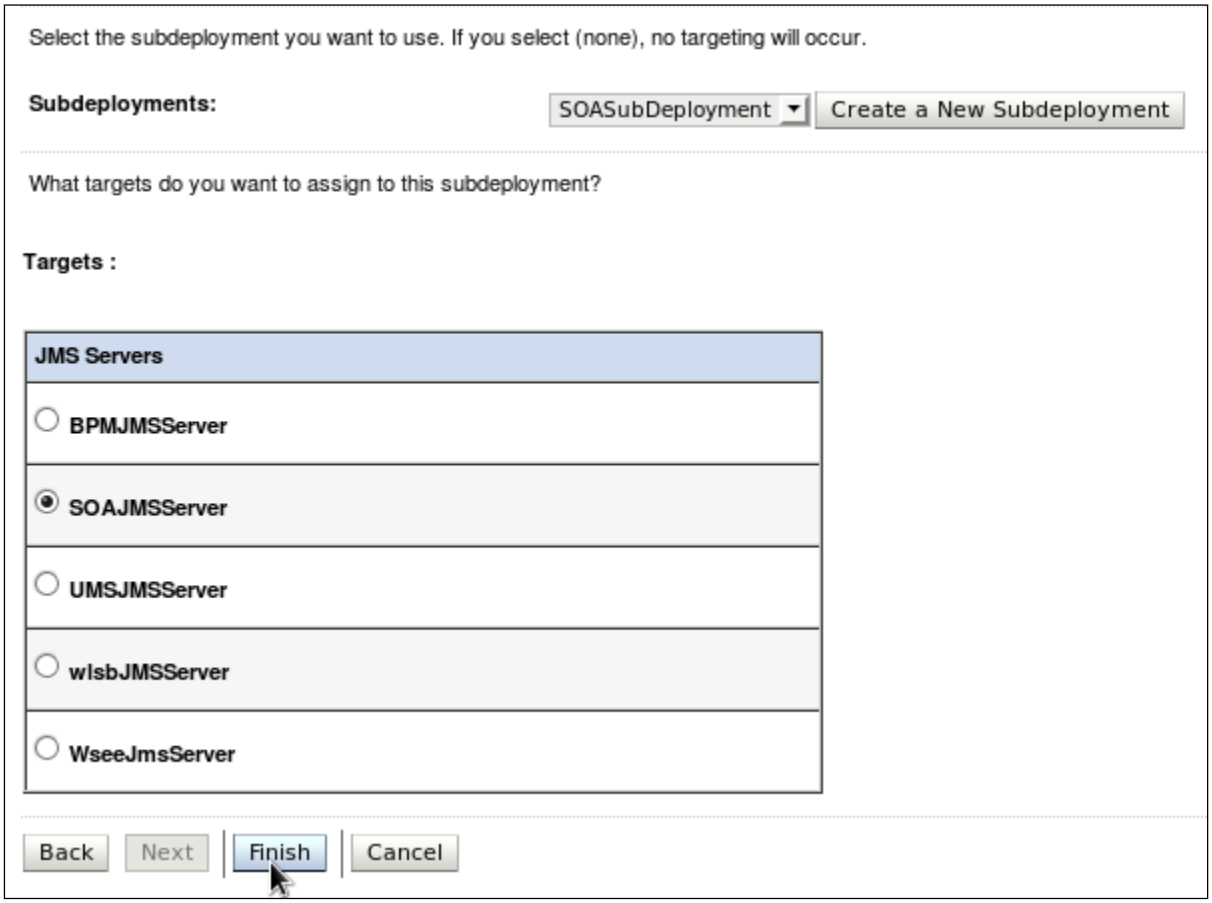


8. On the Create a New JMS System Module Resource page that appears next, perform the following:
   a. Select SOASubDeployment from the Subdeployment drop-down list.
   b. Select the SOAJMSServer option under Targets for the JMS Server section.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

9. Verify your selections and click Finish.



10. On the Response page, confirm whether the new JMS Queue was created successfully.



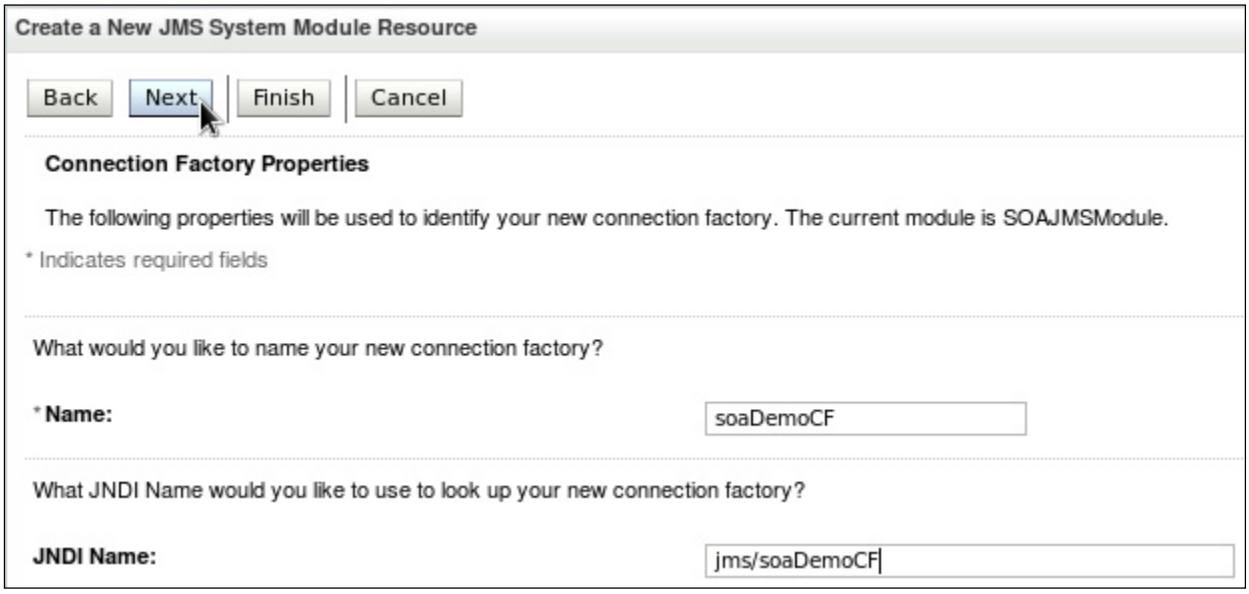
Configuring a JMS Connection Factory

11. On the Settings for SOAJMSModule page, in the Summary of Resources table, click New.
12. On the Create a New JMS System Module Resource page, for the resource type, select the Connection Factory option and click Next.
13. On the Create a New JMS System Module Resource page, in the Connection Factory Properties section, enter:
    - Name: `soaDemoCF`
    - JNDI Name: `jms/soaDemoCF`

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

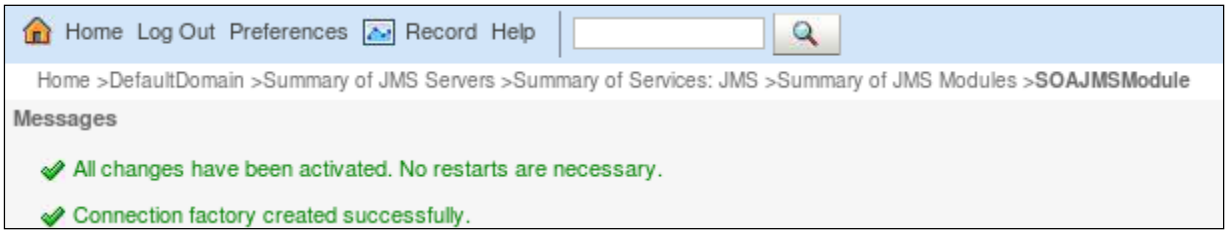
14. Verify your entries and click Next.



15. On the Create a New JMS System Module Resource page, confirm that DefaultServer is the selected target and click Finish.



16. Confirm that the connection factory was successfully created.

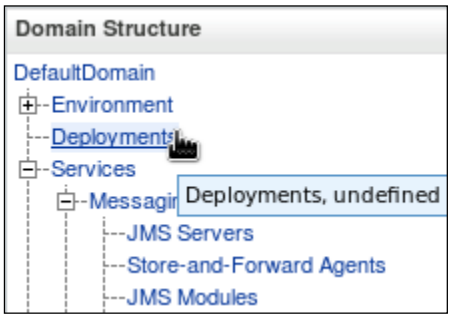


Oracle Internal & Oracle Academy Use Only

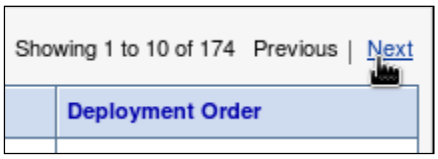
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Creating a JMS Connection Pool and Associate It with the Connection Factory

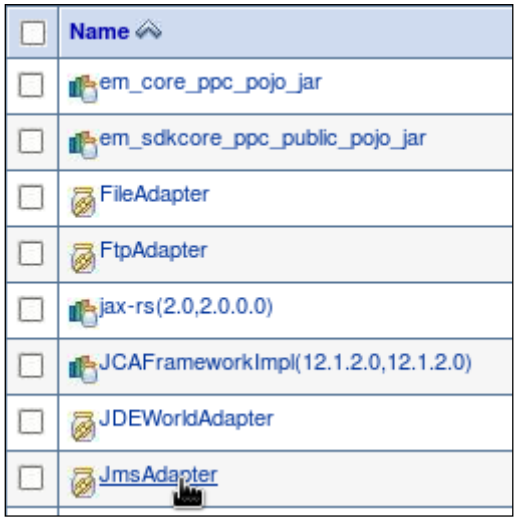
17. In the Domain Structure pane, click Deployments.



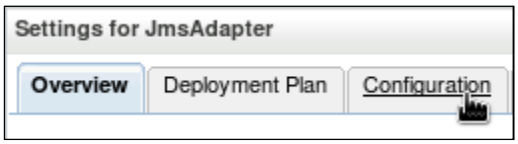
18. On the Summary of Deployments Control tab page, to locate the JMSAdapter entry, in the table heading, click Next. (You click multiple times to navigate this alphabetical list.)



19. On the Summary of Deployments Control tab page, click the JmsAdapter link (*not the check box*).



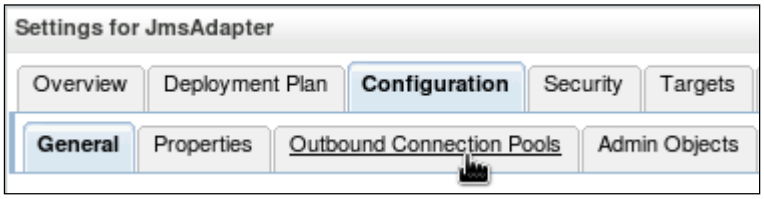
20. On the Settings for JmsAdapter page, click the Configuration tab.



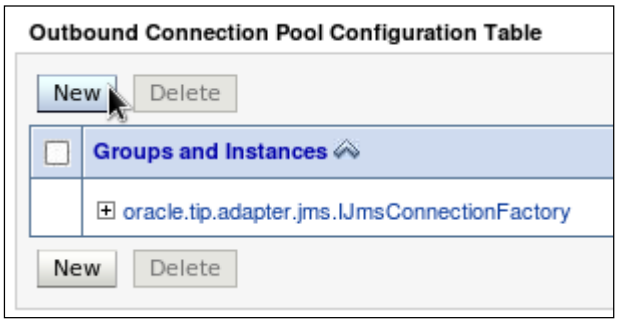
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

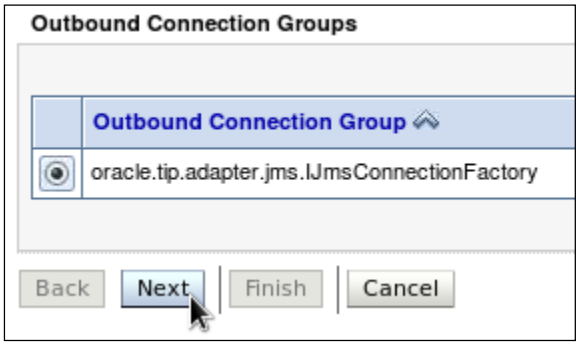
21. On the Settings for JmsAdapter Configuration tab page, click the Outbound Connection Pools tab.



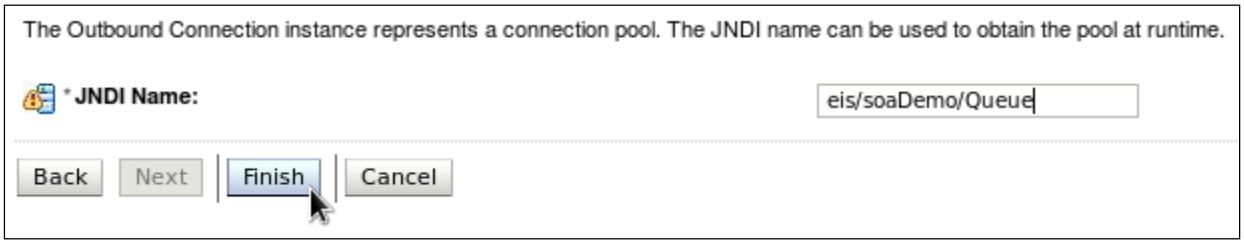
22. On the Settings for JmsAdapter Configuration > Outbound Connection Pools page, click New.



23. On the Create a New Outbound Connection page, select the oracle.tip.adapter.jms.IjmsConnectionFactory option in the Outbound Connections Groups table and click Next.



24. On the Create a New Outbound Connection page, in the JNDI Name field, enter `eis/soaDemo/Queue` and click Finish.



25. On the Save Deployment Plan Assistant page, select the path and enter `PlanJMS.xml` as the plan name.

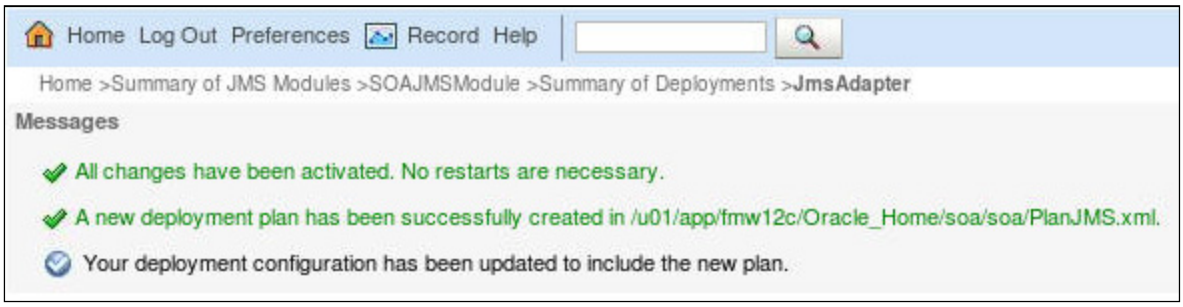
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

26. Verify your entry and click OK.



27. On the Settings for JmsAdapter page, at the top, confirm that the messages indicate that the changes have been activated, and that you do not need to restart the JmsAdapter to reflect the new plan.



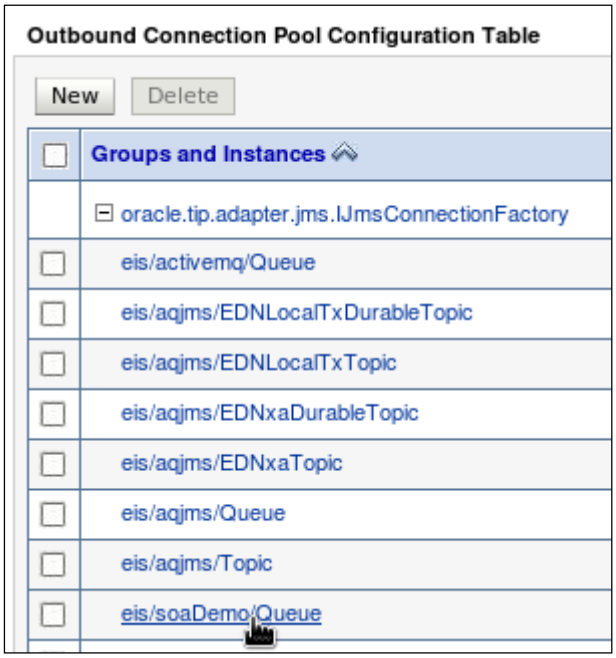
Associating the Connection Pools with the Connection Factory in a New JMS Adapter Configuration Plan

28. On the Settings for JmsAdapter page, click the Configuration tab.

Oracle Internal & Oracle Academy Use Only

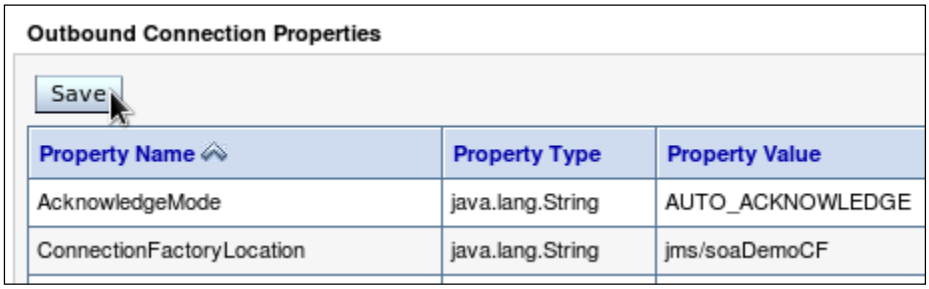
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

29. On the Settings for JmsAdapter Configuration > Outbound Connection Pools tab page, in Outbound Connection Pool Configuration Table, expand the oracle.tip.adapter.jms.IjmsConnectionFactory entry and click the eis/soaDemo/Queue link.



30. On the Settings for oracle.tip.adapter.jms.IjmsConnectionFactory Properties tab page, in the Outbound Connection Properties table, click in the Property Value cells to enter the following Property Name value:
    - ConnectionFactoryLocation: `jms/soaDemoCF`

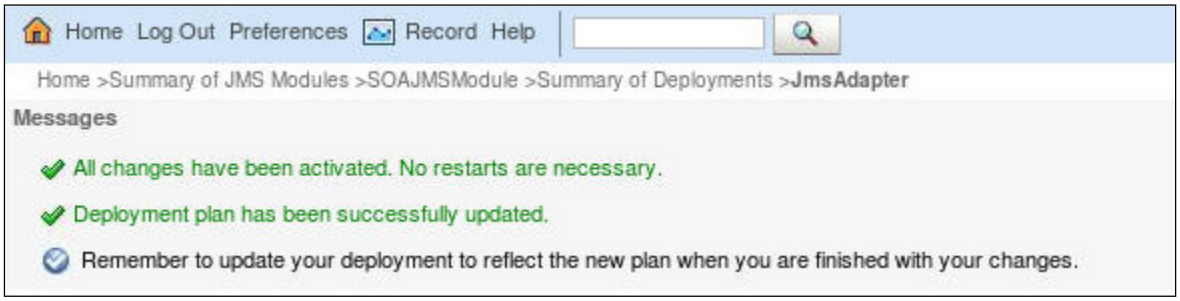
    Note: Ensure that you press Enter after entering the connection factory location text value! Do not use the Tab key.
31. On the Settings for oracle.tip.adapter.jms.IjmsConnectionFactory Properties tab page, click Save.



Oracle Internal & Oracle Academy Use Only

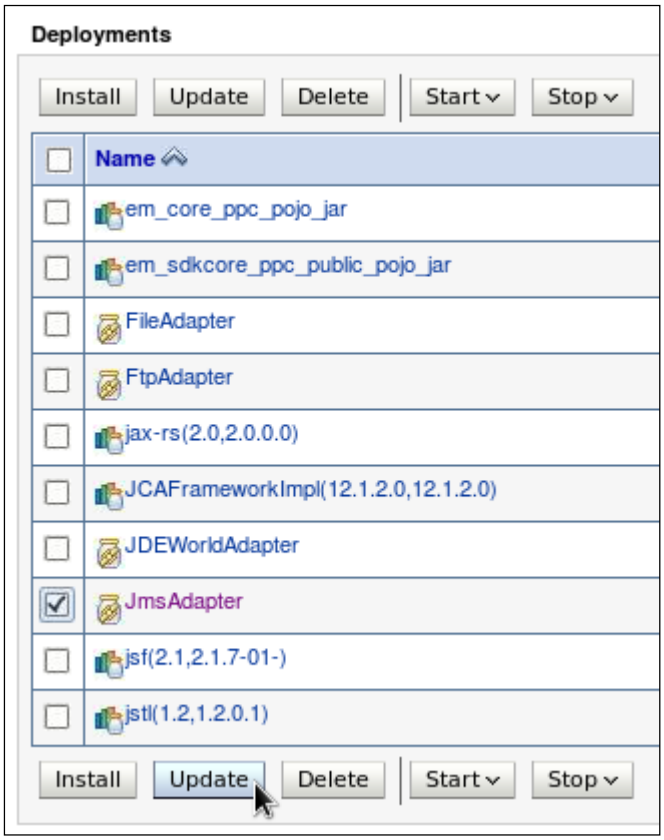
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

32. When the messages are displayed, confirm that the changes were successful.



Redeploying the JMS Adapter with the Newly Configured Resources

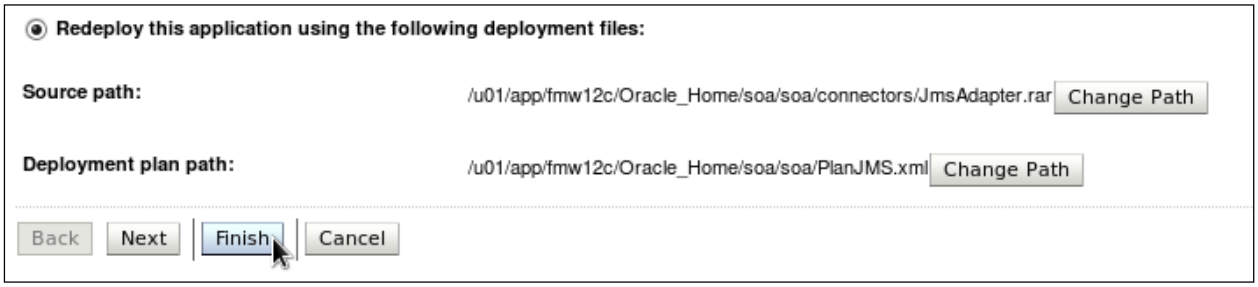
33. In the Domain Structure panel, click Deployments.
34. On the Summary of Deployments page, in the Deployments table, click Next.
35. On the Summary of Deployments page, in the Deployments table, select the check box next to JmsAdapter and click Update.



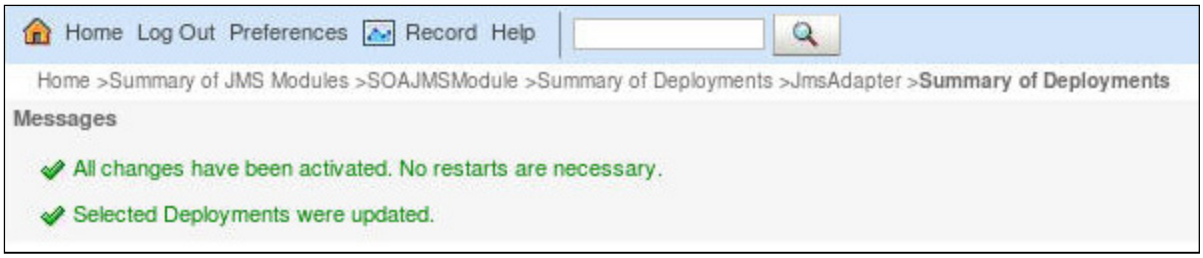
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

36. On the Update Application Assistant page, select the “Redeploy this application using the following deployment files” option and click Finish.



37. When the Messages response is displayed, confirm that the deployments were updated.



38. Minimize your browser.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-6: Creating a Composite Application and JMS Adapter

Overview

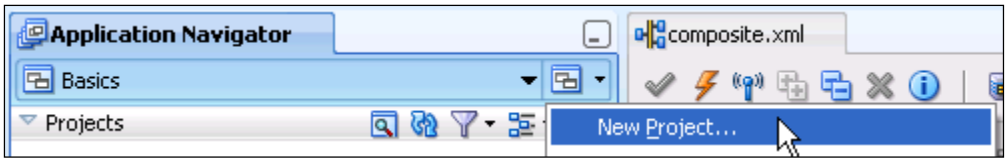
In this practice, you create a new SOA project. You then create and configure a Mediator and a JMS adapter.

Assumptions

This practice assumes that you have completed all work through Practice 5-5 successfully.

Tasks

1. In JDeveloper, create a new empty SOA project named `JMSProducer` in the Basics application.



   The composite overview window is opened.
2. Add a JMS adapter to the External References swimlane.

   The Adapter Configuration wizard appears.
3. Configure the JMS adapter by using the instructions in the following table:

Step	Screen	Choices or Values
a.	JMS Adapter Reference	Name: `jmsProducer` Click Next.
b.	JMS Provider	Select the Oracle WebLogic JMS option. Click Next.
c.	Service Connection	Connection: `IntegratedWebLogicServer` Click Next.
d.	Adapter Interface	Accept the default “Define operation and schema (specified later)” option. Click Next.
e.	Operation	Operation Type: Select the `Produce Message` option. Operation Name: `Produce_Message` Click Next.
f.	Produce Operation Parameters	Browse for and select the Queue and JNDI names from the SOAJMSModule folder in the Select Destination window. Destination Name (Queue): `jms/soaDemoQueue` JNDI Name: `eis/soaDemo/Queue` Accept default settings for the other fields. Click Next.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Step	Screen	Choices or Values
g.	Messages	Use the Browse and Type Chooser windows to import the file `/home/oracle/labs/files/xsd/fulfillment.xsd` into the project. Deselect the "Maintain original directory structure for imported files" check box. Expand the Project Schema Files > `fulfillment.xsd` entry and select the `ShippingRequest` element. Click OK. Click Next.
h.	Finish	Click Finish.

The JMS adapter is added to the External References swimlane.

4. Add a Mediator to the project.

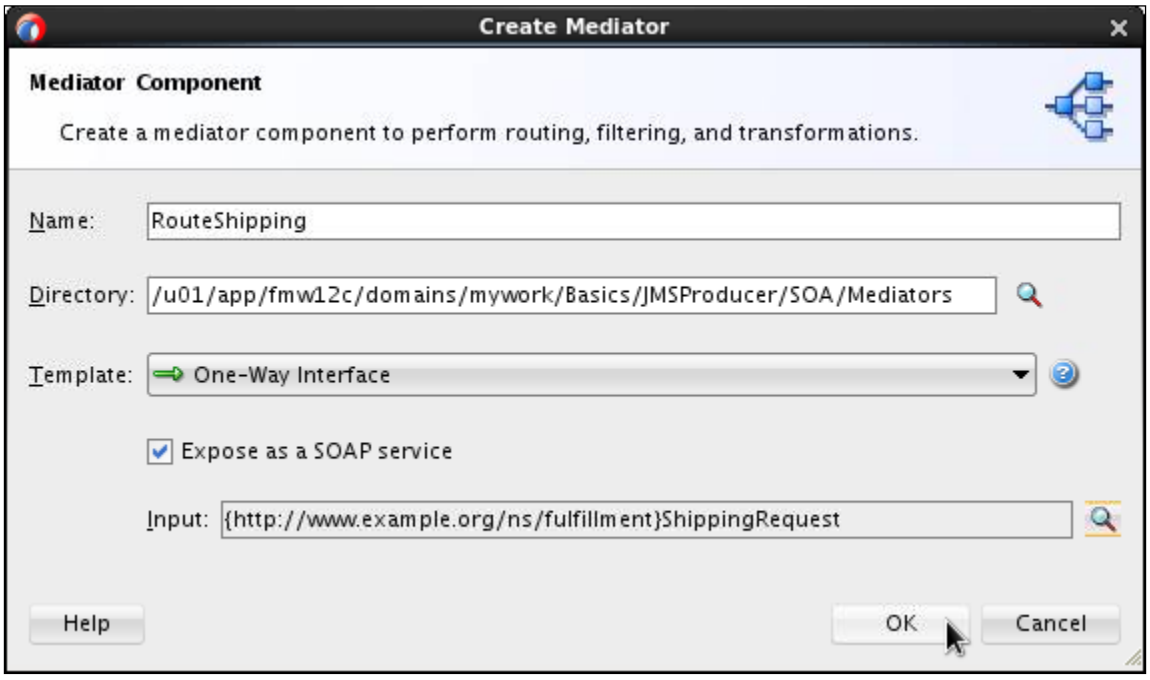
   The Create Mediator wizard opens.

   a. Provide the name `RouteShipping` and select the One-Way Interface template.

      The "Expose as a SOAP service" check box and Input field are displayed.

   b. To update the Input field, use the Browse and Type Chooser windows to expand the Project Schema Files > `fulfillment.xsd` entry and select the `ShippingRequest` element.

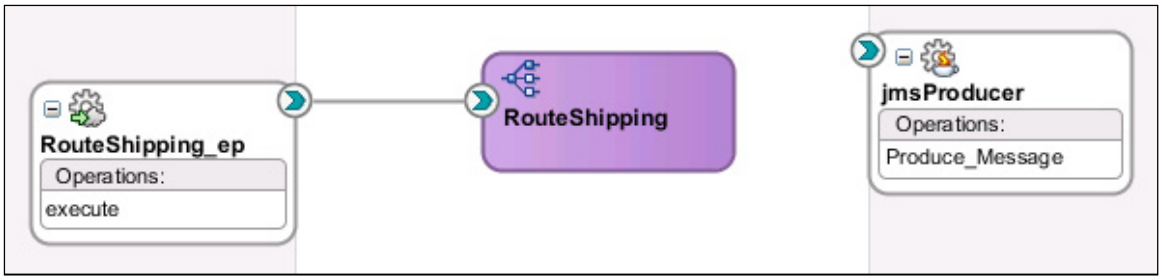
   c. Compare your work to the following screenshot and click OK:



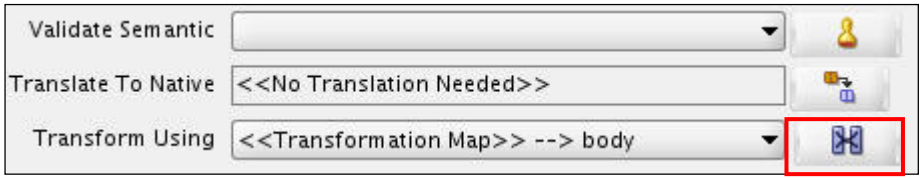
The project window is updated with the new Mediator that is wired to an exposed service named `RouteShipping_ep`.

Oracle Internal & Oracle Academy Use Only

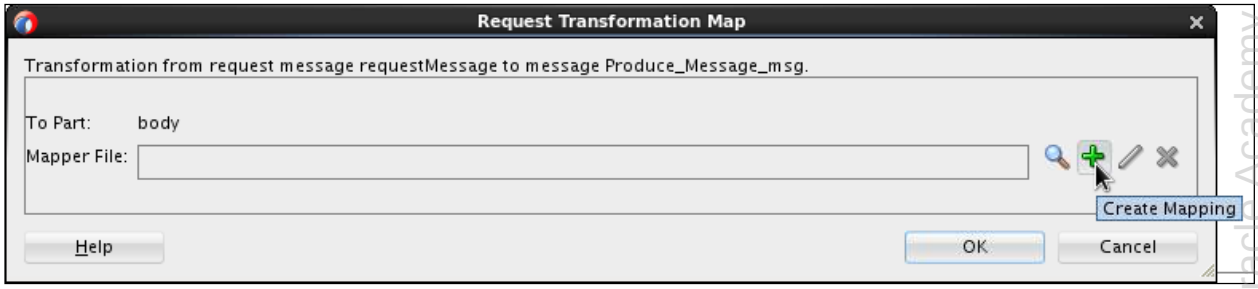
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



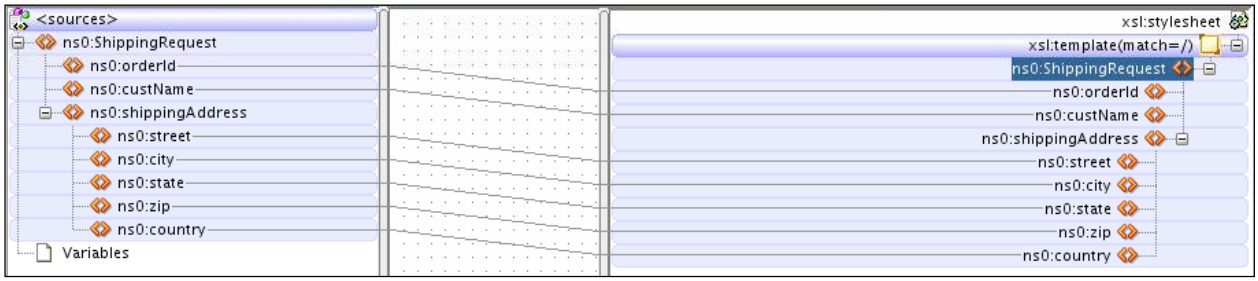
5. Add a wire from the Mediator to the JMS adapter.
6. Create an XSLT mapping.
   a. Right-click the Mediator and select Edit to open the `RouteShipping.mplan` editor.
   b. Click the “Select an existing mapper file or create a new one” icon.



   c. In the Request Transformation Map dialog box, click the Create Mapping icon.



   The Create Transformation Map dialog box is displayed.
   d. Accept the default file name supplied. Click OK.
   e. Click OK to close the Request Transformation Map dialog box.
   The XSLT Mapper window opens.
   f. Map the ShippingRequest node to ShippingRequest.
   The Auto Map Preferences window opens.
   g. Click OK to accept all defaults.
   h. Compare your work to the following screenshot to verify that all lower-level nodes are mapped.



7. Save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-7: Deploying and Testing the Application

Overview

In this practice, you deploy and test the JMSProducer composite application.

Assumptions

This practice assumes that you have completed all work through Practice 5-6 successfully.

Tasks

1. In the JDeveloper Application Navigator, right-click the JMSProducer project and select Deploy > JMSProducer.

   The Deploy JMSProducer wizard opens.

2. Use the instructions in the following table to complete the deployment:

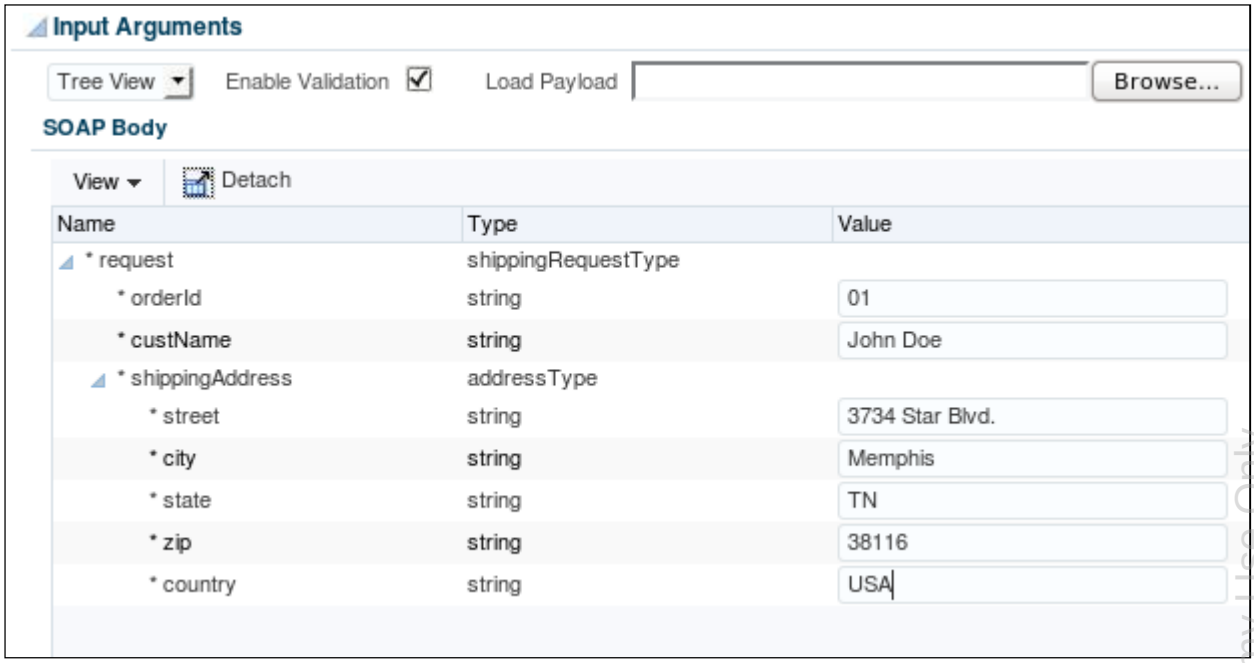
Step	Window	Choices or Values
a.	Deployment Action	Deploy to Application Server. Click Next.
b.	Deploy Configuration	Select the “Overwrite any existing composites with the same revision ID” check box. Click Next.
c.	Select Server	`IntegratedWebLogicServer` Click Next.
d.	SOA Servers	Click Next.
e.	Summary	Click Finish.

3. Deployment processing starts. Monitor deployment progress and check for successful compilation in the SOA – Log window. Verify that deployment is successful in the Deployment – Log window.
4. Test the deployed application.
   a. In a web browser window, access Enterprise Manager at http://localhost:7101/em.
   b. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “JMSProducer [1.0]” link.
   c. On the “JMSProducer [1.0]” home page, click Test.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

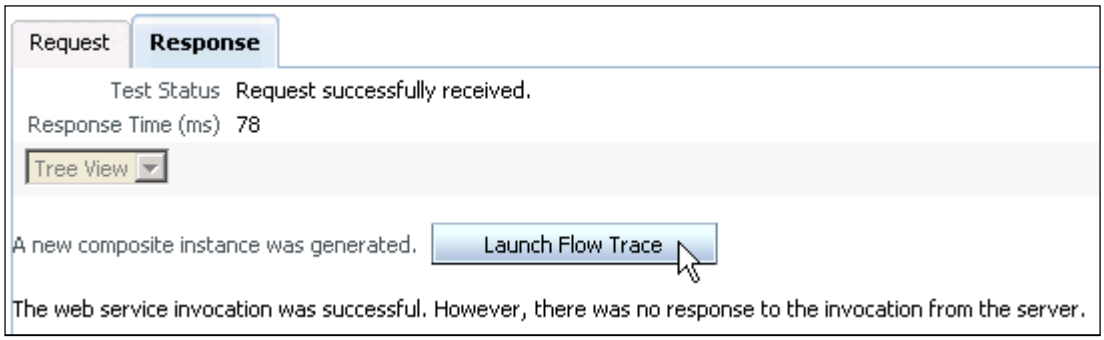
d. On the Test Web Service page, scroll down to the Input Arguments section to the Request tab and supply some test data of your choice.



e. Click Test Web Service.

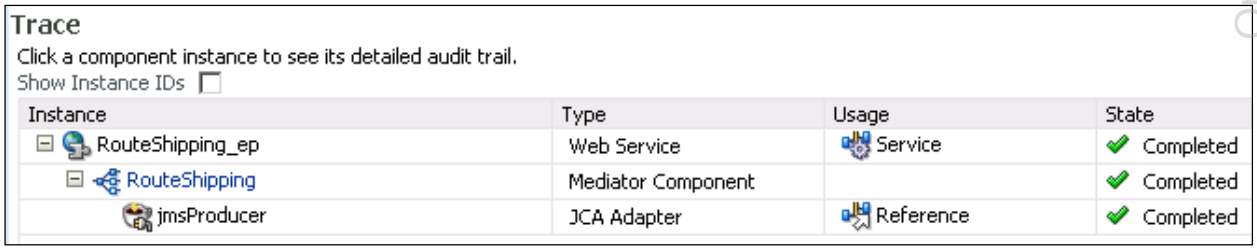
The Response tab is displayed.

f. Verify that the test reports success and click Launch Flow Trace.



g. In the Flow Trace window, verify that each component displays a state of Completed.

h. Click the RouteShipping instance to view the audit trail for that component.



i. Close Enterprise Manager.

Note: In the next practice, you may create a JMS Consumer that retrieves the message from the Queue and publishes it to a file.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-8: Optional: Creating a JMS Consumer Application

Overview

In this optional, unstructured practice, you create a new SOA project that is configured to consume the messages that are sent to the queue in Practice 5-7. The messages pass through the application unchanged and are written to a file.

Assumptions

This practice assumes the following:

- JDeveloper is open.
- IntegratedWebLogicServer is started.
- You have successfully completed all steps in Practice 5-7, and your test of that practice resulted in the output that was described in the instructions.
- You wish to test your understanding of the work that you have completed thus far with a less-structured practice.

Tasks

1. Before you begin, consider the following:
   - Which component will you use to consume messages from the JMS queue?
   - Which component will you use to write those messages to a file?
   - Would you need to use any other components? Which ones? Why?
   - Would you need to reference any external files? Which ones? Why?
2. Create a new empty SOA project named `JMSConsumer` in the Basics application.
3. Add a JMS adapter to the project.
   - Into which swimlane should you add the adapter?
   - Configure the adapter to consume the messages that are sent to the queue in Practice 5-7.

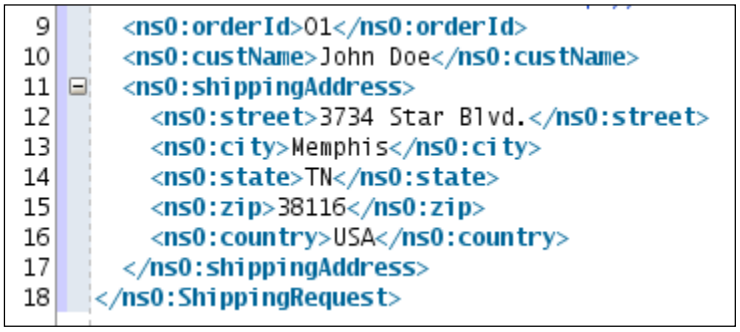
     Hint: To reference the queue from the JMSProducer practice, use the same Destination and JNDI names that you used in that practice.
4. Add and configure other required components.
   - Make sure that the output files are written to `/home/oracle/labs/output/jms`.
5. Wire the components together.
6. Deploy the application.

   Any messages in the queue from Practice 5-7 should be consumed and written to the file immediately.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

7. Open the output file in JDeveloper. The message payload should look similar to the following screenshot:



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 5-9: Optional: Setting Message Properties and Using Message Selectors

Overview

In this optional practice, you modify your previous SOA project so that JMSProducer sets a user-defined message property to a specific value. You then modify JMSConsumer to use a message selector so that only a subset of the published messages are received and written to a file.

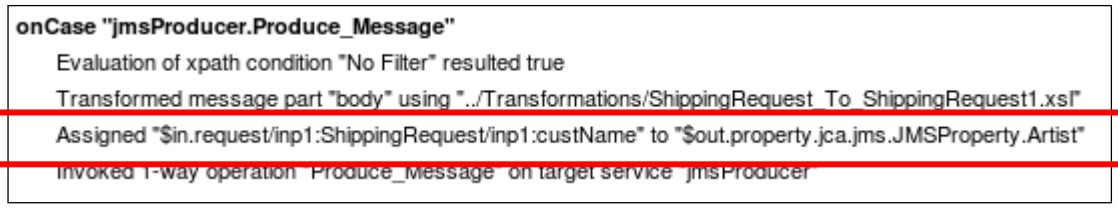
Assumptions

This practice assumes the following:

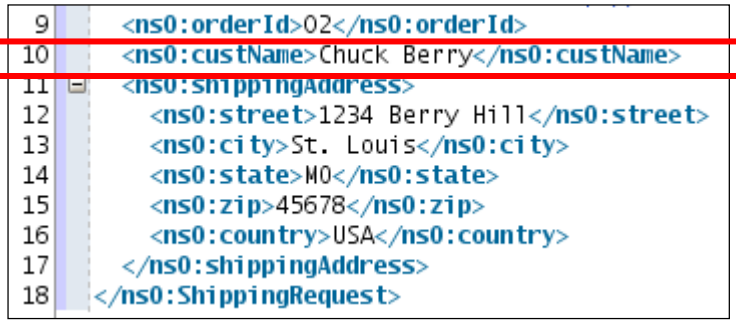
- JDeveloper is open.
- IntegratedWebLogicServer is started.
- You have successfully completed all steps, including optional Practice 5-8, and your test of that practice resulted in the output that was described in the instructions.
- You want to further your knowledge of setting JMS Message Properties and using Message Selectors to filter JMS messages.

Tasks

1. Open JMSProducer Mediator.
2. Assign a user-defined message property.
   - From: $in.request/inpl:Shippingrequest/inpl:custName
   - To: $out.property.jca.jms.JMSProperty.Artist
3. Redeploy and test.
   - Verify that the property is set in Flow Trace.



   - Verify that the message is written to file.



4. Set the message selector.
   a. Open JMSConsumer Adapter.
   b. Set a Message Selector: `Artist LIKE '%Berry'`

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

5. Redeploy and test.
   a. To conduct a positive test, set the customer name to Chuck Berry. This message should be received and written to file by JMSConsumer. (See the screenshots for task 3 in the preceding section.
   b. To conduct a negative Test, set the customer name to Elvis Presley. This message is published to the queue, but is not received by JMSConsumer.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 6: Introduction to BPEL

Chapter 6

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 6: Overview

Practices Overview

In this practice, you learn how to use the following BPEL activities:

- Receive and Reply
- Invoke
- Assign
- Define Partners

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 6-1: Creating a New Application

Overview

In this practice, you create a composite application that includes a simple BPEL process. This process accepts input and calls the credit card validation (`CCValidate`) project that you built earlier.

Assumptions

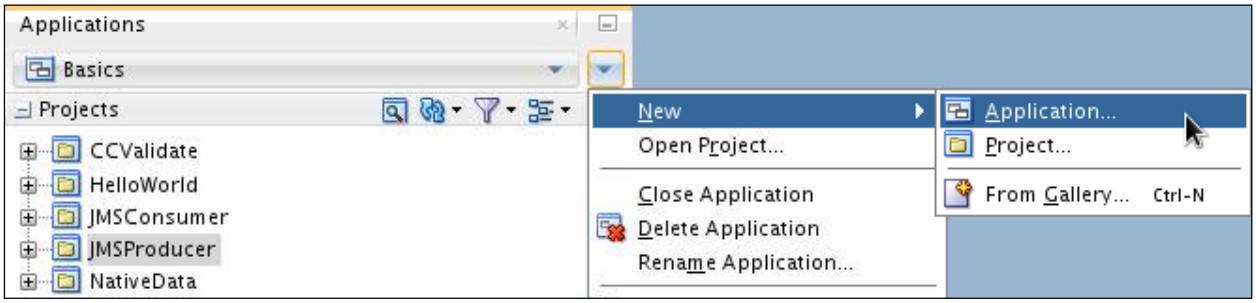
This practice assumes the following:

- The JDeveloper IDE is open.
- You have successfully built, deployed, and tested the project (`CCValidate`) in Practice 5.

Tasks

Creating a New Application and Project

1. In this section, you create a new workspace named BPELProjects, and then create a composite application in this workspace.
2. In the JDeveloper Application Navigator, select New > Application from the **Applications** menu.



3. Use the instructions in the following table to create the application workspace with a SOA project that contains a composite application. Accept the default values for all fields that are not specified in the “Choices or Values” column.

Window	Choices or Values
New Gallery	Application Template: SOA Application Click OK.
Name your application	Application Name: `BPELProjects` Click Next.
Project Name	Project Name: `HelloBPEL` Click Next.
Project SOA Settings	Composite Template: Composite with BPEL Process Click Finish.

The Create BPEL Process wizard appears.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

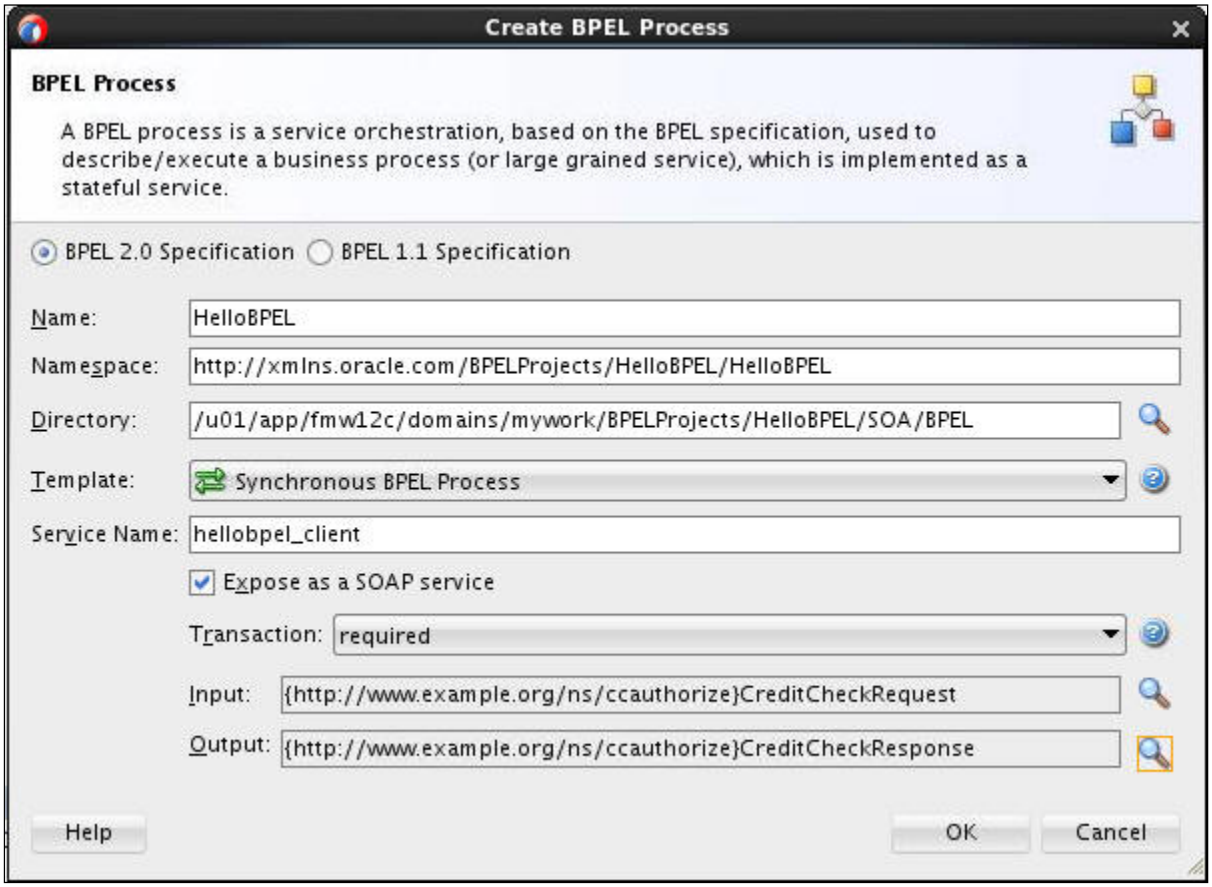
4. Complete the steps in the following table to create the BPEL Process:

Window	Choices or Values
BPEL Process	Name: `HelloBPEL` Template: Synchronous BPEL Process Click the Browse icon to the right of Input. The Type Chooser opens.
Type Chooser	Click the Import Schema File icon.
Import Schema File	With File System selected, navigate to `/home/oracle/labs/files/xsd`. Select `creditcheck.xsd`. Click OK.
Localize Files	Deselect the “Maintain original directory structure…” check box. Click OK.
Type Chooser	Select the message element CreditCheckRequest. Click OK.
Create BPEL Process	Click the Browse icon to the right of Output.
Type Chooser	Select the message element CreditCheckResponse. Click OK.

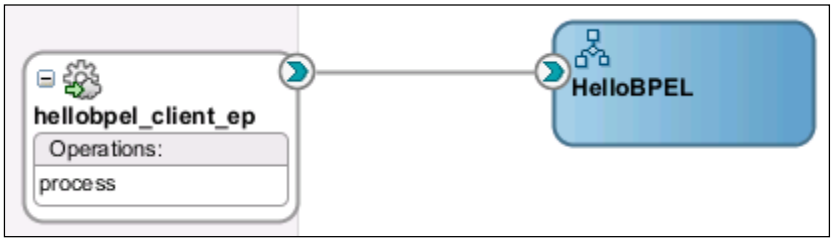
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

5. Compare your work to the following screenshot:



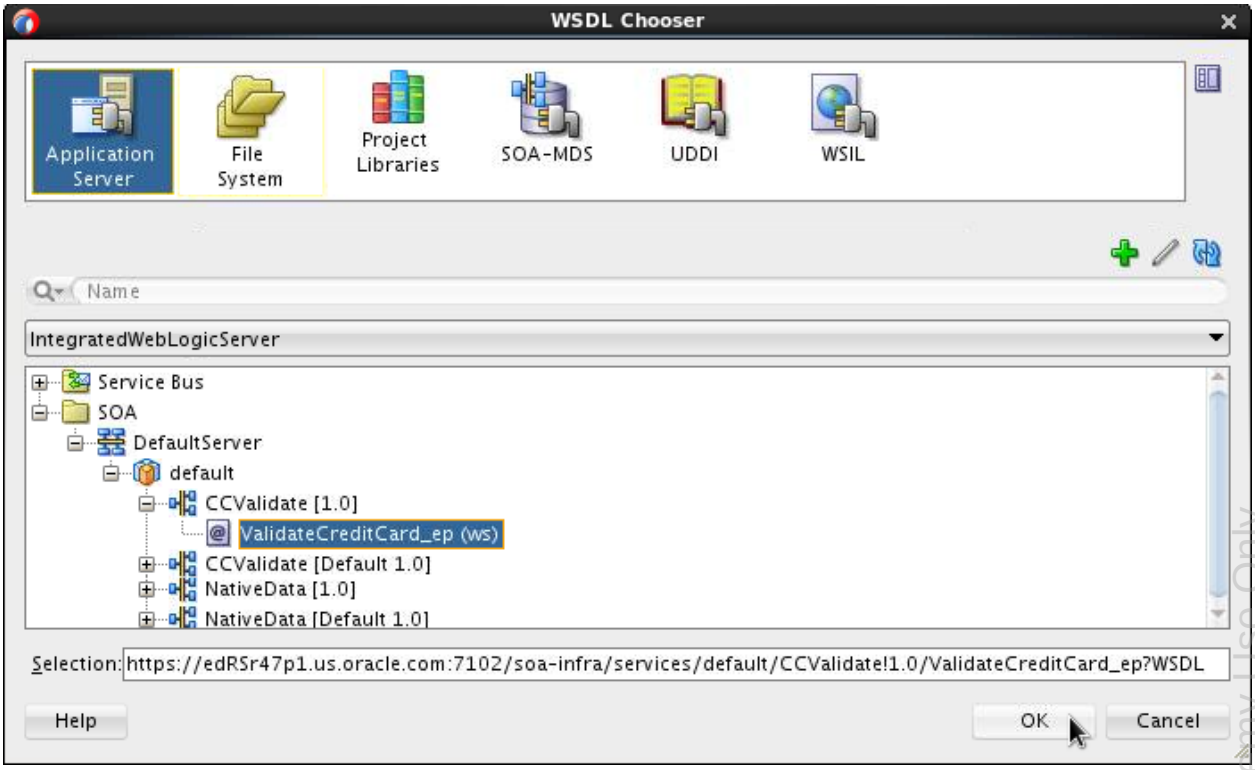
6. Click OK.
   The composite application design window is updated and the BPEL editor is opened.
7. Click the composite overview (HelloBPEL) tab and verify your composite.



8. Add and configure a SOAP web service to the composite as an external reference.
   a. Drag a SOAP web service to the External Reference swimlane.
      The Create Web Service window opens.
   b. Name the service `ValidateCC`.
   c. Define the WSDL URL.
      1) Click the Find Existing WSDLs button.
         The WSDL Chooser opens.
      2) With Application Server selected, navigate to SOA > DefaultServer > default > CCValidate [1.0] > ValidateCreditCard_ep (ws).

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



3) Click OK to accept the choice and close the WSDL Chooser.

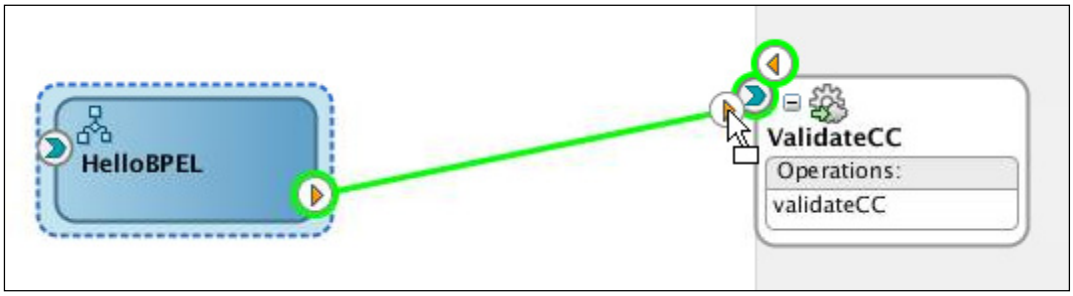
d. Click OK to close the Create Web Service window.
The Localize Files window opens.

e. Deselect the “Maintain original directory structure…” check box.

f. Click OK.

g. Save your work.

9. Create a wire from the BPEL Process to the new SOAP web service.

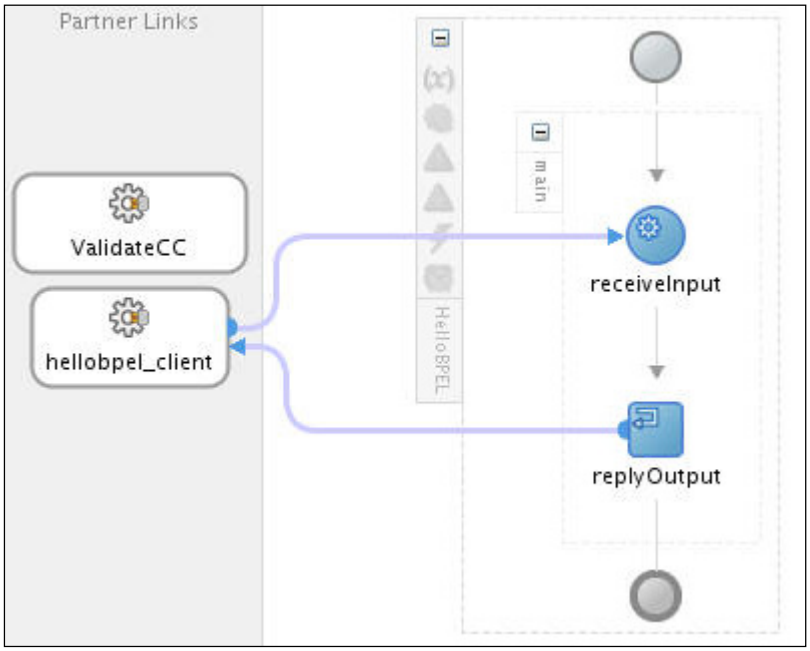


Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

10. Click the `HelloBPEL.bpel` tab.

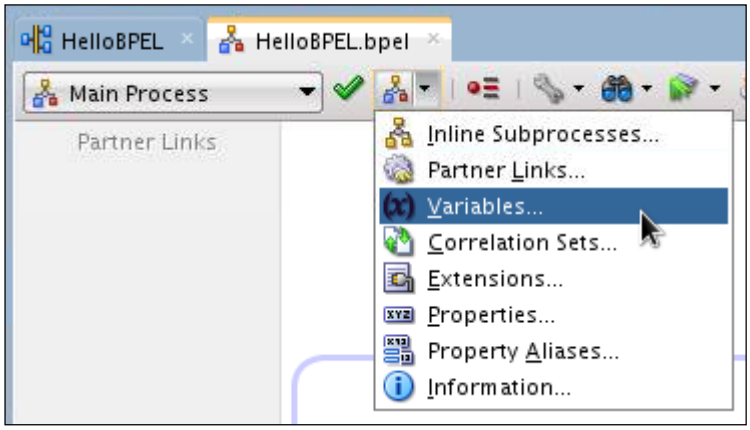
    The business process is created with the Receive and Reply activities.



11. Optional Step: Pause to check your understanding.

    If you are new to BPEL, this is a good time to pause and check your understanding of the elements of the business process that are already in place.

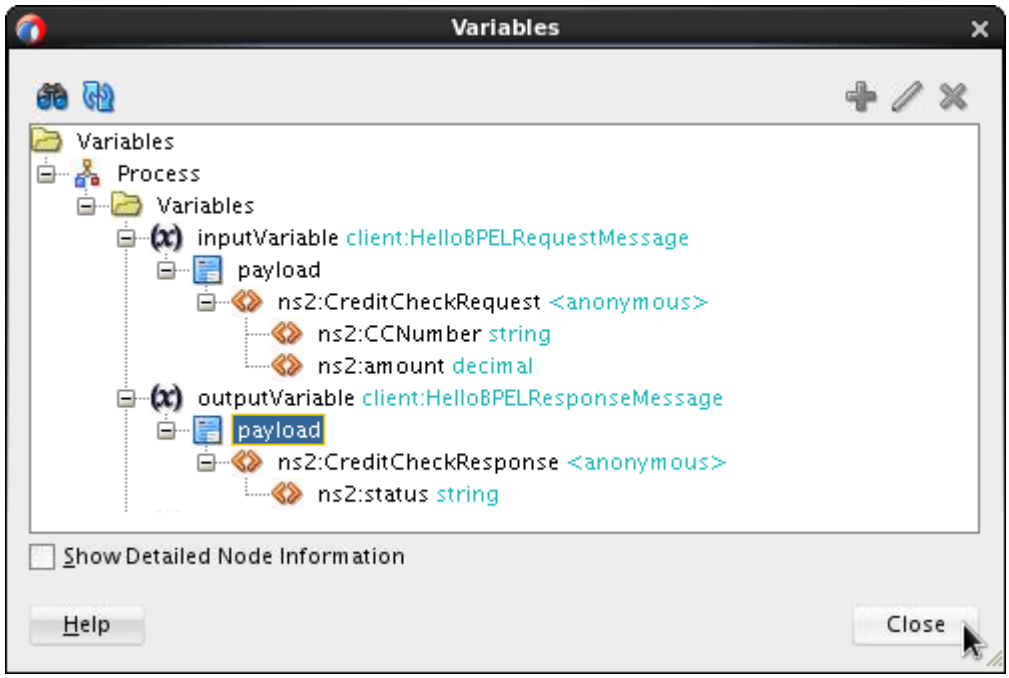
    a. From the BPEL editor menu, select Property Structure > Variables.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

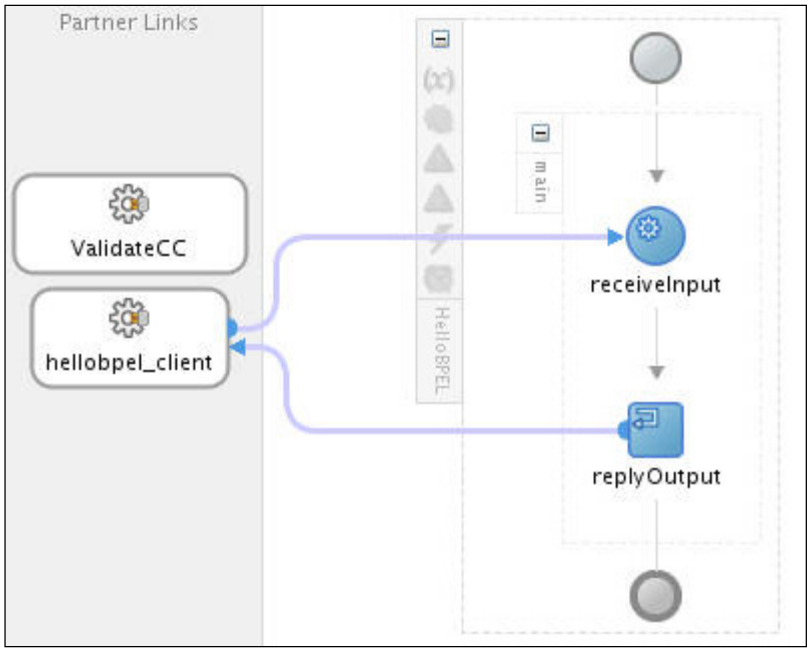
The process input and output variables are shown.



b. Consider the answer to the following questions:
   - What action did you take to create them?
   - Why were both the input and output variables created?

c. Close the variables pane.

d. Identify the Service Interface and the Activities in the following screenshot:

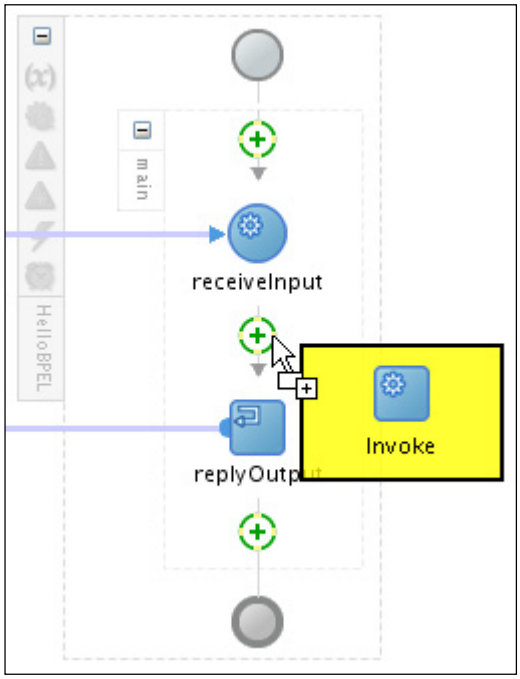


e. If you are comfortable with the vocabulary and concepts that you have just reviewed, continue building your BPEL process.

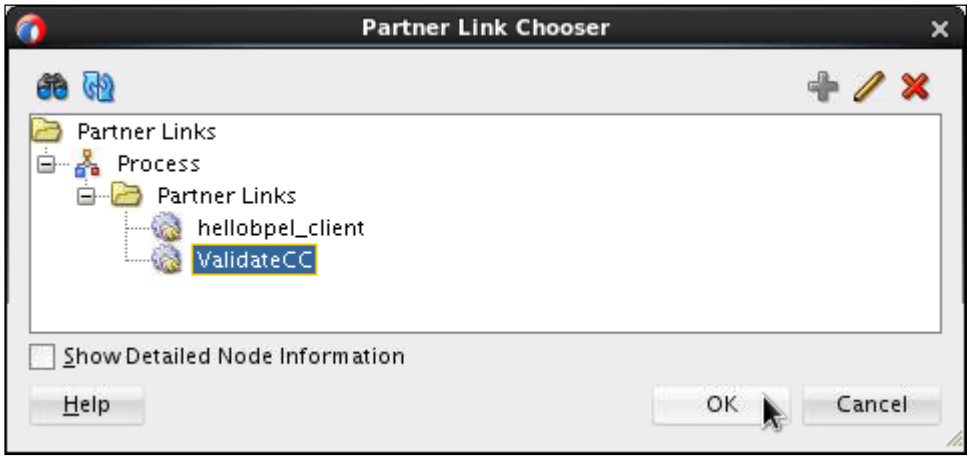
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

12. Create and configure an Invoke activity in the business process.
    This activity calls the `ValidateCC` project that was deployed in Practice 5.
    a. Add an Invoke activity to the process between the Receive and Reply activities.



    b. Right-click the Invoke activity and select Edit.
    c. Rename the activity `Invoke_ValidateCC`.
    d. Click the Browse icon to the right of the Partner Link field.
    The Partner Link Chooser dialog box opens.
    e. Select the Partner Link ValidateCC.



    f. Click OK.
    You are returned to the Invoke configuration window.
    g. Click the Add (green plus) icon to create an input variable.
    h. Click OK to accept the default parameters.
    i. Click the Output tab.
    j. Click the Add icon to create an output variable.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

k. Click OK to accept the default parameters.

l. Click OK to complete the Invoke configuration.

13. Create and configure an Assign activity to populate the input variable for the ValidateCC service.

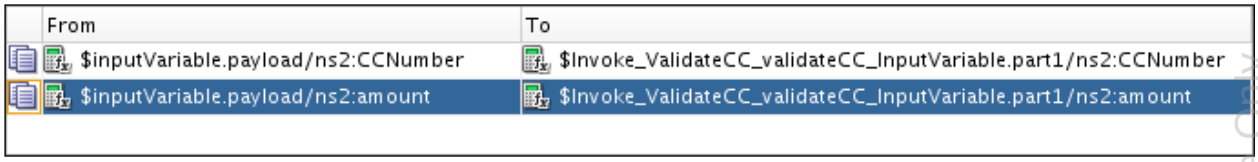
a. Add an Assign activity above `Invoke_ValidateCC`.

b. Name the activity `Assign_ValidateCC`.

c. Right-click Assign and select Edit.

d. Map the `CCNumber` node of `inputVariable` to the `CCNumber` node of `Invoke_ValidateCC_validatecc_InputVariable`.

e. Map the `amount` node of `inputVariable` to the `amount` node of `Invoke_ValidateCC_validatecc_InputVariable`.



f. Click OK.

14. Add an Assign to map the ValidateCC service output to the process response message.

a. Add an Assign below the Invoke.

b. Right-click Assign and select Edit.

c. Name the activity `Assign_Response`.

d. Map the `status` node of `Invoke_ValidateCC_validatecc_OutputVariable` to the `status` node of `outputVariable`.

e. Verify your work and click OK.



f. Save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 6-2: Deploying and Testing the Application

Overview

In this practice, you deploy and test the HelloBPEL composite application.

Assumptions

This practice assumes that you have completed all previous steps in this practice successfully and that JDeveloper is open.

Tasks

1. In the JDeveloper Application Navigator window, deploy the HelloBPEL composite application to IntegratedWebLogicServer.
   - Remember to select the “Overwrite any existing composites with the same revision ID” check box.
   - Monitor the “Deployment – Log” window to ensure that deployment is successful.
2. Update the Audit Level to Development.

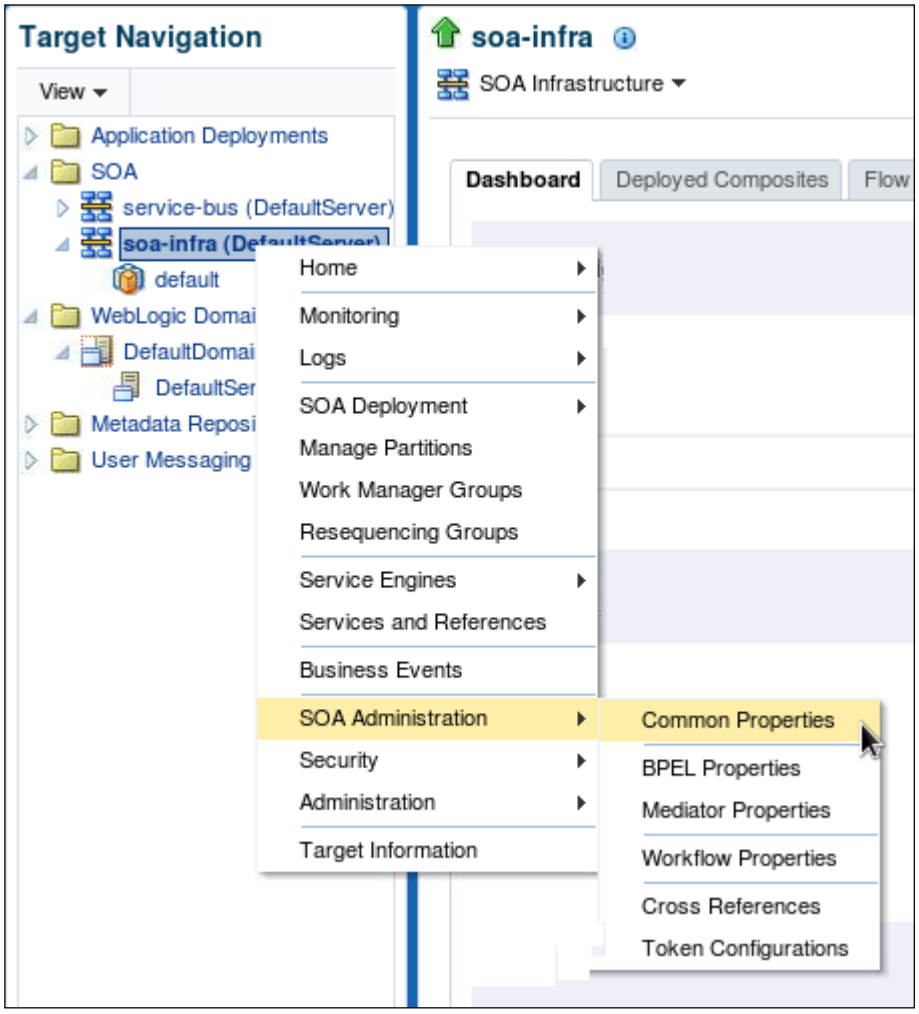
   Note: This change causes more detail to be visible in the Enterprise Manager test tool, which may be of value when debugging this and the subsequent practices.

   a. In a web browser window, access Enterprise Manager at http://localhost:7101/em.

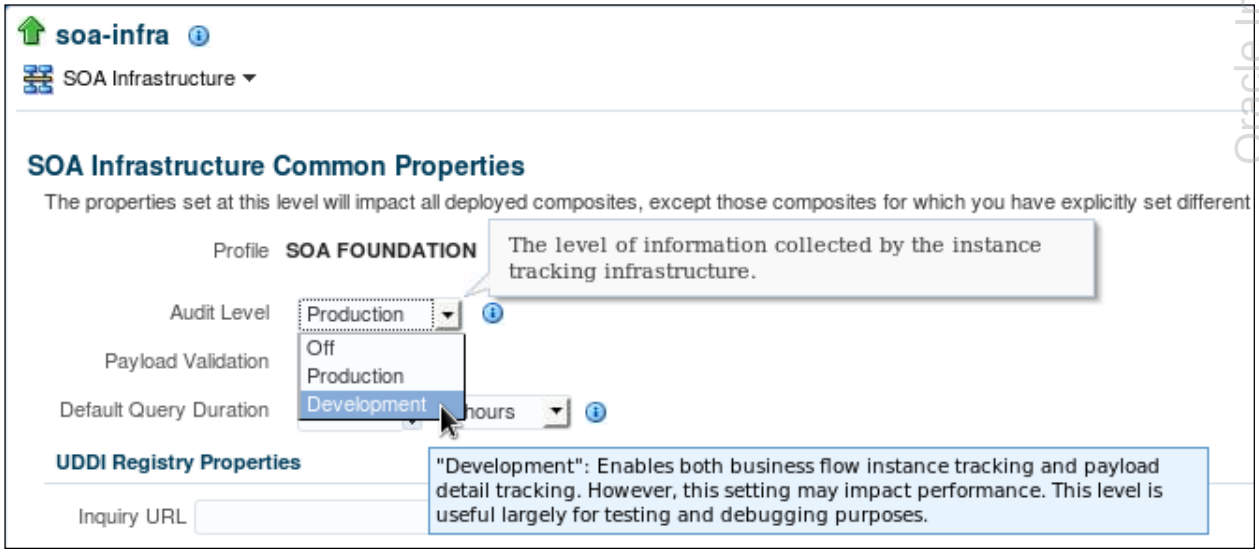
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

b. In the Target Navigation pane, right-click soa-infra and select SOA Administration > Common Properties.



c. In the Common Properties pane, use the drop-down list to update the Audit Level to Development.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

d. Click Apply.
e. In the Confirmation dialog box, click Yes.

3. Test the project.
   a. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the HelloBPEL [1.0] link.
   b. On the HelloBPEL [1.0] home page, click Test.
   c. On the Test Web Service page, scroll down to the Input Arguments section of the Request tab and enter the following values:
      - CCNumber: 1234-1234-1234-1234
      - Amount: 1000
   d. Click Test Web Service.
      The Response tab is displayed.
   e. Verify that the status value returned is `VALID`.
4. Perform a second test to display the INVALID response.
   a. On the Request tab, modify the CCNumber value to be `4321-4321-4321-4321`.
   b. Leave the amount at `1000`.
   c. Click Test Web Service.
   d. On the Response tab page, verify that the status string value returned is `INVALID`.

Points to Consider

This test and the outcome are identical to the test for the CCValidate project that you built earlier. We could have defined the Database adapter as a partner link in this process. It seems like an extra step to have called the composite application from this process.

- When would it make sense to incorporate credit card validation directly into a business process?
- When would this architecture (calling one application from another) make sense?

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 7: More BPEL Activities

Chapter 7

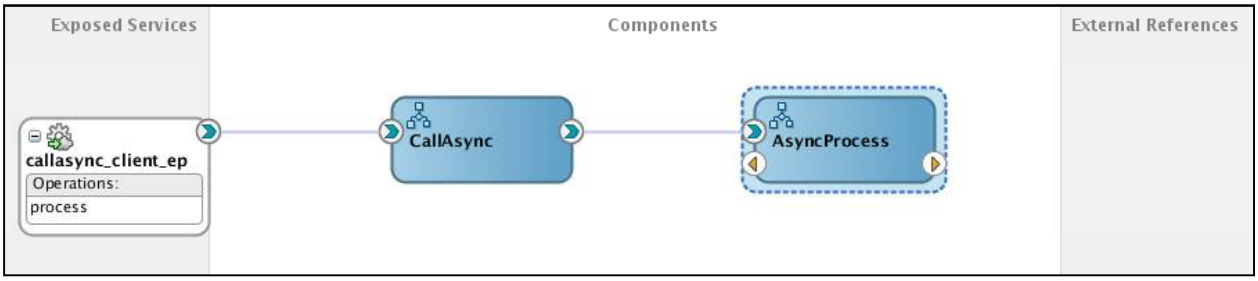
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 7: Overview

Practices Overview

This practice demonstrates calling an asynchronous service. To do this, you build a composite application with two BPEL processes. The first of these is an asynchronous BPEL process. The process uses the While and Wait activities to loop for a specified amount of time. The time is specified by a value that is passed in the incoming message. The second process calls the asynchronous service, and specifies how long that process should loop. The calling process then uses a Pick activity with both an onMessage and an onAlarm branch. The onAlarm branch specifies how long to wait for a response. If the called process returns a message before onAlarm is activated, the onMessage branch executes. If not, the onAlarm branch is triggered instead.



Before You Begin

The path references in this and other practices include XML namespace prefixes (for example, the `ns2` in **`ns2:`**`product`). If you have built each of the components in the order prescribed in the instructions, your namespace references will match. If you have built the components in a different order, or have deleted, and then re-created the components, your namespace prefixes may not match what is listed in these instructions. In that case, *use the namespace references in your project. Do **NOT** change those references to match the book*.

Best Practices

In this practice, you build a synchronous BPEL process, which calls an asynchronous BPEL process. This is not a common design pattern in real-world implementations. It suits our goals for this practice very well, but it would not be a desirable choice for most applications outside the classroom.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 7-1: Creating a New Project

Overview

In this practice, you create a new composite application that includes a BPEL project.

Assumptions

This practice assumes that the JDeveloper IDE is open.

Tasks

Creating a New Project

In this section, you create a new composite application in the BPELProjects workspace.

1. Create a composite application.
   a. In the JDeveloper Application Navigator Project menu, select New > Project.
      The Project wizard opens.
   b. Complete the steps in the following table:

Window	Choices or Values
New Gallery	SOA Project Click OK.
Name your project	Project Name: `AsyncDemo` Click Next.
Project SOA Settings	Composite Template: Composite With BPEL Process Click Finish.

   The Create BPEL Process wizard opens.

2. Use the instructions in the following table to configure the BPEL process:

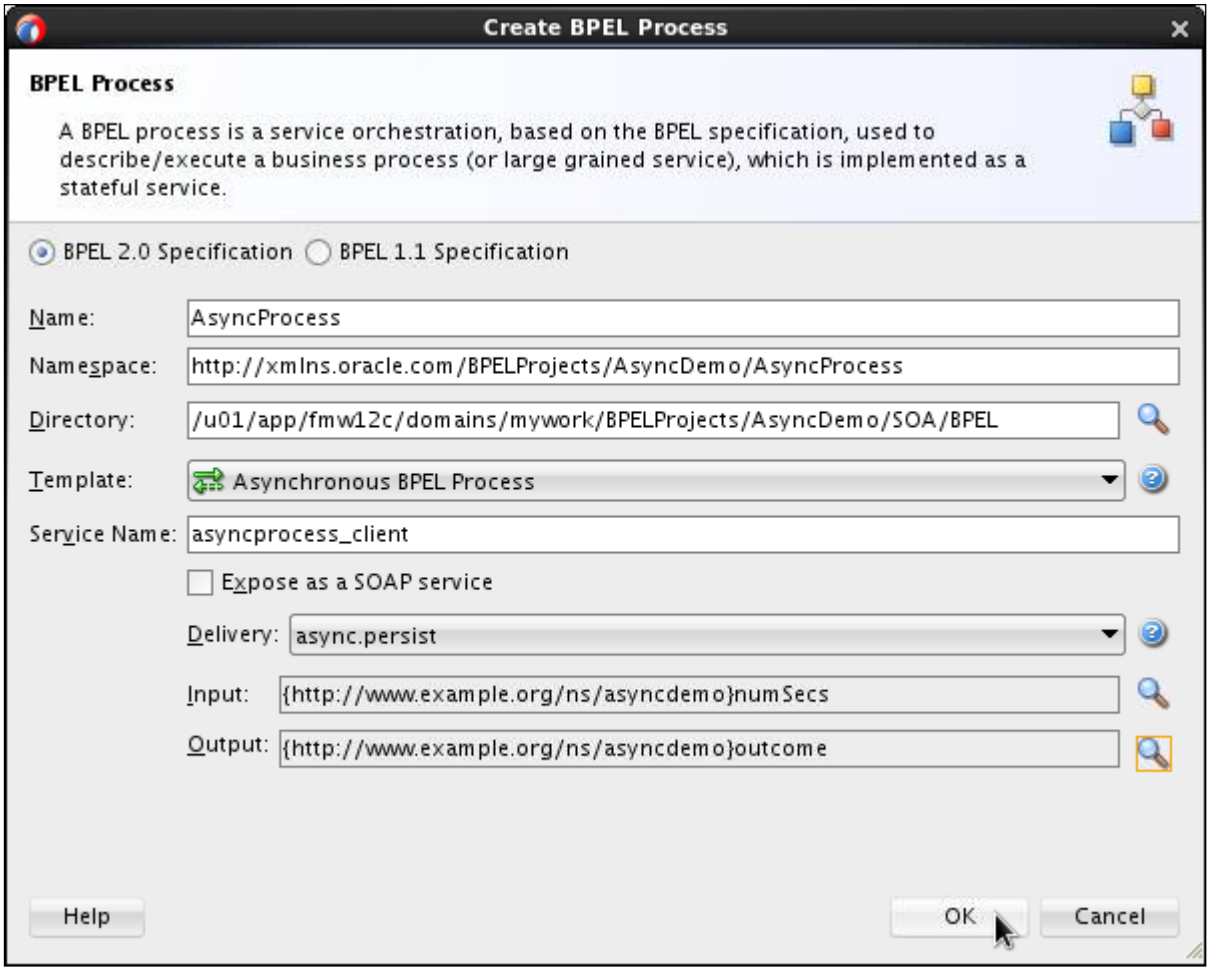
Step	Window	Choices or Values
a.	BPEL Process	Name: `AsyncProcess` Template: Asynchronous BPEL Process (default) Deselect the “Expose as a SOAP Service” check box. Click the Browse Input Elements icon. The Type Chooser opens.
b.	Type Chooser	Click the Import Schema File icon.
c.	Import Schema file	Select File System. Navigate to `/home/oracle/labs/files/xsd`. Select `asyncDemo.xsd`. Click OK.
d.	Localize Files	Deselect the “Maintain original directory …” check box. Click OK.
e.	Type Chooser	Select the message element `numSecs`. Click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Step	Window	Choices or Values
f.	Create BPEL Process	Click the Browse Output Elements icon to the right of Output.
g.	Type Chooser	Select the `asyncDemo.xsd` message element `outcome`. Click OK.

3. Compare your work to the following screenshot and click OK:



The BPEL Editor opens.

4. Save your work.

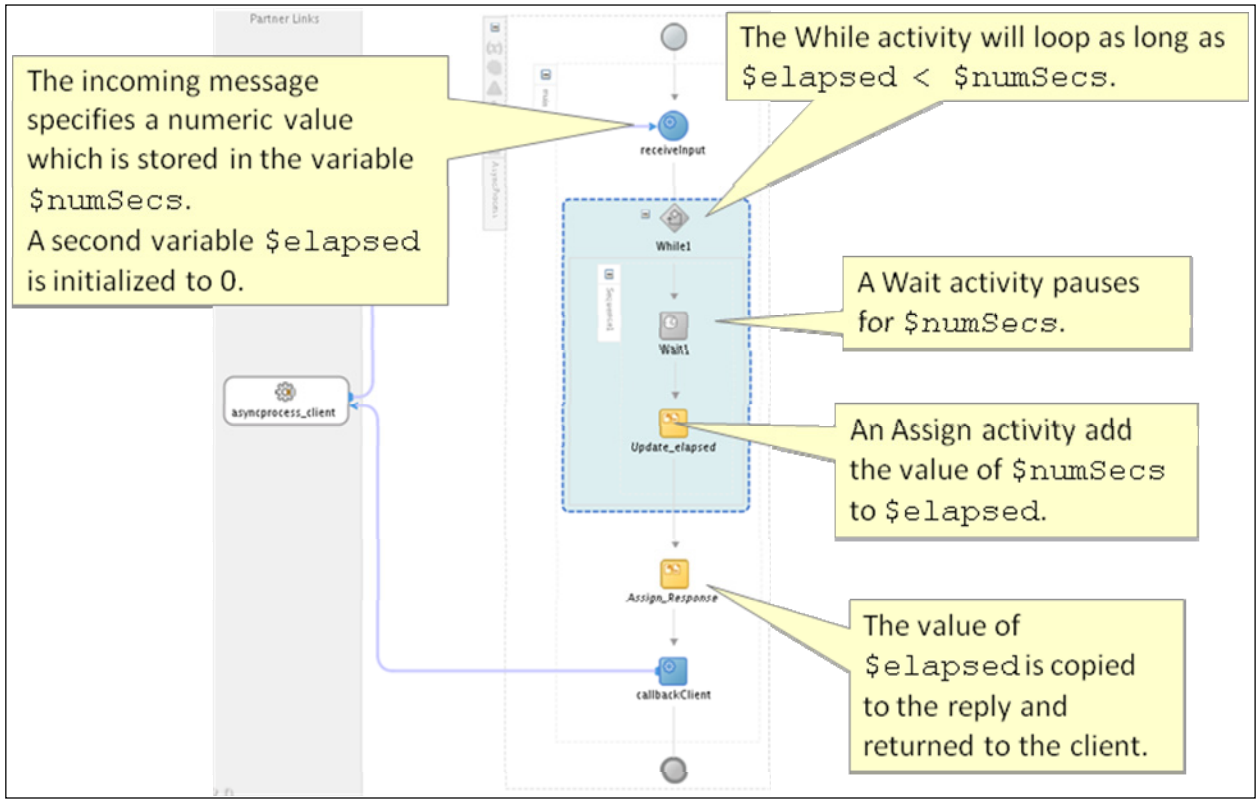
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 7-2: Configuring the Asynchronous Service

Overview

In this practice, you create an asynchronous service that includes a BPEL process. The process uses the While and Wait activities to loop for a specified amount of time. The time is specified by a value that is passed in the incoming message.



Assumptions

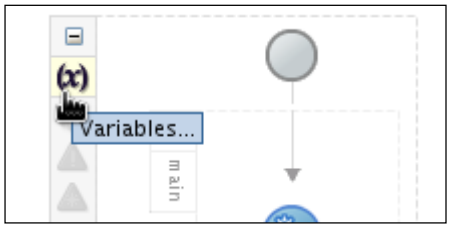
This practice assumes that JDeveloper is open, and that you have completed Practice 7-1 successfully.

Tasks

Creating a Counter Variable

The counter variable counts the number of iterations of the While loop.

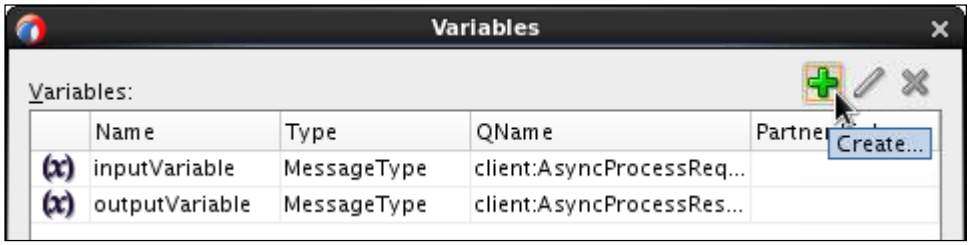
1. Create and initialize a variable named `elapsed` of type `int`.
   a. In the main process scope, click the Variables icon.



Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

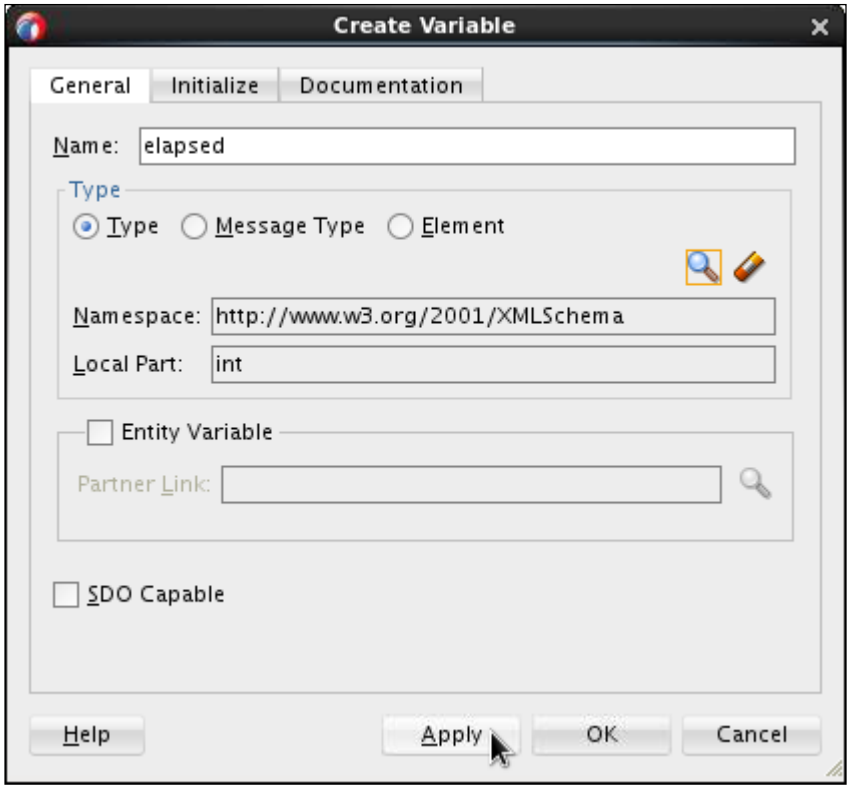
b. In the Variables dialog box, click the Create icon.



The Create Variable dialog box is displayed.

c. To define the variable, perform the following steps:

1) Enter `elapsed` in the Name field.
2) Select the Type option.
3) Click the Browse icon next to the Type option to open the Type Chooser window.
4) In the Type Chooser window, select the “XML Schema Simple Types” > int entry.
5) Click OK.



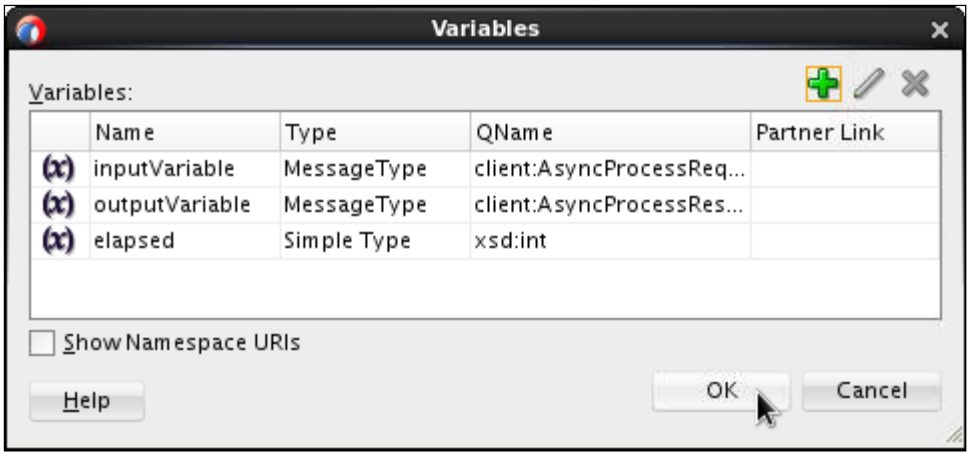
6) Click Apply. (Do not click OK.)

d. To define the variable, perform the following steps:

1) Click the Initialize tab.
2) Select Data Source: Expression
3) In the Expression pane, enter the number `0`.
4) Click OK to close the Create Variable dialog box.

Oracle Internal & Oracle Academy Use Only

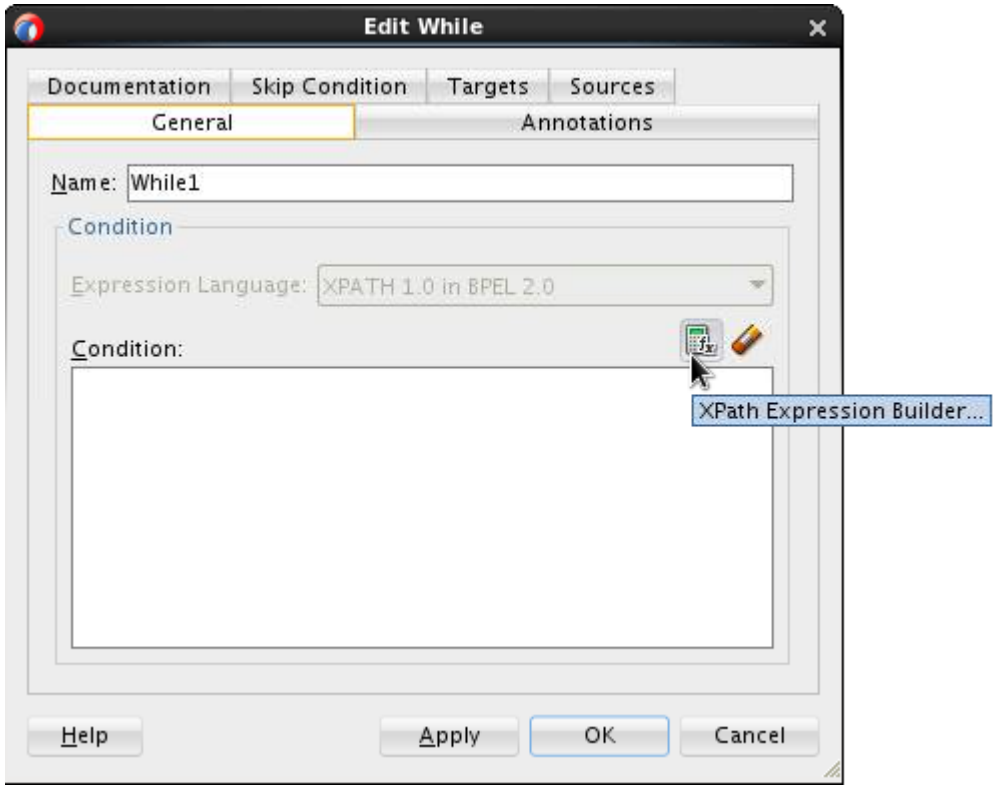
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



e. Verify your work and click OK to close the Variables dialog box.

Creating a While Loop

2. Create and configure a While activity so that the loop continues when `$elapsed < $inputVariable`.
   a. Add a While activity after the receiveInput activity.
   b. Right-click the While icon and select Edit.
   c. On the While > General tabbed page, click the XPath Expression Builder icon.



   The Expression Builder window opens.
   d. Select the BPEL variable `elapsed`.
   e. Click Insert Into Expression.
   f. In the Expression pane, enter "`<`" (excluding the quotation marks).

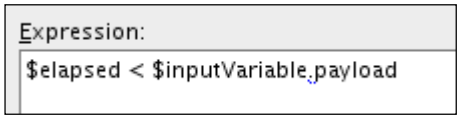
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

g. Select the BPEL variable `inputVariable.payload`.

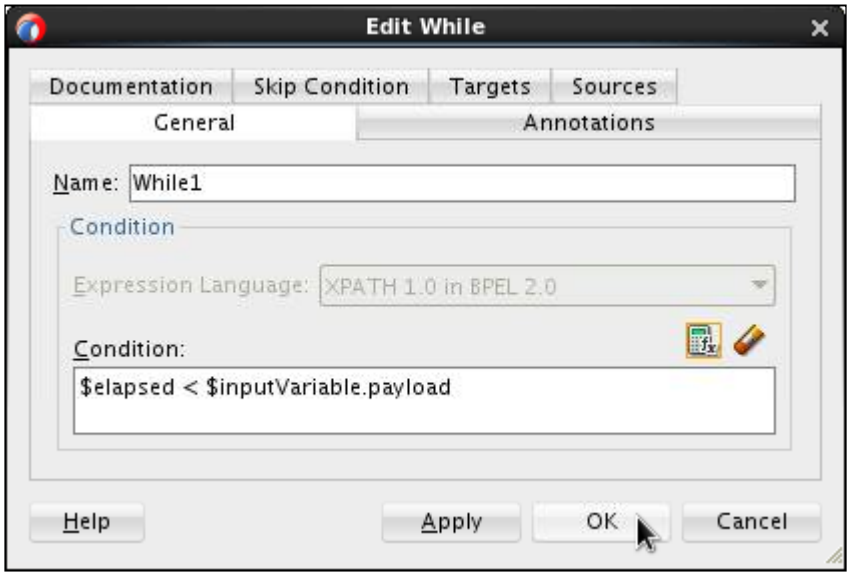
h. Click Insert Into Expression.

i. Verify your work and click OK.



The Expression Builder window closes.

j. Verify the While loop and click OK.

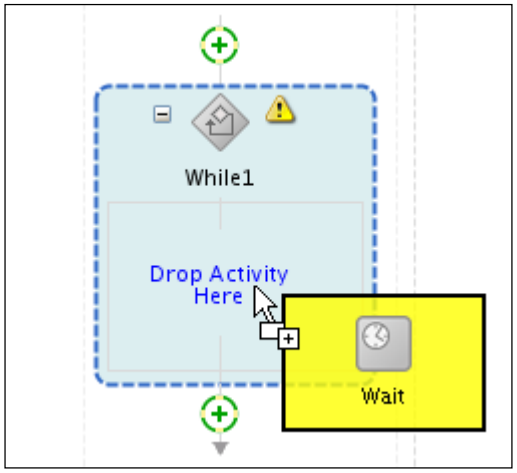


k. Save your work.

Creating a Wait Activity

3. Create and configure a Wait activity.

a. Drag a Wait activity into the While activity.



The Wait activity collapses.

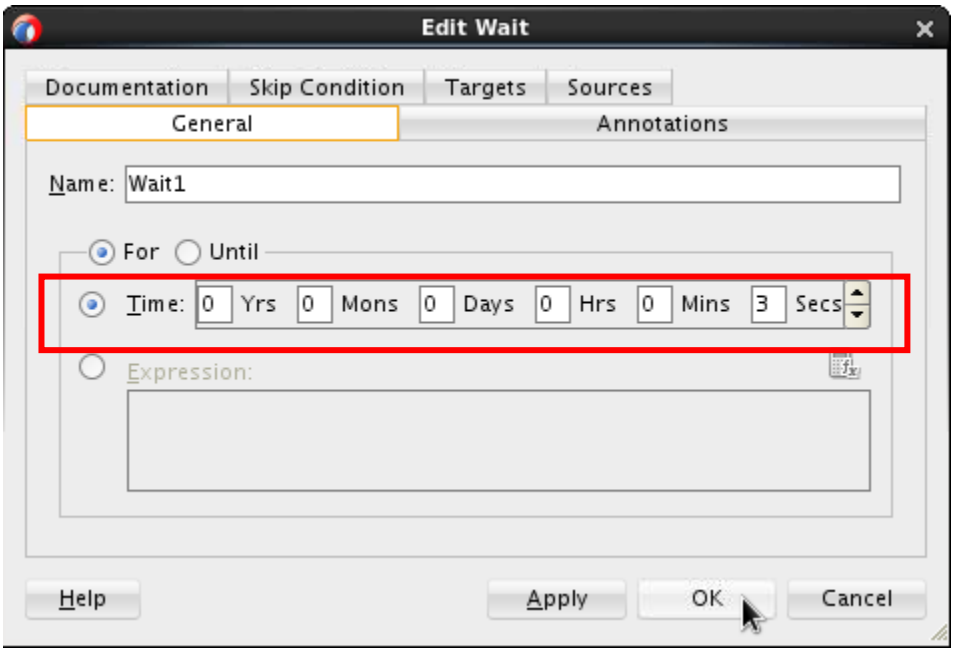
b. Right-click the Wait activity and select Edit.
The Wait properties dialog box is displayed.

c. Enter a value of `3` in the Secs field.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

d. Verify your work and click OK.

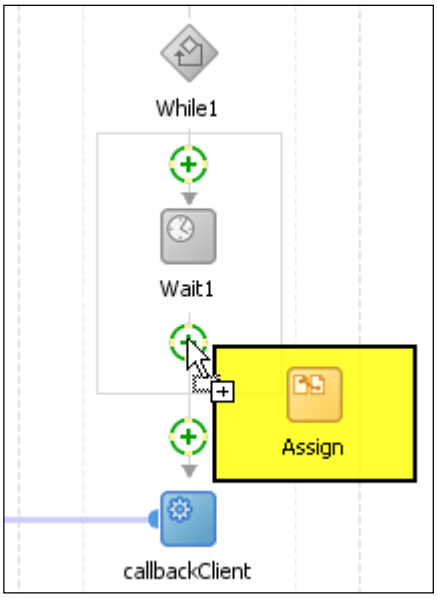


The Wait properties dialog box closes.

Updating Variables with Each Iteration of the Loop

In this step, you update the value of the counter `$elapsed`.

4. Create and configure an Assign activity as the last step in the While sequence.
   a. Add an Assign activity within the While activity (after the Wait activity).

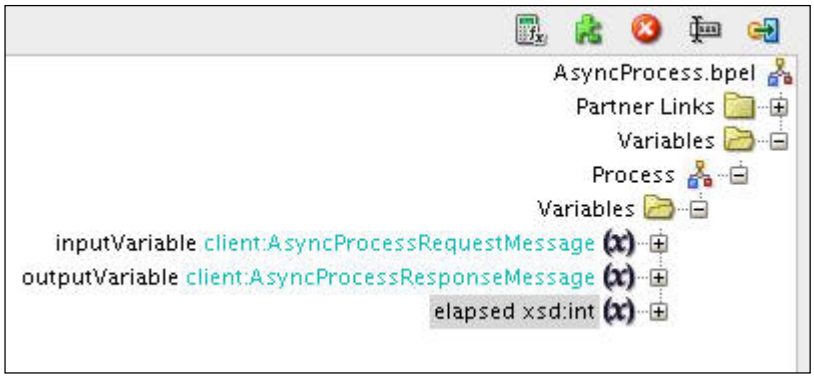


   b. Rename the Assign activity Update_elapsed.
   c. Right-click Update_elapsed and select Edit.

Oracle Internal & Oracle Academy Use Only

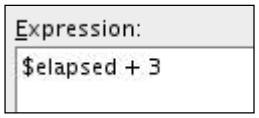
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

d. Drag the Expression Builder icon onto the `$elapsed` variable.



The Expression Builder is displayed.

e. In the Expression pane, add the text `$elapsed + 3`.



f. Verify your work and click OK.
The Expression Builder is closed.

g. Verify the Assign activity and click OK to close the Assign editor.

5. Save your work.

6. Assign a value to the response message.

a. Add an Assign activity after the While activity.

b. Name the activity Assign_Response.

c. Use the Expression Builder to assign the XPath expression:

```
concat('total of ', string($elapsed), ' seconds elapsed.')
```

to `$outputVariable.payload`.

d. Verify and save your work.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 7-3: Creating a BPEL Process to Make an Async Call

In this practice, you build another BPEL process. The second process calls the asynchronous service, and specifies how long the process should loop. The calling process then uses a Pick activity with both an onMessage and an onAlarm branch. The onAlarm branch specifies how long to wait for a response. If the called process returns a message before onAlarm is activated, the onMessage branch executes. If not, the onAlarm branch is triggered instead.

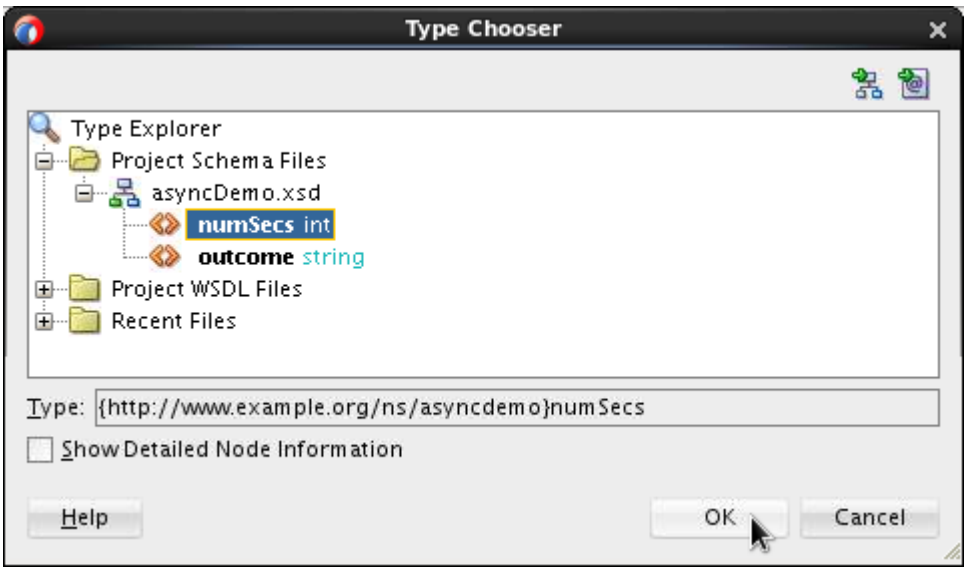
Assumptions

This practice assumes that JDeveloper is open, and that you have completed Practice 7-2 successfully.

Tasks

Adding a BPEL Process to the Application

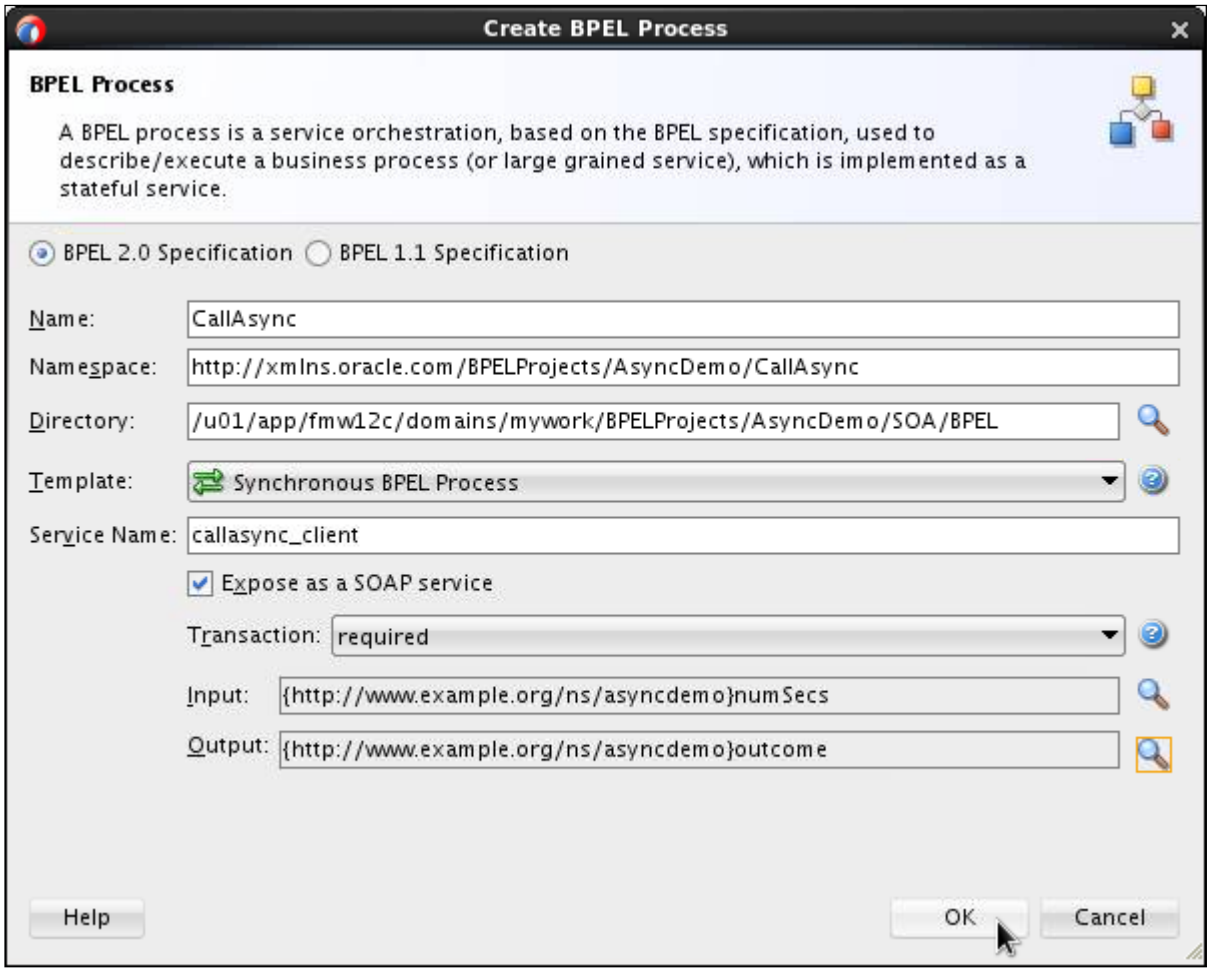
1. Create and configure a BPEL service component.
   a. In JDeveloper, click the composite overview tab.
   b. Add a BPEL process to the Components swimlane.
      The Create BPEL Process dialog box is displayed.
   c. Name the process `CallAsync`.
   d. Select the Synchronous BPEL template.
   e. To define the input, click the Browse Input Elements button.
   f. In the Type Chooser, navigate to the `numSecs` element of `asyncDemo.xsd`.



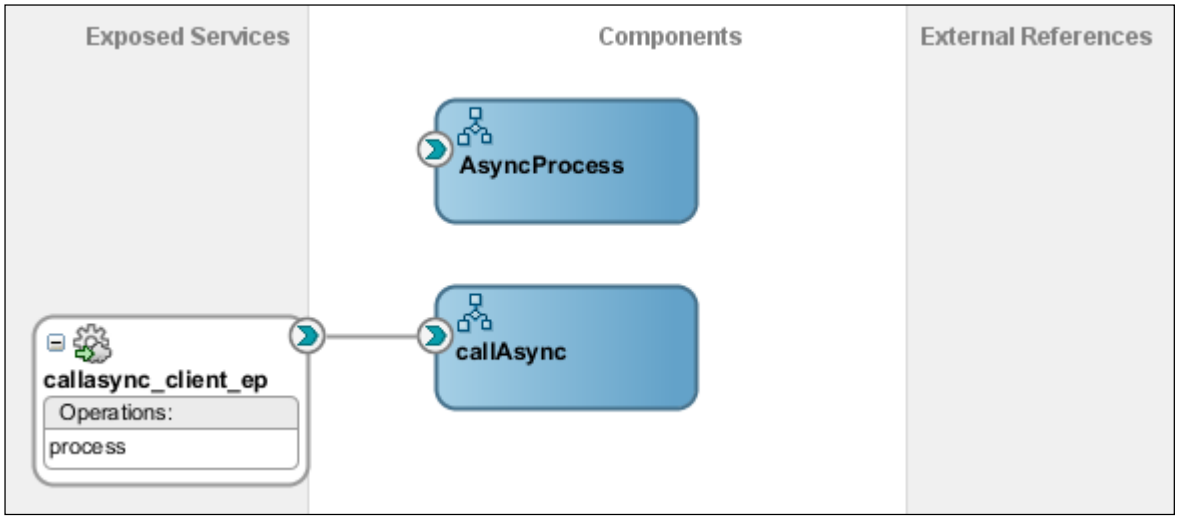
   g. Click the Browse Output Elements icon.
   h. In the Type Chooser, navigate to the `outcome` element of `asyncDemo.xsd`.
   i. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



2. Verify your composite application and save your work.



3. Wire callAsync to AsyncProcess.

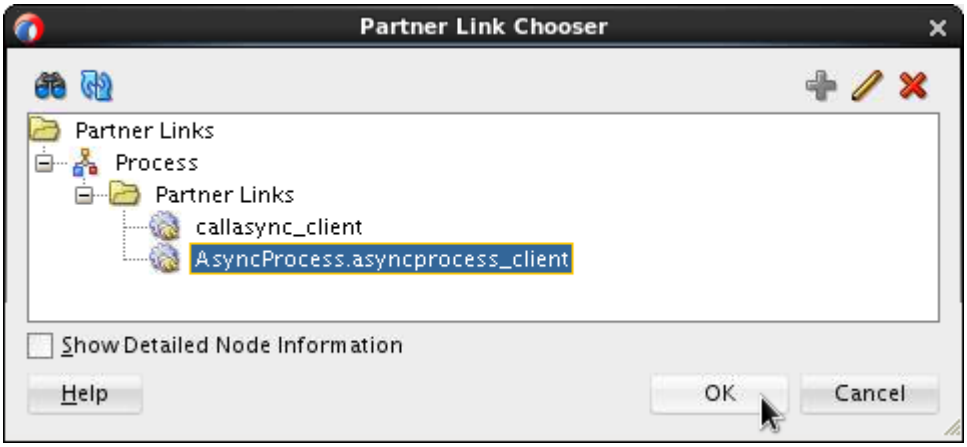
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Invoking the Asynchronous Service

In this step, you create an Invoke activity that calls the web service.

4. Create and configure an Invoke activity.
   a. Right-click the BPEL component callAsync and select Edit.
   b. Add an Invoke activity to the process below the Receive activity.
   c. Right-click the Invoke activity and select Edit.
   d. Name the activity `Invoke_Async`.
   e. Click the Browse Partner Links button and select the partner link `AsyncProcess.asyncprocess_client`.

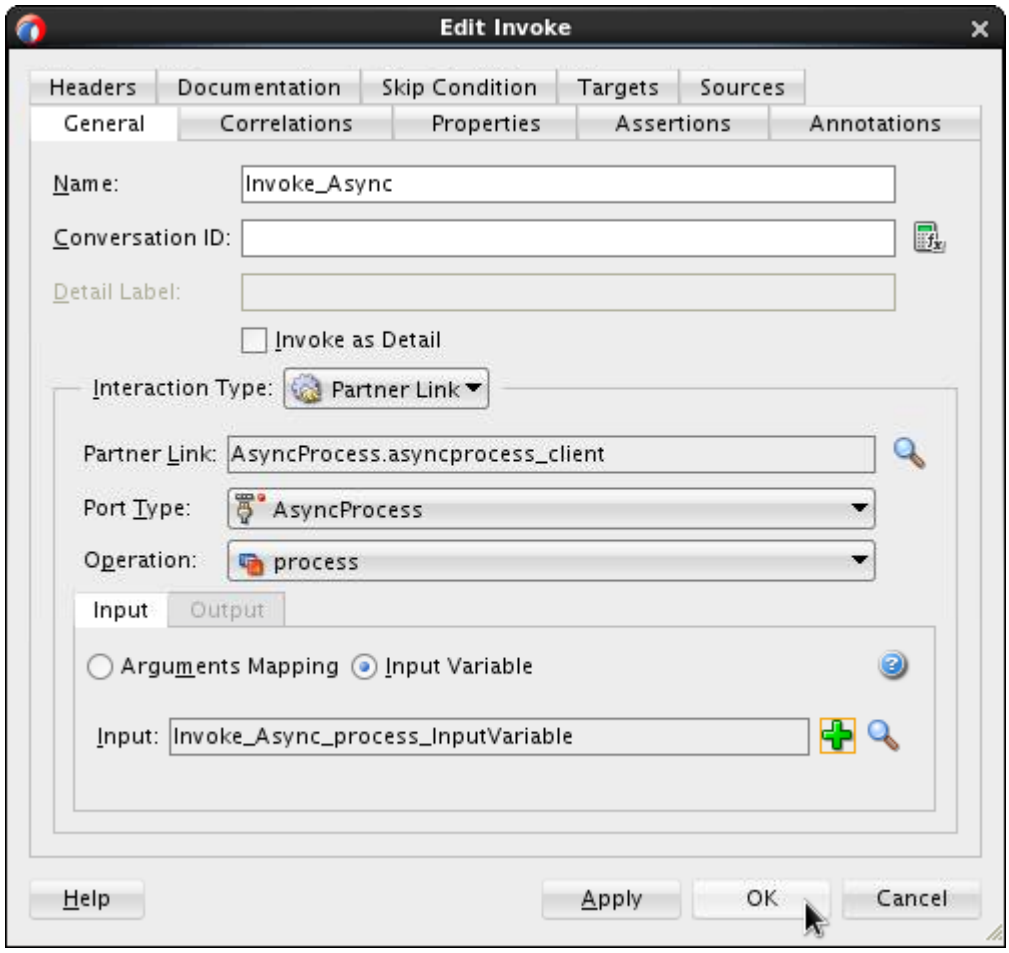


   f. Click OK.
   g. Create an Input variable. (Accept the default name.)

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

h. Verify your work and click OK.

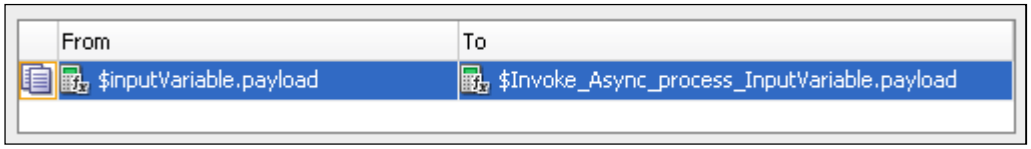


i. Save your work.

Populating the Input Variable for the Service Invocation

In this step, you create an Assign activity to populate the Input variable for the web service invocation.

5. Create and configure the `Assign_Async` activity.
   a. Add an Assign activity above `Invoke_Async`.
   b. Name the activity `Assign_Async`.
   c. Right-click `Assign_Async` and select Edit.
      The Edit Assign window opens.
   d. Map variable `inputVariable.payload` to `$Invoke_Async_process_InputVariable.payload`.



   e. Click OK.
   f. Save your work.

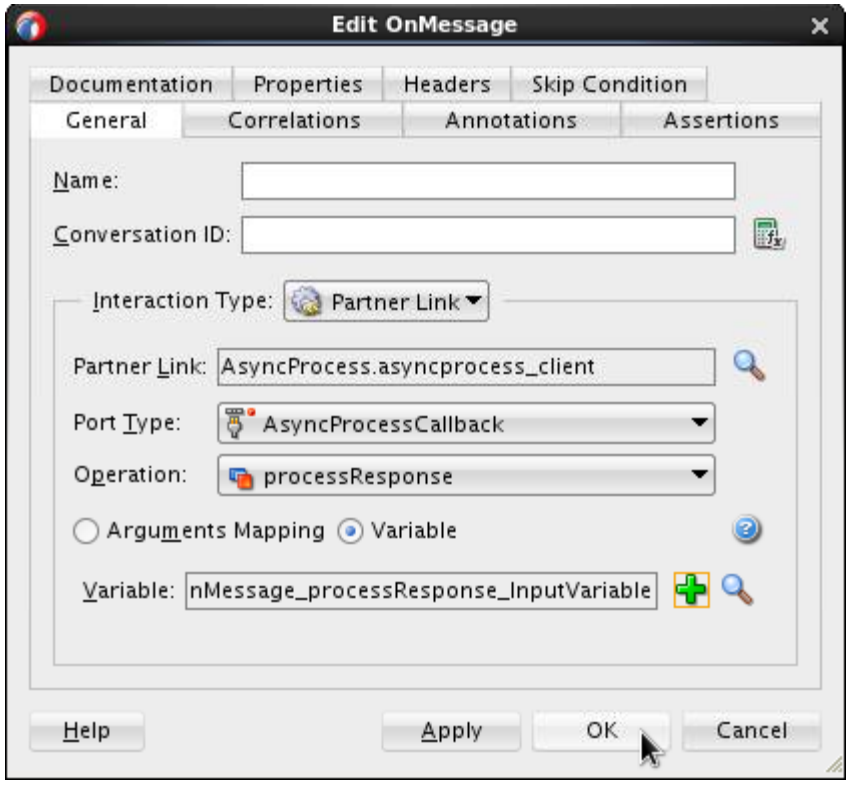
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Processing the Service Response

In this section, you create and configure a Pick activity. This activity waits for a specified amount of time to receive the callback from the async service.

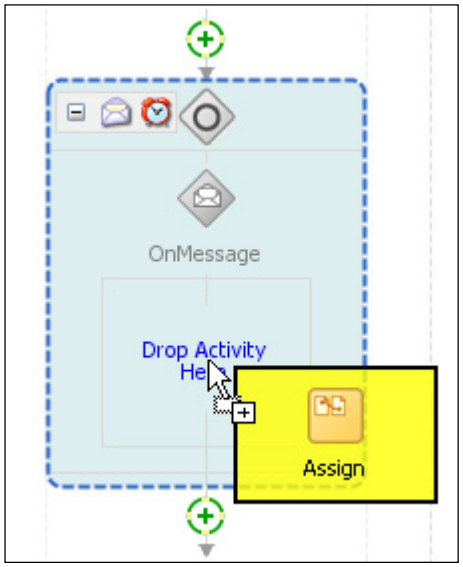
6. Create a Pick Activity.
   a. Add a Pick activity to the BPEL process after the Invoke_Async activity.
   b. Right-click the new Pick activity and select Edit.
      The Pick settings window opens.
   c. Rename the activity `Pick_Operation`.
   d. Click OK.
7. Configure the `onMessage` branch.
   a. Right-click the `onMessage` branch icon and select Edit.
   b. Browse to the partner link AsyncProcess.asyncprocess_client.
   c. Auto-create a variable.
   d. Verify your work and click OK.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

8. Create and configure `Assign_Message`.
   a. Expand the `onMessage` branch of the `Pick_Operation` activity.
   b. Add an Assign activity named `Assign_Message` to the branch.



   c. Right-click `Assign_Message` and select Edit.
   d. Map `$OnMessage_processResponse_InputVariable.payload` to `outputVariable.payload`.

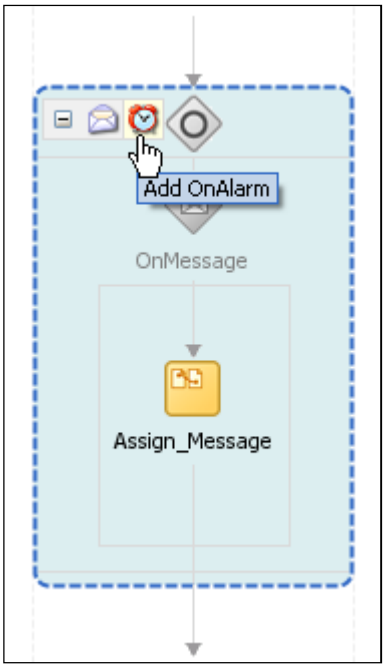
      The Edit Assign window displays the following:



   e. Verify your work and click OK.
9. Create and configure an `onAlarm` branch.
   a. Click the Pick activity.
   b. Click the Add OnAlarm button.

Oracle Internal & Oracle Academy Use Only

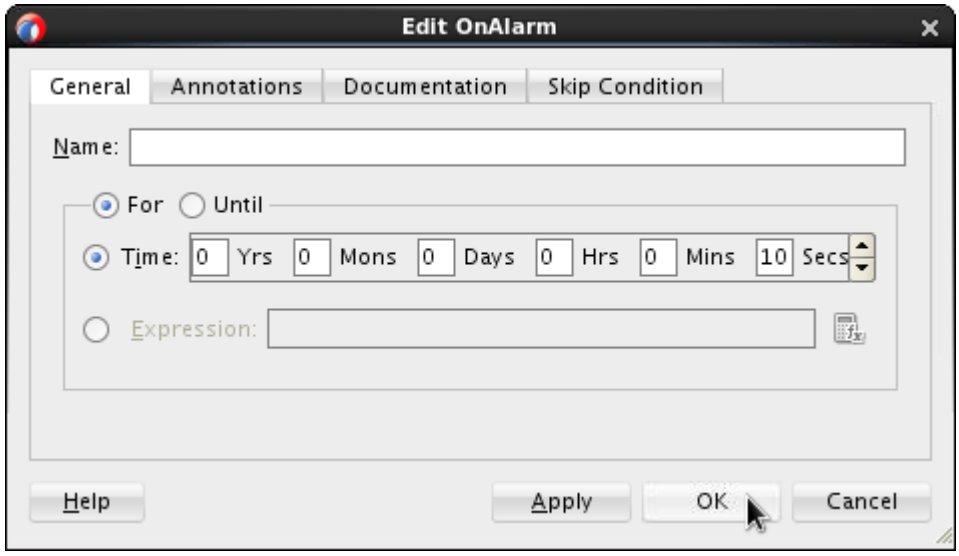
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



An on Alarm branch is added to the Pick activity.

c. Right-click the onAlarm branch and select Edit.
The onAlarm properties dialog box is displayed.

d. Configure the timeout to 10 seconds.



e. Verify your work and click OK.

10. Assign the text “timeout” to the process output variable.
Refer to step 7, if needed.

a. Add an Assign activity to the onAlarm branch of the Pick activity.

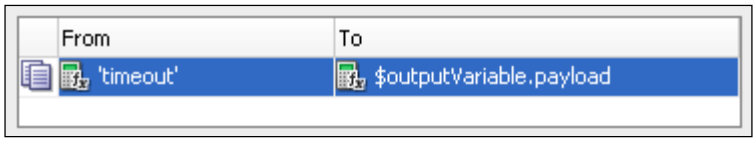
b. Name the activity `Assign_Timeout`.

c. Configure `Assign_Timeout` with a copy operation that stores the string literal `'timeout'` in the variable `$outputVariable.payload`.

d. Verify the copy operation and click OK.

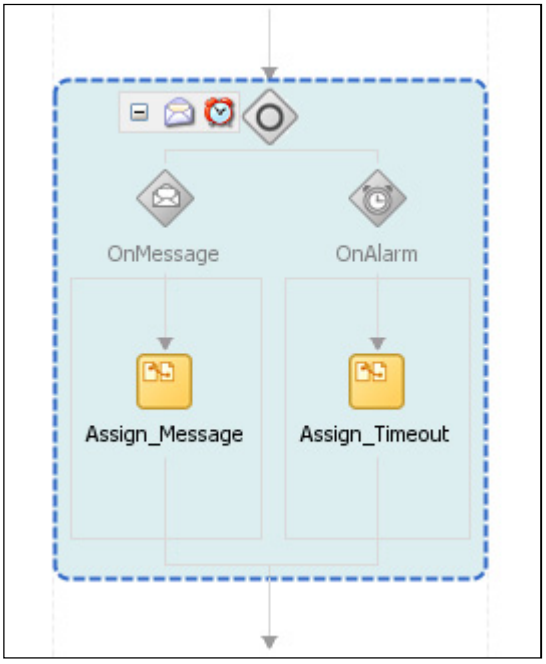
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



The Assign Editor dialog box is closed.

11. Verify the Pick activity and save your work.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 7-4 Deploying and Testing Your Work

Overview

In this practice, you deploy and test the completed `AsyncDemo` composite application.

Assumptions

This practice assumes the following:

- You have completed all previous practices successfully.
- JDeveloper is open.
- The WebLogic Application Server is running.

Tasks

Deploying the Application

1. In the JDeveloper Application Navigator window, deploy the `AsyncDemo` composite application to IntegratedWebLogicServer.
   - Remember to select the “Overwrite any existing composites with the same revision ID” check box.
   - Monitor the “Deployment – Log” window to ensure that deployment is successful.

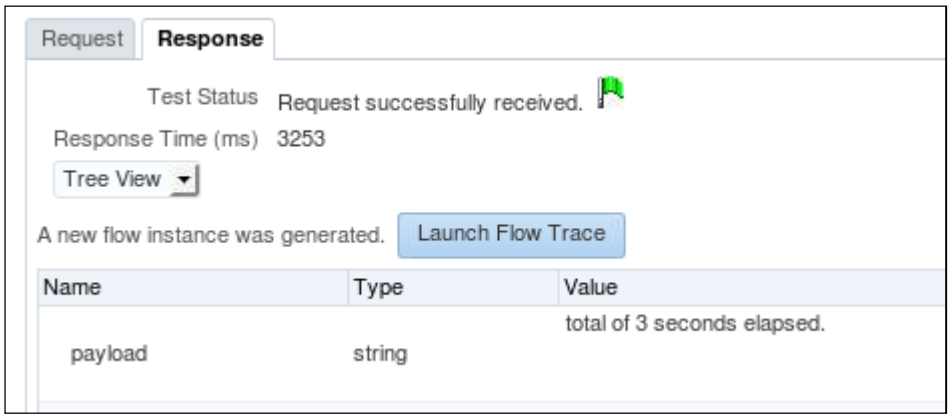
Testing the Application

2. Set up the test.
   a. Log in to Oracle Enterprise Manager (http://localhost:7101/em).
   b. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree and click the “AsyncDemo [1.0]” link.
   c. On the “AsyncDemo [1.0]” home page, click Test.
   d. On the Test Web Service page, scroll down to the Input Arguments section of the Request tab.
   e. On the Request tab, enter a value of `2` for the payload.

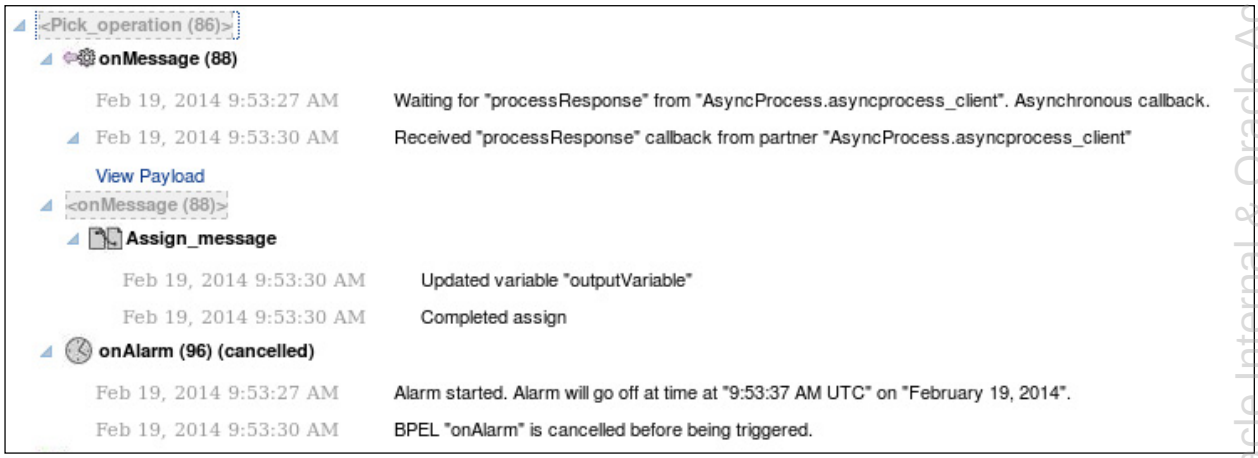
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

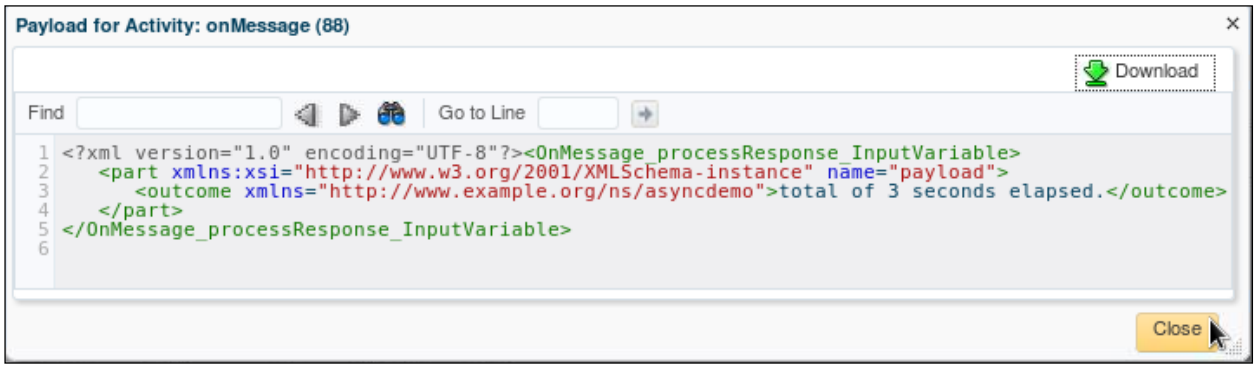
3. Execute the test.
   a. Click Test Web Service.
      After a brief delay, the message “total of 3 seconds elapsed” is displayed on the Response tab.



   b. On the Response tab, click the Launch Flow Trace button.
   c. Select the CallAsync process.
   d. Expand the Pick_Operation and each of its subnodes.
      You see the onMessage node receive the callback and the Assign_message activity update the output variable accordingly. The onAlarm node is cancelled because a message was received before the alarm was triggered.



   e. Click View Payload in onMessage.
      The callback message that you defined for AsyncProcess is displayed.

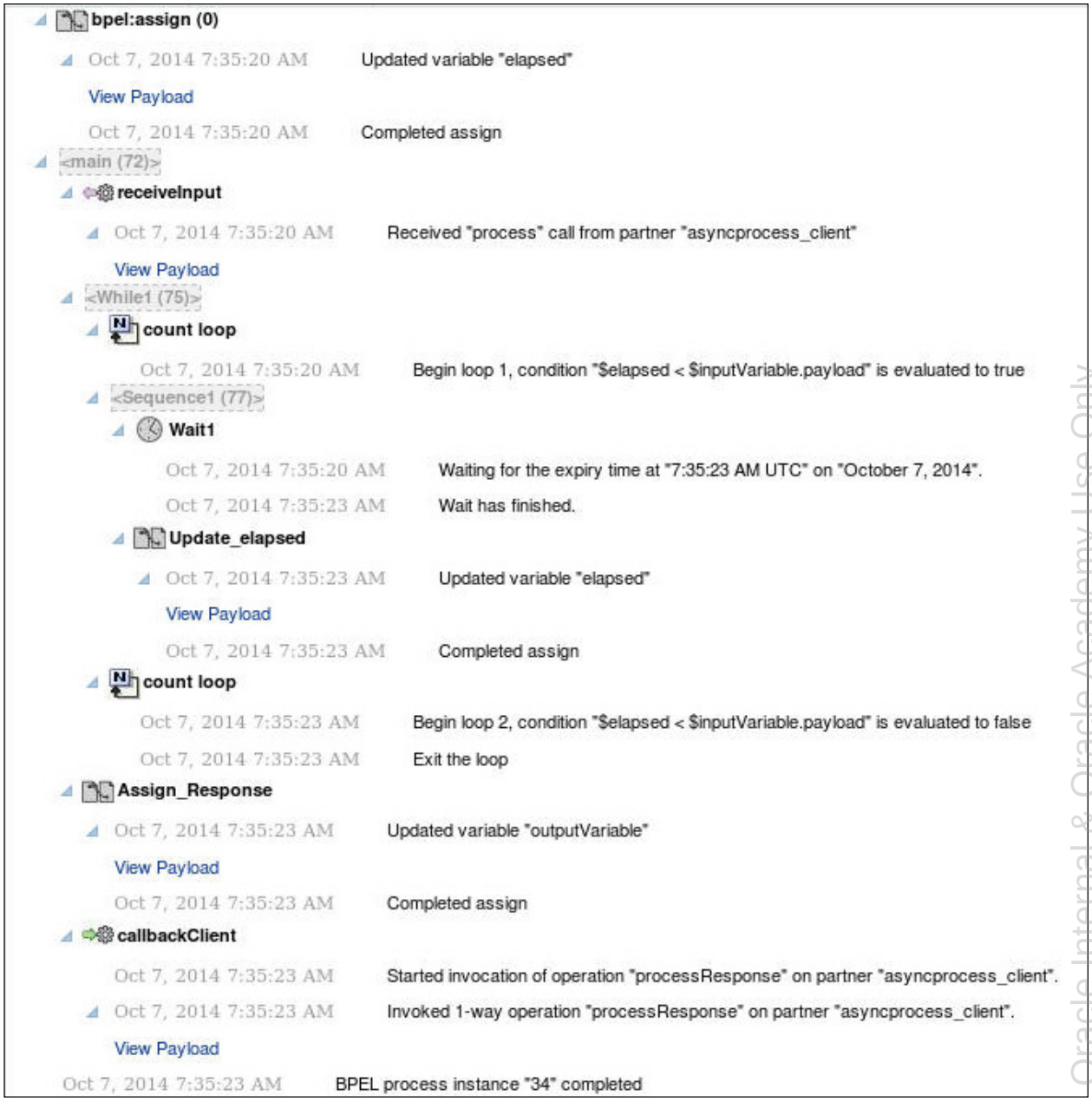


Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

f. Navigate to the Flow Trace for AsyncProcess.

g. Expand the While and Wait nodes.



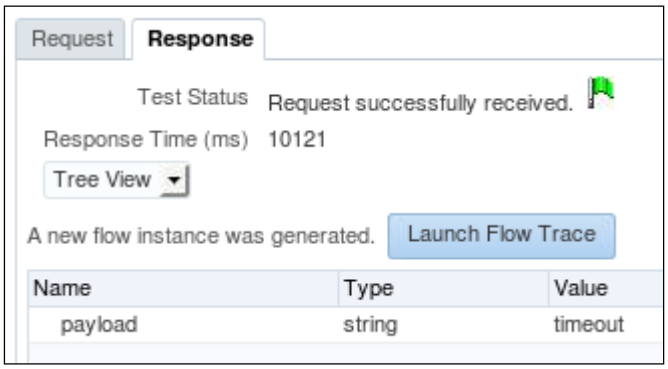
h. Close the Flow Trace window.

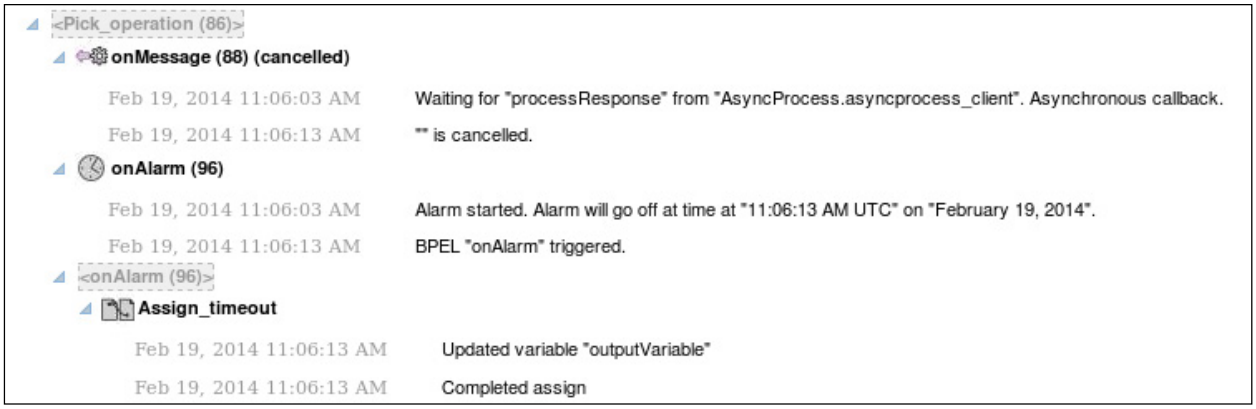
4. Refer to step 2 to run the test again. This time, supply the value `12` as the input.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

5. Refer to step 3 to explore the test results.
   The results for the CallAsync process are different than in the previous test.
   - Is this expected? Why or why not?

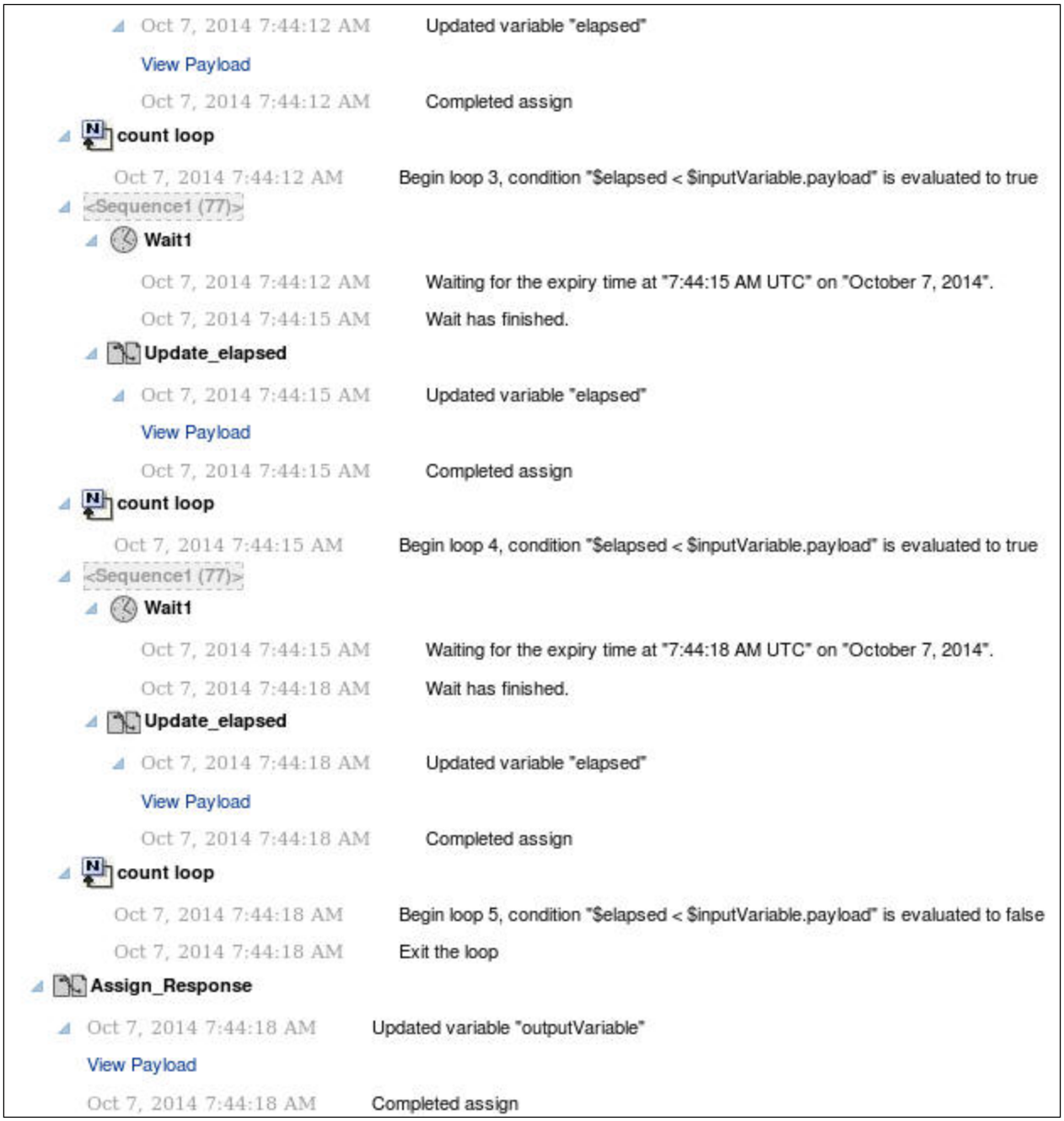




   Except for the value in the outcome, the results for AsyncProcess seem to be the same as the previous test.
   - Is this expected? Why or why not?
   - What happened to the callback message?

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.





6. Close the Flow Trace and minimize your browser window.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 8: Handling Faults in Composite Applications

Chapter 8

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 8: Overview

Practices Overview

In this practice, you first add a supplied project (`BookingSystem`) to your BPELProjects application, and then deploy the project. You then create and configure a second project (`Enroll`) that interacts with `BookingSystem`, and include a fault-handling configuration that is invoked if the interaction with `BookingSystem` results in a fault condition.

The `BookingSystem` project provides a simple mechanism to simulate a training enrollment booking system. It receives enrollment-related messages from `Enroll` project. In most cases, it responds with positive confirmations, but provided with certain data values, it returns a fault. You deliberately trigger this behavior to test the fault-handling configuration that you have built in the `Enroll` project.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 8-1: Opening and Deploying the `BookingSystem` Project

Overview

In this practice, you add the supplied `BookingSystem` project to your BPELProjects application, and then deploy it.

Assumptions

This practice makes no assumptions.

Tasks

1. Stop the Integrated WebLogic Server and close JDeveloper.

   Note: This is not strictly necessary, but "bouncing" the server and JDeveloper periodically is a good idea. This is a good time to do it.

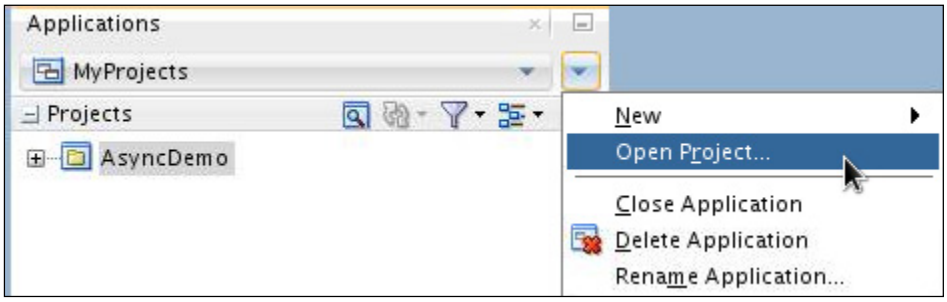
2. Open a terminal window and issue the following commands to copy three additional projects into application BPELProjects:

   ```
   cd /u01/app/fmw12c/domains/mywork/BPELProjects
   cp -R /home/oracle/labs/apps/projects/BookingSystem .
   cp -R /home/oracle/labs/apps/projects/CallGreeting .
   cp -R /home/oracle/labs/apps/projects/SayHello .
   ```

3. Reopen JDeveloper and restart the Integrated WebLogic Server.

4. Verify that the BPELProjects application is selected.

5. From the Application Navigator menu, select Open Project.



6. Navigate to `/u01/app/fmw12c/domains/mywork/BPELProjects/BookingSystem/BookingSystem.jpr`.

7. Deploy the project to IntegratedWebLogicServer.

   Note: Remember to select the "Overwrite any existing composites with the same revision ID" check box.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 8-2: Creating and Configuring the Enroll Project

Overview

In this practice, you configure the BPEL process that calls the booking system to process the enrollment and payment information. The configuration includes the creation and configuration of fault-handling routines.

Assumptions

This practice assumes the following:

- JDeveloper is open.
- Practice 8-1 has been completed successfully.

Tasks

Creating a Composite Application

In this section, you create a new composite application. You define an exposed service and an external reference, and add a BPEL service component.

1. In the BPELProjects application, create a new empty SOA project named `Enroll`.

   The `composite.xml` file is created and opened for editing.

2. Create and configure an exposed service.
   a. Add a SOAP web service to the Exposed Service swimlane.
   b. Provide the name `Enroll`.
   c. Generate the WSDL from a schema.
   d. Select the Synchronous Interface Type.
   e. Define the Input:
      1) Add a new message part.
      2) Name the message part `request`.
      3) To specify the URL, import the file `/home/oracle/labs/files/xsd/enrollment.xsd`.
      4) Copy the file to the current project.

         Note: Be sure to deselect the “Maintain original directory structure” check box before copying.

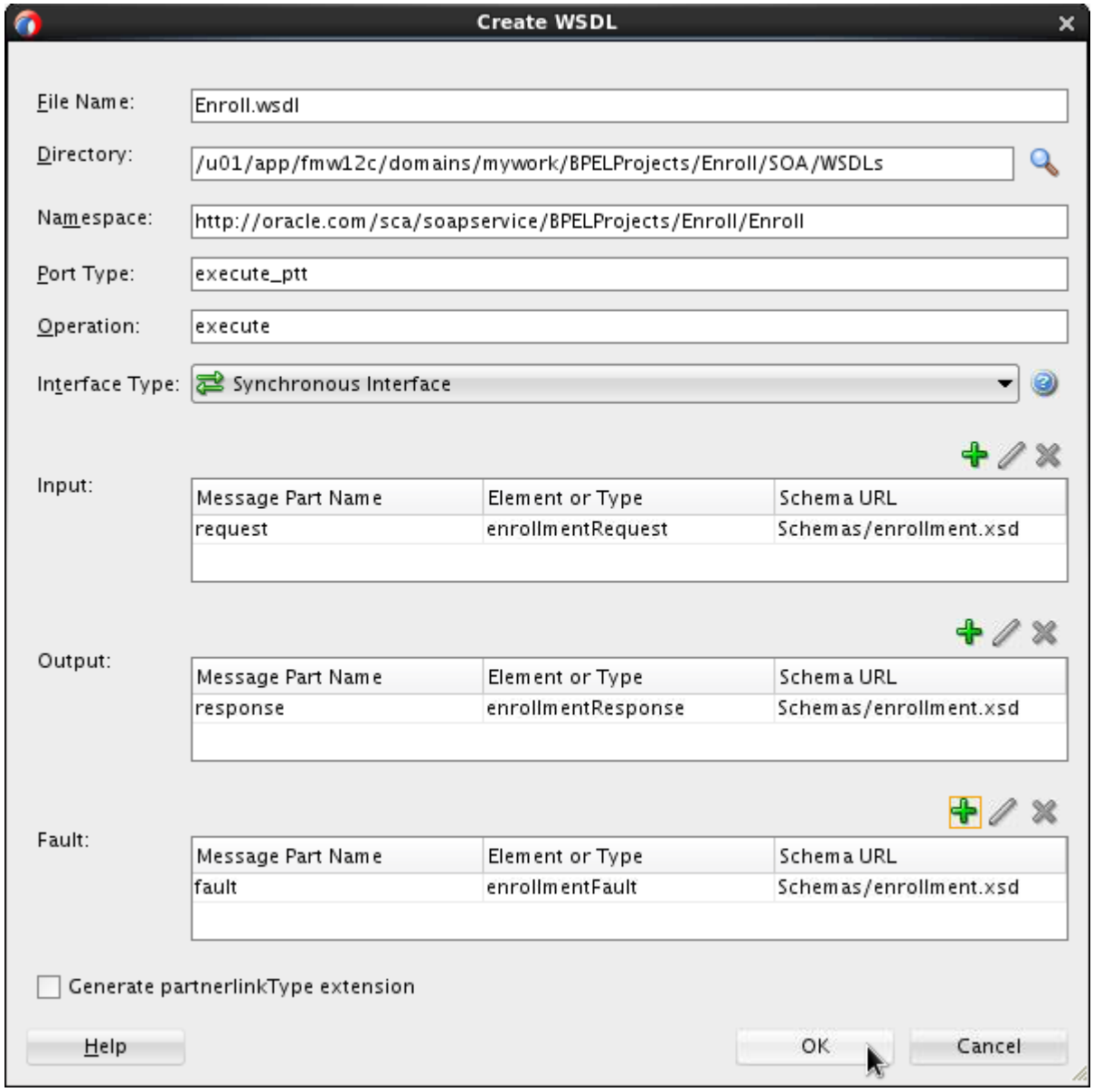
      5) Select the `enrollmentRequest` element.
   f. Define the Output:
      1) Add a new message part.
      2) Name the message part `response`.
      3) To specify the URL, browse to `enrollment.xsd`. Select the element `enrollmentResponse`.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

g. Repeat the previous step to define the fault message part by using the element `enrollmentFault` from the `enrollment.xsd` file.

h. Verify and save your work.



3. Create and configure an external reference.

a. Add a SOAP web service to the External Reference swimlane.

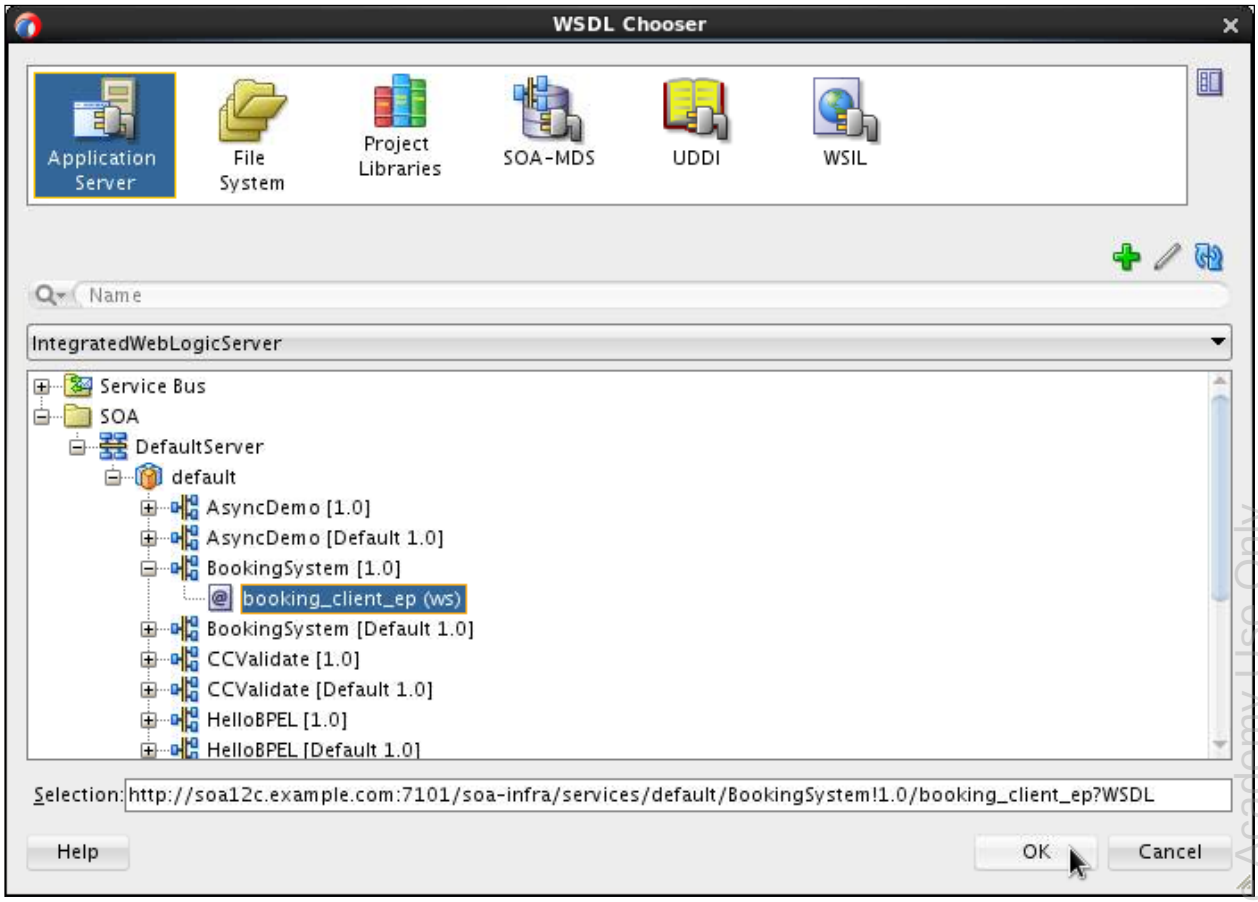
b. Name the reference `BookingService`.

c. To specify the WSDL URL, click the Find Existing WSDLs icon.

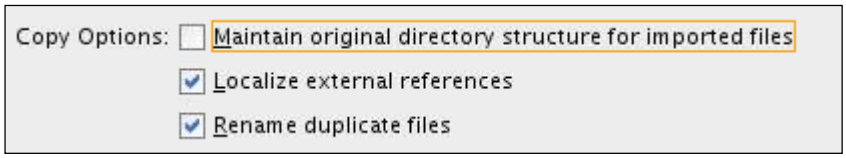
d. In the WSDL Chooser, select Application Server. Navigate to the deployed application `BookingSystem`. Select `booking_client_ep` and click OK.

Oracle Internal & Oracle Academy Use Only

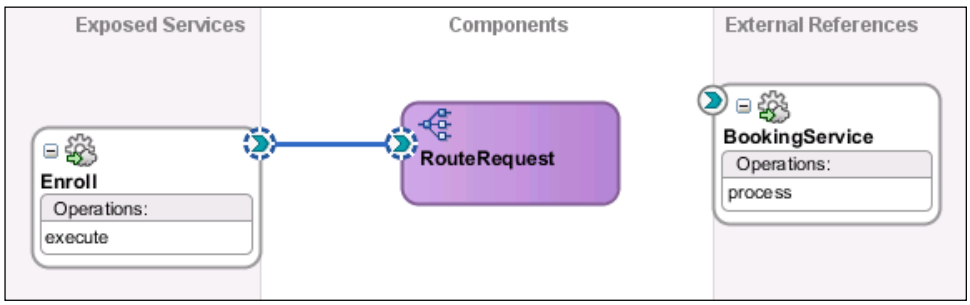
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



e. In the Localize Files dialog box, deselect the "Maintain original directory structure for imported files" check box, and then click OK.



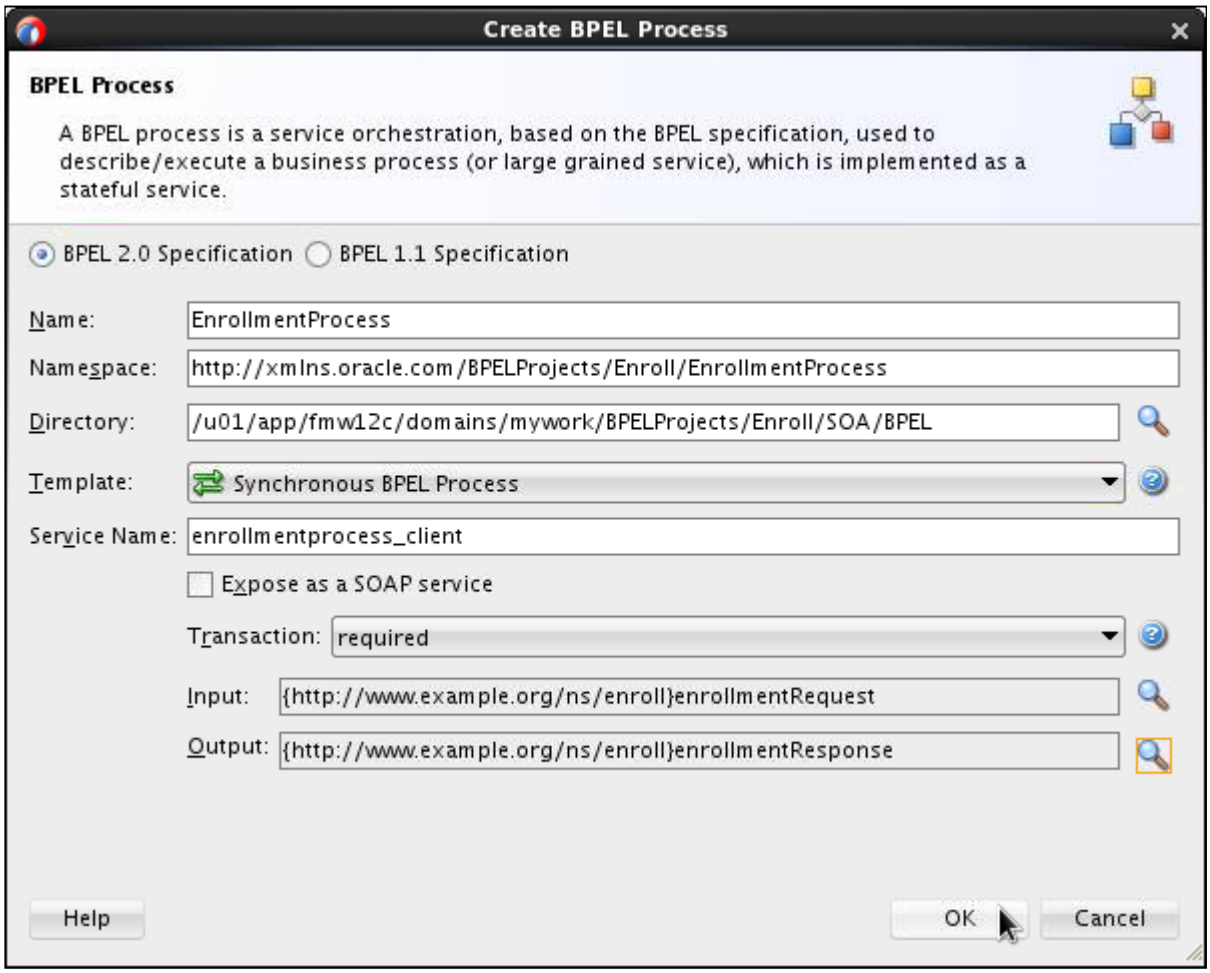
4. Create a Mediator component.
   a. Add a Mediator to the Components swimlane.
   b. Name the mediator `RouteRequest`.
   c. Click OK to accept the Define Interface Later option.
   d. Add a wire from the Exposed Service Enroll to the Mediator RouteRequest.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

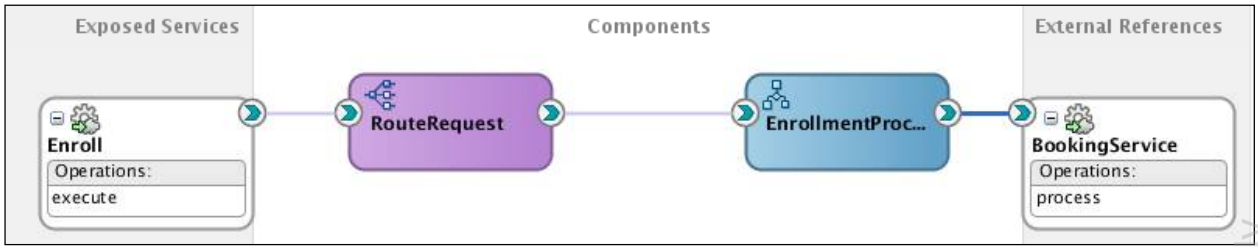
5. Create and configure a BPEL service component.
   a. Add a BPEL process to the Components swimlane.
   b. Name the process `EnrollmentProcess`.
   c. Select the Synchronous BPEL template.
   d. Deselect the “Expose as a SOAP service” check box.
   e. To define the input, use the Browse button to navigate to the `enrollmentRequest` element of `enrollment.xsd`.
   f. To define the output, browse to the `enrollmentResponse` element of `enrollment.xsd`.
   g. Verify and save your work.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

6. Configure additional wires in `composite.xml`.
   a. Create a wire from the Mediator RouteRequest to the BPEL process `EnrollmentProcess`.
   b. Create a wire from the BPEL process `EnrollmentProcess` to the external reference `BookingService`.
   c. Verify and save your work.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 8-3: Configuring the EnrollmentProcess Process

Overview

In this practice, you create and configure the activities that comprise the BPEL Process.

Assumptions

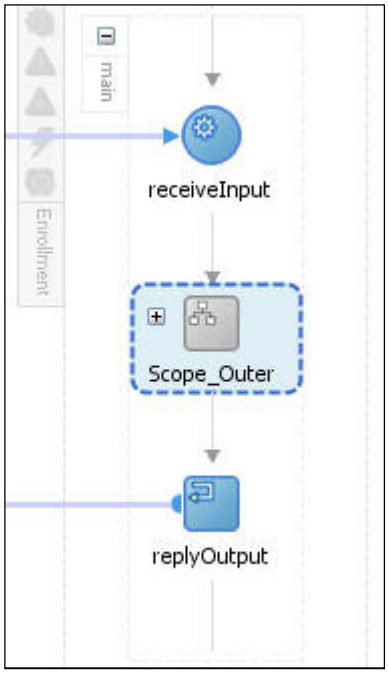
This practice assumes that you have completed Practice 8-2 successfully.

Tasks

1. Right-click the BPEL process EnrollmentProcess and select Edit.

   The model editor opens.

2. Create and configure a Scope activity.
   a. Add a Scope activity between the receiveInput and replyOutput activities.
   b. Name the Scope `Scope_Outer`.



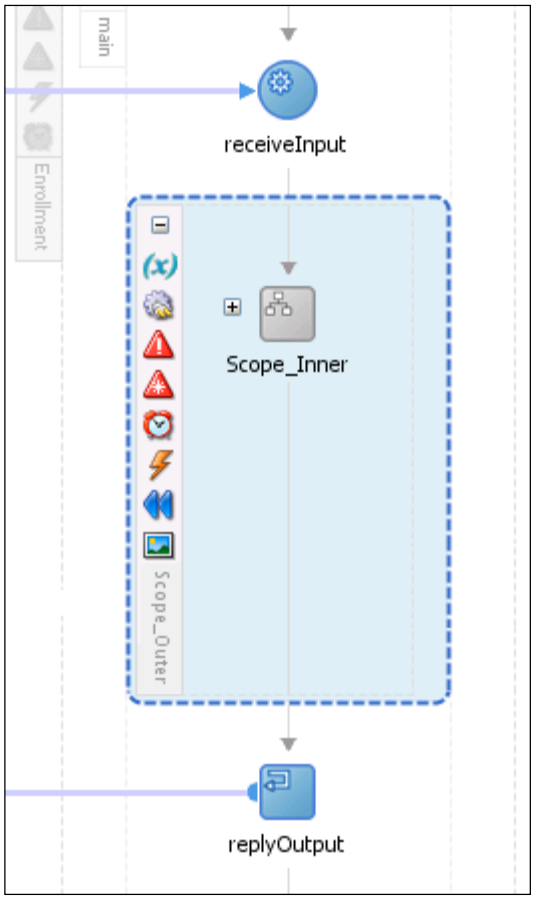
Defining Enrollment Invocation

In this section, you define the activities that are directly related to the invocation of the enrollment request.

3. Create and configure an inner Scope.
   a. Expand `Scope_Outer`.
   b. Add a Scope within `Scope_Outer`.
   c. Name the new Scope `Scope_Inner`.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



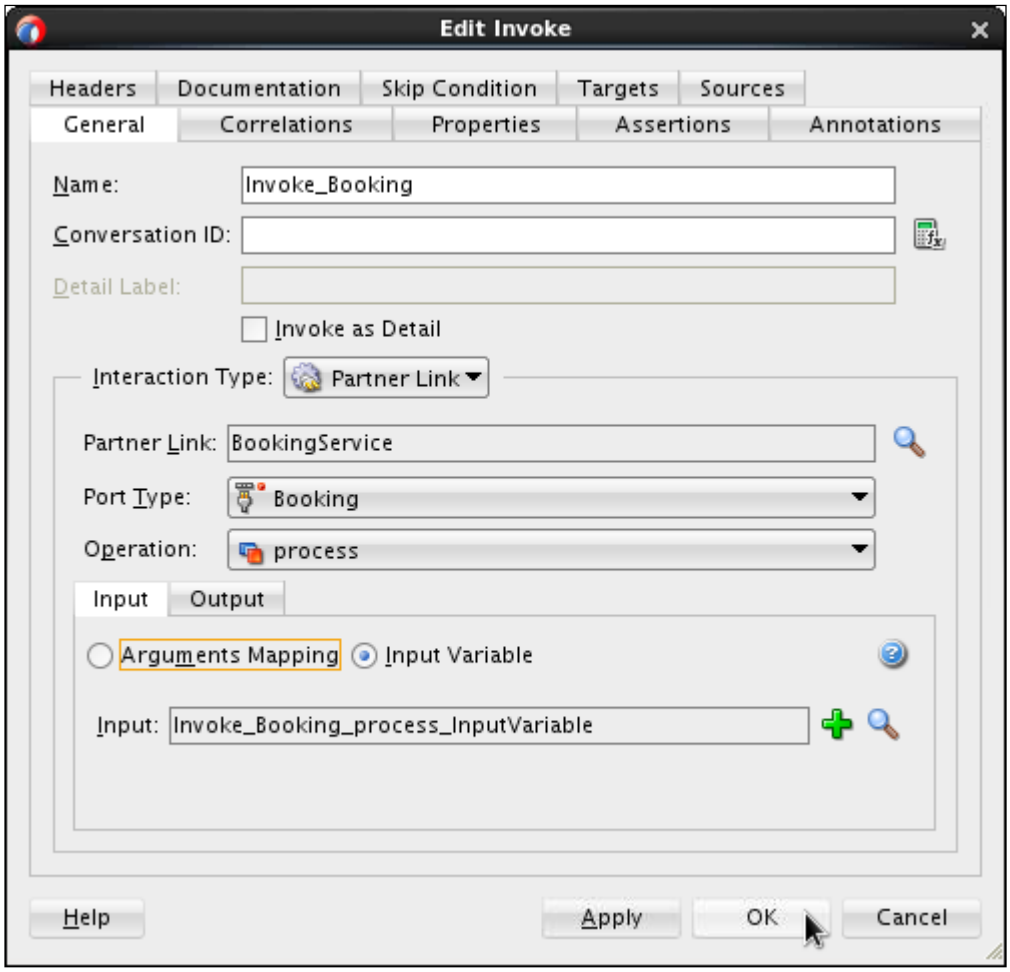
4. Create and configure an Invoke activity.
   a. Expand `Scope_Inner`.
   b. Add an Invoke activity to `Scope_Inner`.
   c. Right-click the Invoke activity and select Edit.
   d. Name the Invoke `Invoke_Booking`.
   e. To specify the partner link, use the Browse icon to navigate to BookingService.
   f. Create a new Input variable.
   g. Create a new Output variable.

      Note: Within a scope, you have the option of creating either global or local variables. You opt to use global variables in this practice.

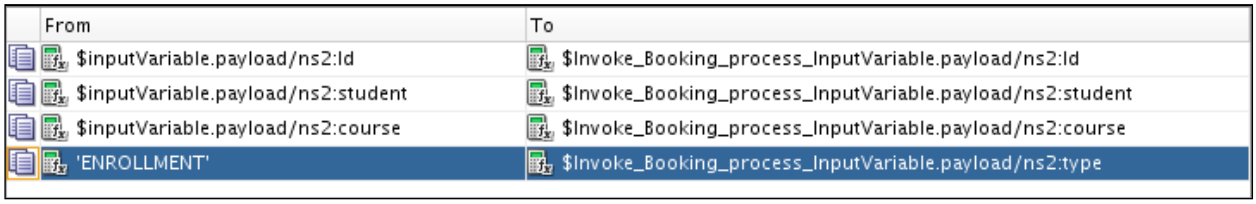
   h. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



5. Create and configure an Assign activity.
   a. Add an Assign activity just above `Invoke_Booking` (inside `Scope_Inner`).
   b. Name the activity `setBooking`.
   c. Right-click the new activity and select Edit.
   d. Map the `inputVariable` elements `Id`, `student`, and `course` to their corresponding elements in the `Invoke_Booking_process_InputVariable`.
   e. Use Expression Builder to map the text `'ENROLLMENT'` to the `type` node of `Invoke_Booking_process_InputVariable`.
   f. Verify and save your work.



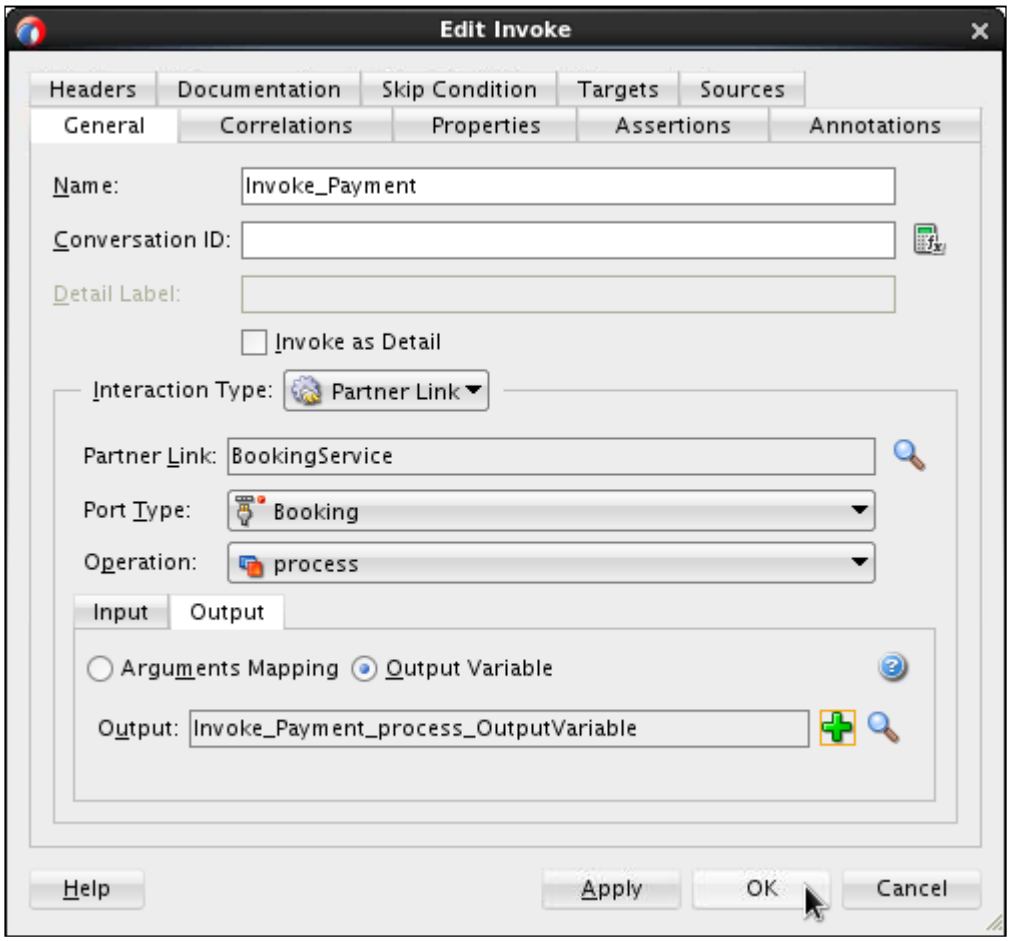
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Defining Payment Invocation

In this section, you create and configure the activities that are directly related to the invocation of the payment processing.

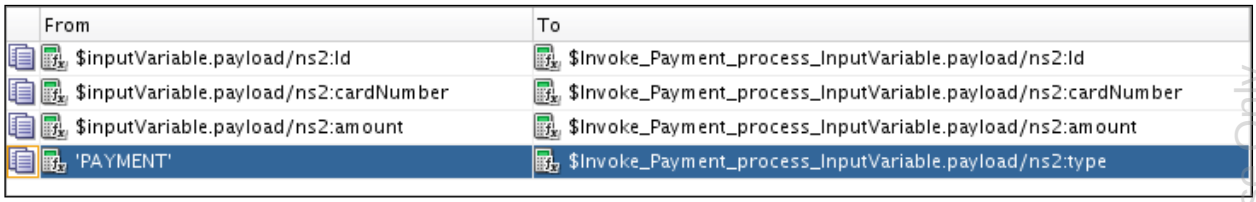
6. Create and configure an Invoke activity.
   a. Collapse `Scope_Inner`.
   b. Add an Invoke activity to `Scope_Outer` immediately following `Scope_Inner`.
   c. Right-click the new Invoke activity and select Edit.
   d. Name the activity `Invoke_Payment`.
   e. To specify the partner link, use the Browse icon to navigate to BookingService.
   f. Create a new Input variable.
   g. Create a new Output variable.
   h. Verify and save your work.



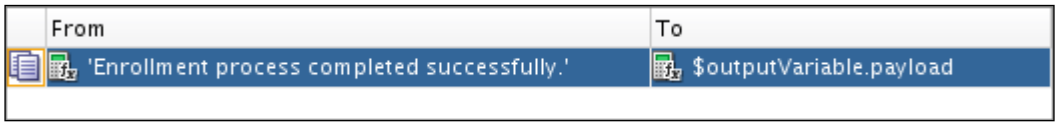
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

7. Create and configure an Assign activity.
   a. Add an Assign activity just above `Invoke_Payment`.
   b. Name the activity `setPayment`.
   c. Right-click the new activity and select Edit.
   d. Map the `inputVariable` elements `Id`, `cardNumber`, and `amount` to their corresponding elements in `Invoke_Payment_process_InputVariable`.
   e. Use Expression Builder to map the text `'PAYMENT'` to the `Invoke_Payment_process_InputVariable type` field.
   f. Verify, and then save your work.



8. Create and configure an Assign activity.
   a. Add an Assign activity just below `Invoke_Payment`.
   b. Name the activity `setSuccess`.
   c. Right-click the new activity and select Edit.
   d. Use Expression Builder to map the text `'Enrollment process completed successfully'` to the `enrollmentResponse` node of `outputVariable`.
   e. Verify, and then save your work.



Adding Compensation Handling to `Scope_Inner`

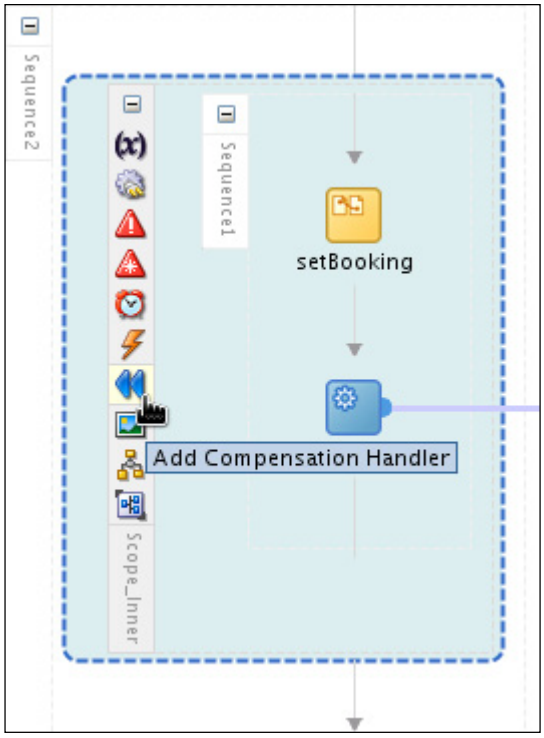
At this point, you have created and configured the "happy path," where the enrollment and payment information are processed successfully. Recall, however, that the practice scenario describes the possibility of a fault being raised in the payment processing, and the need for handling that fault. Specifically, by the time payment processing fails, space will already have been reserved in the class for the enrollment. In this section, you configure the compensation handler to send an offsetting enrollment cancellation that will free up the reserved seat.

9. Create and configure a compensation handler on `Scope_Inner`.
   a. Expand `Scope_Inner`.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

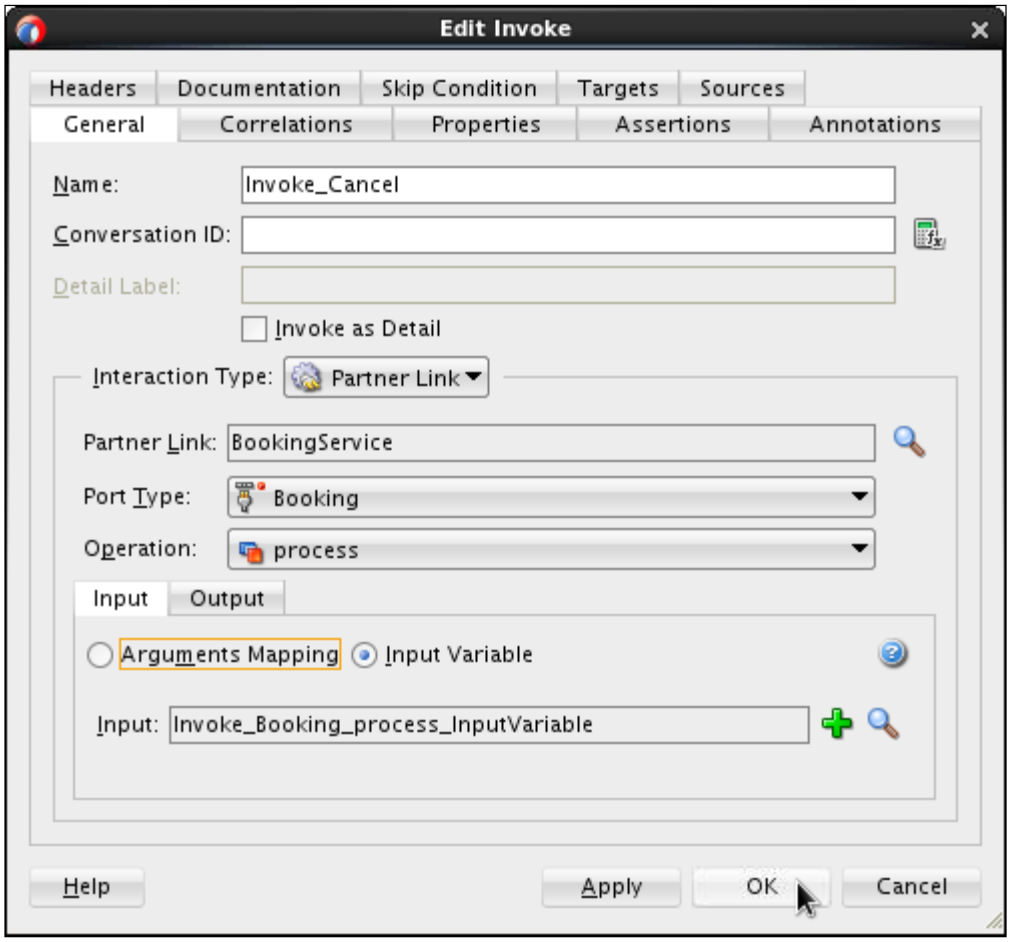
b. On the left edge of the Scope, click the Add Compensation Handler icon as shown in the following diagram:



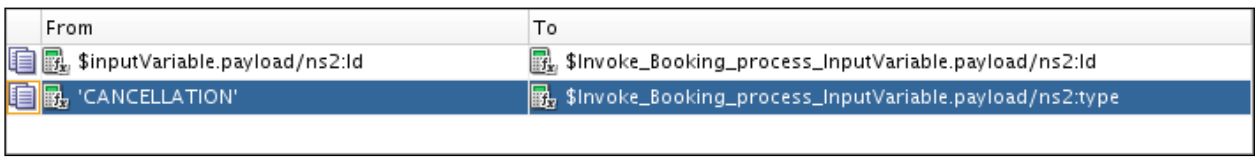
10. Create and configure an Invoke activity.
    a. Add an Invoke activity to the compensation handler.
    b. Right-click the new Invoke and select Edit.
    c. Name the activity `Invoke_Cancel`.
    d. To specify the partner link, use the Browse icon to navigate to BookingService.
    e. Use the Browse icon to *reuse* `Invoke_Booking_process_InputVariable`.
    f. Use the Browse icon to *reuse* the `Invoke_Booking_process_OutputVariable`.
    g. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



11. Create and configure an Assign activity.
    a. Add an Assign activity just above `Invoke_Cancel`.
    b. Name the activity `setCancel`.
    c. Right-click the new activity and select Edit.
    d. Map the `inputVariable` element ID to the corresponding element in `Invoke_Booking_process_InputVariable`.
    e. Use Expression Builder to map the text `'CANCELLATION'` to the `Invoke_Booking_process_InputVariable type` field.
    f. Verify and save your work.



Oracle Internal & Oracle Academy Use Only

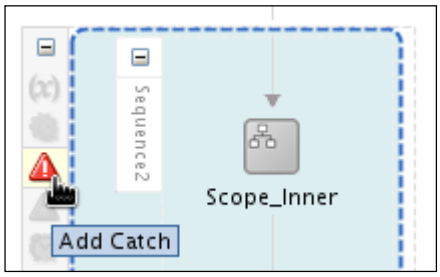
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Adding Error Handling to `Scope_Outer`

12. Create and configure a Catch branch.

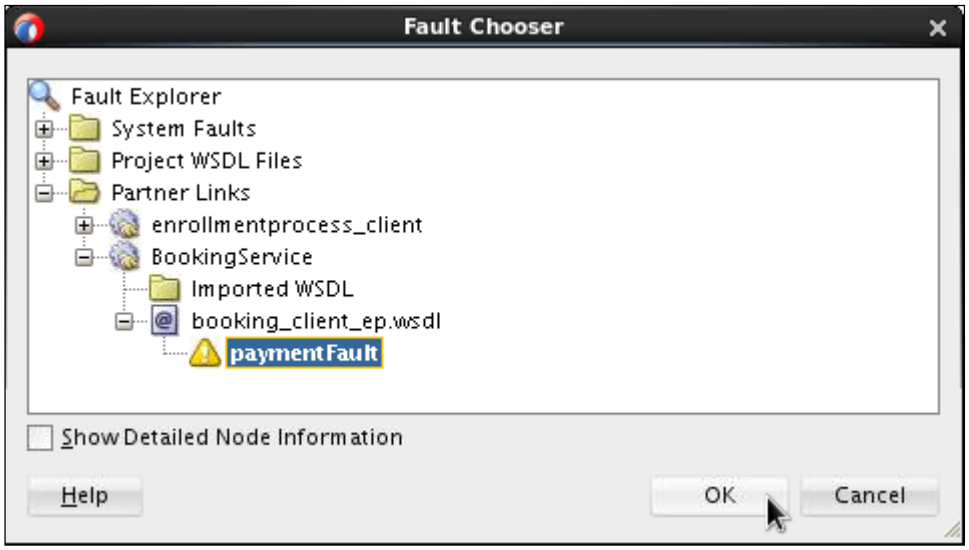
    a. Collapse `Scope_Inner`.

    b. On the left edge of `Scope_Outer`, right-click the Add Catch icon and select Edit.



    The Edit Catch window is displayed.

    c. Use the Browse icon to navigate to Partner Links > BookingService > booking_client_ep.wsdl > `paymentFault`.



    This specifies which fault to catch. The fault is thrown by the BookingService application, and is described in its `.wsdl` document.

    d. Specify `faultVariable` in the Fault Variable field.

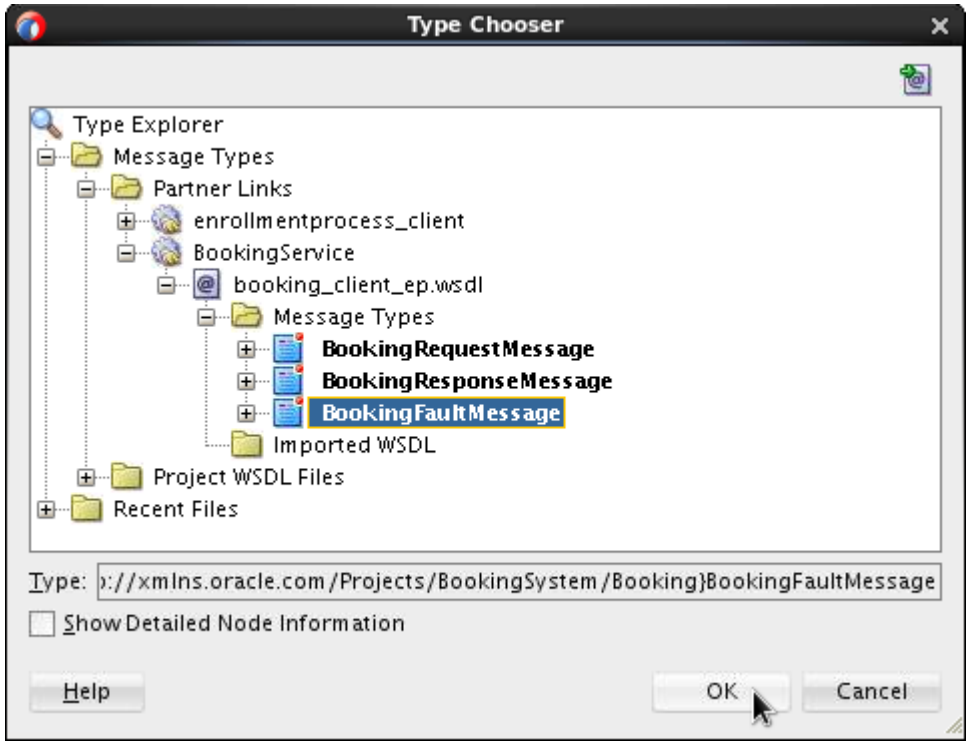
    This is the variable that holds the fault message.

    e. Select the Message Type radio button.

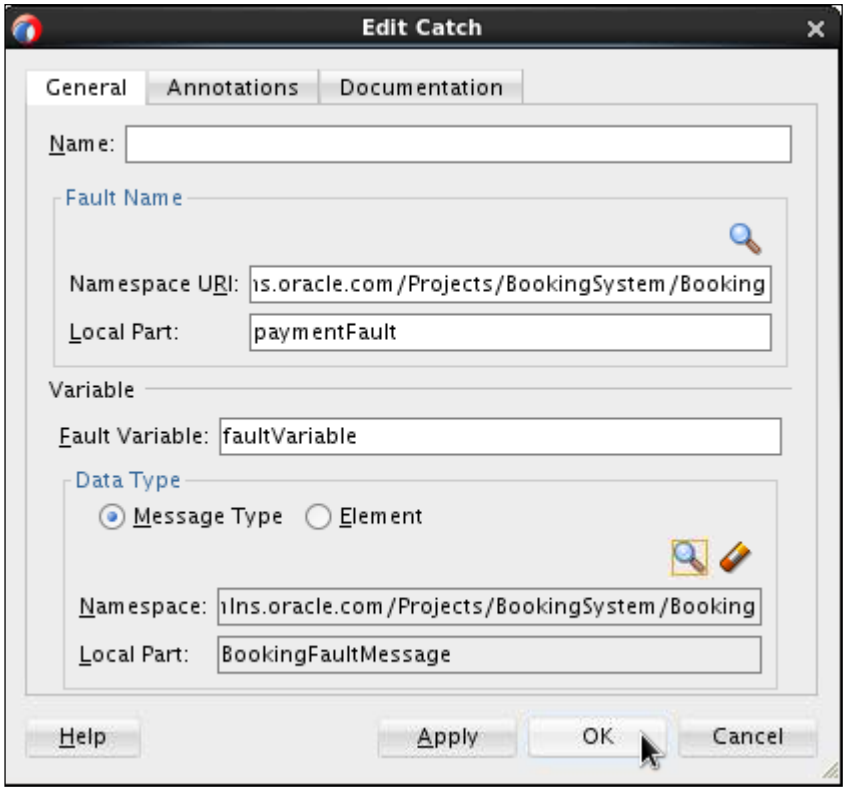
    f. Use the Browse icon to navigate to the partner link `BookingService` file `booking_client_ep.wsdl` and specify the Message Type `BookingFaultMessage`.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



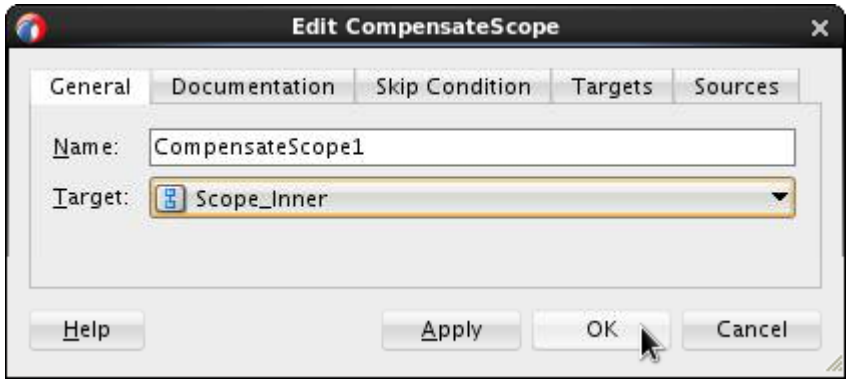
g. Verify and save your work.



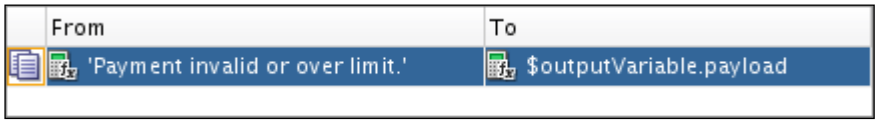
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

13. Create and configure a CompensateScope activity.
    a. Add a CompensateScope activity within Catch.
    b. Right-click the activity and select Edit.
    c. From the Target drop-down list, select `Scope_Inner`.
    d. Verify and save your work.



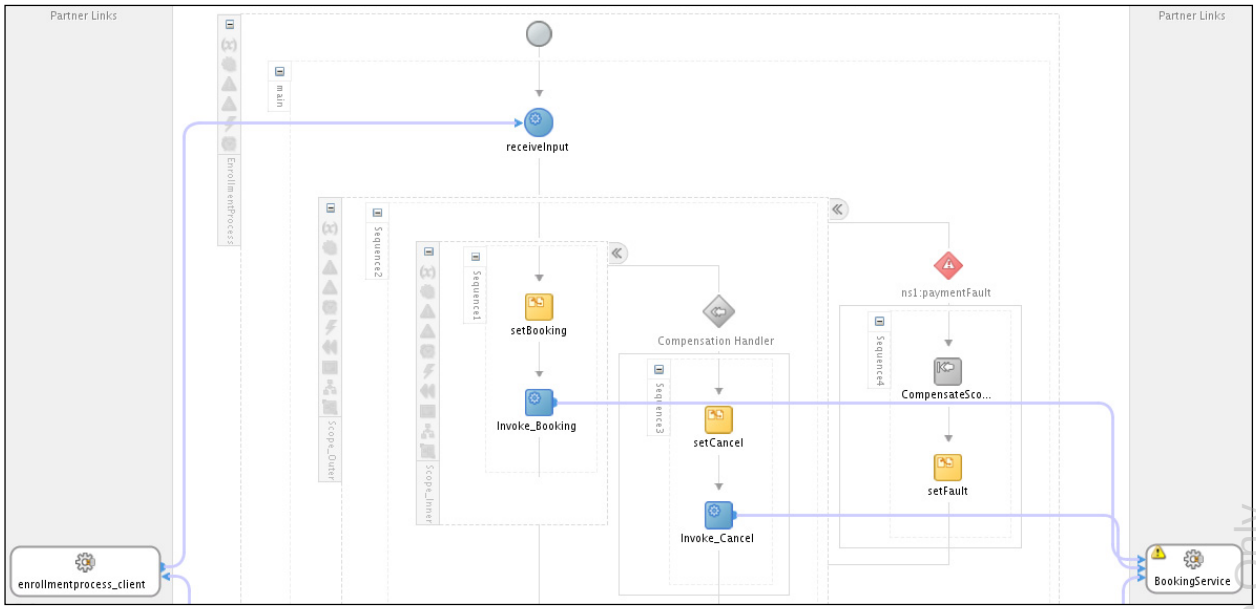
14. Create and configure an Assign activity.
    a. Add an Assign activity just below CompensateScope.
    b. Name the activity `setFault`.
    c. Right-click the new activity and select Edit.
    d. Use Expression Builder to map the text `'Payment invalid or over limit.'` to the `outputVariable enrollmentResponse` field.
    e. Verify and save your work.



15. Compare your BPEL process to the following partial image. (The fault-handling components are shown here. The “successful” path is not shown in its entirety.)

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Completing the Configuration of Mediator

Configuration of the BPEL process is complete. The only step that remains is to configure the routing rules in the Mediator.

16. Create and configure the transformation for the enrollment request.
    a. In the composite overview window, right-click the Mediator `RouteRequest` and select Edit.
    b. To the right of the first Transform Using field, click the Mapper file icon.
    c. Create a new XSLT mapper file.
    d. In the mapper, map `enrollmentRequest` to `enrollmentRequest`.

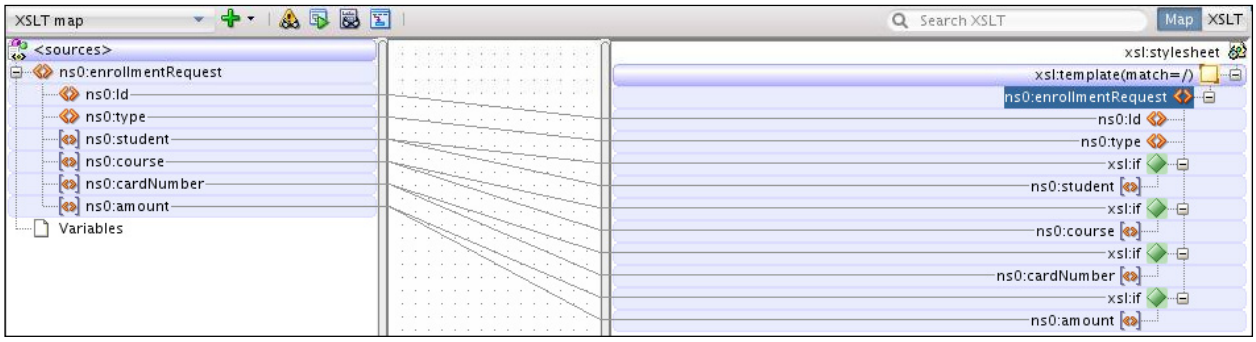
       Note: This mapping passes the data unchanged from the Mediator to the BPEL Process. This is possible because both the components specify the same input message in their `wsdl` files.

       The AutoMapper dialog box opens.
    e. Click OK to accept all defaults.

       Each of the leaf nodes is mapped from source to target.
    f. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



g. Close the editor.

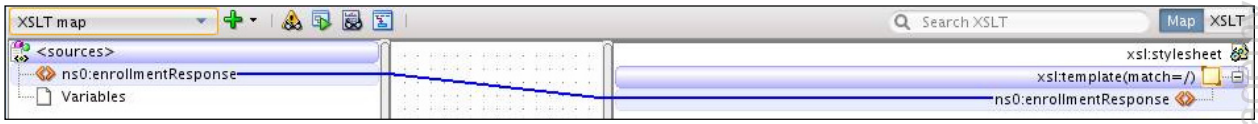
17. Create and configure the transformation for the enrollment reply.

a. In the Synchronous Reply section, to the right of the Transform Using field, click the Mapper file icon.

b. Create a new mapper file.

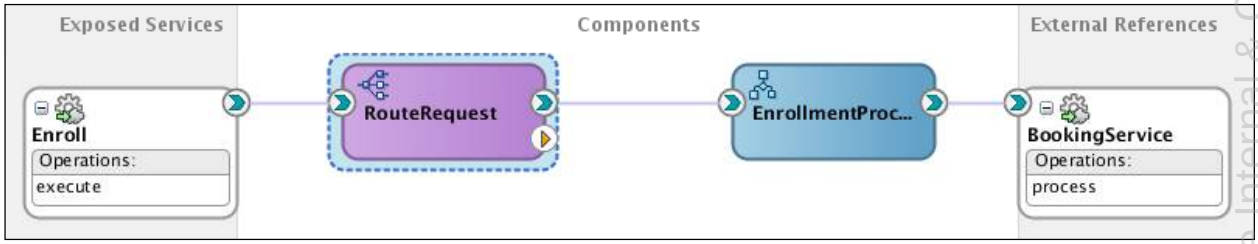
c. In the mapper, map `enrollmentResponse` to `enrollmentResponse`.

d. Verify and save your work.



18. Close the `.mplan` and `.bpel` file editors.

19. In the `compsite.xml` editor, verify your composite application.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 8-4: Deploying and Testing the Application

Overview

In this practice, you deploy the `Enroll` application and test the fault-handling configuration.

Assumptions

This practice assumes that you have completed practices 8-1 through 8-3 successfully.

Tasks

Deploying the `Enroll` Composite Application

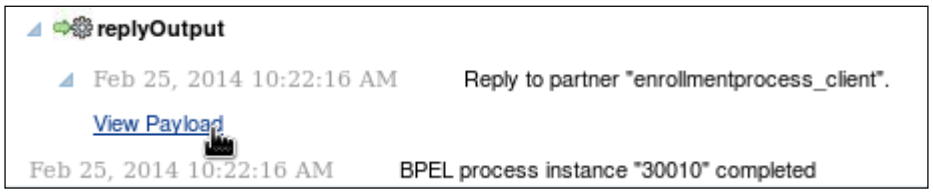
1. Prepare for the test by deploying the `Enroll` application to IntegratedWebLogicServer.

   Note: Remember to select the “Overwrite any existing composites with the same revision ID” check box.

Testing the Composite Application

2. Open Oracle Enterprise Manager (http://localhost:7101/em).
   a. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the Enroll [1.0] link.
   b. On the Enroll [1.0] home page, click Test.
   c. On the Test Web Service page, navigate to the Request tab. Use the Browse button to replace the default message with the contents of the file `/home/oracle/labs/files/xml_in/enrollment_input.xml`.
   d. Click Test Web Service.

      The Response tab is displayed.
   e. Click Launch Flow Trace.
   f. Click Enrollment Process.
   g. Under the replyOutput node, click View Payload.



   h. Verify that the status value returned is “Enrollment process completed successfully.”

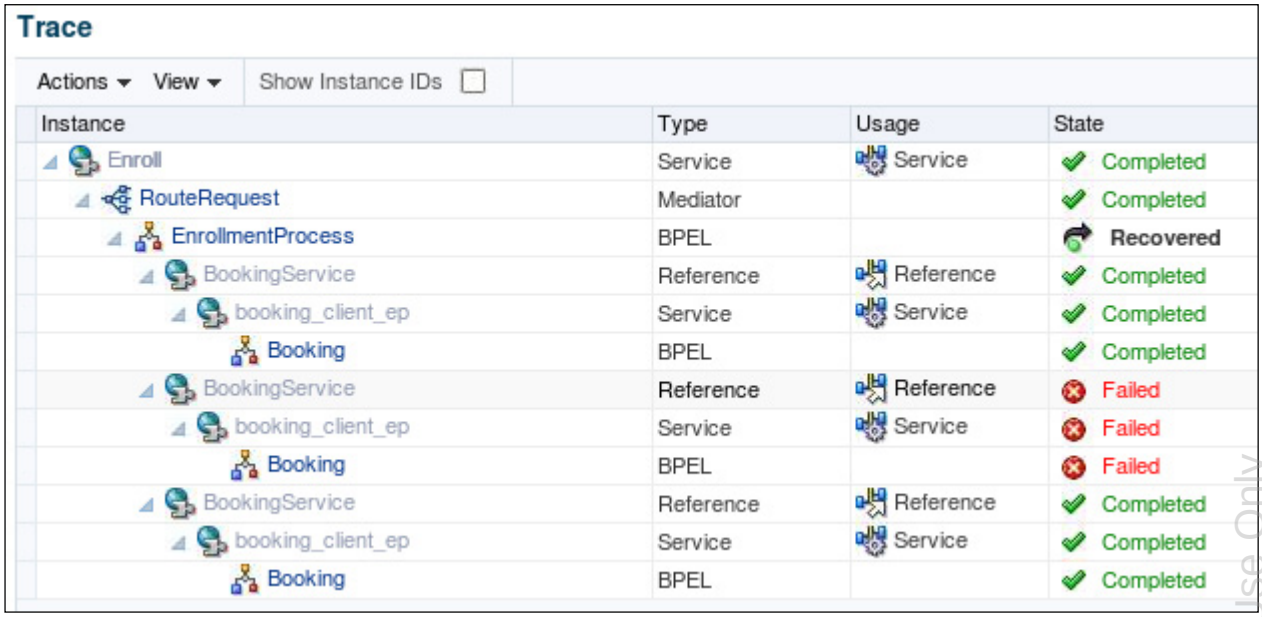
```
<?xml version="1.0" encoding="UTF-8"?><outputVariable>
   <part xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" name="payload">
      <enrollmentResponse xmlns="http://www.example.org/ns/enroll">Enrollment process completed successfully.</enrollmentResponse>
   </part>
</outputVariable>
```

   i. Close the Flow Trace windows.
3. To perform a second test (and generate a fault), click the Request tab.
   a. On the Request tab page, modify the amount to be any number greater than 5000.
   b. Click Test Web Service.

Oracle Internal & Oracle Academy Use Only

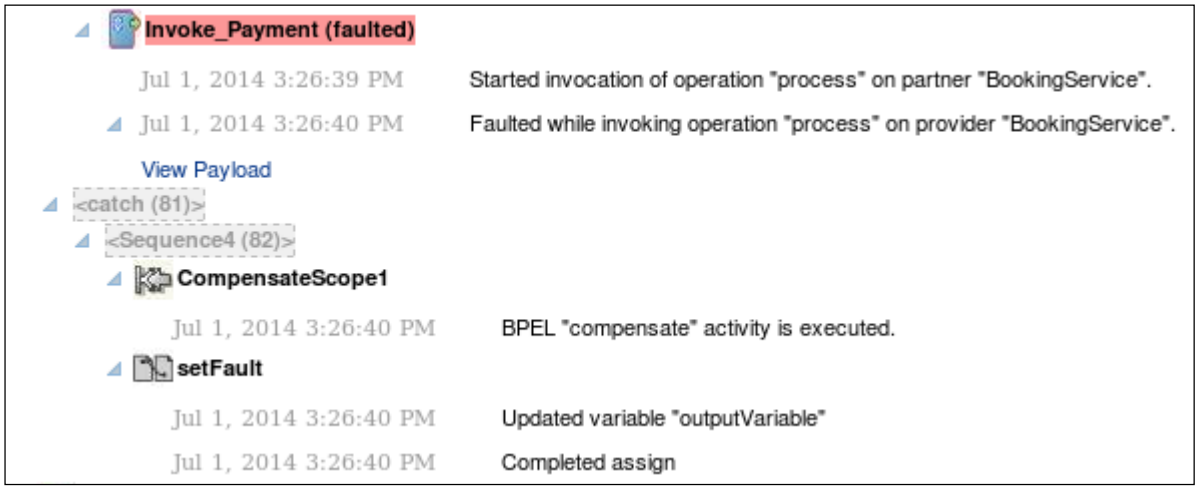
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

c. On the Response tab page, click Launch Flow Trace.



d. Click EnrollmentProcess.

The Flow Trace shows that the Invoke faulted and that execution passed to CompensateScope, and within CompensateScope, to setFault.



e. View the payload for Invoke_Payment. The response message matches what is configured in setFault.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

```
<?xml version="1.0" encoding="UTF-8"?><messages>
   <input>
<Invoke_Payment_process_InputVariable>
         <part xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" name="payload">
            <enrollmentRequest xmlns="http://www.example.org/ns/enroll">
               <Id>10</Id>
               <type>PAYMENT</type>
               <cardNumber>9090-8080-7070-6060</cardNumber>
               <amount>5100</amount>
            </enrollmentRequest>
         </part>
      </Invoke_Payment_process_InputVariable>
   </input>
   <fault>
<bpelFault>
         <faultType>1</faultType>
         <paymentFault xmlns="http://xmlns.oracle.com/Projects/BookingSystem/Booking">
            <part name="payload">
               <enrollmentFault xmlns="http://www.example.org/ns/enroll">Payment invalid or excessive.</enrollmentFault>
            </part>
         </paymentFault>
      </bpelFault>
   </fault>
   <faultType>
<message>1</message>
   </faultType>
</messages>

```

f. Close the Flow Trace windows and minimize your browser.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 8-5: Using the Fault Policy Wizard

Overview

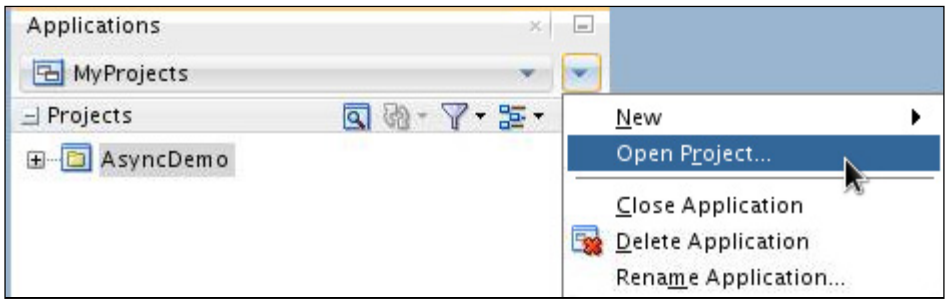
In this practice, you design a fault policy with the Fault Policy wizard and associate the fault policy with the fault policy binding file.

Assumptions

This practice makes no assumptions.

Tasks

1. Open and deploy SayHello and CallGreeting.
   a. Verify that the BPELProjects application is selected.
   b. From the Application Navigator menu, select Open Project.



   c. Navigate to `/u01/app/fmw12c/domains/mywork/BPELProjects/ SayHello/SayHello.jpr`.
   d. Repeat steps b and c to open the CallGreeting project.
   e. Deploy SayHello to IntegratedWebLogicServer.

      Remember to select the “Overwrite any existing composites with the same revision ID” check box.
   f. Repeat step e to deploy the CallGreeting project.
2. Test the applications.

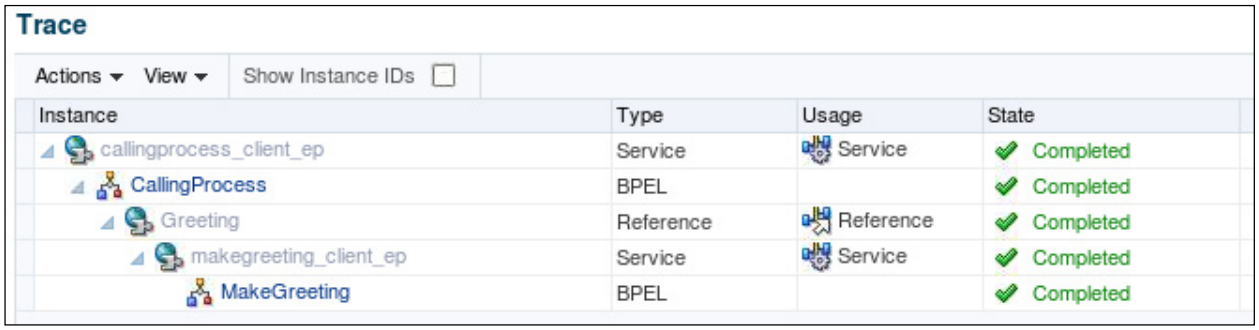
   Note: The SayHello project is the classic “Hello World” project. It concatenates the string “Hello” and an input string that is received as part of the project invocation. The CallGreeting [asynchronous] project accepts an input string that you provide, and then passes that string when it invokes SayHello. In the following test, you verify that the applications run without error and do not [yet] throw any faults.
   a. Open the Oracle Enterprise Manager page (http://localhost:7101/em).
   b. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “CallGreeting [1.0]” link.
   c. On the “CallGreeting [1.0]” home page, click Test.
   d. On the Test Web Service page, scroll down to the Input Arguments section to the Request tab.
   e. Provide your first name (or any other text you like) as input.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

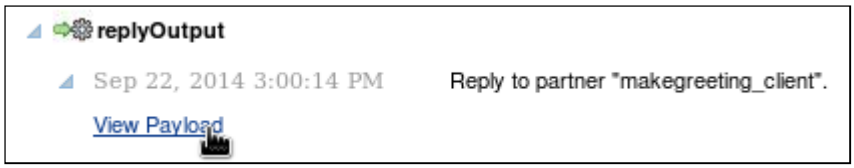
f. Click Test Web Service.

g. On the Response tab, click the Launch Flow Trace link. Verify that each step has completed without error.

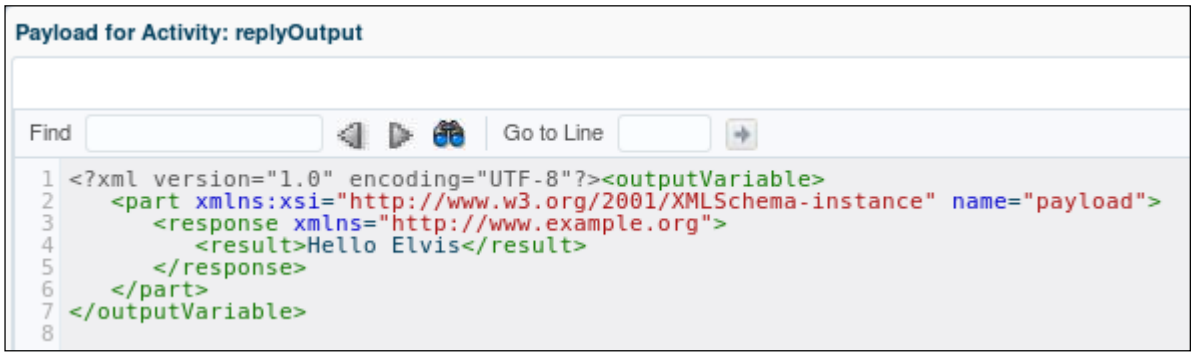


h. In the Trace, click MakeGreeting.

i. In the Audit Trail, click ViewPayload.



j. Verify that the payload includes the concatenated text.



Generating a Remote Fault

3. Shut down the SayHello project.

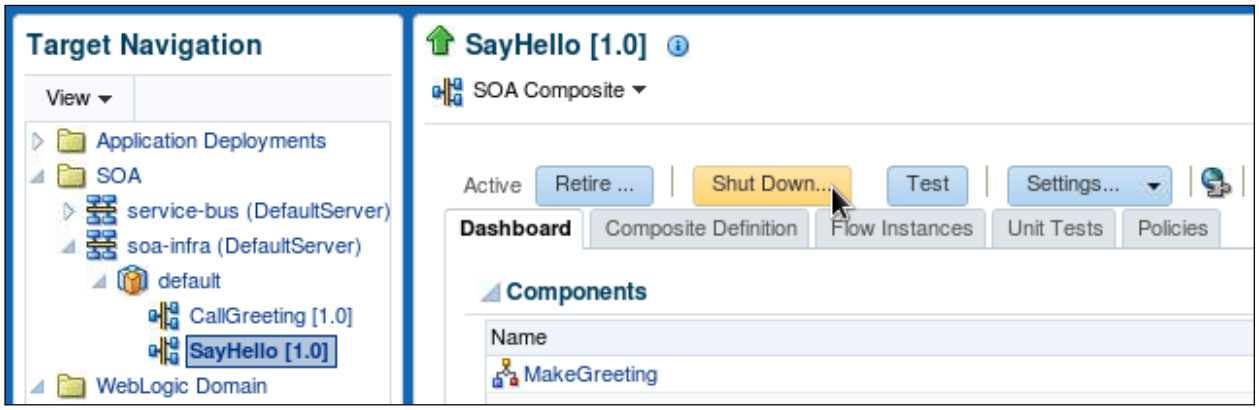
a. In the Enterprise Manager target navigation pane, expand soa-infra > default.

b. Select the SayHello project.

c. In the SayHello pane, click Shut Down.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



d. In the Confirmation dialog box, click Yes.

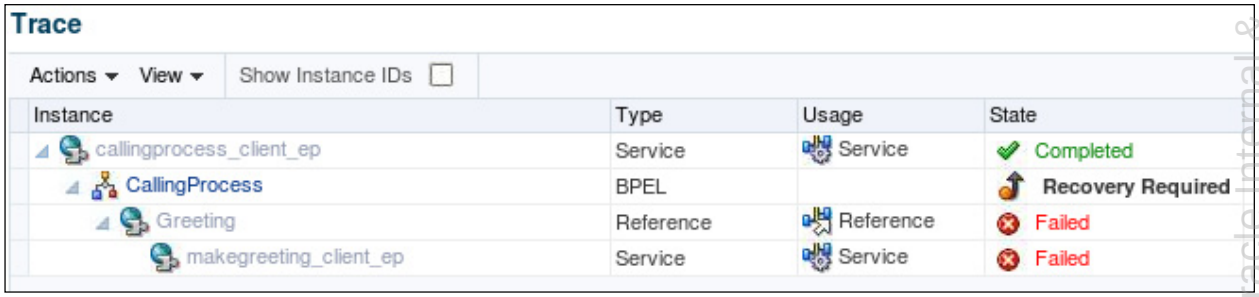


The project is shut down.

4. Generate a fault condition.

a. Repeat the preceding task 2, steps b–f to test the CallGreeting composite.

b. On the Response tab, click the Launch Flow Trace link. Note the state of each step of the instance.



c. Above the Trace pane, on the Faults tab, notice the fault information.



Creating the Fault Policy

In this section, you use the wizard to create a Fault Policy.

5. In JDeveloper, open the composite overview for the CallGreeting project.

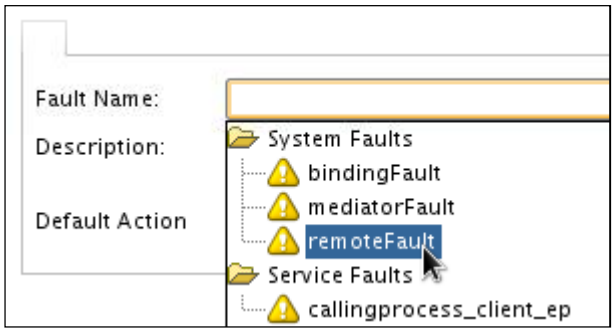
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

6. Create a fault policy document.
   a. To open the wizard, select File > New> From Gallery.
      The New Gallery dialog box opens.
   b. In the Categories pane, select SOA Tier > Faults.
   c. In the Items pane, select Fault Policy Document and click OK.



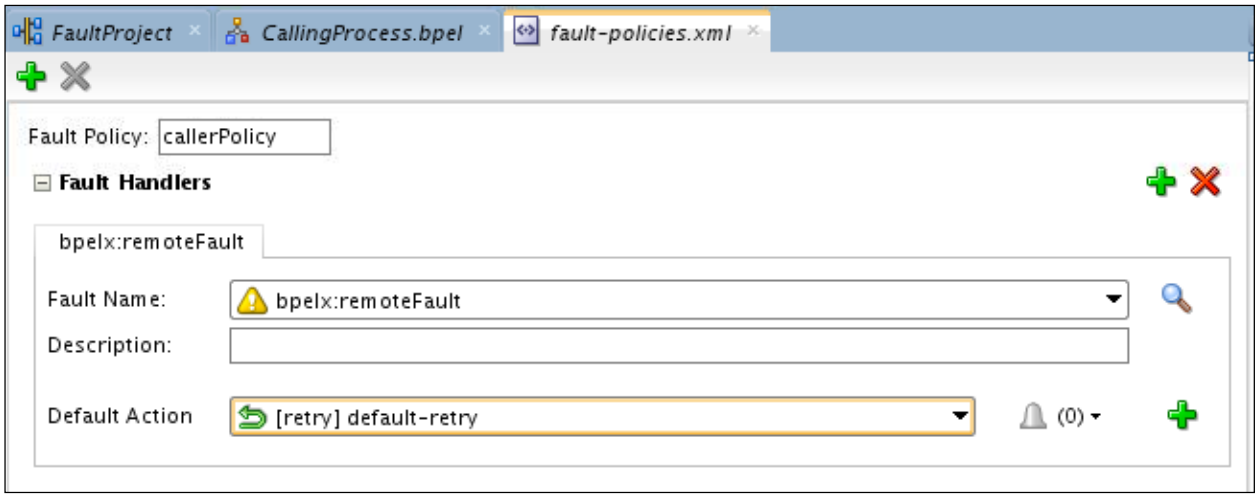
   The Fault Policies editor opens.
7. Define a fault policy.
   a. Name the policy `callerPolicy`.
   b. Select remoteFault from the Fault Name drop-down list.



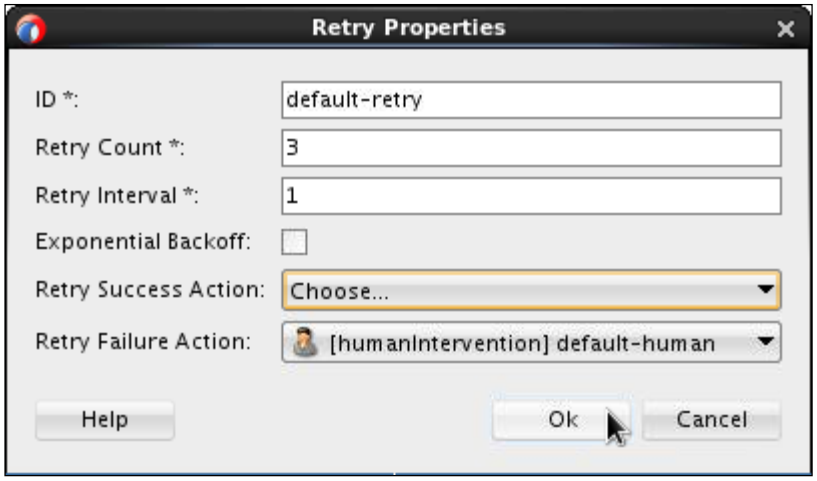
   c. Select [retry] default-retry from the Default Action drop-down list.
      This defines what actions the policy will take when it is activated.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



8. Edit the properties of the retry action.
   a. Click the **Actions** tab and highlight **default-retry**.
   b. Click the Pencil icon to edit the action.
      1) Retry Count: 3
      2) Retry Interval: 1
      3) Retry Failure Action: [humanIntervention] default-human
   c. Verify your work and click OK.



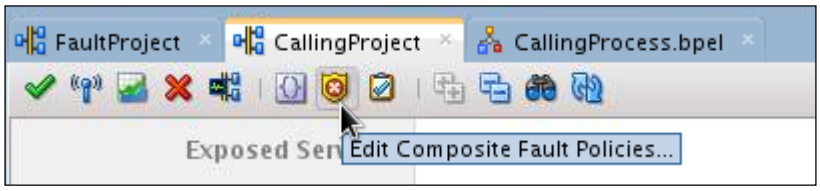
   The policy is now created.
   d. Save your work and close the Policy wizard.

   Notice that the `fault-policies.xml` file is created in the project under SOA.

Oracle Internal & Oracle Academy Use Only

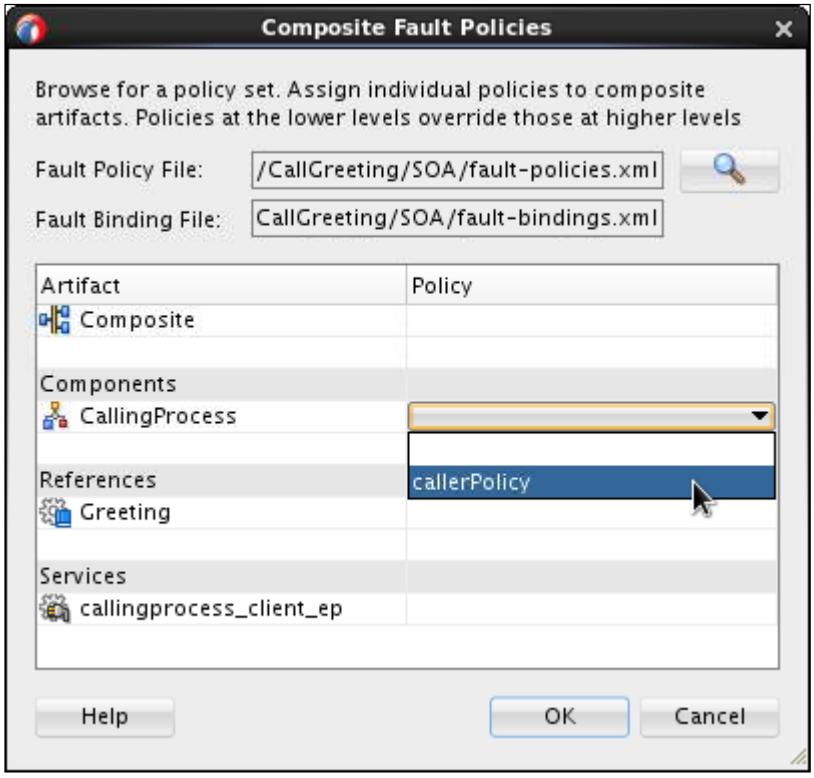
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

9. Associate the fault policy with the BPEL process CallingProcess.
   a. On the Composite Editor toolbar, click the Edit Composite Fault Policies icon.



   The Composite Fault Policies screen is displayed. Here you can associate fault policies with the composite or the components therein. In this practice, you associate the policy with CallingProcess.

   b. Select the policy from the drop-down list to the right of the CallingProcess component.



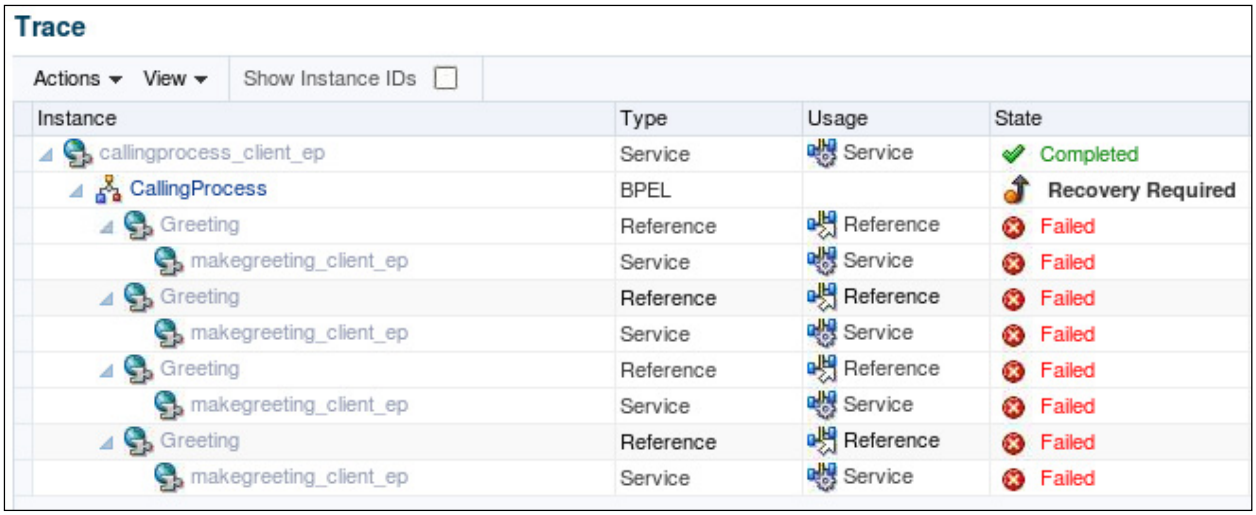
   c. Click OK.

10. Redeploy and test the application.
    a. Redeploy the CallGreeting project to activate the fault policy.
    b. Repeat the preceding step 2 to test the CallGreeting composite.
    c. On the Response tab, click the Launch Flow Trace link.

       This time, the trace indicates that the instance was retried three times after the initial failure.

Oracle Internal & Oracle Academy Use Only

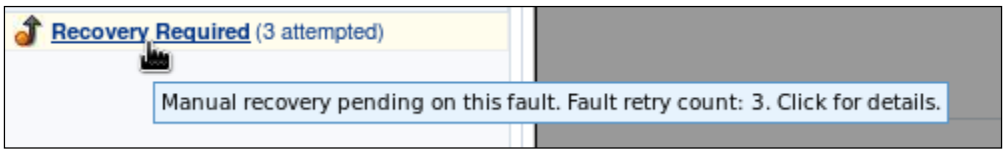
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



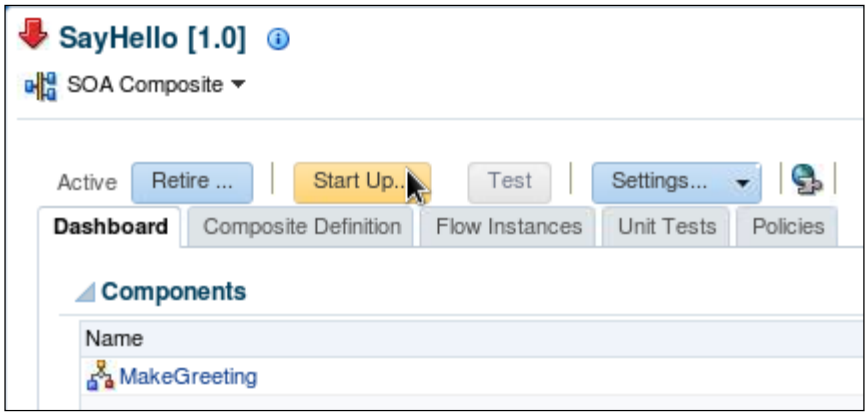
Manually Recovering the Fault

11. Hover your cursor over the text “Recovery Required” on the Faults tab.

    A message is displayed confirming that manual recovery is pending.



12. Activate the SayHello project.
    a. In the Enterprise Manager target navigation pane, expand soa-infra > default.
    b. Select SayHello project.
    c. In the SayHello pane, click Start Up.

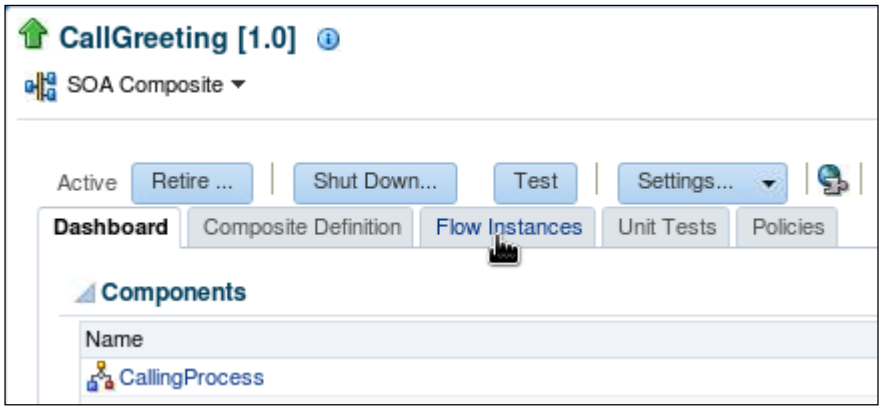


    d. Click Yes to confirm that you want to start the SayHello composite.

13. Manually recover the fault.
    a. In the Enterprise Manager target navigation pane, expand soa-infra > default.
    b. Select CallGreeting project.
    c. In the CallGreeting pane, click Flow Instances.

Oracle Internal & Oracle Academy Use Only

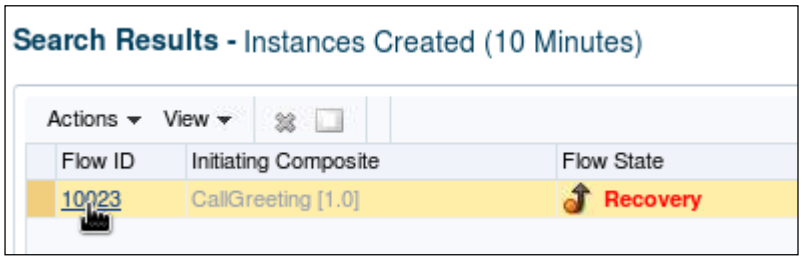
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



d. In the Search pane, click Search.



e. In the Search Results pane, click the Flow ID.



The Flow Trace for the instance is displayed.

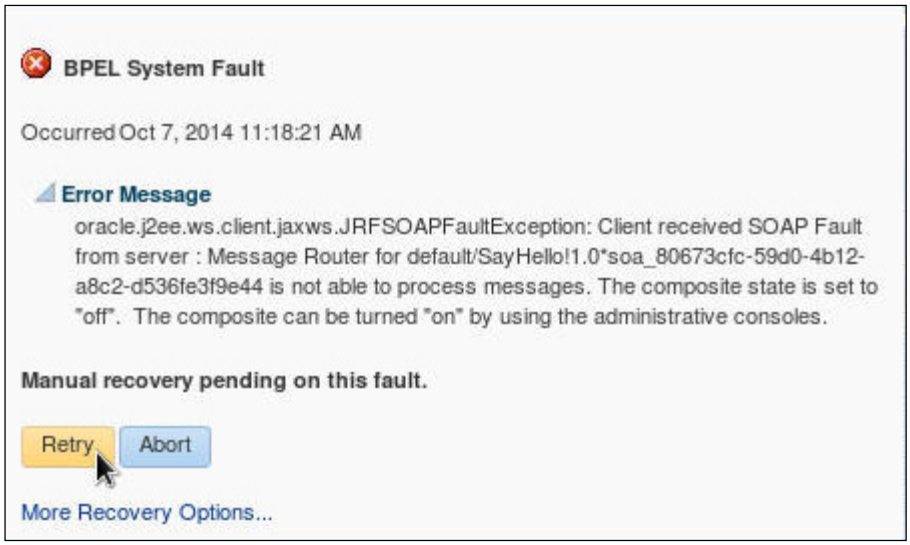
f. On the Faults tab, click the Recovery Required link.

Details of the fault are displayed along with the Retry and Abort options.

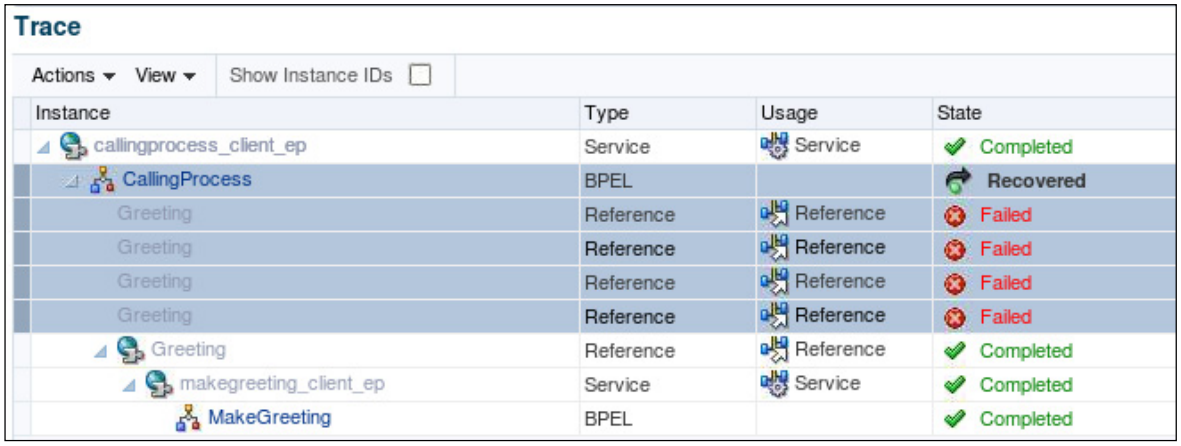
g. Click Retry.

Oracle Internal & Oracle Academy Use Only

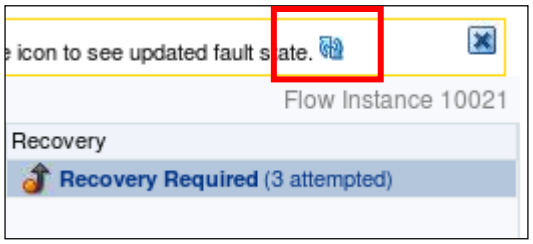
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



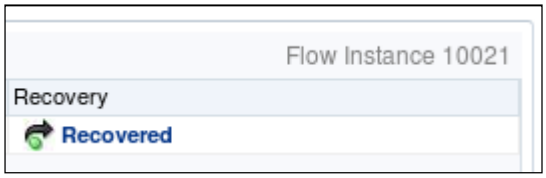
The Trace is updated, indicating the completion of the instance.



h. On the Faults tab, click the Refresh icon.



The Recovery status is updated.



i. Close the open Flow Trace window.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 9: Implementing Human Workflow

Chapter 9

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 9: Overview

Practices Overview

In this practice, you implement a human task component and email notification functionality in a composite application.

The human task component allows the user to review order details, and then approve or reject the order. Email notification is implemented to notify the customer about the status of the order.

Note: Use *exact* names (both spelling and case are important) as specified for your project and components. A later practice titled “Sharing Functionality” will expect to find these names in use.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 9-1: Seeding the Demo User Community

Overview

In this section, you deploy a web application that contains the demo users for the internal LDAP database of the WebLogic Server. It also contains a servlet that installs these users for you.

Assumptions

This practice assumes that the Integrated WebLogic Server is running.

Tasks

1. Deploy the web application to seed the LDAP database with users.
   a. Right-click the Terminal icon on your desktop and select Open.

      A terminal window opens.

   b. In the terminal window, initiate the deployment script by issuing the following commands:

      ```
      cd /home/oracle/labs/scripts/util
      sh deploySeedApp.sh
      ```

      The script confirms that deployment is completed.

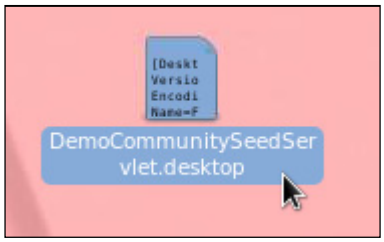
      ```
      Task 0 initiated: [Deployer:149026]deploy application SOATestDemoApp on DefaultS
      erver.
      Task 0 completed: [Deployer:149026]deploy application SOATestDemoApp on DefaultS
      erver.
      Target state: deploy completed on Server DefaultServer

      [oracle@EDRSR47P1 util]$
      ```

2. Run the DemoCommunitySeedServlet within the application that you just deployed to seed the demo users to the internal LDAP database.

   Note: The deployment script also added an icon to your desktop to access the newly deployed application.

   a. Click the DemoCommunitySeedServlet icon on your desktop.

      An Untrusted Application Launcher prompt is displayed.

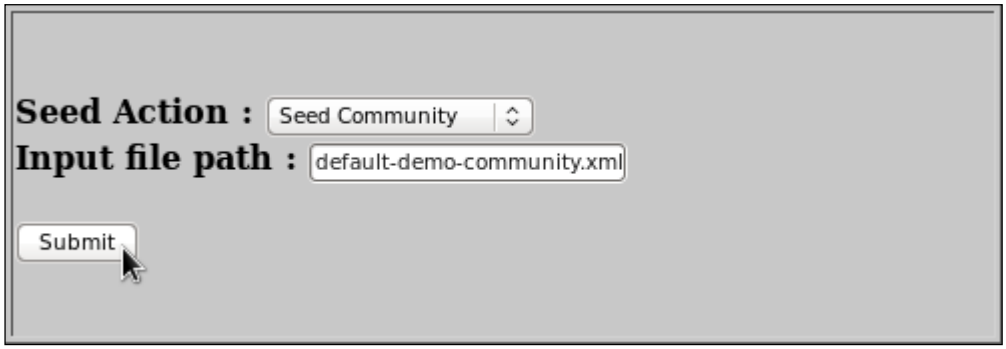
   b. Click either Launch Anyway or Mark as Trusted to proceed.

      A browser window is opened to http://localhost:7101/integration/SOADemoCommunity/DemoCommunitySeedServlet. The Seed Demo Community form appears.

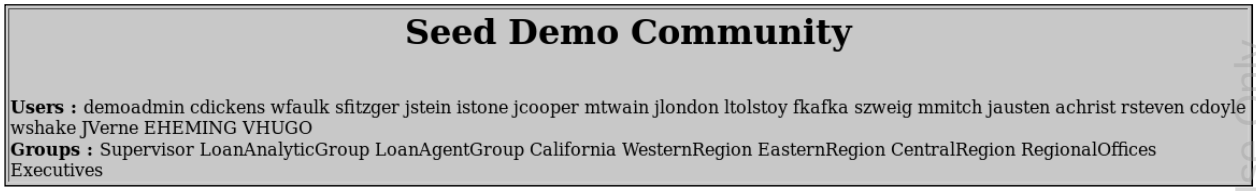
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

c. Click the Submit button.



A confirmation page lists the users and groups that were added.



3. Close the Seed Demo Community tab of your browser.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 9-2: Configuring the Email Driver

Overview

In this practice, you enable Email notifications to work in the SOA Server for Human Tasks and BPEL Email notification.

Assumptions

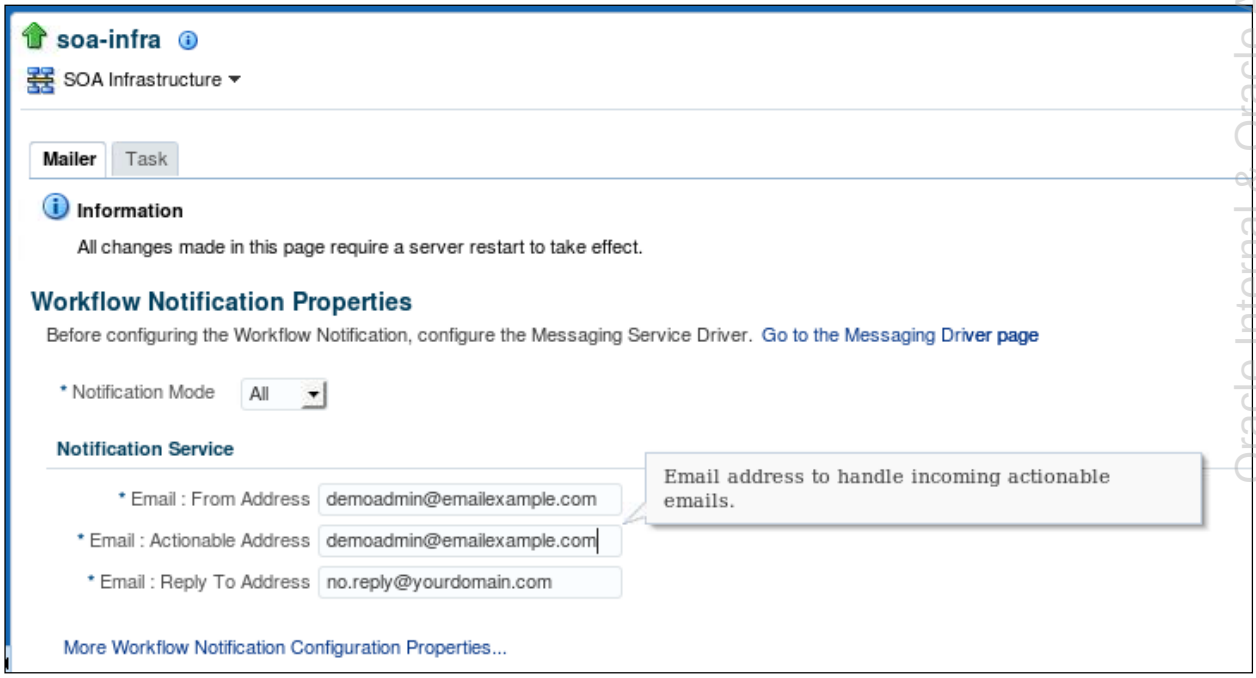
This practice makes no assumptions.

Tasks

1. Open the Oracle Enterprise Manager page (http://localhost:7101/em).
   a. In the Target Navigation pane, expand the SOA folder. Right-click soa-infra and select SOA Administration > Workflow Properties.
   b. On the Workflow Notification Properties page, set and enter the following values:

Notification Mode	ALL
Email: From Address	demoadmin@emailexample.com
Email: Actionable Address	demoadmin@emailexample.com
Email: Reply To Address	no.reply@yourdomain.com (default)

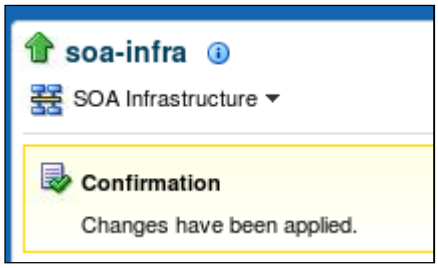
   c. Verify your work and click Apply.



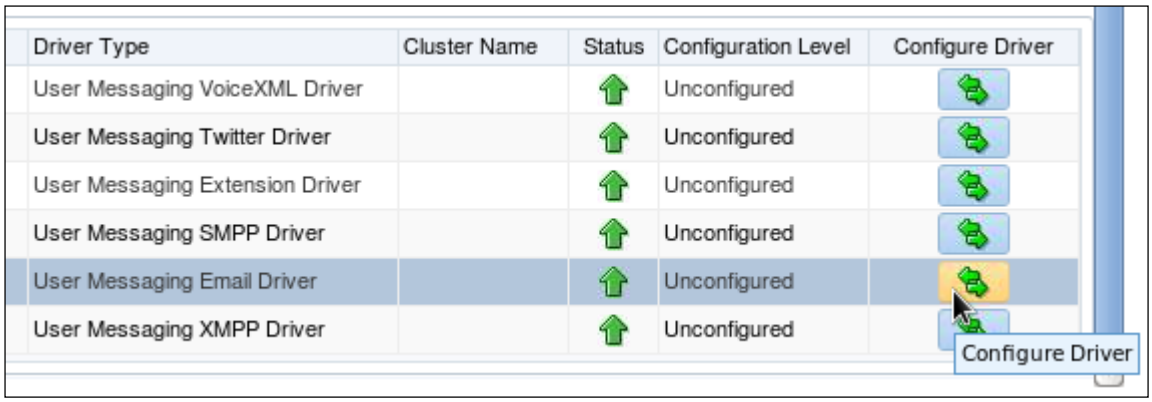
   d. In the Confirmation window, click Yes.
   e. On the SOA Infrastructure > Workflow Notification Properties page, verify that you get a confirmation that the changes have been applied.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



2. Configure the email driver.
   a. Click the Go to the Messaging Driver page link.
   b. On the **usermessagingserver** page, under Associated Drivers in the Local tabbed page, click the **Configure Driver** (bi-directional arrows) icon in the row for the User Messaging Email Driver (Driver Type).



   c. Click Create.
   d. Name the configuration `bcaEmail`.
   e. Scroll down in the Driver-Specific Configuration section as needed to set the following Driver-Specific Configuration field values:

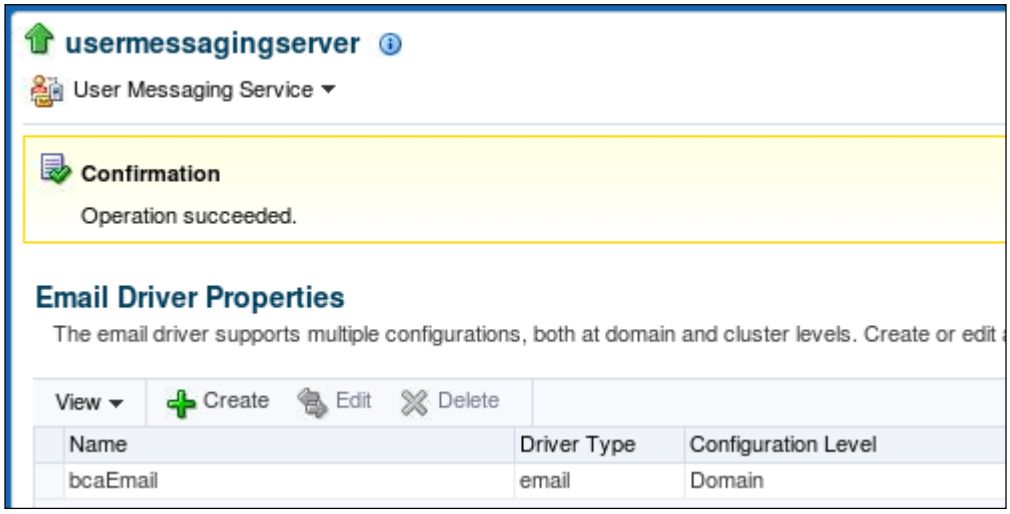
E-Mail Receiving Protocol	`POP3`
OutgoingMailServer	`emailexample.com`
OutgoingMailServerPort	`25` [This is the default]
Default From Address	`demoadmin@emailexample.com`
IncomingMailServer	`emailexample.com`
IncomingMailServerPort	`110`
IncomingMailIDs	`demoadmin@emailexample.com`
IncomingUserIDs	`demoadmin`

   f. For IncomingUserPasswords: set the following:
      - Type of Password: **Indirect Password, Create New User** [default]
      - Indirect Username/Key: **`demoadmin`**
      - Password: **`welcome1`** (This is displayed as asterisks when entered.)
   g. Click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

h. Verify that you receive a confirmation that changes have been applied.



Testing Your Changes

3. Stop and restart the Integrated WebLogic Server to cause your changes to take effect.
4. Test the email driver.
   a. In the Enterprise Manager Target Navigation pane, right-click soa-infra and select Service Engines > Human Workflow.
   b. On the Human Workflow Engine page, click the Notification Management tab.
   c. Click Send Test Notification.
   d. In the Send Test Notification dialog box, supply the following values:
      - Send To: `jcooper@emailexample.com`
      - Subject: `Test Message`
      - Content: `Can you see me?`
      - Channel: Email
   e. Verify your settings and click Send.

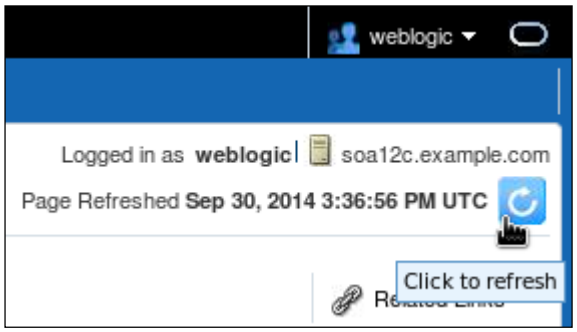


   f. The response SENT is displayed in the lower left section of the Send Test Notification pane.

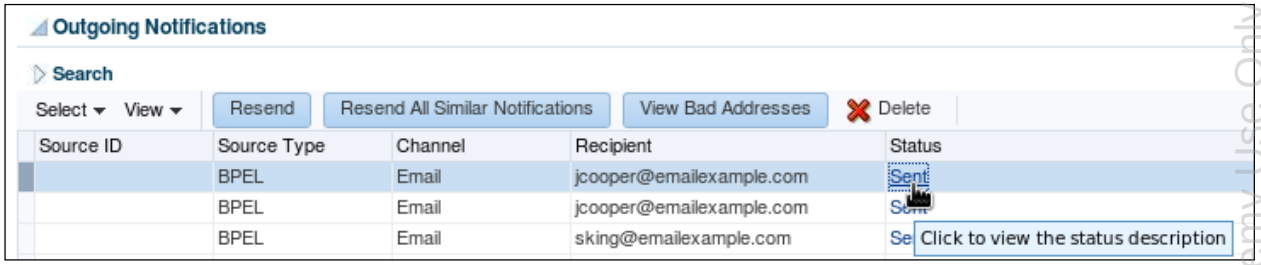
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

5. Review the sent message status.
   a. Refresh the Notification Management page.



   b. In the Outgoing Notifications section, click the Sent link of the topmost message.

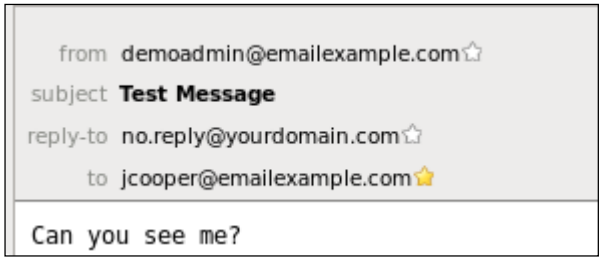


   The Status Description box is displayed.



6. View the email message that was sent to `jcooper`.
   a. Right-click the Thunderbird desktop icon on your desktop and select Open.
   b. In the Thunderbird window, in the All Folders pane, click the `jcooper@emailexample.com` account name.
   c. Click Get Mail on the Thunderbird toolbar.

      An email appears in the inbox.
   d. In the Inbox, click the email to see the message body.



   e. Close or minimize Thunderbird.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 9-3: Creating a Composite Application

Overview

In this practice, you create a new project and create a human task component.

Assumptions

This practice makes no assumptions.

Note: Use *exact* names (both spelling and case are important) as specified for your project and components. A later practice will expect to find these names in use.

Tasks

Creating a Composite Application

In this section, you create a new composite application. You define an exposed service, a human task, and a BPEL service component.

1. In the BPELProjects application, create a new empty SOA project named `Approval`.

   The `composite.xml` file is created and opened for editing.

2. Create a human task component in the assembly model.
   a. In the composite overview window, add a human task component to the Components swimlane.
   b. In the Create Human Task window, enter `ManualApproval` as the Name. Accept the default values for other settings. Click OK.

      Note: The `ManualApproval.componentType` and `ManualApproval.task` files are added to the project. The human task implementation details are stored in the `ManualApproval.task` file, which contains the human task configuration settings.

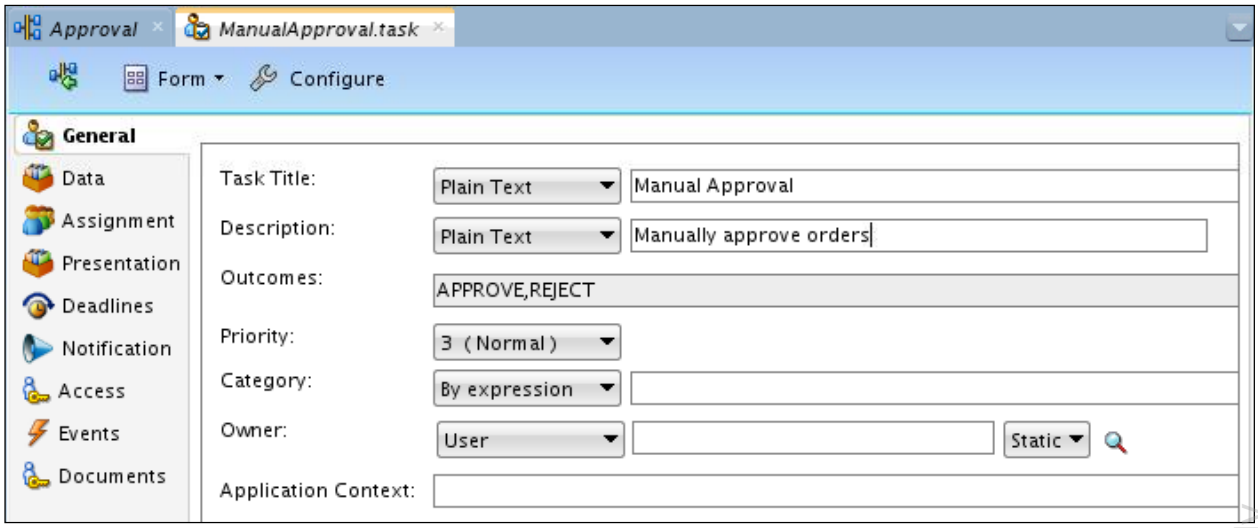
3. Configure the new human task component.
   a. Right-click the `ManualApproval` human task component icon and select Edit.

      The Human Task Editor opens the `ManualApproval.task` file.

   b. Configure the Title.
      1) Click the General tab.
      2) Select `Plain Text` from the drop-down menu.
      3) Enter `Manual Approval`.
   c. In the Description field, enter: `Manually approve orders`.
   d. Verify your work:

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



4. Configure the data parameters.
   a. Click the Data tab.
   b. Click the Create icon and select Add Other Parameter from the drop-down menu.

      The Add Task Parameter window opens.
   c. Select the Element radio button.
   d. Click the “Browse for Complex Types” icon to open the Type Chooser.

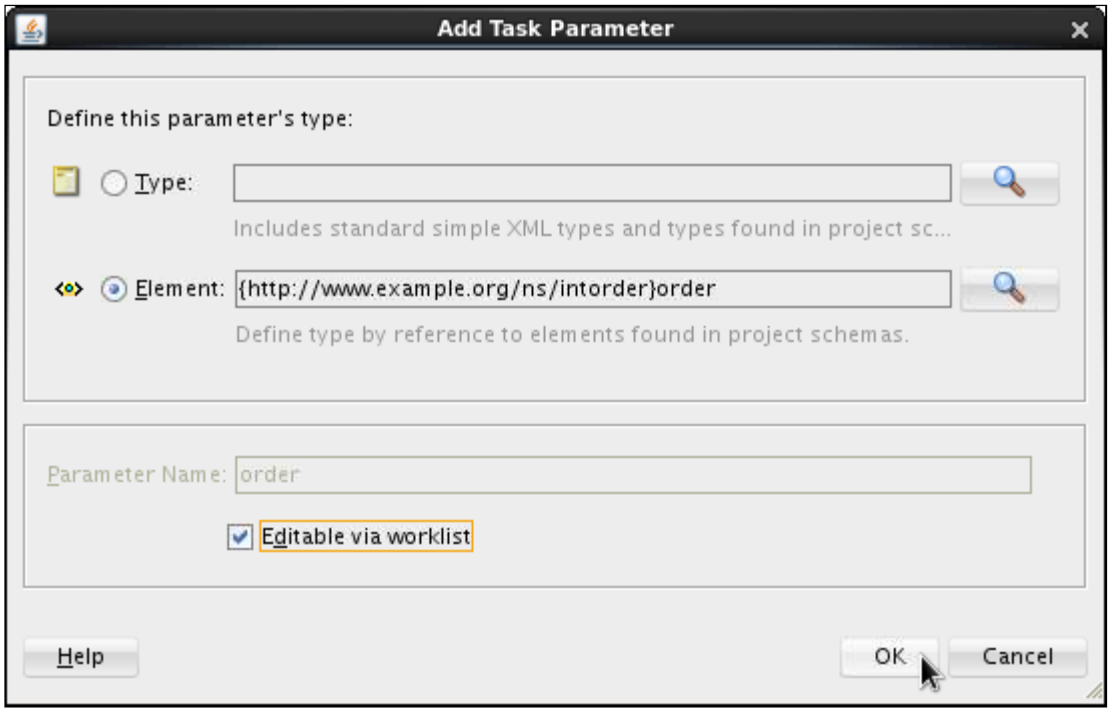
      Note: If necessary, point the cursor to an icon to verify its name for selection.
   e. Click the import icon to navigate to `/home/oracle/labs/files/xsd/internalorder.xsd`.

      Note: Be sure to deselect the “Maintain original directory structure” check box before copying.
   f. Select the `order` element and click OK.

      You are returned to the Add Task Parameter window.
   g. Select the “Editable via worklist” check box.
   h. Use the following screenshot to verify your work, and then save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

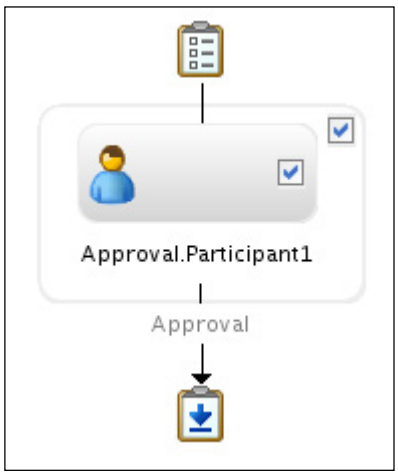


Configuring the Task Assignment and Routing Policy

The following steps associate the task with a particular participant.

5. Define a stage and associate it with a participant.
   a. Click the Assignment tab.
   b. Double-click the heading text `Stage1` in the shaded box.
   c. Name the stage `Approval` and click OK.
   d. Drag a Single Participant from the Workflow Editor section of the Component Palette onto the participant box.

      Approval.Participant1 is added to the box.



   e. Double-click Approval.Participant1.

      The Edit Participant Type window opens.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

f. In the Participant Names table, click the Add icon and select Add User.

A user row is added to the table.

g. Click the Ellipsis icon in the Value column.

The Identity Lookup window opens.

Note: In the next step, you connect to an LDAP Server that provides a list of employees defined in the system. You then assign the task to one of these employees.

6. Look up and select the user `jcooper` as the participant.

a. Select the Application Server IntegratedWebLogicServer (Resource Palette Connection).

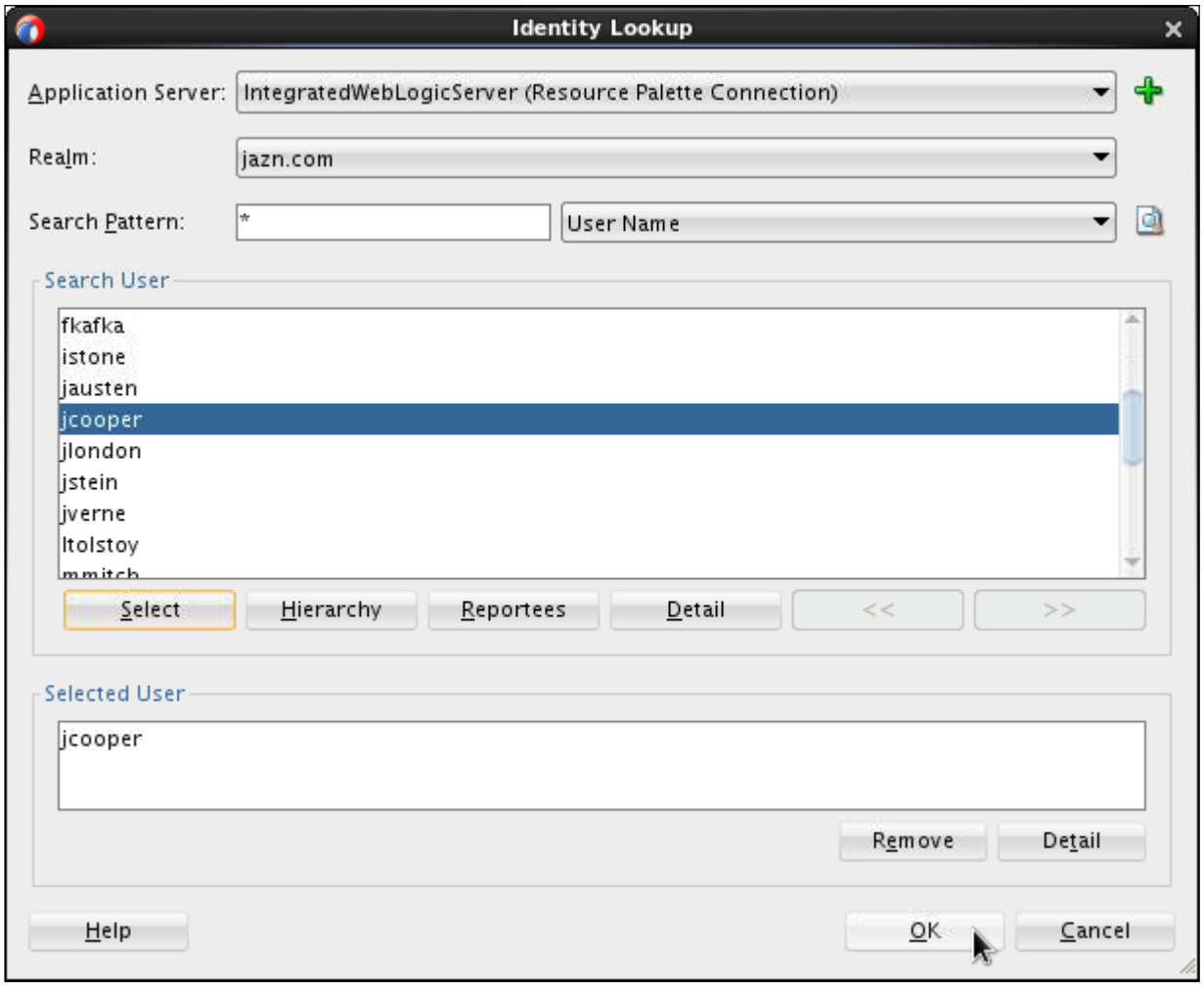
b. Click the Lookup icon to the right of the Username drop-down menu.

The Search User area is populated with a list of usernames registered in the LDAP Server.

c. Highlight the `jcooper` user and click Select.

The `jcooper` user is displayed in the Selected User area.

d. Verify your work and click OK.

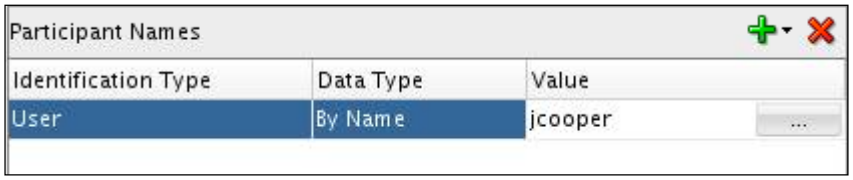


You are returned to the Edit Participant Type window.

Oracle Internal & Oracle Academy Use Only

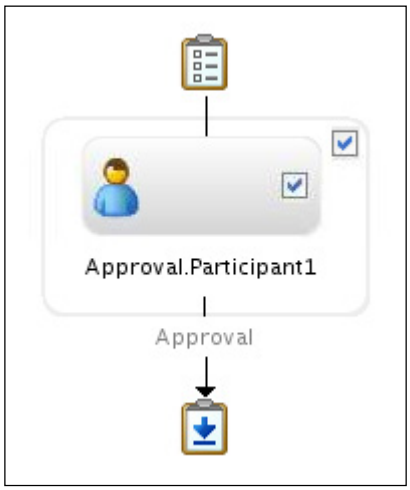
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

e. Verify that `jcooper` appears in the Participant Names table row and click OK.



You are returned to the `ManualApproval.task` window.

f. Verify that the Assignment and Routing Policy: diagram contains the `Approval.Participant1` value.



7. Ensure that the notification settings are enabled to send an email to Assignees when the task is assigned to the user.
   a. Click the Notification tab.
   b. In the Assign row, click the Edit icon (pencil) in the Notification Header column to examine the text used for the email subject.
8. Save your work and close the `ManualApproval.task` window.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 9-4: Implementing Manual Approval in the BPEL Process

Overview

In this practice, you create and configure a BPEL process that receives incoming order data. It invokes the Manual Approval human task that you just completed to seek approval. The process then returns the result of that manual approval to the calling client. In this practice, you also generate an ADF Task Form project for the human task to display the task parameter information in the worklist application.

Assumptions

This practice assumes that you have completed Practice 9-1 successfully.

Tasks

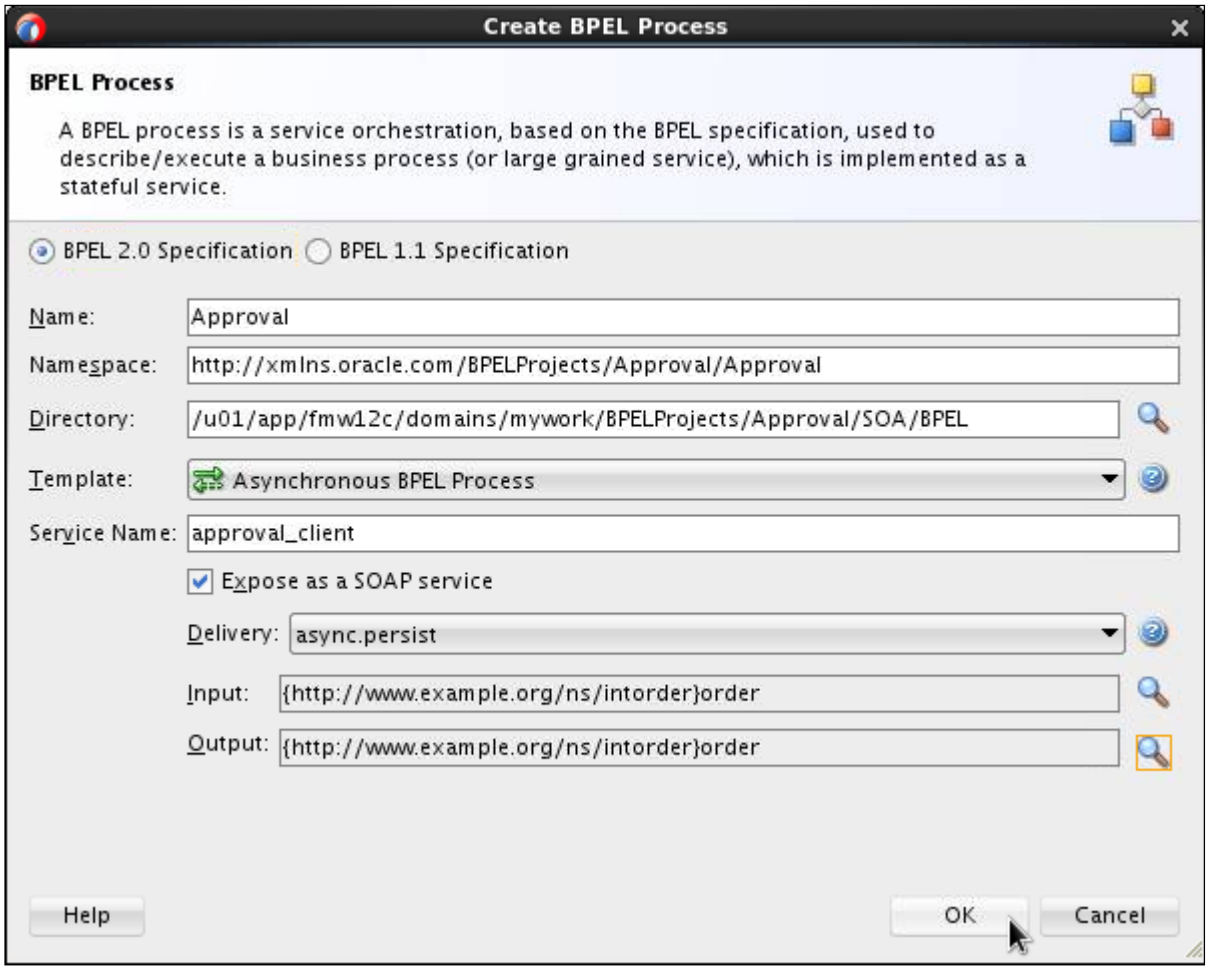
Creating and Configuring a BPEL Process

In this section, you create a BPEL process and configure its activities.

1. Create and configure a BPEL service component.
   a. Add a BPEL process to the Components swimlane in the `composite.xml` editor.
   b. Name the process `Approval`.
   c. Verify that the Asynchronous Template is chosen (default).
   d. To define the input, use the Browse button to navigate to the `order` element of `internalorder.xsd`.
   e. To define the output, browse to the `order` element of `internalorder.xsd`.
   f. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



g. Save your work.

2. Create and configure an Assign activity to copy `inputVariable` to `outputVariable`.
   a. Right-click the BPEL Process Approval and select Edit.
   b. Add an Assign activity to the process between the Receive and Callback activities.
   c. Name the activity `Assign_Output`.
   d. Right-click the new activity and select Edit.
   e. Map the `inputVariable` element `payload` to its corresponding element in `outputVariable`.
   f. Verify and save your work.
3. Create and configure a Human Task activity in the business process.

   Note: This task is associated with the human task component that you created earlier.
   a. Add a Human Task (from the SOA Component section of the Component Palette) to the process immediately following `Assign_Output`.
   b. Right-click the activity and select Edit.

      The Edit Human Task window opens.
   c. Select the Task Definition `ManualApproval` from the drop-down menu.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Additional configuration fields are exposed.

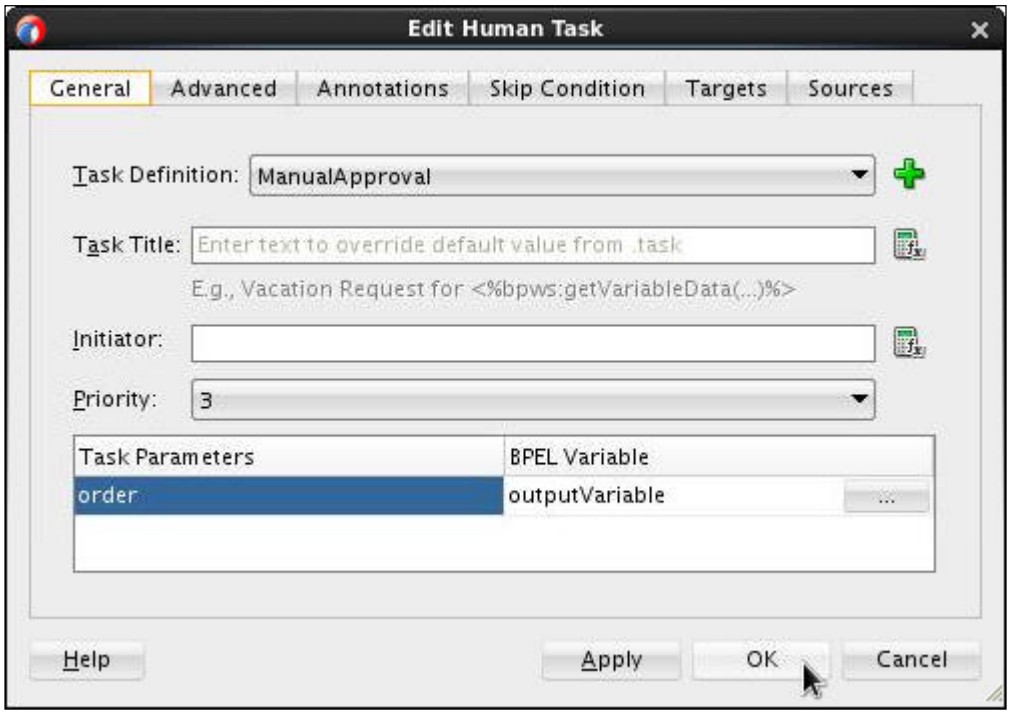
d. Click the Ellipsis (…) icon in the BPEL Variable field for the Task Parameter called `order`.

   The Task Parameter window opens.

e. Select the Variables > **output**Variable > payload > `order` element.

f. Click OK to close the Task Parameter window.

g. Verify your work and click OK.

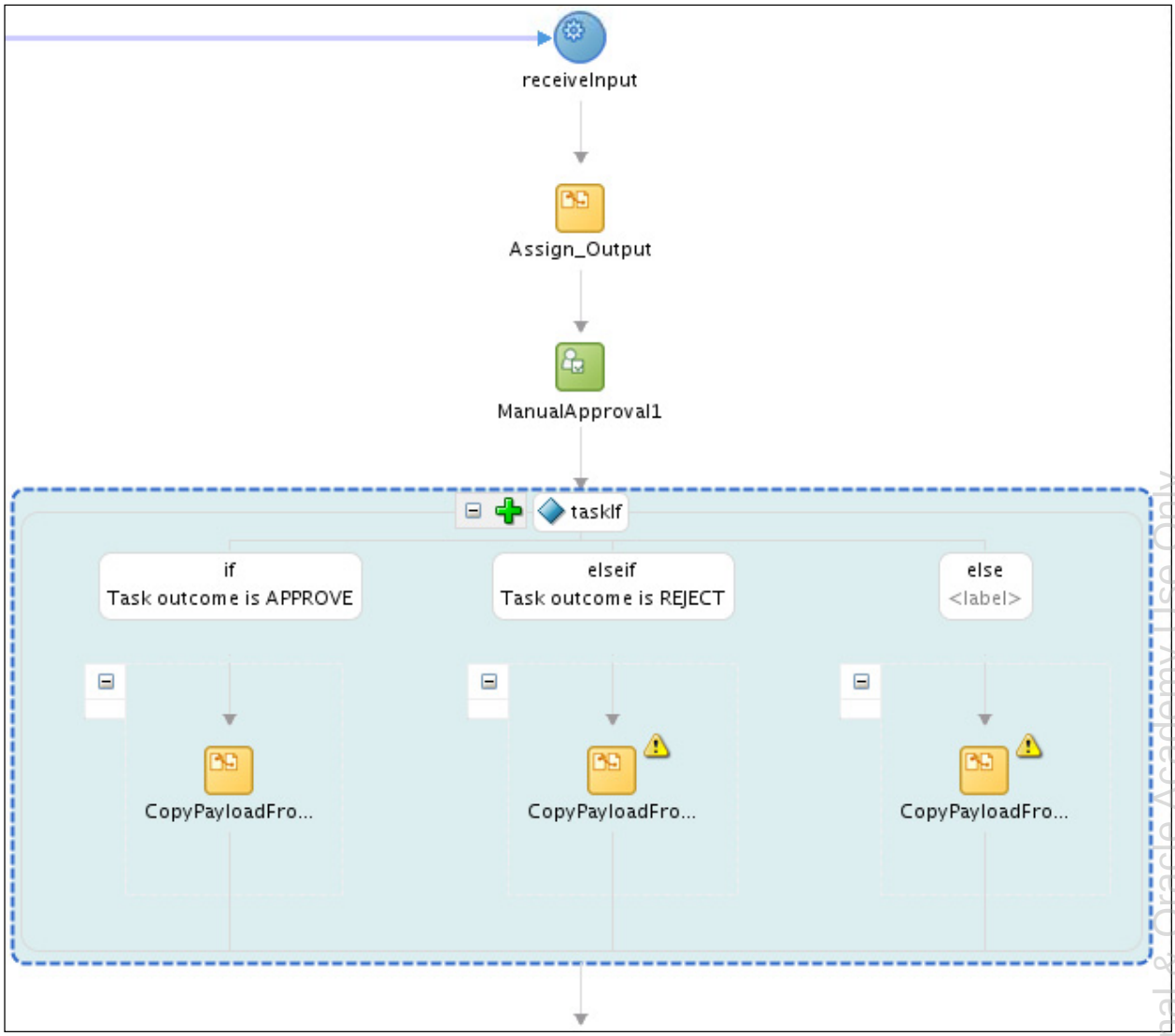


An If activity is added to the process to process the possible outcomes of the manual approval.

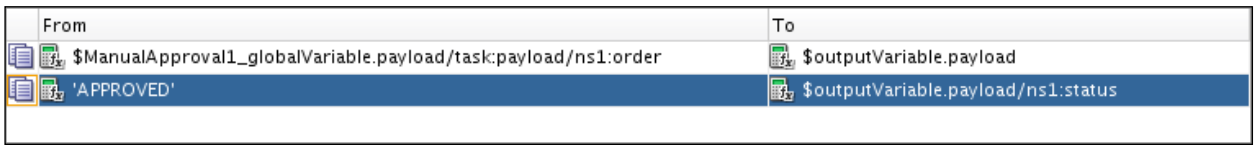
Note: These outcomes are part of the configuration of the human task component.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



4. Configure the `taskIf` branches to process the various possible task outcomes.
   a. Expand the `taskIf` that appears after the `ManualApproval1` Human Task activity.
   b. Expand the CopyPayloadFromTask Assign activity in the APPROVE branch of the If.
   c. Use the Expression Builder icon to insert the word `'APPROVED'` into the `status` node of `outputVariable`.



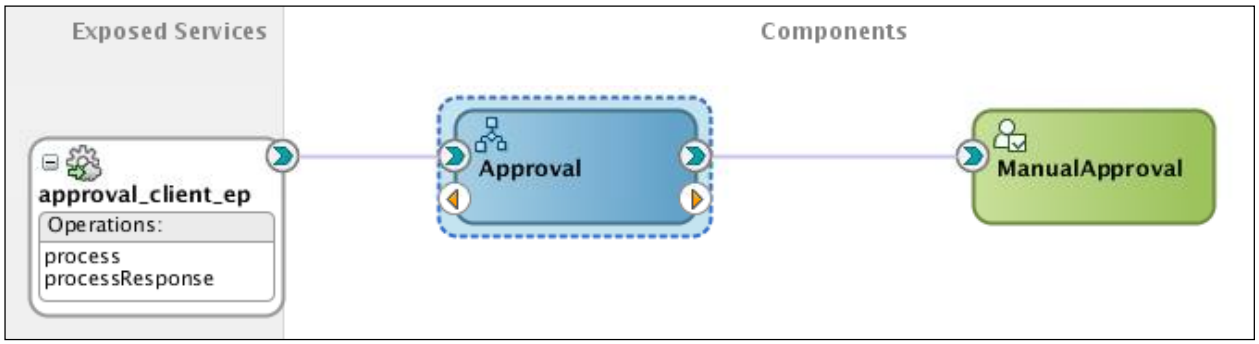
   d. Repeat steps b and c to insert the word `'REJECTED'` into the `status` node of the `elseif` REJECT branch of the If.
   e. Repeat steps b and c to insert the word `'FAILED'` into the `status` node of the `else` branch of the If.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

f. Save your work.

g. Click the Approval tab and verify that a wire is created between the BPEL process `Approval` and the human task `ManualApproval`.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 9-5: Generating the ADF Task Form for the Human Task

Overview

In this section, you use the human task to generate a default ADF Task Form Project, which contains JSF code to display the task parameter information in the worklist application. The worklist application displays the generated Task Form that is viewed by the task assignee to approve or reject the orders that are sent for manual approval.

Note: Because generation of the ADF Task Form Project is CPU-intensive and memory-intensive, you begin this section by shutting down Enterprise Manager and any other unnecessary applications.

Assumptions

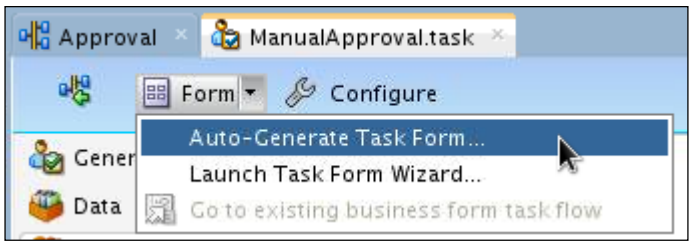
This practice assumes that you have completed all work through Practice 9-4 successfully.

Tasks

1. Shut down all unused applications, including your web browser, the Integrated WebLogic Server, and any other applications currently open, except JDeveloper.
2. Generate the Human Task Form Project.
   a. In the composite overview window, right-click `ManualApproval` and select Edit.

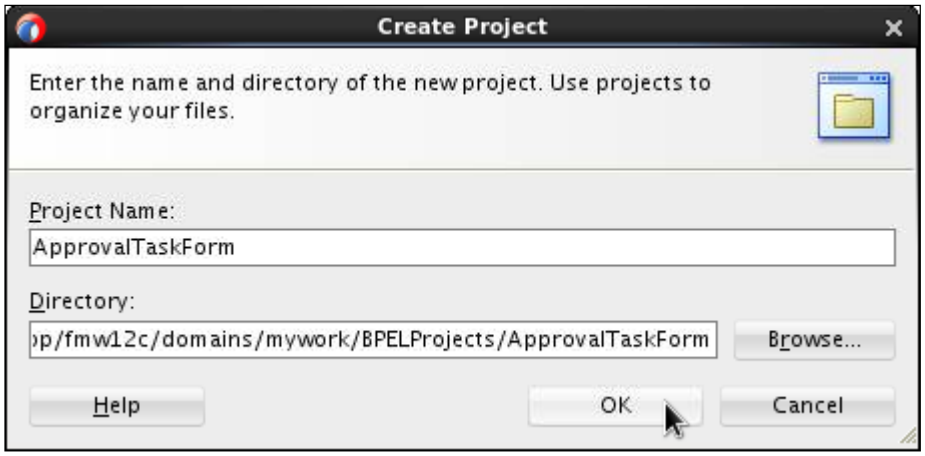
      The `ManualApproval.task` window opens.

   b. From the Form drop-down menu, select Auto-Generate Task Form.



      The Create Project window opens.

   c. Name the project `ApprovalTaskForm` and click OK.

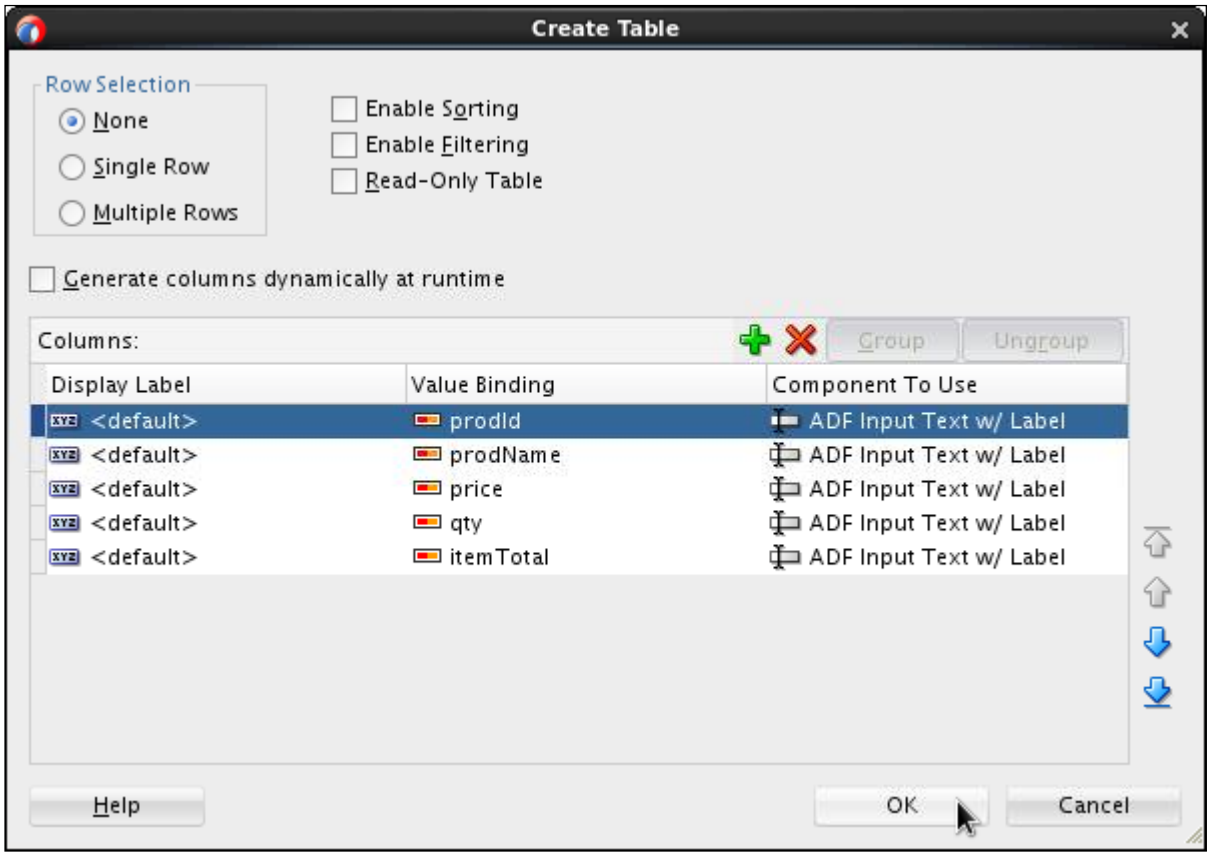


Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

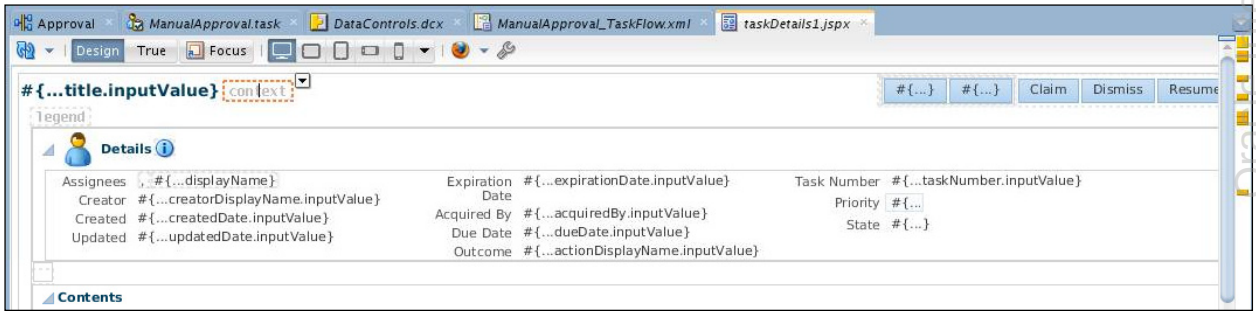
Note: JDeveloper starts to consume 100% of the CPU at this time to begin the generation process.

The Create Table window is displayed.



d. To accept the default settings, click OK.

Note: JDeveloper again consumes 100% of the CPU. When the form is complete, the `taskDetails1.jspx` window displays the generated task form in the designer.



e. Save your work.

f. Close the `taskDetails1.jspx`, `ManualApproval_TaskFlow.xml` and `DataControls.dcx` windows.

3. Restart the Integrated WebLogic Server from the JDeveloper *Run* menu.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 9-6: Deploying and Testing the Application

Overview

In this practice, you deploy the `Approval` and `ApprovalTaskForm` applications. You then test the applications, including the manual approval of an order and the sending and receipt of email to the approver.

Assumptions

This practice assumes that you have completed all work up to this point successfully.

Tasks

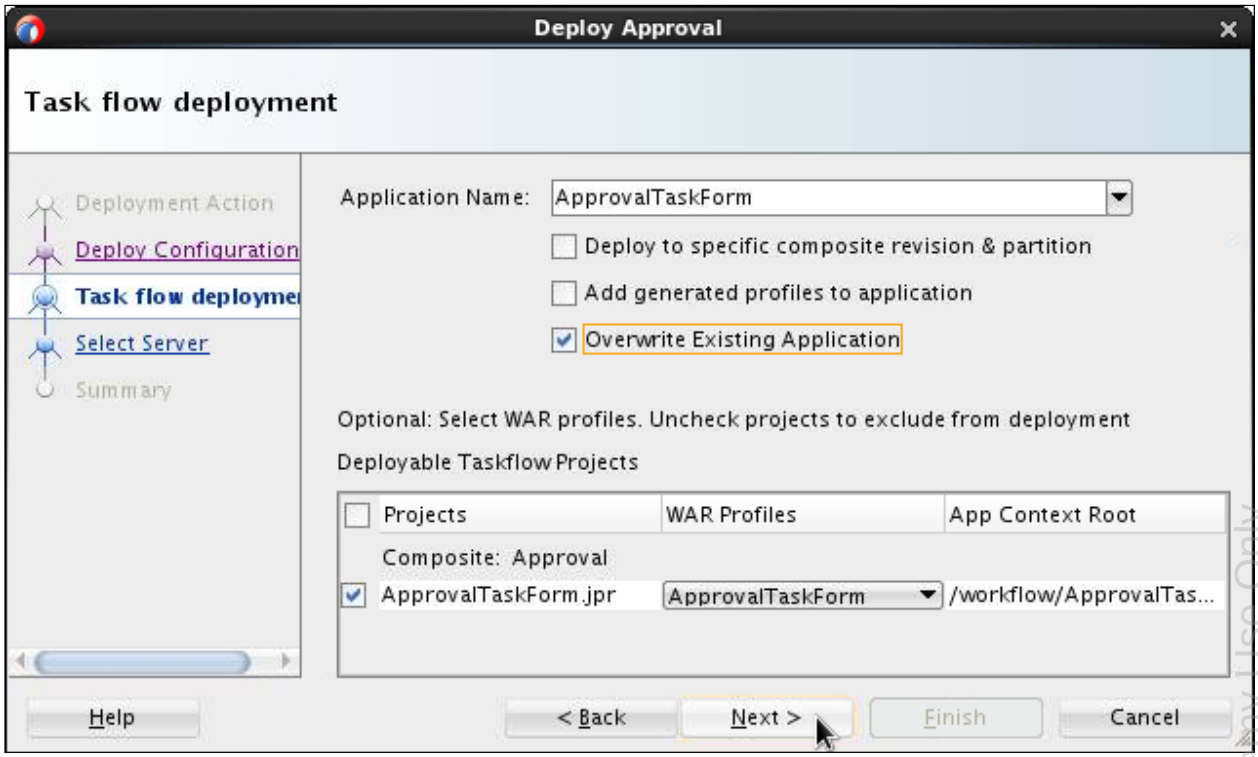
Deploying the `Approval` Composite Application

In this section, you deploy the `Approval` and `ApprovalTaskForm` applications. Note that this deployment sequence differs from what you have seen so far in this course.

1. Deploy the `Approval` application to IntegratedWebLogicServer.
   a. In JDeveloper, right-click the application `Approval` and select Deploy > Approval.
   b. Select “Deploy to Application Server.”
   c. Select the “Overwrite any existing composites with the same revision ID” check box.
      The Task Flow Deployment screen appears.
   d. Select the following options:
      - Application Name: `ApprovalTaskForm`
      - Overwrite Existing Application check box
      - Deployable Taskflow Projects: ApprovalTaskForm.jpr
   e. Verify your settings and click Next.

Oracle Internal & Oracle Academy Use Only

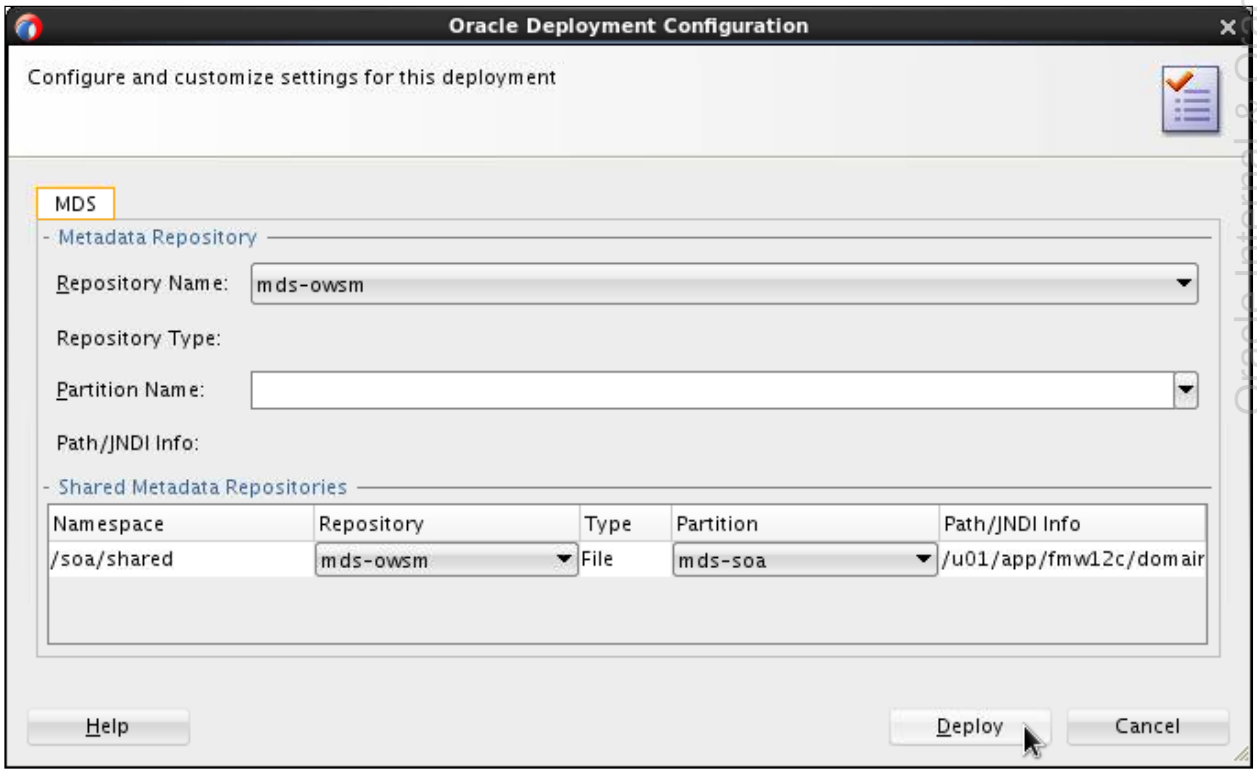
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



f. Select the application server IntegratedWebLogicServer.

g. Click Finish.

The Oracle Deployment Configuration window appears.

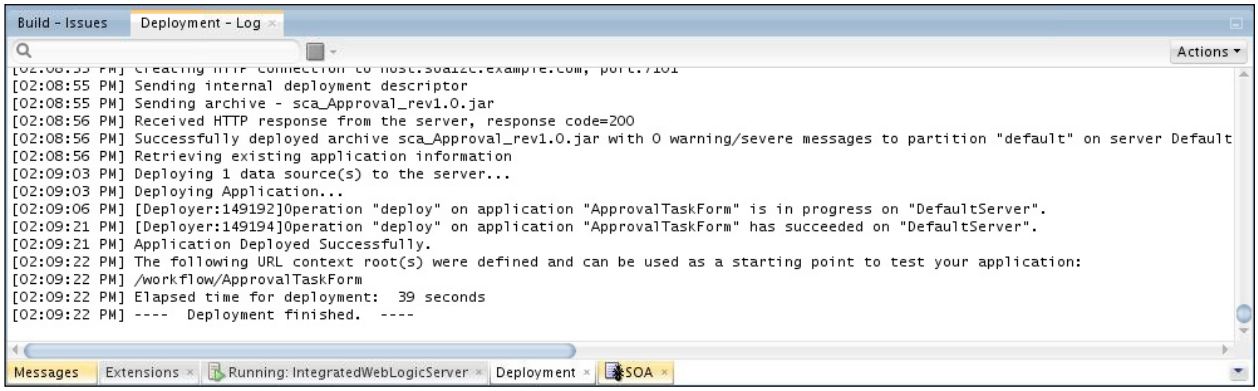


h. Click Deploy.

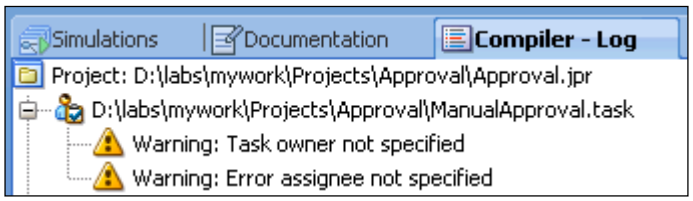
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

i. In the Deployment – Log window, verify that deployment is successful.



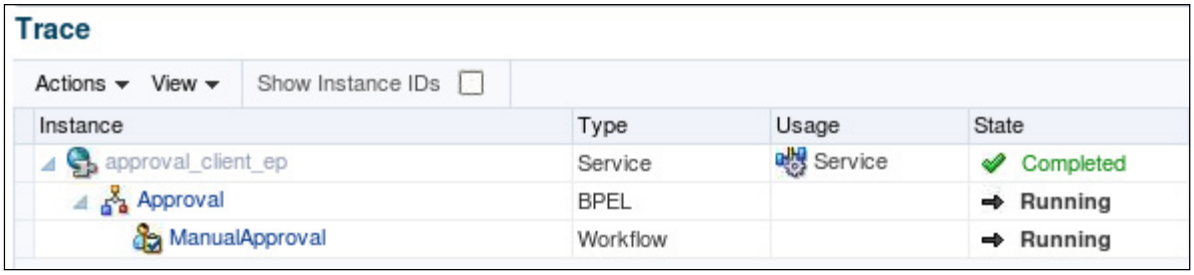
Note: You may ignore the following compiler warnings:



Testing the Composite Application

2. Initiate the test.
   a. Open Oracle Enterprise Manager (http://localhost:7101/em).
   b. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “Approval [1.0]” link.
   c. On the “Approval [1.0]” home page, click Test.
   d. Use the Browse button to replace the sample XML data with the contents of the file `/home/oracle/labs/files/xml_in/approval_input.xml`.
   e. Click Test Web Service.

      The Response tab is displayed.
   f. On the Response tab, click the Launch Flow Trace button.
3. On the Flow Trace page, examine the Trace tree that is shown in the following image and answer the questions that follow the image:



Note: If the process Flow Trace tree does not resemble the preceding image, wait a few seconds and refresh the page before you continue. You may need to refresh the page a few times.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Using the Flow Trace tree, answer the following questions:

a. Has the entire process completed? Explain your answer.

b. In this case, what does the last row in the Trace tree that contains the `ManualApproval` Human Workflow component indicate?

Q	Answers to Step 3.
a.	No, the process is still running. If you look in the Trace tree, you can see that the rows containing Approval and ManualApproval indicate their state as Running.
b.	The ManualApproval row indicates that the process is executing the Human Workflow component, which is not yet complete because it is still in the Running state.

4. To examine the `ManualApproval` component state, perform the following steps:

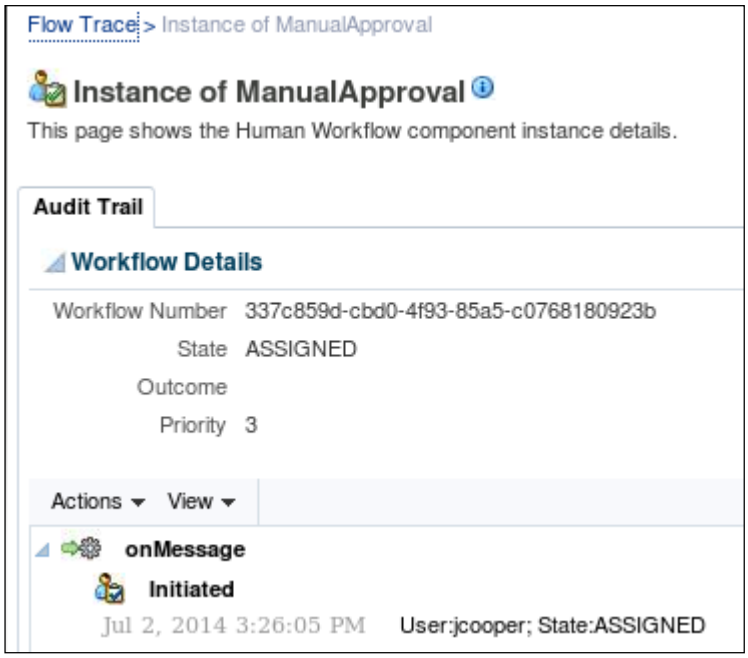
   a. In the web browser window with the Trace Flow page, click the ManualApproval Human Task component link.

   b. On the "Instance of ManualApproval" page, view the information and answer the following questions:

      1) What is the state of the human workflow task?

      2) Who is the task assignee?

Q	Answers to Step 4b).
a.	The task is in the ASSIGNED state. The following image shows the Instance of ManualApproval.
b.	The assignee is `jcooper` (refer to the following image).



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

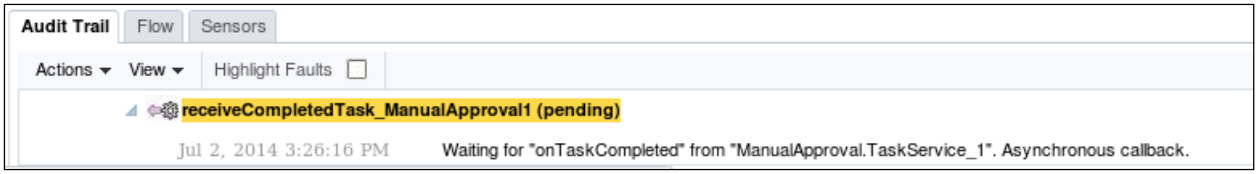
c. On the “Instance of ManualApproval” page, click the Flow Trace locator link at the top of the page.



5. To determine why the application is still running, click the Approval BPEL Component link in the web browser window with the Trace Flow page.
6. On the “Instance of Approval” page, view the process details and answer the following questions:
   a. What is the name of the last activity executed?
   b. Explain what the last activity execute state represents at this time.

   Note: *Do not* close the “Instance of Approval” page yet.

Q	Answers for Step 8.
a.	The name of the last activity executed is `receiveCompletedTask_ManualApproval1`.
b.	The `receiveCompletedTask_ManualApproval1` is in a *pending* state. The activity is waiting for an asynchronous callback message from the human workflow task that indicates that it has been completed. The implications are that the human workflow process is not complete.



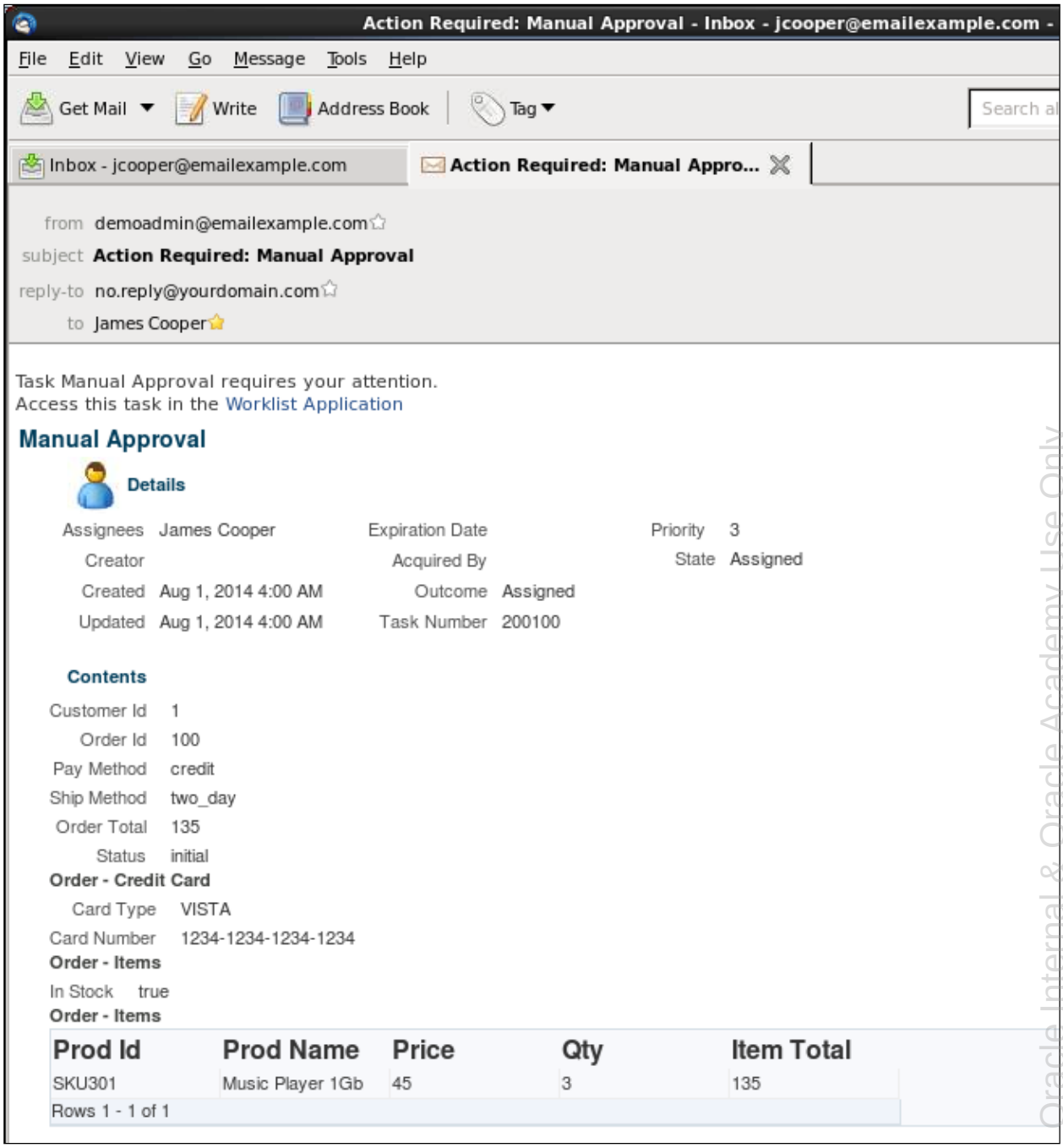
Viewing the Email Messages and the Worklist Application

7. View the email message that is sent to the task assignee `jcooper`.
   a. Right-click the Thunderbird desktop icon on your desktop and select Open.
   b. In the Thunderbird window, in the All Folders pane, click the `jcooper@emailexample.com` account name.
   c. Click Get Mail on the Thunderbird toolbar.

      An email appears in the Inbox.
   d. In the Inbox, verify the presence of an email message and click the entry to see the message body.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



8. To open the worklist application and log in as `jcooper` to make changes to the shipping method and approve the order, perform the following steps:

   a. To access the worklist application in a Firefox web browser window, enter the URL http://soa12c.example.com:7101/integration/worklistapp.

      Tips:

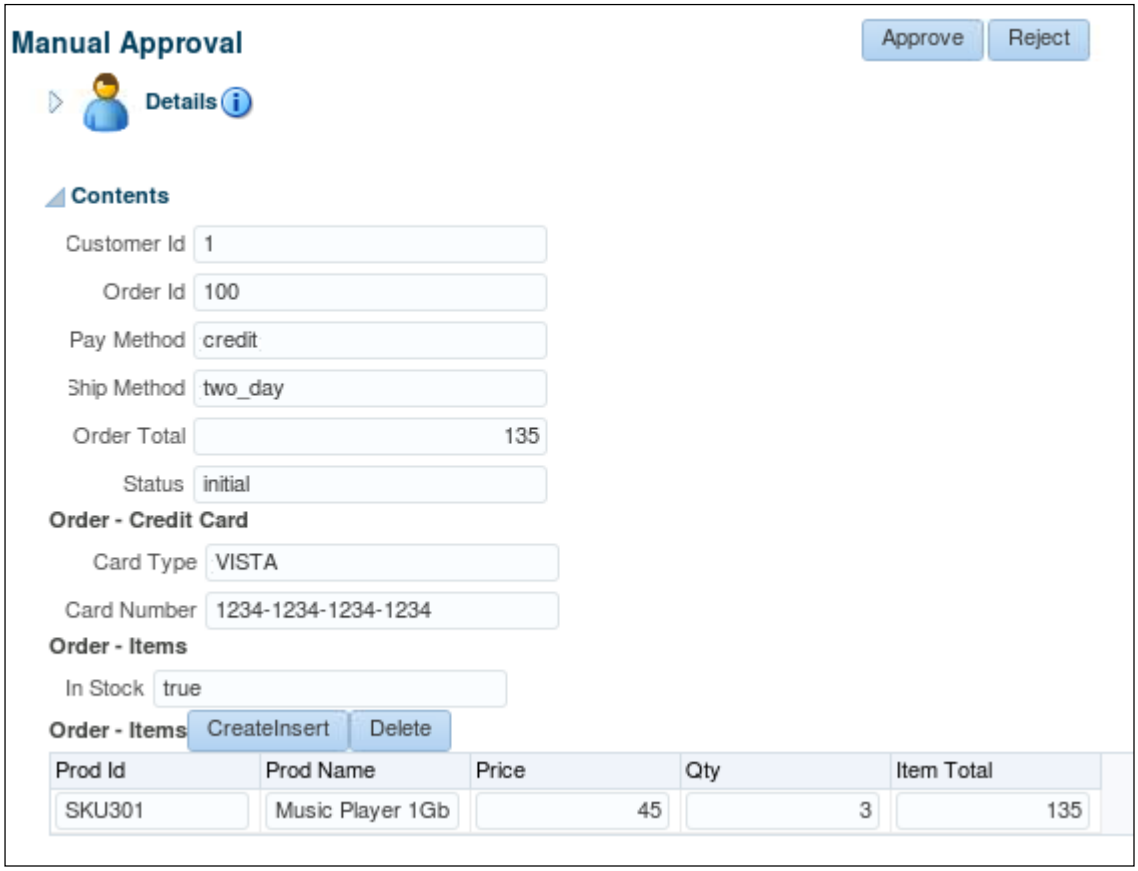
      - **Do *not* substitute "localhost" for the fully qualified host name in the URL.**
      - **Use only the Firefox browser.**

   b. On the Oracle BPM Worklist application page, enter `jcooper` as the Username and `welcome1` as the Password, and click Login.

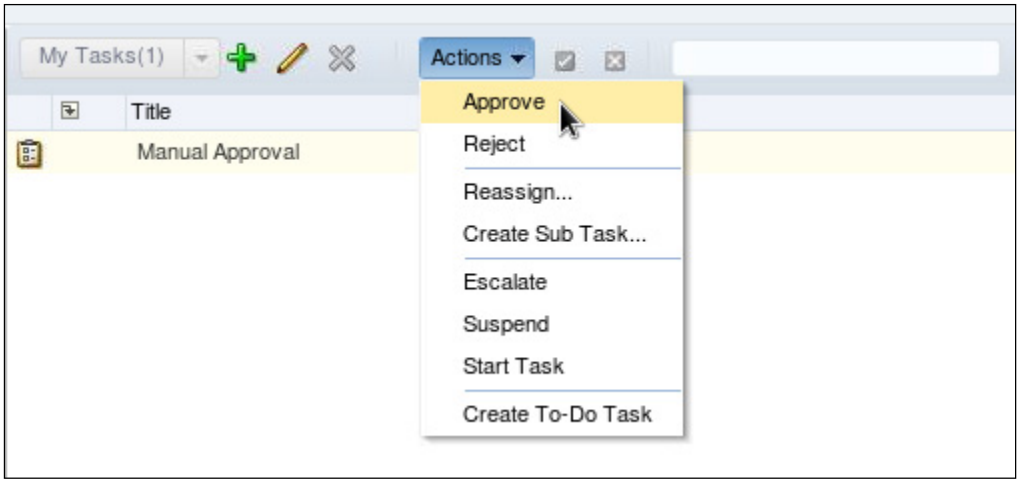
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

c. On the Worklist home page, the Inbox is selected and the task with the title “Manual Approval” should be present in the top-right pane on the page. Click the “Manual Approval” task entry to display the order details.



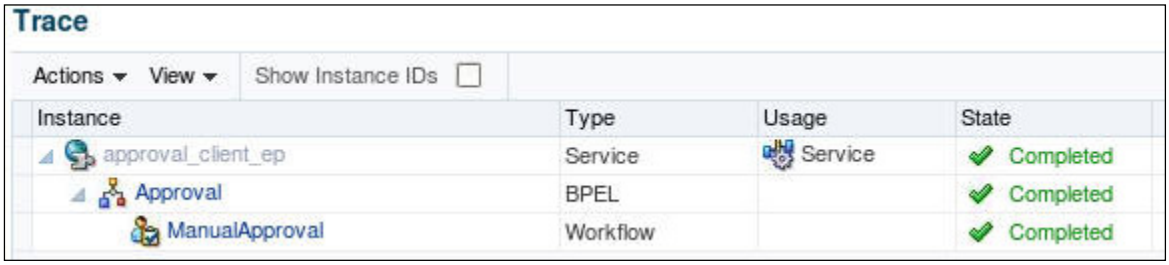
9. Click Approve (on either the task form as shown in the preceding screenshot or the toolbar shown as follows) to approve the order and allow the process to complete.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

10. Return to the Flow Trace and click Approval. Examine the steps of the business process and verify that they completed successfully.



11. If you want, and if time permits, run another test and, in the worklist application, reject the order.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 9-7: Sending Order Status Email Notification to a Customer [Optional]

Overview

In this practice, you first add a Database adapter to the composite application. You then modify the `Approval` BPEL Process. You create a Scope, and then add activities to the Scope to look up the customer email address from a database and send an email notification to the customer.

Assumptions

This practice assumes that you have completed Practice 9-5 successfully.

Tasks

Creating and Configuring a Database Adapter

1. In the assembly model (`composite.xml`) for the project `Approval`, add a Database adapter to the External References swimlane.

   The Adapter Configuration wizard opens.

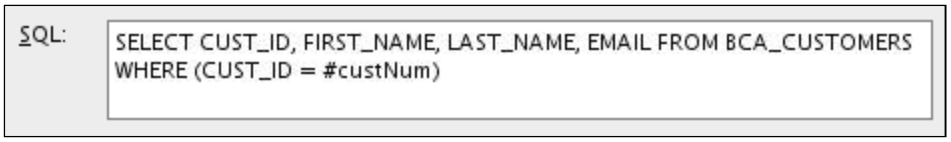
2. To configure the Database adapter, complete the following steps:

Step	Screen	Choices or Values
a.	Database Adapter Reference	Name: `CustomerLookup`
b.	Service Connection	Click Browse. Select SOA connection. Click Copy Connection. Verify the following settings: • Connection: **`SOA`** • User Name: **`soainfra`** • Driver: **`org.apache.derby.jdbc.clientDriver`** • Connect String: **`jdbc:derby//localhost:1527/soainfra`** • JNDI Name: **`eis/DB/SOA`**
c.	Operation Type	Select the “Perform an operation on a Table” option. Ensure that the Select check box is selected. Deselect other operations.
d.	Select Table	Click Import Tables.
e.	Import Tables	Set the BCA_% filter.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Step	Screen	Choices or Values
		Click Query.
f.	Import Tables	Copy the `BCA_CUSTOMERS` table from the Available list to the Selected list. Click OK.
g.	Select Table	Verify that the `BCA_CUSTOMERS` table is listed.
h.	Relationships	Click Next.
i.	Attribute Filtering	Select the firstName, lastName, and email attributes. Deselect the remaining options. **Note:** The custID (key attribute) cannot be deselected. Click Next.
j.	Define Selection Criteria	Next to the Parameters section, click Add.
k.	Parameter Name	Enter `custNum`. Click OK.
l.	Define Selection Criteria	Next to the SQL field, click Edit.
m.	Expression Builder	Click Add.
n.	Expression Builder	Form the condition `custId` EQUAL `custNum`, where: • First Argument Query Key: `custID` • Operator: `EQUAL` • Second Argument: Select the Parameter option, and select `custNum`. Click OK.
o.	Define Selection Criteria	Verify that your SQL statement is correct by comparing it with the following screenshot.
p.	Advanced Options	Click Next.
q.	JCA Endpoint Properties	Click Next.
r.	Finish	Click Finish.



3. Save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Modifying the Approval BPEL Process

4. In the composite overview window, map the BPEL component Approval to CustomerLookup.

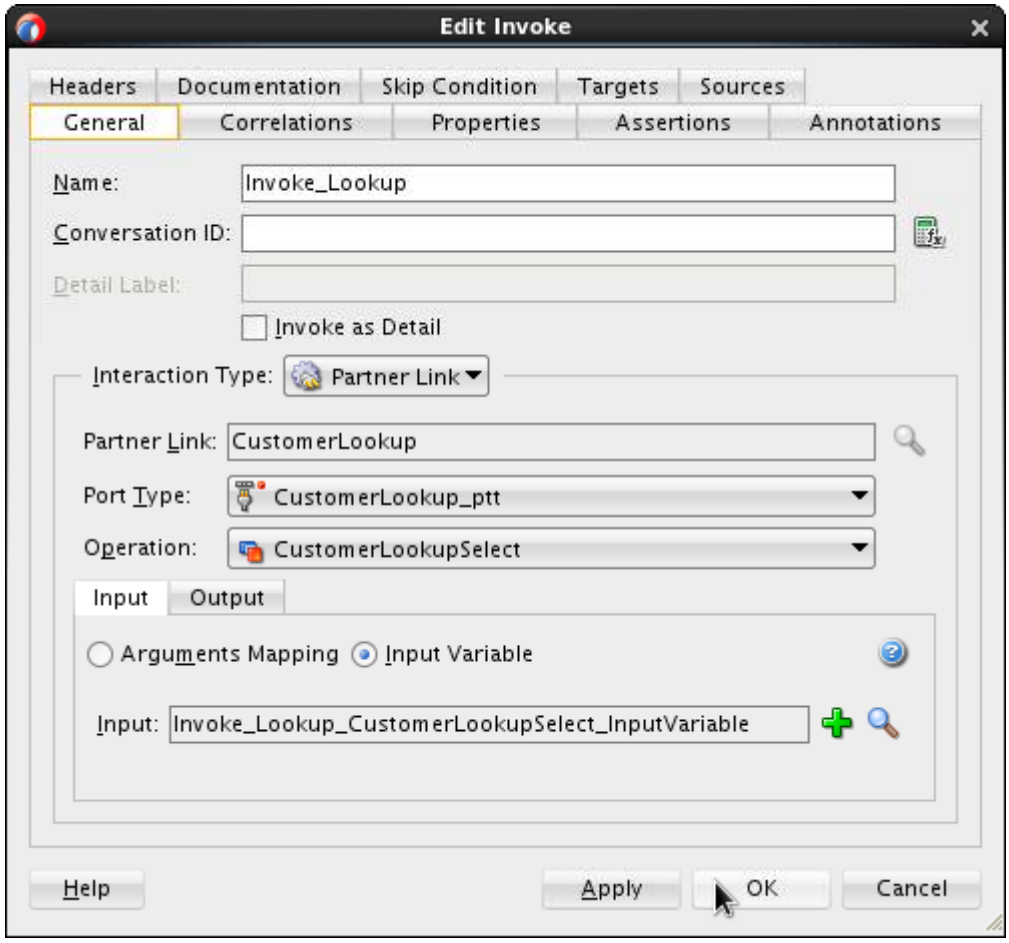
   This causes CustomerLookup to appear as a partner link in the BPEL editor.

5. Create and configure a Scope activity in Approval.
   a. If it is not already open for edit, right-click the Approval component and select Edit.
   b. For visual clarity, collapse the `taskIf` activity.
   c. Add a new Scope just above the `callbackClient` activity.
   d. Name the new Scope `Scope_Email`.
6. Create and configure an Invoke activity to look up the customer email address.
   a. Expand `Scope_Email`.
   b. Add an Invoke activity to `Scope_Email`.
   c. To specify the partner link, drag and drop Invoke to the partner link CustomerLookup.

      The Edit Invoke dialog box appears.
   d. Name the Invoke `Invoke_Lookup`.
   e. Create a new Input variable.
   f. Create a new Output variable.
   g. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



7. Create and configure an Assign activity.
   a. Add an Assign activity within `Scope_Email` just above `Invoke_Lookup`.
   b. Name the activity `Assign_Lookup`.
   c. Right-click the new activity and select Edit.
   d. Map the `inputVariable` element `customerId` to the `custNum` element of the `Invoke_Lookup... inputVariable`.
   e. Verify and save your work.



8. Create and configure an `Email` activity.
   a. Add an `Email` activity (Oracle Extension) within `Scope_Email` just below `Invoke_Lookup`.
   b. Name the activity `Email_Customer`.
   c. Right-click the `Email` activity and select Edit.

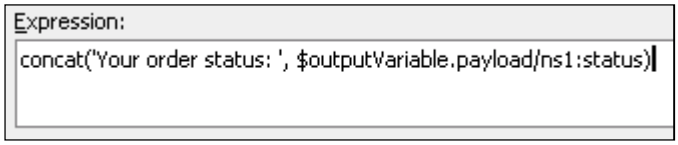
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

d. To specify the To field, use Expression Builder to reference the `Invoke_Lookup` Output variable `email` field.



e. To specify the Subject field, use Expression Builder to construct the value `concat('Your order status: ',$outputVariable.payload/ns1:status)`.

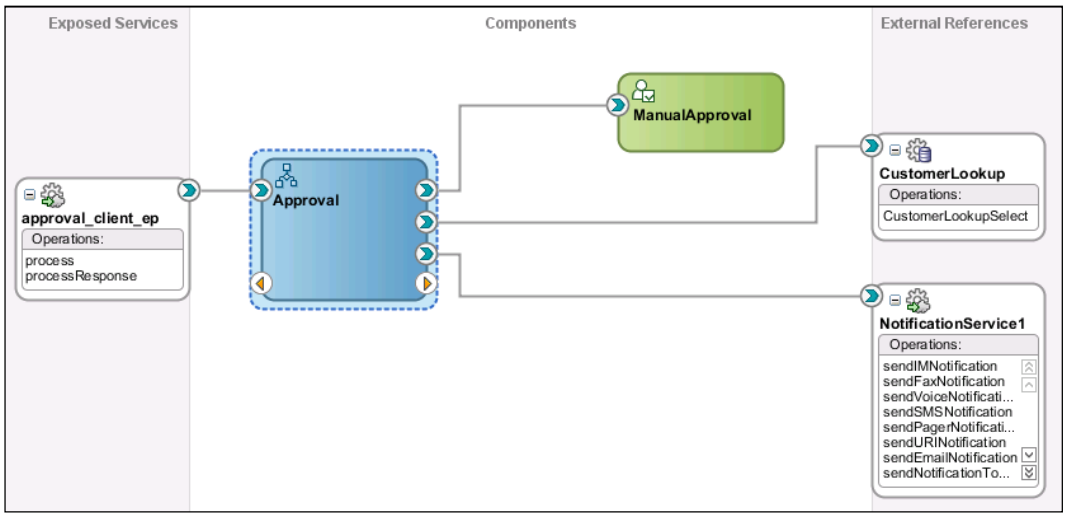


f. To specify the Body field, you can use a mix of text and XPath expressions. XPath expressions must be enclosed with `<%` and `%>` begin and end tags. The body of the email can be clear text or HTML. For this practice, it is recommended that you copy and paste the following example from the file `/home/oracle/labs/files/xml/email-body.xml`.

```
concat('Your order status is
',$outputVariable.payload/ns1:status,'. Thank you for shopping
with us. For more information contact customer support on 1-800-
MUSICFANS.')
```

g. Save your work and close the BPEL editor.

9. Verify your assembly model.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 9-8: Redeploying and Testing the Application

Overview

In this practice, you redeploy the `Approval` and `ApprovalTaskForm` applications. You repeat the test of the applications, including the manual approval of an order and the sending and receipt of email to **both the approver and the end customer**.

Assumptions

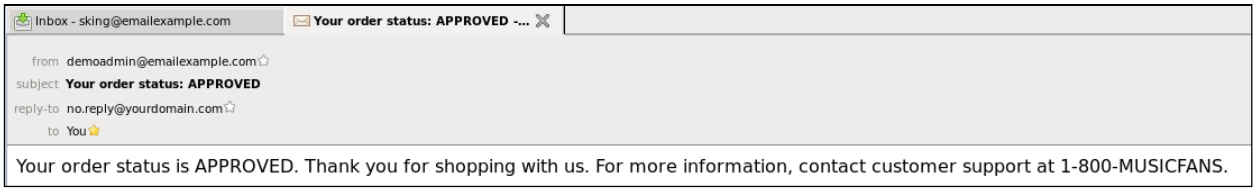
This practice assumes that you have completed all work up to this point successfully.

Tasks

Undeploying the Application

1. Complete all steps in Practice 9-6 to redeploy and test the application.
2. Return to the email program. Select the user `sking` and click the toolbar button “Get Mail.”

   Note: It may take a few moments for the email to arrive in the Inbox.
3. Verify that the customer email was sent correctly.



4. When you have finished, you can close the various editors that are open within JDeveloper.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 10: Sharing Functionality in Oracle SOA Suite

Chapter 10

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 10: Overview

Practices Overview

In this practice, you first import a collection of templates into the SOA Design Time Repository. You create a project, and use a BPEL component template to accelerate development of the project. You then modify the project. These modifications include the creation and calling of a BPEL in-line subprocess. You then test the finished project.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 10-1 Using a Component Template

Overview

In this section, you import the contents of a previously exported Design Time Repository. These contents are in a `.jar` file and they include several project and component templates. In this and subsequent practices, you use these templates to accelerate the building of larger projects.

Assumptions

This practice assumes the following:

- You have completed Practice 5 for the lesson titled “Using JMS and JDBC Adapters” by using *exact* project and component names as specified in that practice, and that the `ValidateCreditCard` application is still deployed to the application server.
- You have completed Practice 9 for the lesson titled “Implementing Human Workflow and Notifications” by using *exact* project and component names as specified in that practice, and that the `Approval` application is still deployed to the application server.

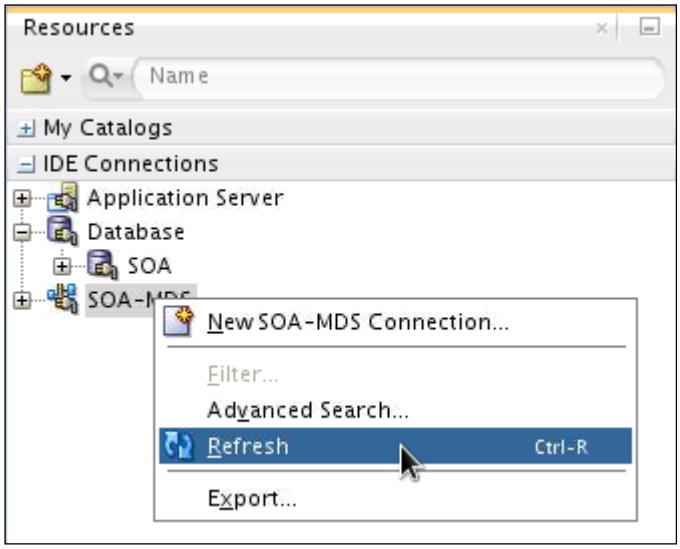
If your environment does not meet these assumptions, deploy the working solutions for the missing practices as described in `/home/oracle/labs/soln/restoringSolutions.pdf`.

Tasks

Importing Templates to the SOA Design Time Repository

1. Import templates from the `.jar` file to the Design Time Repository.

   a. In JDeveloper, in the IDE Connections section of the Resources tab, right-click SOA-MDA and select Refresh.

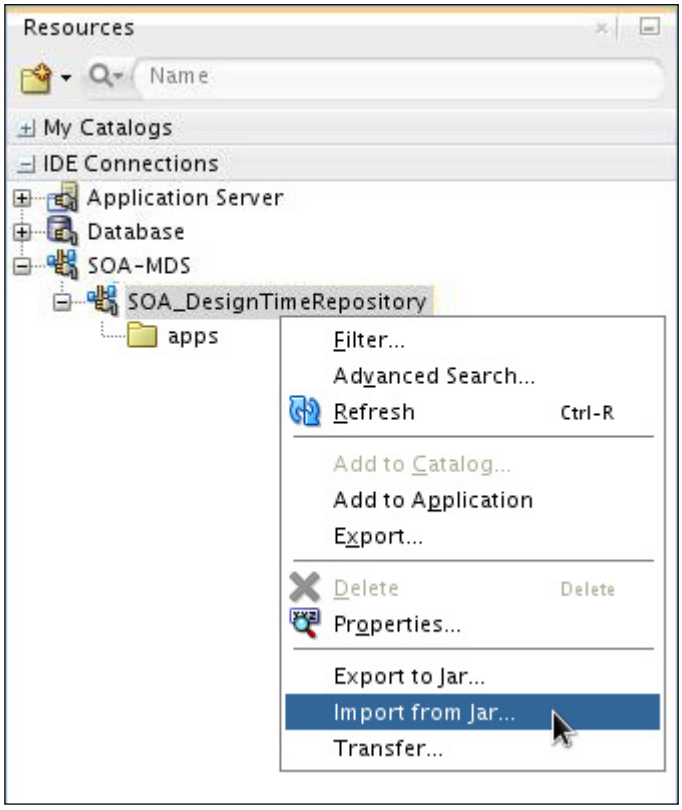


   b. Expand the SOA-MDS folder and right-click SOA_DesignTimeRepository.

Oracle Internal & Oracle Academy Use Only

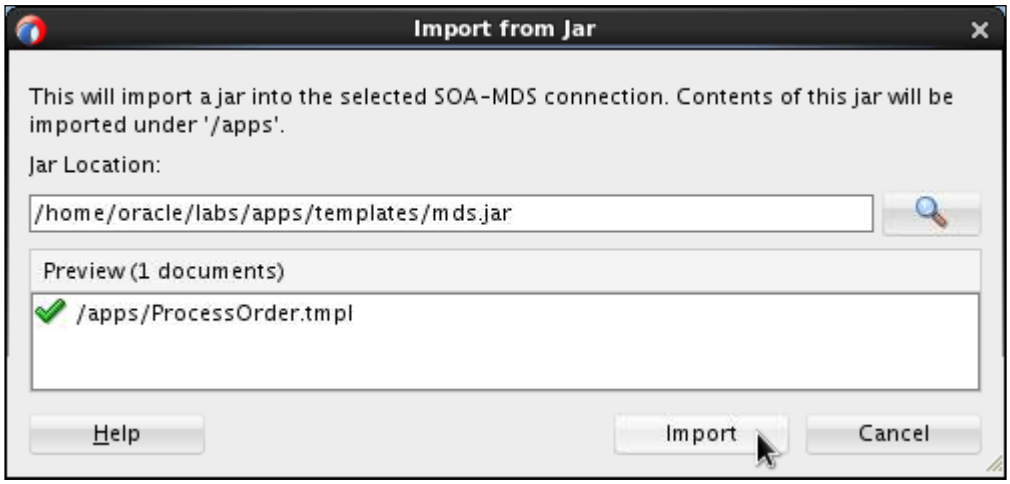
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

c. In the context menu, select Import From Jar.

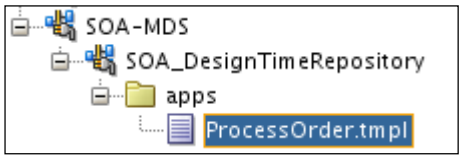


d. Navigate to the Jar location `/home/oracle/labs/apps/templates/mds.jar` and click Open.

e. In the Import from Jar window, click Import.



f. In the IDE Connections pane, expand the /apps folder and verify successful import of the specified template.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Creating a Project

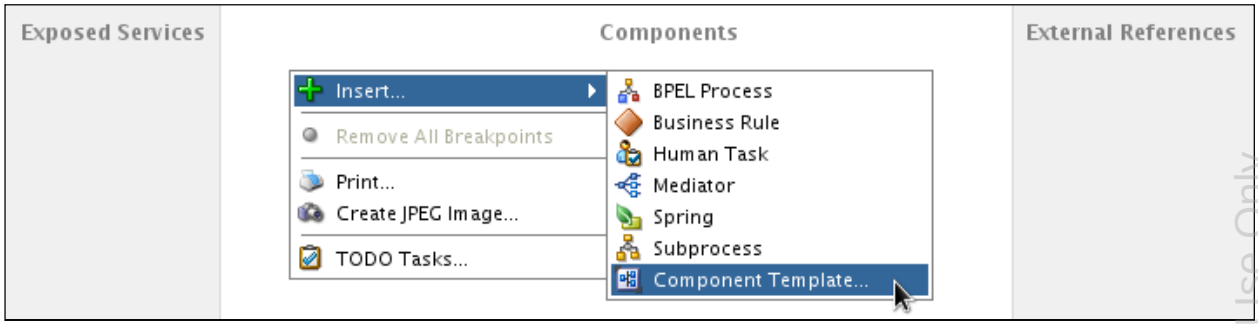
2. In the BPELProjects application, create a new empty SOA project named `OrderFlow`.

   The `composite.xml` file is created and the overview window is opened for editing.

Adding a BPEL Component from a Template

3. Add a BPEL Component from a template.

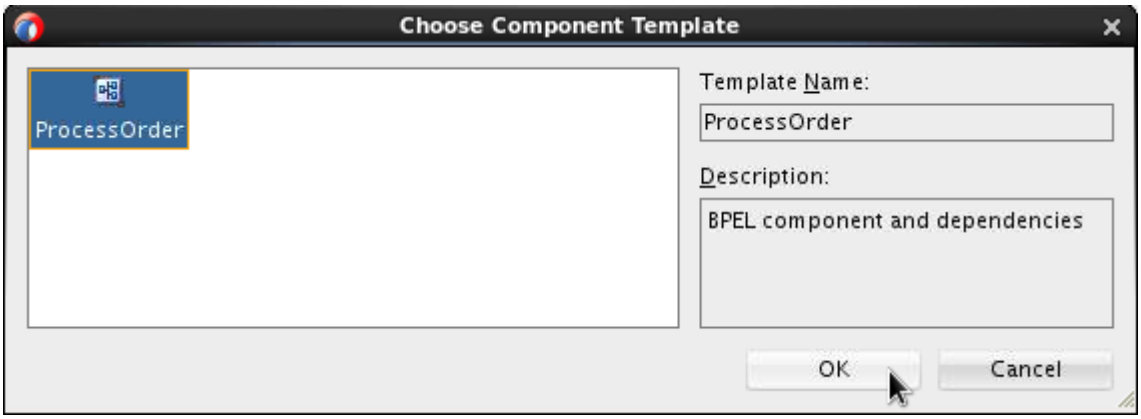
   a. In the component overview window, right-click in the Components swimlane and select Insert > Component Template.



   The Choose Component Template dialog box appears.

   Note: If the Component Template option does not appear, stop the Integrated WebLogic Server. Close and reopen JDeveloper. Restart the Integrated WebLogic Server. You can resume your work while it is restarting.

   b. Select ProcessOrder and click OK.

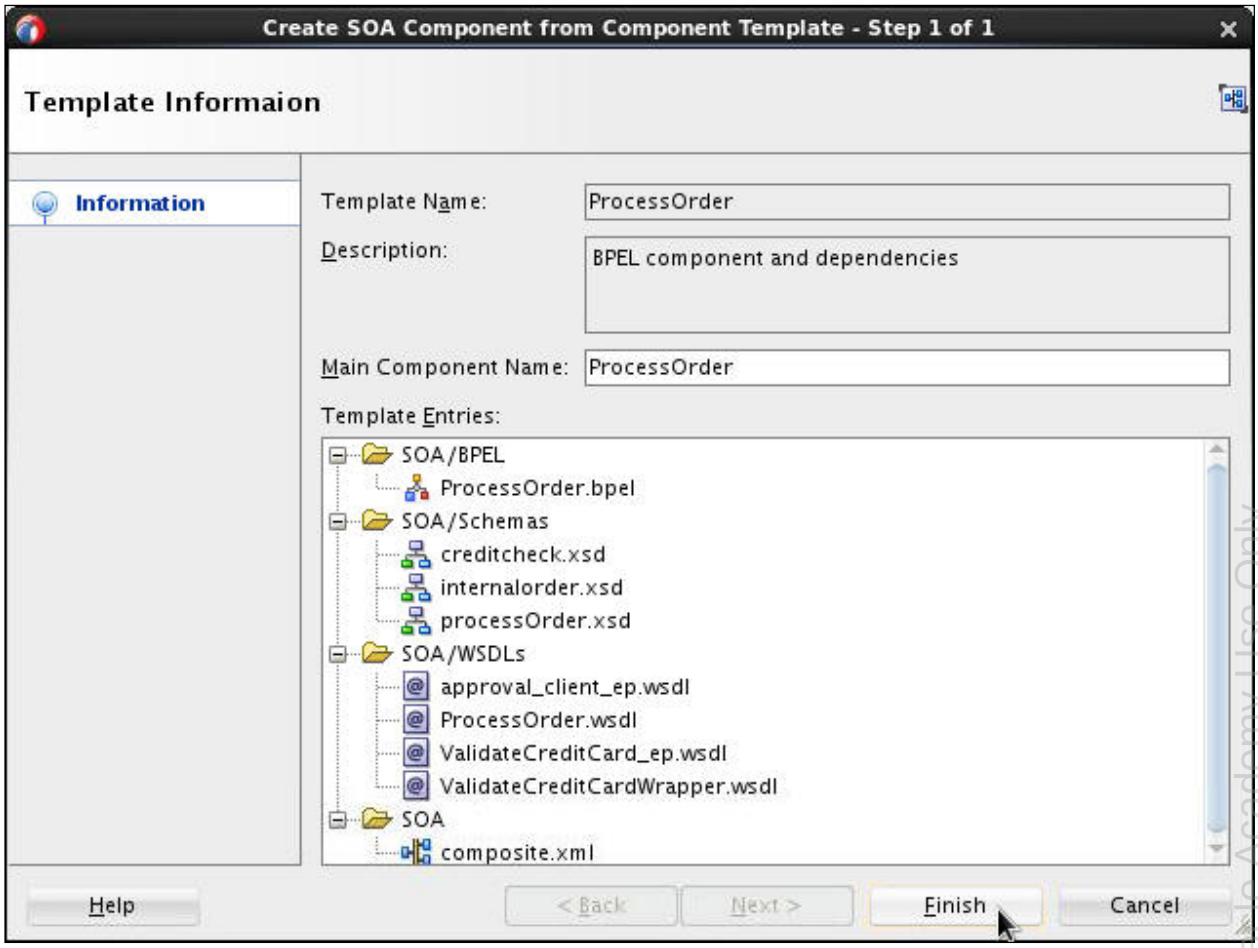


   The Create SOA Component from Component Template dialog box appears.

   c. Examine the template entries and click Finish.

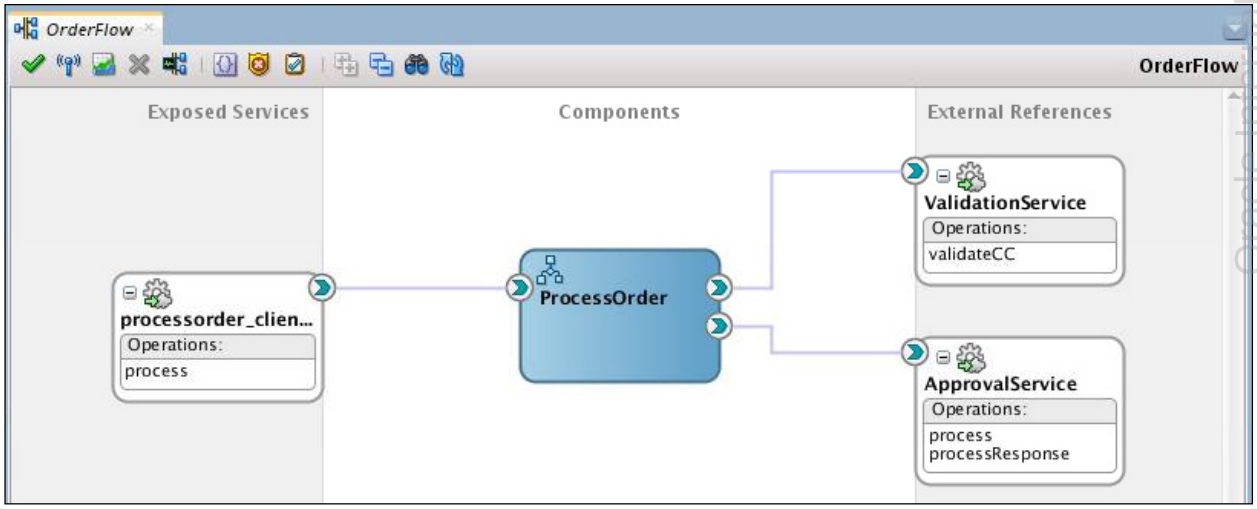
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



The composite overview window is updated.

d. Verify your project and save your work.



Note: Expect many error messages as you save. Recall that JDeveloper performs a validation check each time you save. These errors indicate that the project is not fully configured, which is expected at this point.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Creating and Configuring a Database Adapter

4. In the assembly model (`composite.xml`) for the `OrderFlow` project, add a Database adapter to the External References swimlane.

   The Adapter Configuration wizard opens.

5. To configure the Database adapter, complete the following steps:

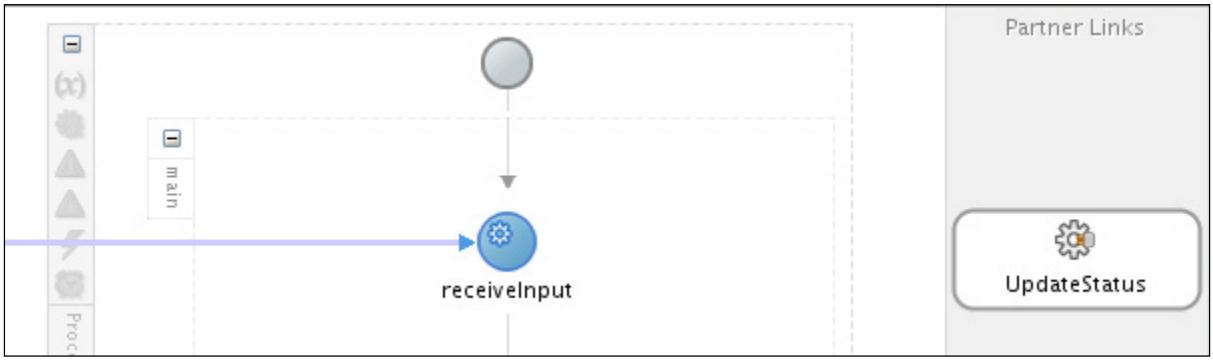
Step	Screen	Choices or Values
a.	Database Adapter Reference	Name: `UpdateStatus`
b.	Service Connection	Verify the following settings: • Connection: **`SOA`** • User Name: **`soainfra`** • Driver: **`org.apache.derby.jdbc.clientDriver`** • Connect String: **`jdbc:derby//localhost:1527/soainfra`** • JNDI Name: **`eis/DB/SOA`**
c.	Operation Type	Select the "Perform an operation on a Table" option. Ensure that the Update Only check box is selected. Deselect the other operations.
d.	Select Table	Click Import Tables.
e.	Import Tables	Set the BCA_% filter. Click Query.
f.	Import Tables	Move the `BCA_ORDERS` table from the Available list to the Selected list. Click OK.
g.	Select Table	Verify that the `BCA_ORDERS` table is listed.
h.	Relationships	Click Next.
i.	Attribute Filtering	Deselect all attributes except `status`.
j.	Advanced Options	Click Next.
k.	JCA Endpoint Properties	Click Next.
l.	Finish	Click Finish.

6. In the Composite Editor, wire the ProcessOrder BPEL process to the UpdateStatus external reference.
7. Open the BPEL process for editing.

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

8. Verify that UpdateStatus is shown in the Partner Links column and save your work.



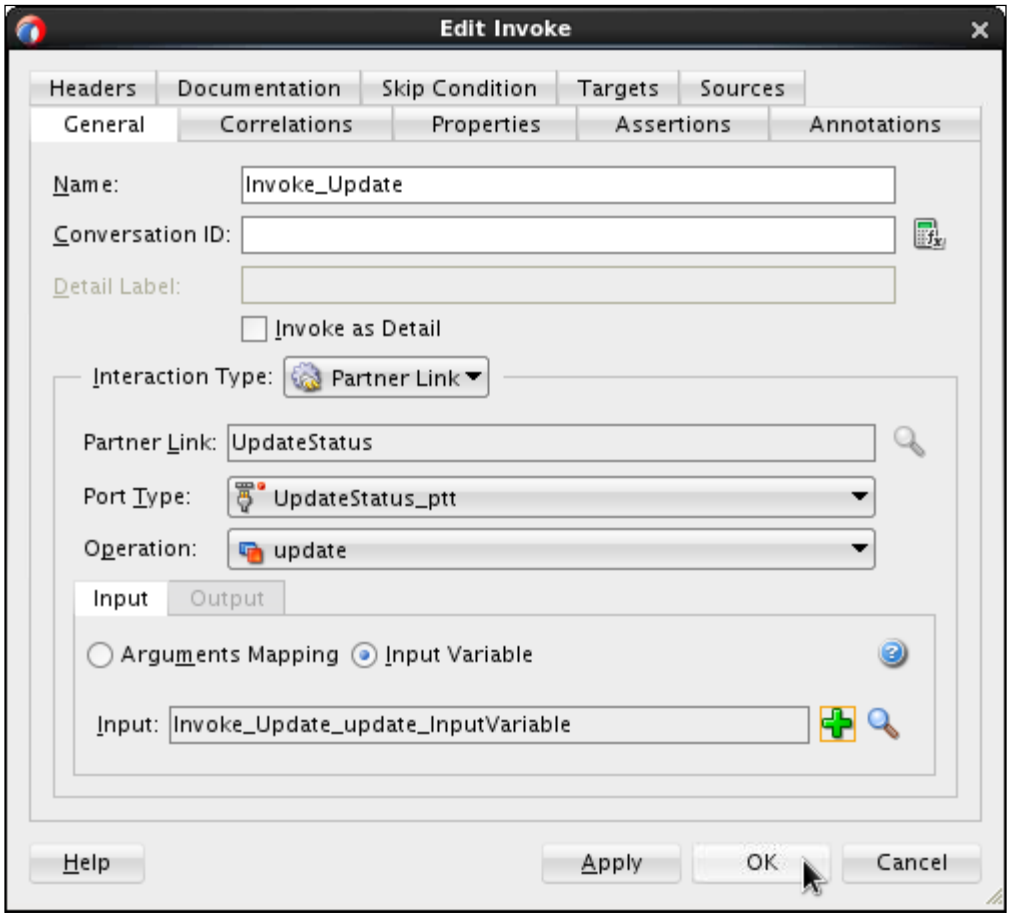
Modifying the BPEL Process

9. Add a new Scope activity to the BPEL process, just below the `GetValidation` activity.
10. Create and configure an Invoke activity.
    a. Add an Invoke activity to the new Scope.
    b. To specify the partner link, drag and drop the Invoke activity to the partner link UpdateStatus.

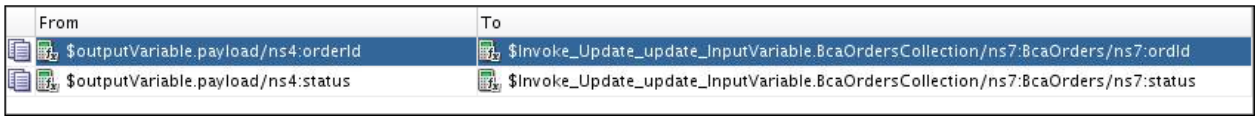
       The Edit Invoke dialog box appears.
    c. Name the new activity `Invoke_Update`.
    d. Create a new Input variable.
    e. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



11. Create and configure an Assign activity.
    a. Add an Assign activity within the Scope activity, just above `Invoke_Update`.
    b. Name the activity `SetUpdate`.
    c. Right-click the new activity and select Edit.
    d. Map the `outputVariable` element `orderId` to the `ordId` element of `Invoke_Update_update_inputVariable`.
    e. Map the `outputVariable` element `status` to the `status` element of `Invoke_Update_update_inputVariable`.
    f. Verify and save your work.



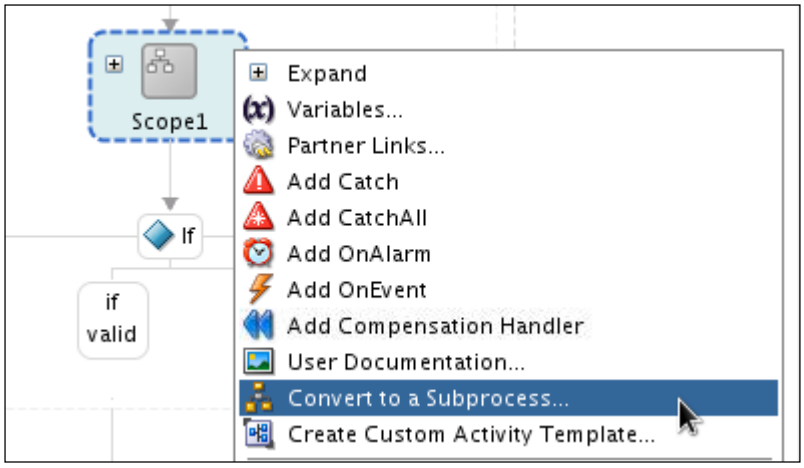
Converting the Scope to a Subprocess

Updating the order status is something that will need to be done multiple times in this business process. For this reason, it makes sense to convert the update function (as defined in the scope activity) to a subprocess.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

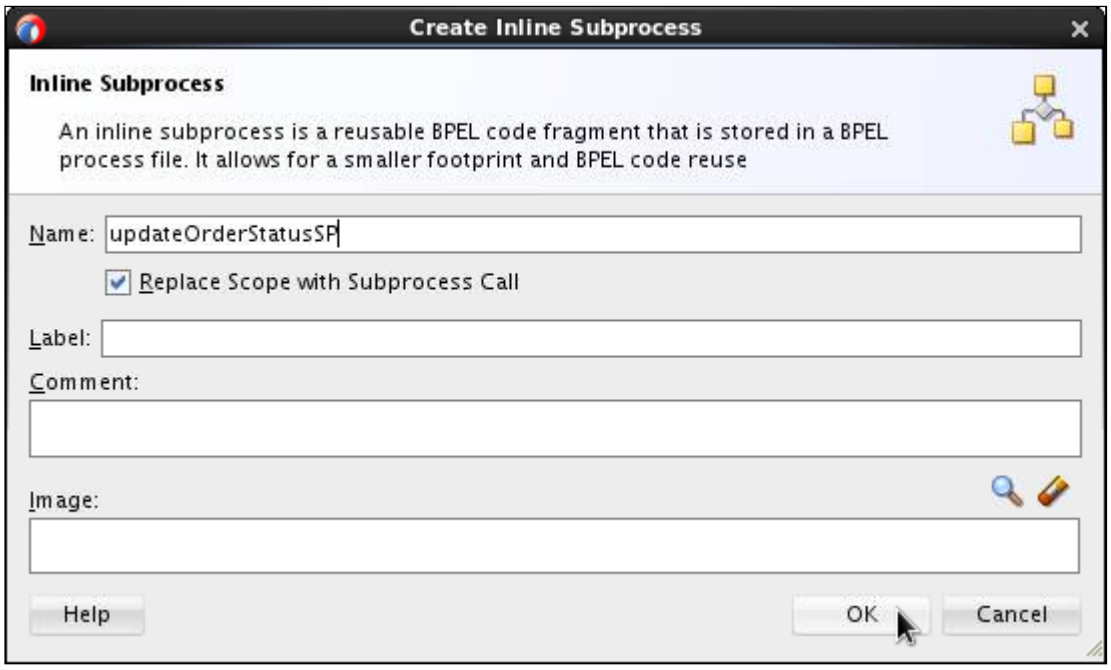
12. Convert the `Scope1` scope to a subprocess.
    a. Collapse the scope and right-click the scope icon.
    b. Select Convert to a Subprocess.



The Create Inline Subprocess dialog box appears.

    c. Name the subprocess updateOrderStatusSP.

Leave the Replace Scope with Subprocess Call check box selected. This automatically replaces the current scope with a Call activity.

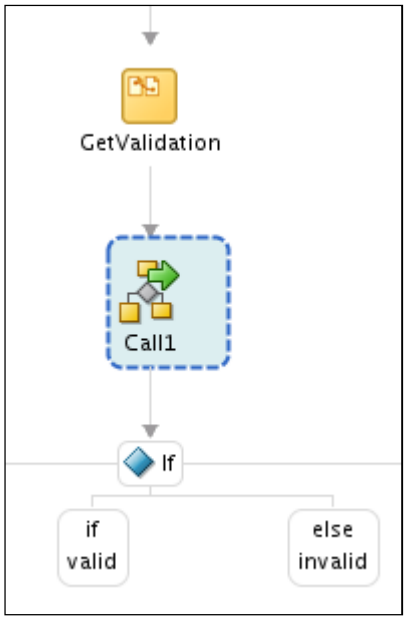


    d. Click OK.

A call activity is displayed in the process.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



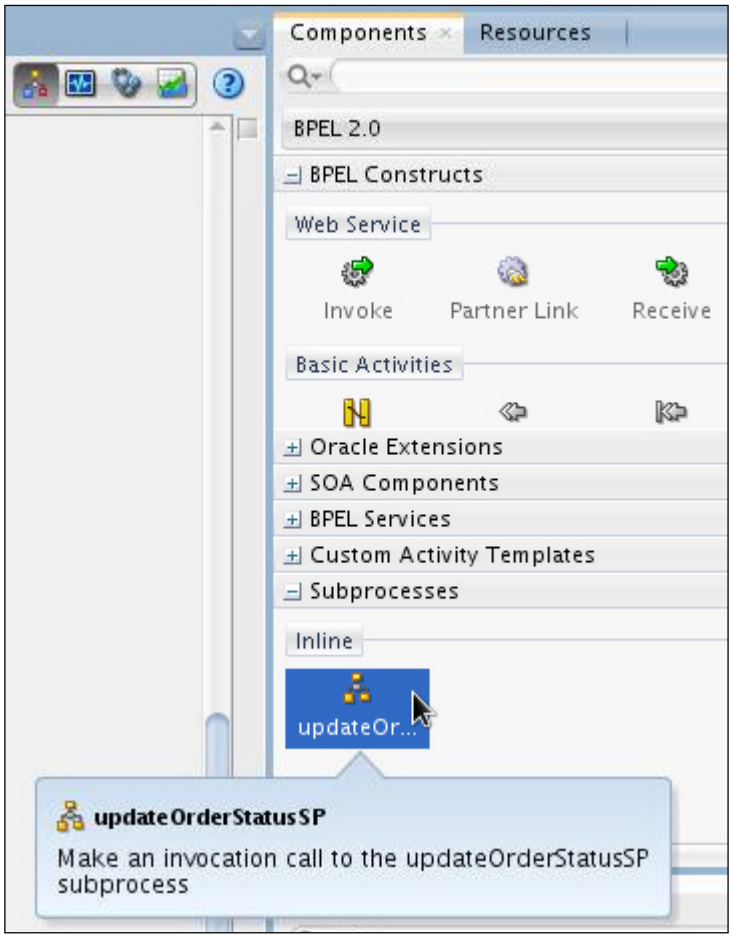
Note: After restarting JDeveloper, the subprocess is also displayed in the Palette under Subprocesses. You can see and edit the definition of the subprocess when switching from the Main Process to the subprocess on top of the BPEL editor. All partner links that are available in the Main Process are displayed, in case you want to add another Invoke. Changes to the subprocess are reflected in every call.

13. Add a call activity after ValidationService.
    a. Expand the Subprocesses section of the Component Palette.

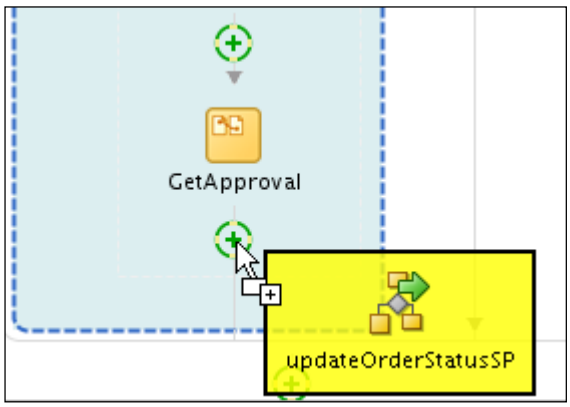
       The updateOrderStatusSP subprocess is displayed.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



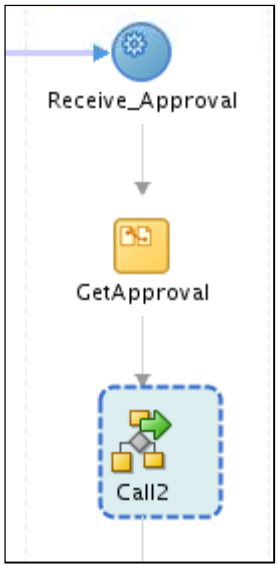
b. Drag the updateOrderStatusSP subprocess from the Component Palette to the BPEL process, within the If branch of the If activity, just below the GetApproval activity.



A new call activity is added to the process. That is all that needs to happen to invoke the subprocess and have the order status updated to whatever value is returned by the call to the approval process (APPROVED or REJECTED).

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



14. Save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 10-2: Deploying and Testing the `OrderFlow` Application

Overview

In this practice, you deploy and test the `OrderFlow` composite application.

Assumptions

This practice assumes that you have completed all work in Practice 10-1 successfully.

Tasks

Deploying the Composite Application

1. Deploy the `OrderFlow` application to IntegratedWebLogicServer.

   Remember to select the “Overwrite any existing composites with the same revision ID” check box.

Testing the Composite Application

In this section, you submit an order that will first have the credit card validated, and if the card is valid, will have the order manually approved. As employee `jcooper`, you log in to the Worklist application to approve the order.

Before you do that, you open a SQL script, the Worklist application, and the Enterprise Manager test window.

2. In JDeveloper, open the `/home/oracle/labs/scripts/SQL/JDBlist_orders.sql` file.
3. In a web browser, open the Worklist application:
   a. Access the Oracle BPM Worklist application (http://soa12c.example.com:7101/integration/worklistapp).
   b. Log in to the application as `jcooper` with the password `welcome1`.
4. On a second web browser tab, open the Oracle Enterprise Manager test page.
   a. Access Enterprise Manager at http://localhost:7101/em.
   b. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “OrderFlow [1.0]” link.
   c. On the “OrderFlow [1.0]” home page, click Test.
   d. On the Test Web Service page, scroll down to the Input Arguments section to the Request tab.
   e. Use the Browse button to replace the sample XML data with the contents of the `/home/oracle/labs/files/xml_in/processOrder.xml` file.
5. Launch the test.
   a. In JDeveloper, with the JDBlist_orders.sql tab selected, click Run Script.

	ORD_ID	CUST_ID	PAY_METHOD	SHIP_METHOD	CARRIER	CARD_TYPE	CARD_NUMBER	TOTAL_PRICE	DATE_ORDERED	DATE_SHIPPED	STATUS
1	1001	1	credit	two_day	(null)	ACARD	5678-5678-5678-5678	6500.00	2014-08-30	(null)	waiting
2	1002	2	credit	next_day	(null)	VCARD	1234-1234-1234-1234	4190.00	2014-08-30	(null)	waiting
3	1003	1	credit	five_day	(null)	ACARD	5678-5678-5678-5678	12000.00	2014-08-30	(null)	waiting

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

b. In Enterprise Manager, click Test Web Service.

c. Switch to the Worklist application tab.

d. Click the Refresh icon.

e. Click the Manual Approval task entry.

   The order details are displayed in the lower-right pane.

f. Click Approve.

g. In JDeveloper, with the JDBlist_orders.sql tab selected, click Run Script.

	ORD_ID	CUST_ID	PAY_METHOD	SHIP_METHOD	CARRIER	CARD_TYPE	CARD_NUMBER	TOTAL_PRICE	DATE_ORDERED	DATE_SHIPPED	STATUS
1	1001	1	credit	two_day	(null)	ACARD	5678-5678-5678-5678	6500.00	2014-08-30	(null)	APPROVED
2	1002	2	credit	next_day	(null)	VCARD	1234-1234-1234-1234	4190.00	2014-08-30	(null)	waiting
3	1003	1	credit	five_day	(null)	ACARD	5678-5678-5678-5678	12000.00	2014-08-30	(null)	waiting

h. Return to the Enterprise Manager tab.

i. On the Response tab, click the Launch Flow Trace link.

j. Examine the audit trail for ProcessOrder.

k. View each of the payload options and observe the changes in the Status field.

Note: If it took more than 30 seconds to complete steps b–f, your results in step g will vary. The status field will read ‘VALID.’ Enterprise Manager will report a test failure because the test will have timed out. For the purpose of this course, you can ignore this and rerun the test.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 11: Using the REST Adapter

Chapter 11

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 11: Overview

Practice Overview

In this practice, you add a REST interface to the HelloBPEL project. You then create a new project that calls HelloBPEL via the new interface.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 11-1: Exposing a REST Service

Overview

In this practice, you open the HelloBPEL project and modify it by adding a REST interface.

Assumptions

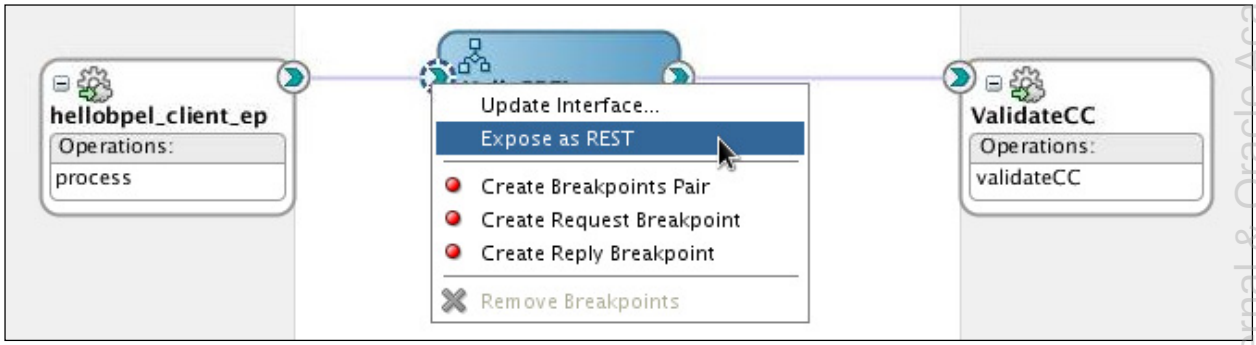
This practice assumes the following:

- You have built and deployed the CCValidate project that was described in Practice 5 for the lesson titled “Using JMS and JDBC Adapters.”
- You have built and deployed the HelloBPEL project that was described in Practice 6 for the lesson titled “Introduction to BPEL.”
- Both these projects were successfully tested and are still deployed.

Tasks

Defining a REST Interface

1. In the BPELProjects application, open the `composite.xml` file for the HelloBPEL project.
2. Define a REST interface for the HelloBPEL project.
   a. Right-click the BPEL process interface and select Expose as REST.



   The Create Rest Binding window is displayed.

   b. Set the following fields:
      - Name: `helloBPEL_RS`
      - Description: REST interface

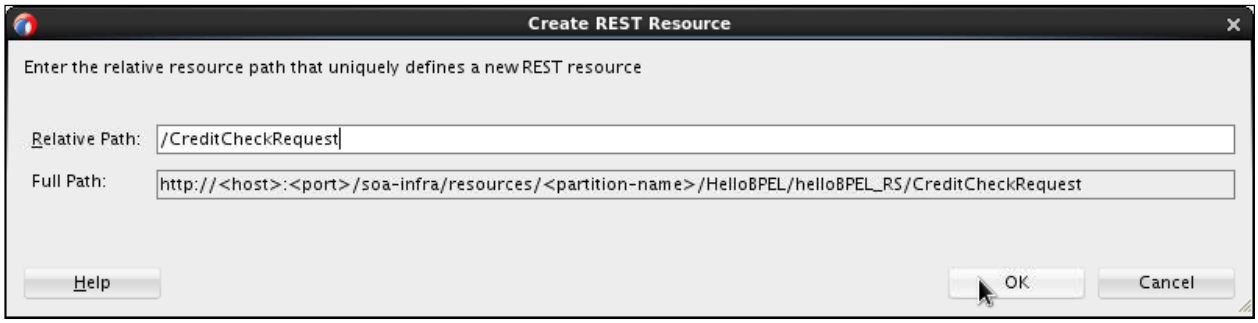
3. Add a resource.
   a. Click the Add a Resource icon.

      The Create REST Resource dialog box is displayed.

   b. Insert `/CreditCheckRequest` as a relative path to the resource.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

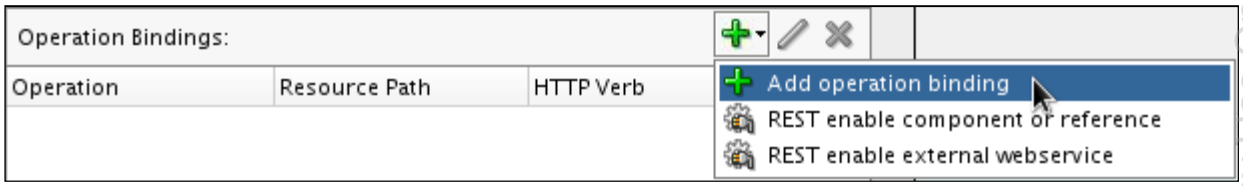


c. Click OK.

d. Select the default resource (/) and click the Delete (X) button.

4. Add an operation binding.

a. Click the Add (+) icon and select “Add operation binding.”



The REST Operation Binding dialog box is displayed.

b. Set the following fields:

- Operation: **validateCard**
- Resource: **/CreditCheckRequest**
- HTTP Verb: **POST**

c. Define the request.

1) Click the “Browse for schema file” icon.

2) In the Type Chooser, select the Request element of the file **`creditcheck.xsd`**.

3) Click OK.

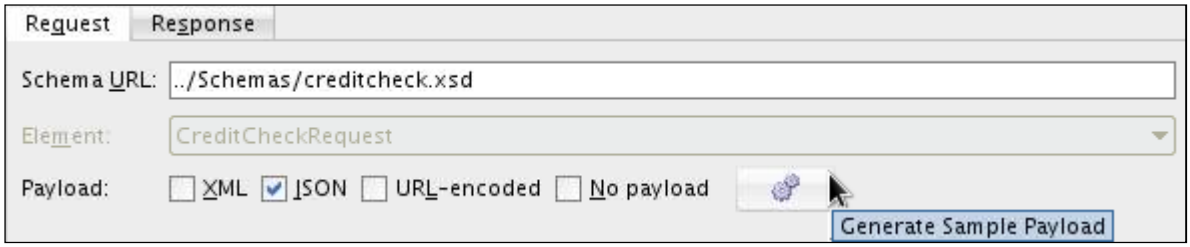
You are returned to the REST Operation Binding dialog box.

5. Generate a sample payload. (You use this to test the application later.)

a. Deselect the XML check box next to Payload.

b. Select the JSON check box.

c. Click the Generate Sample Payload button.



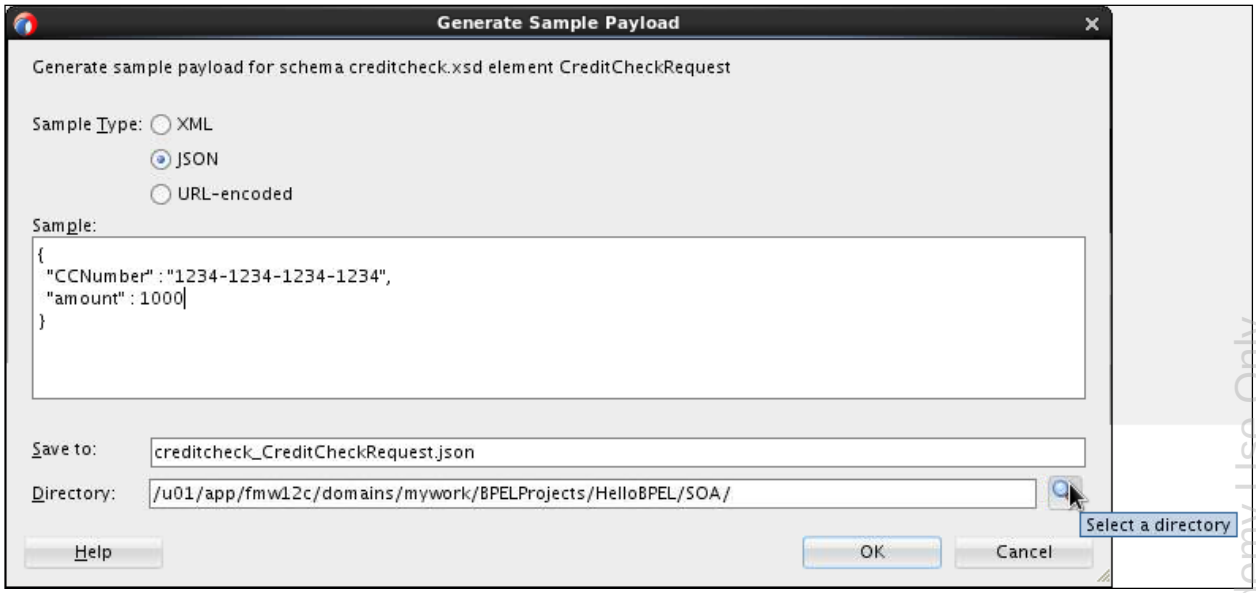
d. In the Generate Sample Payload window, select the JSON option.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

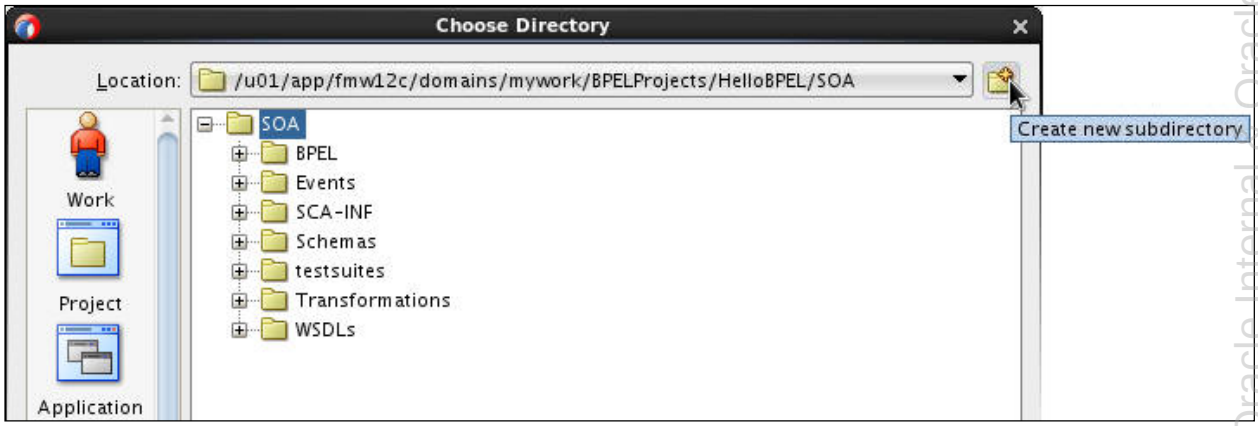
e. Modify the sample values:
   - CCNumber “1234-1234-1234-1234”
   - amount 1000

f. Click the “Select a directory” icon.



g. Navigate to the SOA directory.

h. Click the “Create new subdirectory” icon.



i. Enter `SampleInput` as the subdirectory name.

j. Click OK.

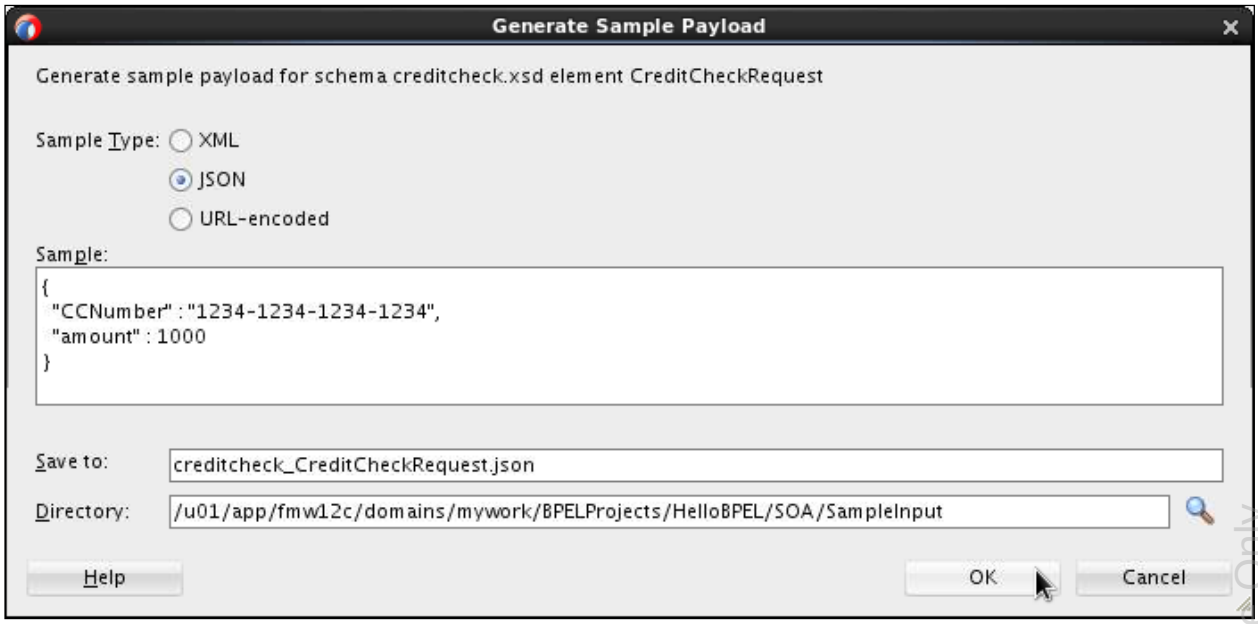
k. Highlight the newly created directory and click Select.

   You are returned to the Generate Sample Payload window.

l. Verify your work and click OK.

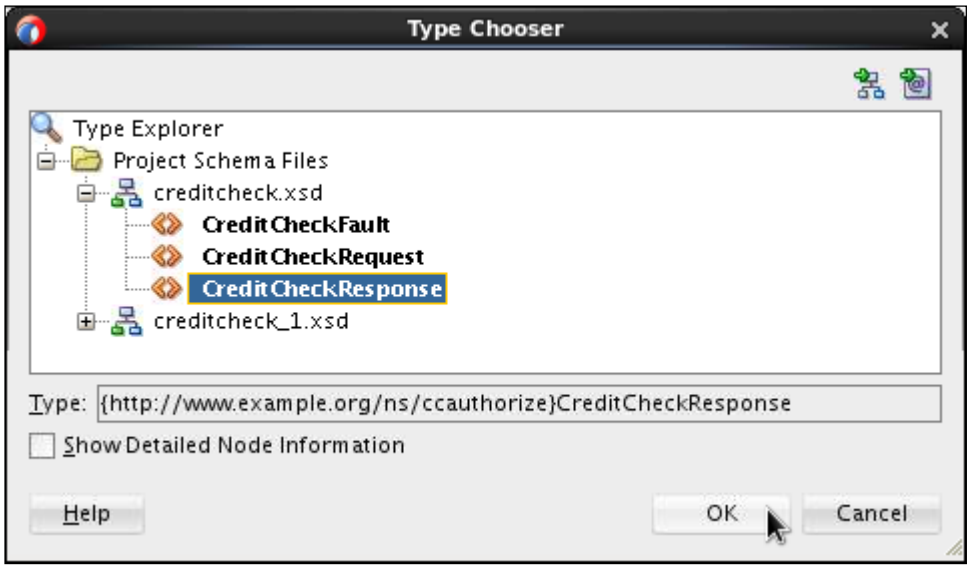
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



You are returned to the REST Operation Binding window.

6. Define the response of the REST service.
   a. Click the Response tab.
   b. Deselect No Payload.
   c. Specify the Payload type JSON.
   d. Click the “Browse for schema file” icon and select creditcheck.xsd > CreditCheckResponse.

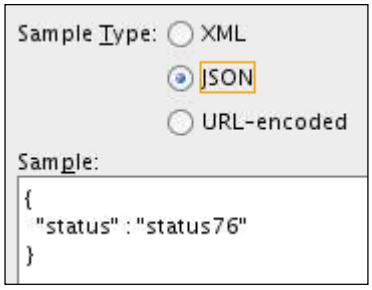


   e. Click OK.
   f. Click Generate Sample Payload.
   g. Select the SampleType JSON.

Oracle Internal & Oracle Academy Use Only

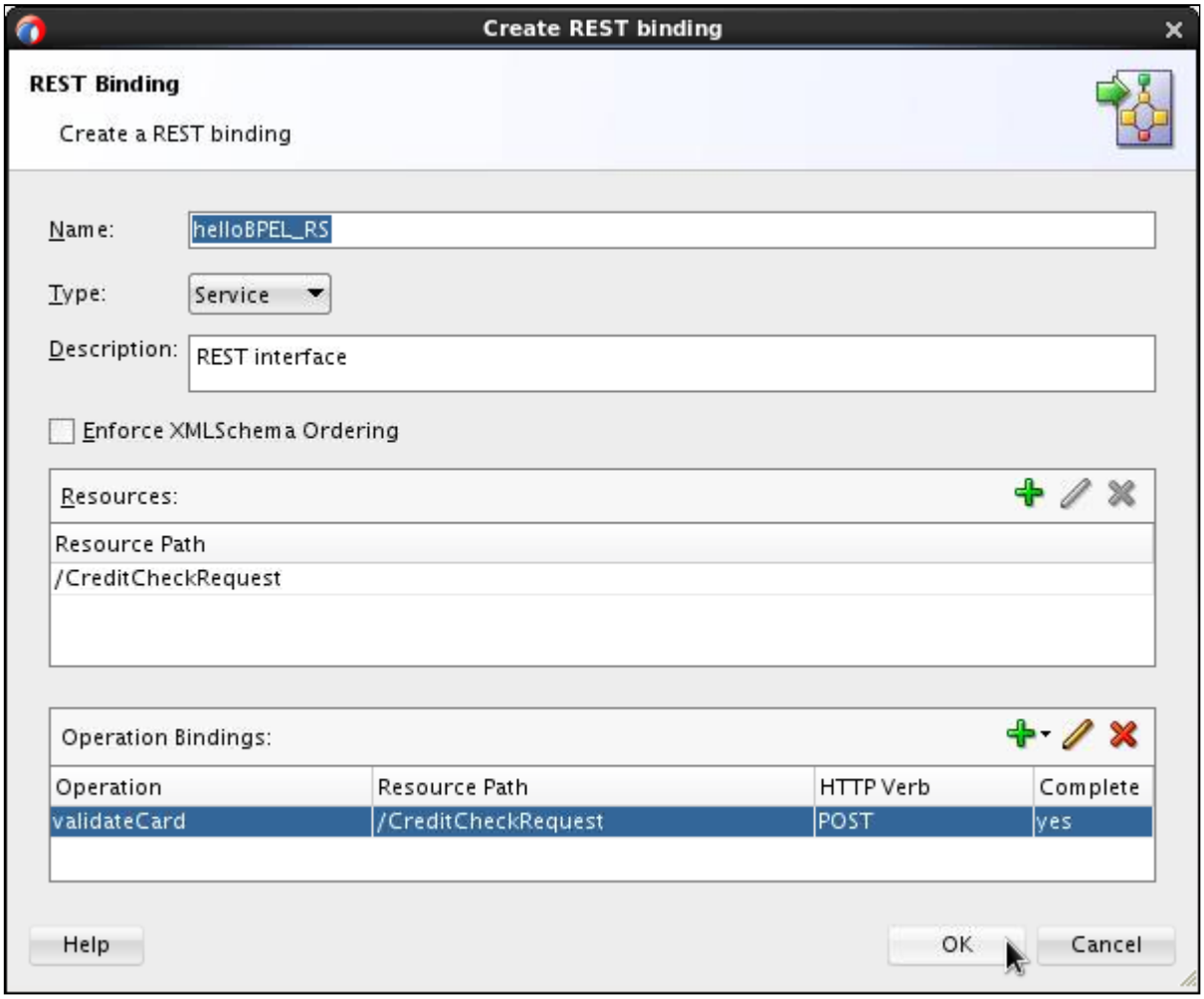
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

h. Verify that your sample is similar to the following screenshot (the sample value may vary):



i. Click OK to close the REST Operation Binding window.

7. Verify your work and click OK to complete the definition of the REST binding.



Adding a Mediator

8. Create and configure a Mediator component.

a. Add a Mediator to the Component column of the composite overview window.

The Create Mediator window opens.

b. Set the Name to `RouteData`.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

c. Accept the “Define Interface Later” template option.

d. Click OK.

9. Wire the Mediator component to the REST interface (helloBPEL_RS) and to the HelloBPEL process.

10. Configure the Request transformation in the Mediator.

a. Right-click the Mediator icon and select Edit.

The `RouteData.mplan` window opens.

b. Expand the Static Routing rule that is created by the wire.

c. In the upper (request) section of the Static Routing rule, click the “Select an existing mapper file or create a new one” icon.

d. In the Request Transformation Map dialog box, click the Create Mapping icon.

The Create Transformation Map dialog box is displayed.

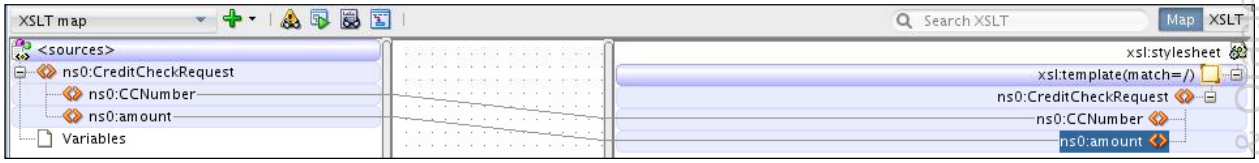
e. Accept the default file name that is supplied. Click OK.

f. Click OK to close the Request Transformation Map dialog box.

The XSLT Mapper window opens.

g. Map the `CCNumber` element in the Source column to the `CCNumber` element in the target column.

h. Map the `amount` element in the Source column to the `amount` element in the target column.



i. Save your work.

j. Close the XSLT editor and return to the `RouteData.mplan` window.

11. Configure the Reply transformation.

a. In the Synchronous Reply section of the routing rule, click the “Select an existing mapper file or create a new one” icon.

b. In the Reply Transformation Map dialog box, click the Create Mapping icon.

The Create Transformation Map dialog box is displayed.

c. Accept the default file name that is supplied. Click OK.

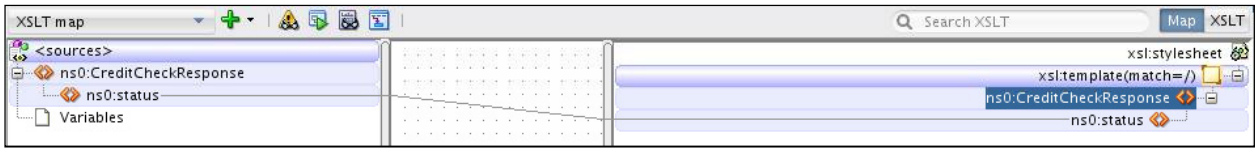
d. Click OK to close the Reply Transformation Map dialog box.

The XSLT Mapper window opens.

e. Map the status element in the Source column to the status element in the target column.

Oracle Internal & Oracle Academy Use Only

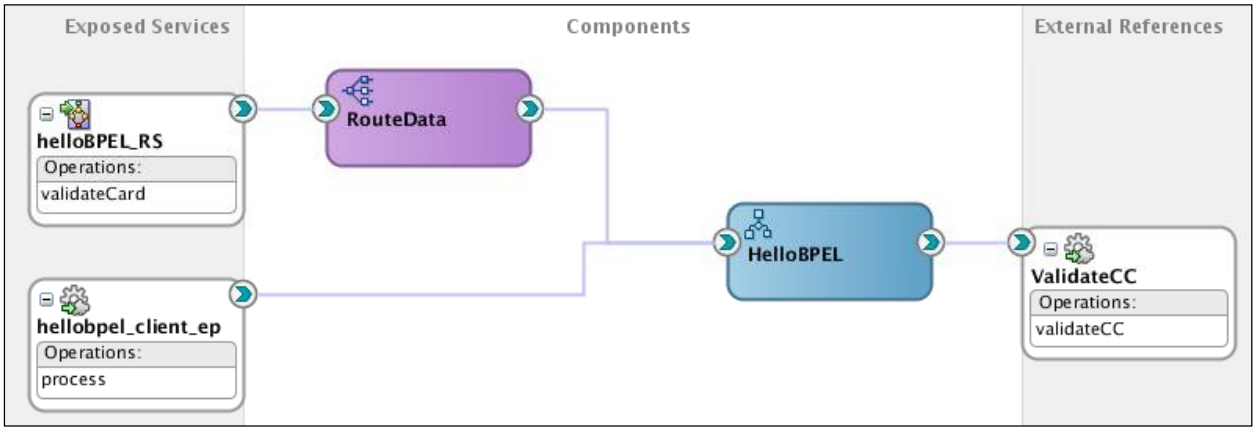
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



f. Save your work.

g. In the JDeveloper window, close the XSLT Mapper and the `RouteData.mplan` windows.

12. Verify that the composite application assembly model resembles the following image:



13. Redeploy the HelloBPEL project to IntegratedWebLogicServer.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 11-2: Creating and Configuring a REST Interface

Overview

In this practice, you create a SOA project that consumes the exposed REST service that you just developed and tested.

Assumptions

This practice assumes that you have completed Practice 11-1 successfully.

Tasks

1. In the BPELProjects application, create a new empty SOA project named CallREST.

   The `composite.xml` file is created and the overview window is opened for editing.

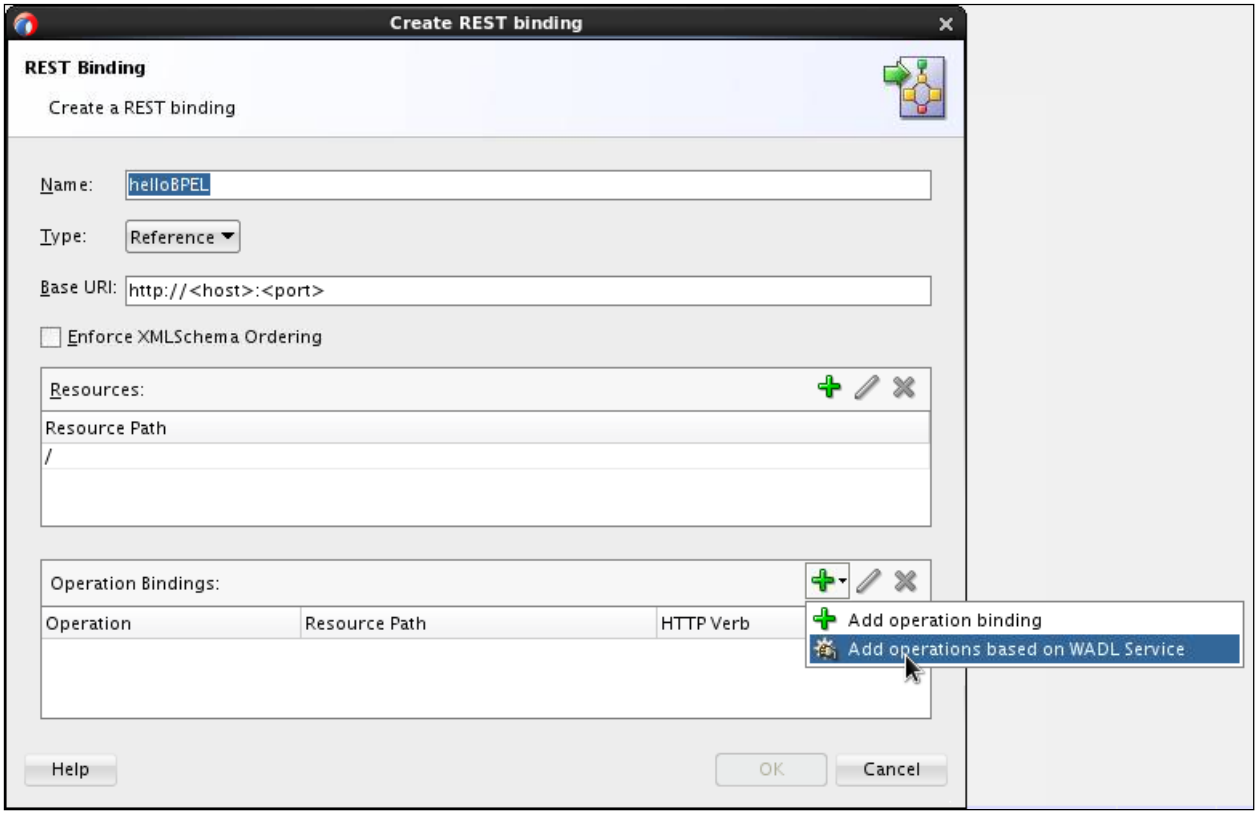
Creating and Configuring a REST Interface

2. Add a REST binding to the External References swimlane.

   The “Create Rest binding” dialog box is displayed.

3. Enter **`helloBPEL`** for the name.

4. Click the Operations Bindings **Add** (+) button and select **Add operations based on WADL Service**.

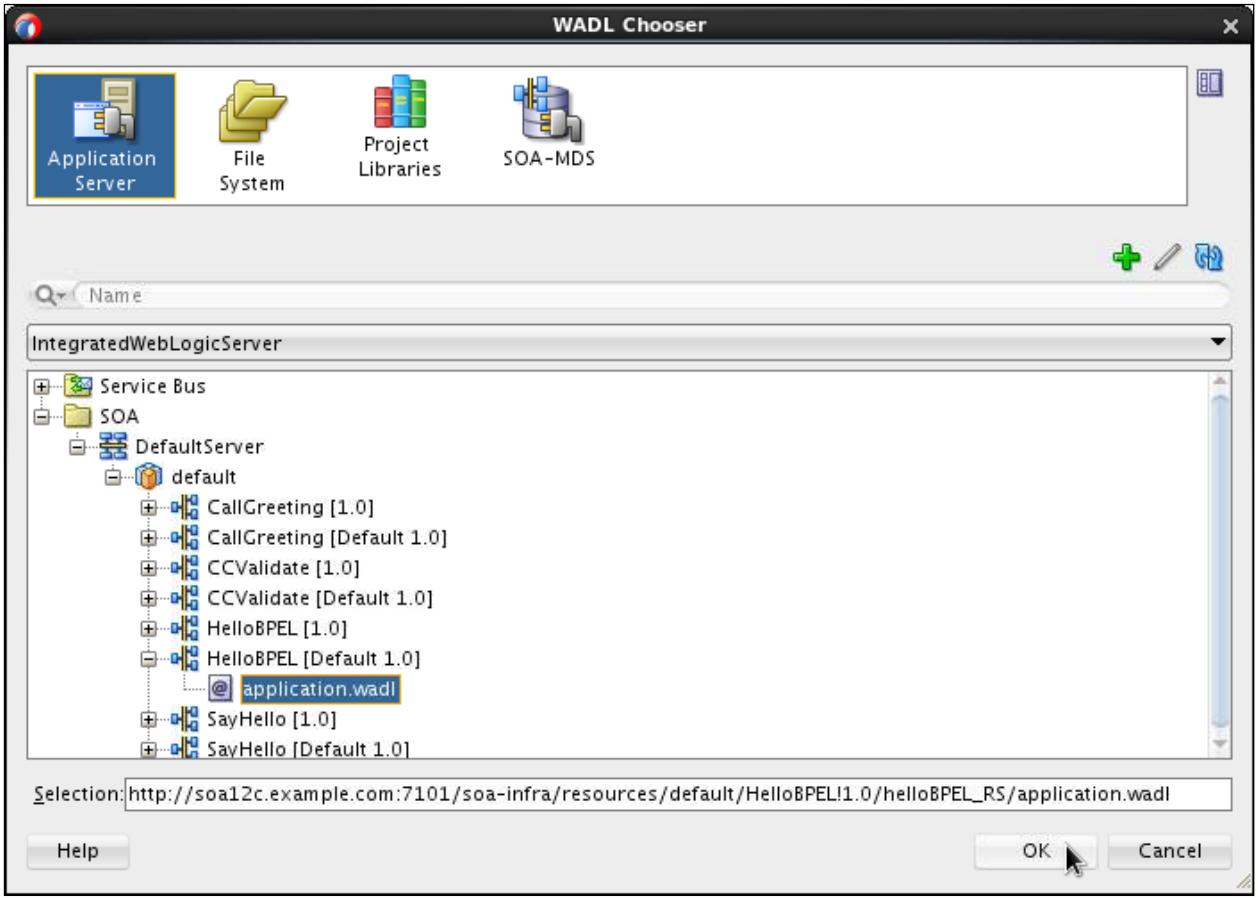


   The WADL Chooser is displayed.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

5. With **Application Server** selected, expand the SOA node and find the **`HelloBPEL`** REST interface.

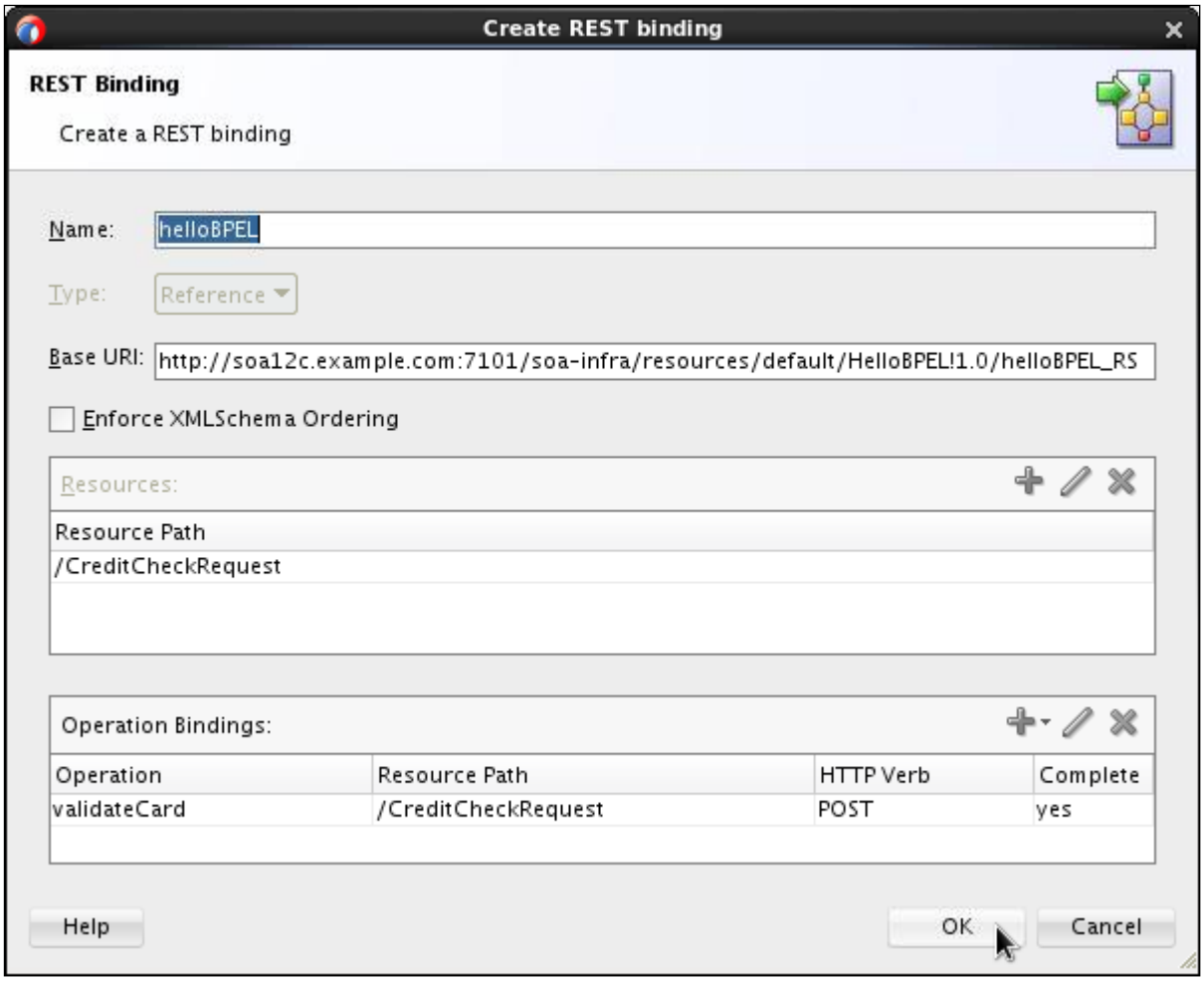


6. Select **`application.wadl`** and click **OK**.

   You are returned to the “Create REST binding” window. The fields are automatically populated based on the WADL file.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



7. Click **OK** to close the "Create REST binding" window.

   The Localize Files dialog box is displayed.

8. Deselect the **Maintain original directory structure for imported files** check box and click OK.

   The definition of the REST binding is completed.

Creating a BPEL Process

In this section, you create a BPEL process that invokes the REST interface that you created.

9. Add a BPEL process to the application.

   The Create BPEL Process dialog box is displayed.

   a. Name: `CallREST`

   b. Template: Synchronous

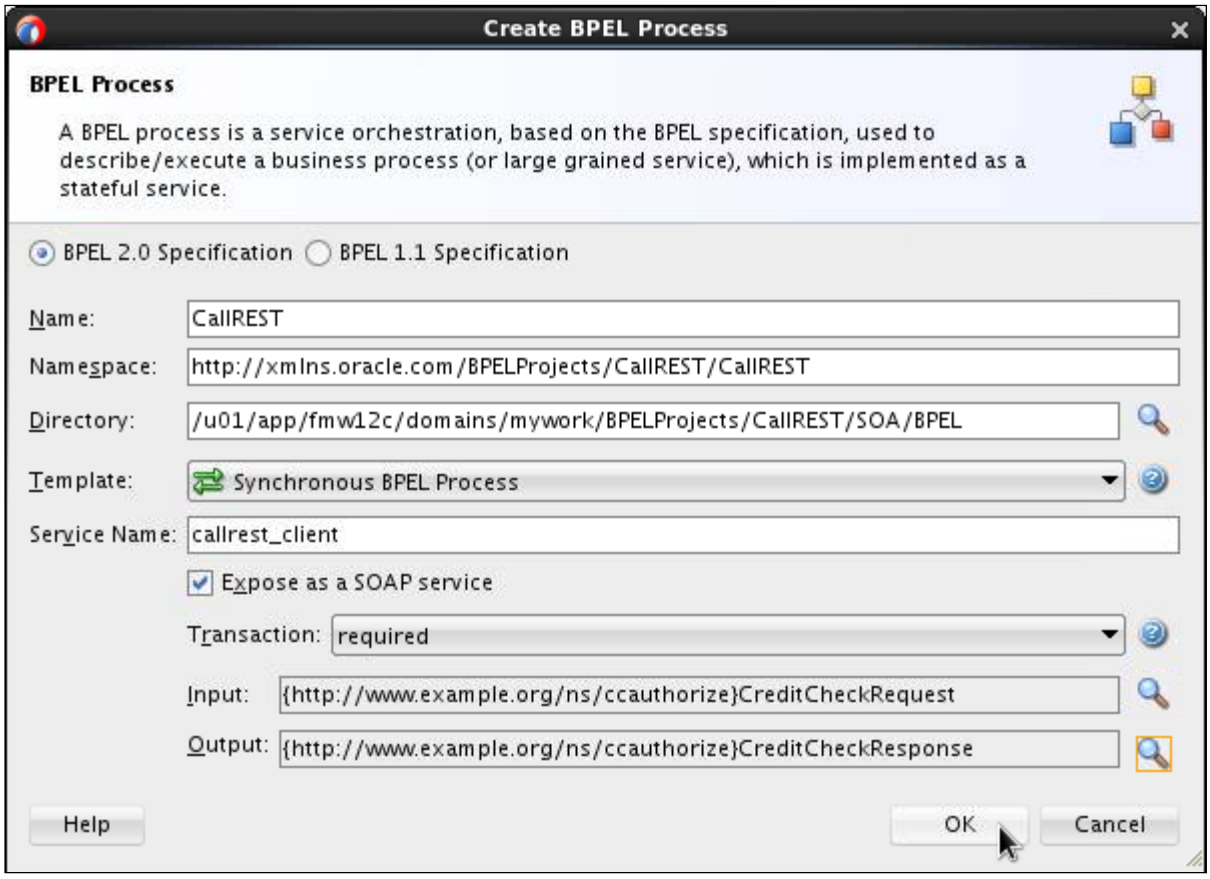
   c. Create an input variable based on the CreditCheckRequest element of the `creditcheck.xsd` file (already in the project).

   d. Create an output variable based on the CreditCheckResponse element of the `creditcheck.xsd` file.

   e. Verify your work and click **OK**.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



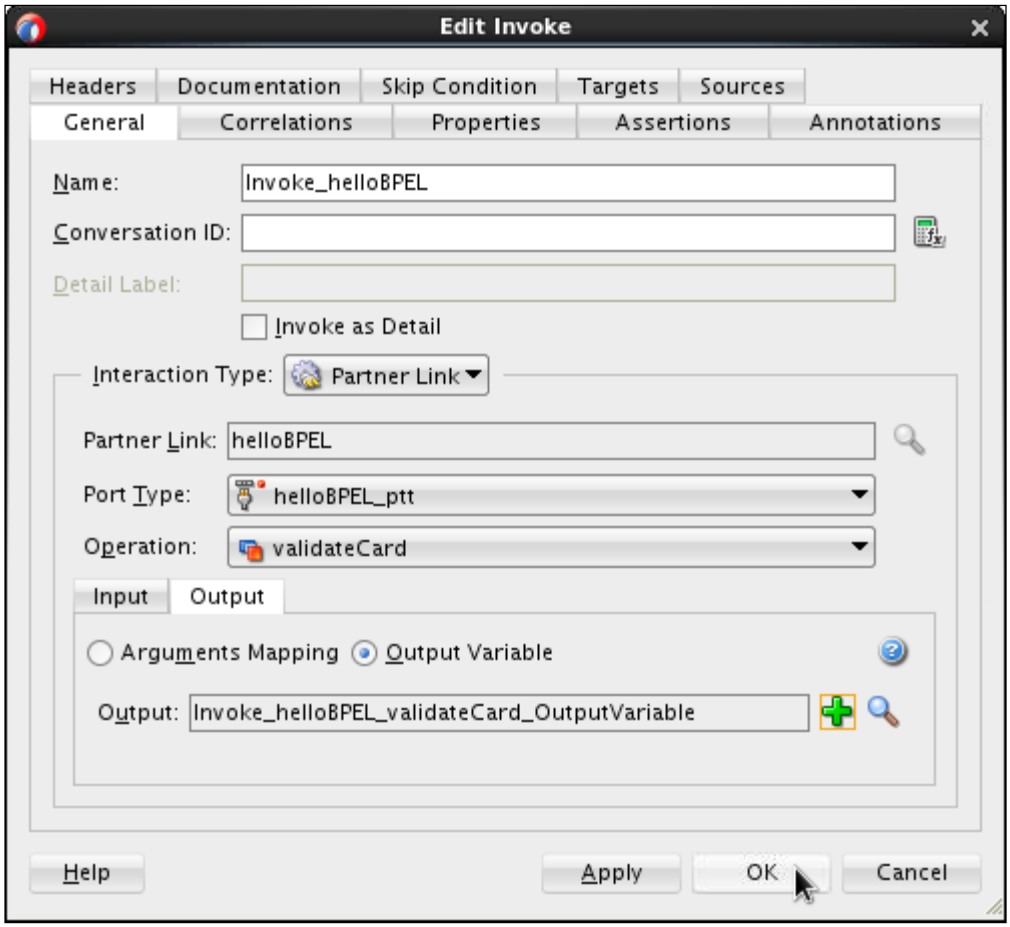
10. Wire the BPEL process to the REST binding.

Defining Activities in the BPEL Process

11. Open the BPEL Process for editing.
12. Create and configure an Invoke activity.
    a. Add an **Invoke** activity to the BPEL process.
    b. Map the Invoke activity to the helloBPEL partner link.
       The Edit Invoke window is displayed.
    c. Name the new activity **`Invoke_helloBPEL`**.
    d. Automatically create input and output messages.
    e. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



13. Create and configure an Assign activity to set the input message for the REST service.
    a. Add an **Assign** activity before the Invoke activity.
    b. Name the new activity **`Assign_CreditCard`**.
    c. In the Assign Editor, map the CCNumber element in the process input variable to the CCNumber element in the input variable for the REST service.
    d. Map the amount element in the process input variable to the amount element in the input variable for the REST service.



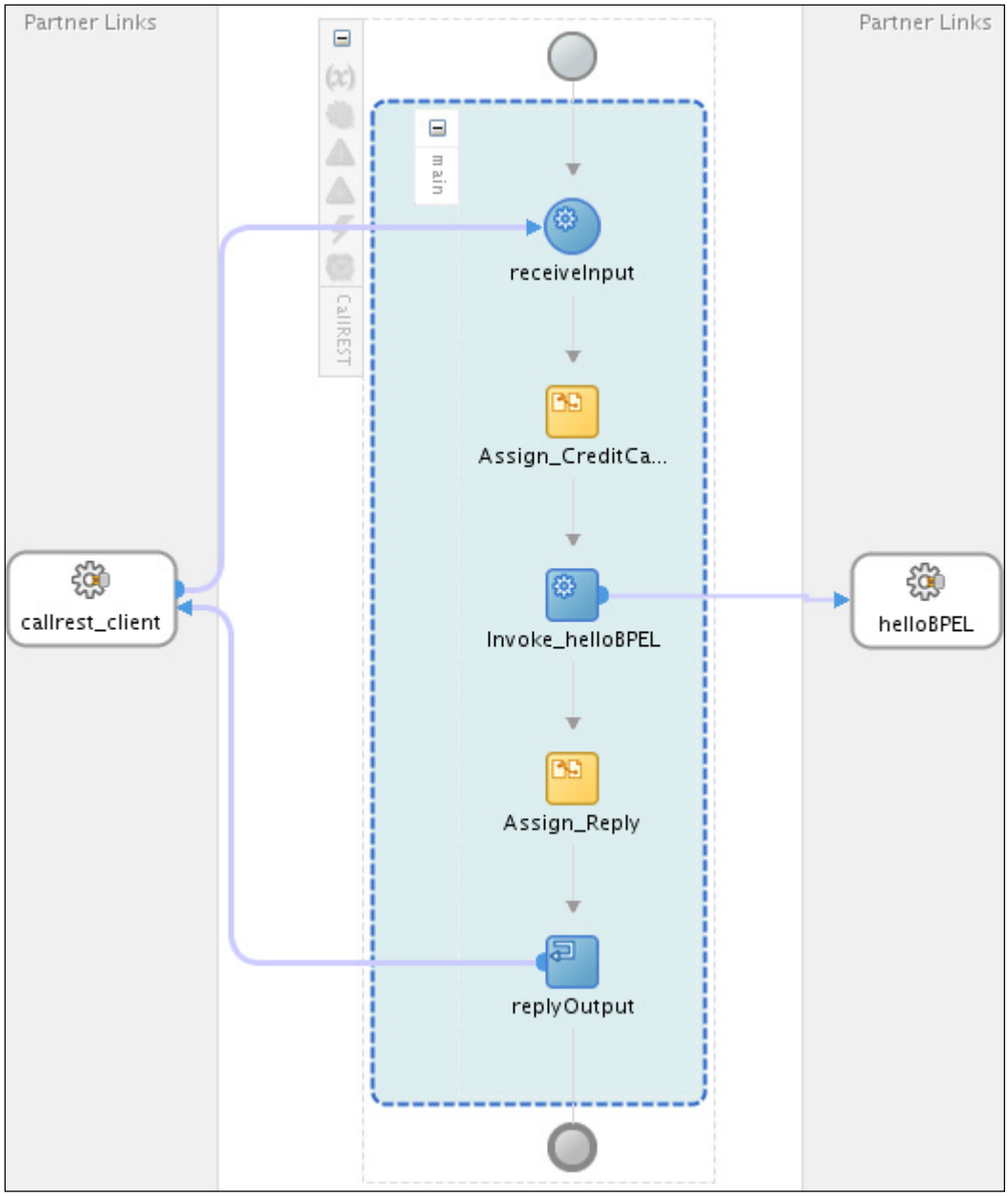
    e. Click OK.
14. Create and configure an Assign activity to set the output message for the BPEL process.
    a. Add an **Assign** activity after the Invoke activity.
    b. Name the new activity **`Assign_Reply`**.
    c. In the Assign Editor, map the status element in the REST service output variable to the status element of the output variable for the BPEL process.
    d. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

From	To
$Invoke_helloBPEL_validateCard_OutputVariable.reply/ns2:status	$outputVariable.payload/ns2:status

15. Verify and save your work.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 11-3: Deploying and Testing the Project

Overview

In this practice, you deploy and test the project.

Assumptions

This practice assumes that you have completed all work through Practice 11-2 successfully.

Tasks

1. Deploy the CallREST application to IntegratedWebLogicServer.

   Remember to select the “Overwrite any existing composites with the same revision ID” check box.

2. In Enterprise Manager, navigate to the test page for the CallREST project.
   a. Open Enterprise Manager.
   b. Navigate to the CallREST project.
   c. On the CallREST home page, click Test.

      The test page opens.

   d. On the Test Web Service page, scroll down to the Input Arguments section to the Request tab. Expand **payload** and enter the following values:

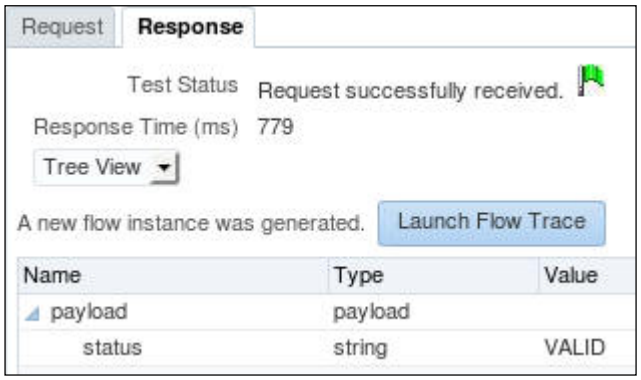
      CCNumber: `1234-1234-1234-1234`

      amount: `1000`

   e. Click Test Web Service.

      The Response tab is displayed.

   f. Verify that the status value returned is VALID, as shown in the following screenshot:



   g. On the Response tab, click the Launch Message Flow Trace link.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

h. In the Flow Trace, click the View Payload link for Assign_Reply in CallREST.

The called project hellobpelRS has successfully processed the message. The REST adapter has converted the JSON to XML, and returned the status of VALID.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 12: Developing Business Rules

Chapter 12

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 12: Overview

Practices Overview

In this lesson, you learn basic vocabulary and concepts for Oracle Business Rules, as well as how to configure business rules and Decision Tables. You also learn how to invoke the configured business rule service component from a BPEL process.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 12-1: Creating and Configuring a Composite Application

Overview

In this practice, you create a composite application that processes order data. The assembly model includes a Business Rules component that determines whether or not the order requires manual approval. The application includes an external reference that exposes the `Approval` application (from Practice 9) as a web service. The assembly model also includes a BPEL process, which invokes the service component and external reference in the course of its execution.

Assumptions

This practice assumes that you have completed Practice 9 for the lesson titled “Implementing Human Workflow and Notifications” successfully, and have deployed the `Approvals` and `ApprovalTaskForm` applications.

Tasks

1. In the BPELProjects application, create a new empty SOA project named `OrderProcessing`.

   The `composite.xml` file is created and the overview window is opened for editing.

Creating and Configuring an External Reference to the `Approval` Project

In this section, you add an External Reference to the `Approval` project that you created and deployed in the previous practice. The application is invoked as a web service from a BPEL process within this composite.

2. Create and configure a web service in the assembly model (`composite.xml`) as an External Reference.
   a. Add a SOAP web service to the External Reference swimlane.

      The Create Web Service window opens.

   b. Name the service `ApprovalService`.

   c. To define the WSDL URL, perform the following steps:

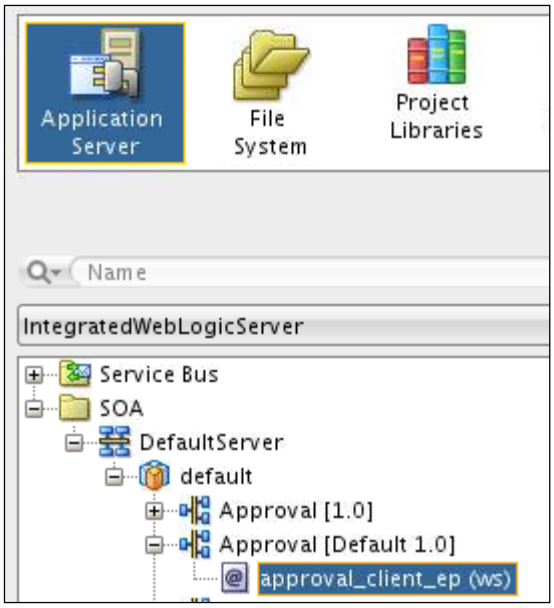
      1) Click the Find Existing WSDL button.

         The WSDL Chooser opens.

      2) With Application Server selected, navigate to SOA > Default Server > default > Approval [Default 1.0] > approval_client_ep (ws).

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



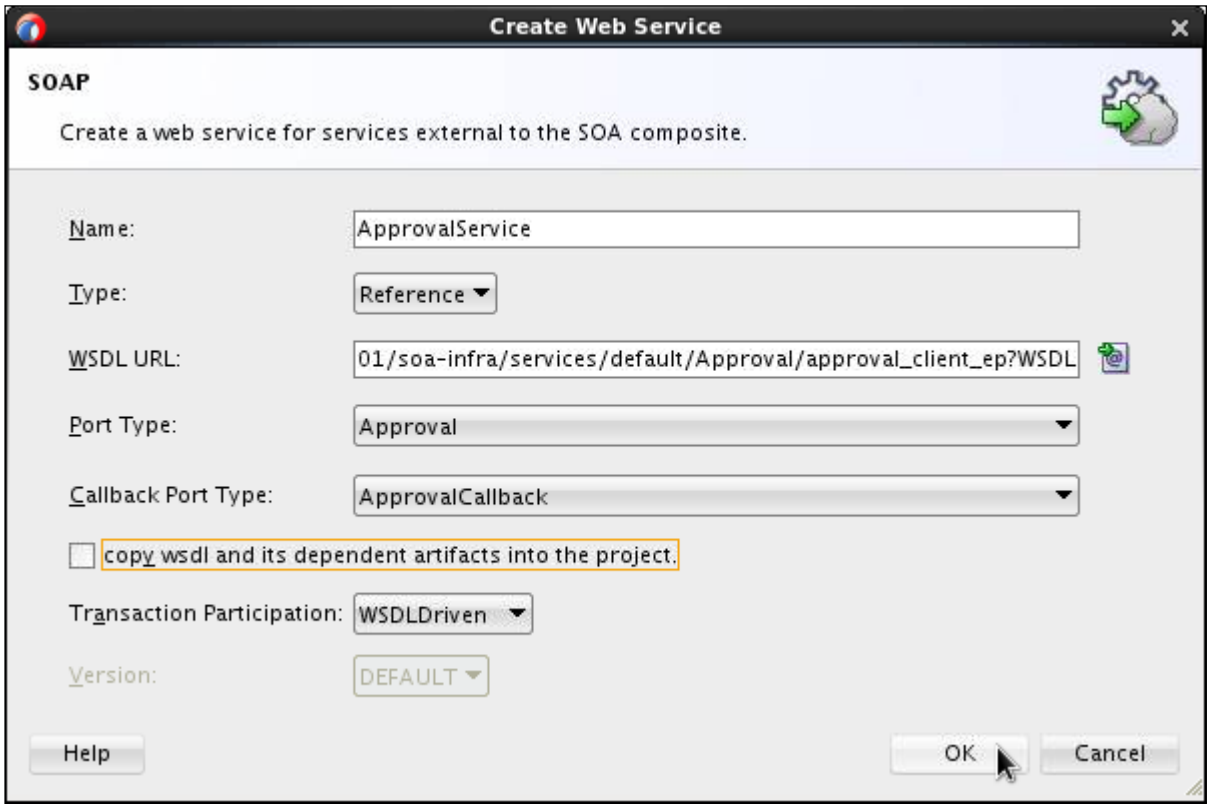
3) Click OK to close the WSDL Chooser.

d. Set the Port Type to Approval.

e. Set the Callback Port Type to ApprovalCallback.

f. Deselect the “copy wsdl and its dependent artifacts into the project” check box.

g. Verify your work and click OK.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Creating the `ApprovalRules` Business Rules Component

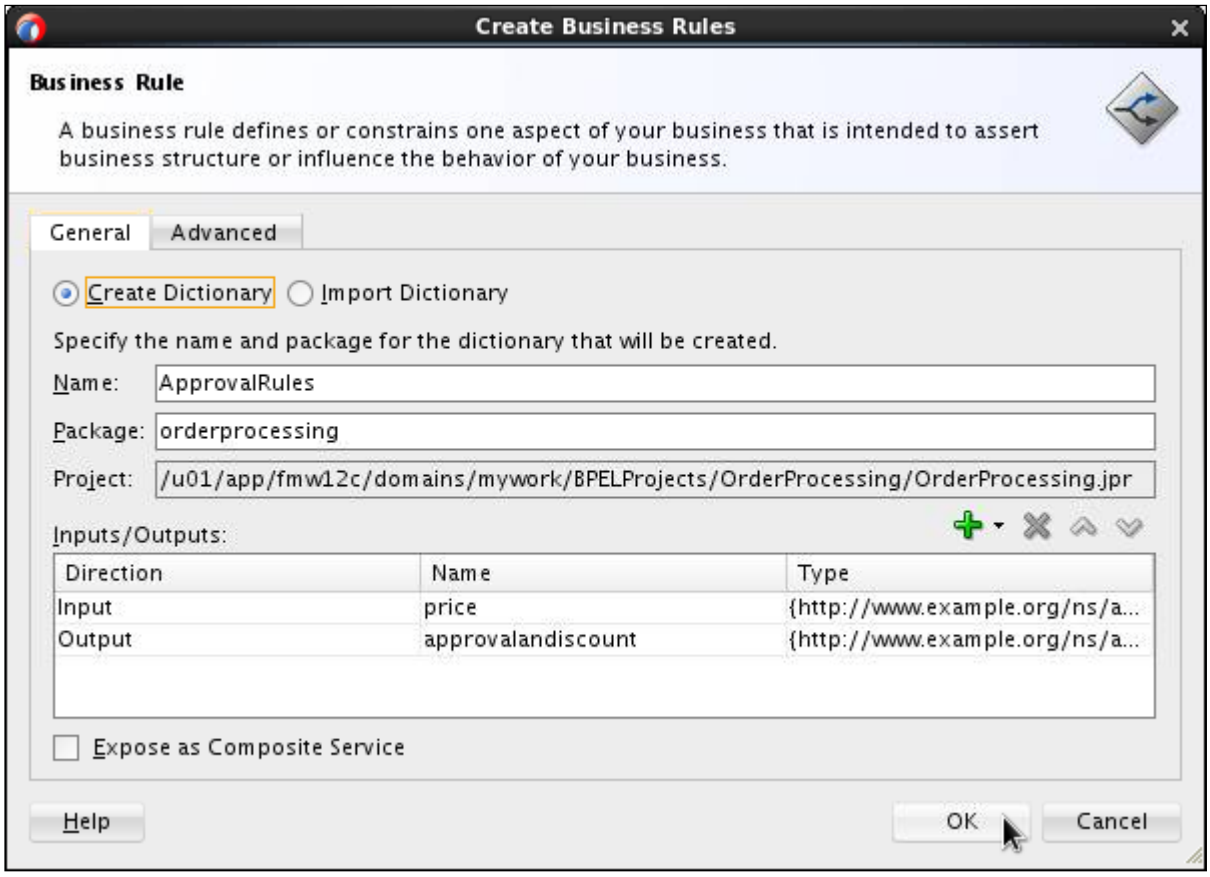
In this section, you create a Business Rules component that implements the if/then logic to determine whether or not the current order needs human authorization. The decision is based on the total value of the order (for example, “if total is greater than 4000, require approval”). The Business Rules component is invoked from a BPEL process within this application.

3. Create and configure a Business Rules component in the assembly model.
   a. Add a Business Rules component to the Components swimlane.
   b. Name the component `ApprovalRules`.
   c. Add input.
      1) Select Create > Input.
      2) Import `/home/oracle/labs/files/xsd/orderapproval.xsd`.

         Remember to deselect the “Maintain original directory structure for imported files” check box.
      3) Select the `price` element.
   d. Add output.
      1) Select Create > Output.
      2) Navigate to the `approvalandiscount` element of `orderapproval.xsd`.
   e. Define a name for the decision service.
      1) Click the Advanced tab.
      2) Name the service `ApprovalRulesService` (case-sensitive).
   f. Toggle back to the General tab.
   g. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



h. Save your work.

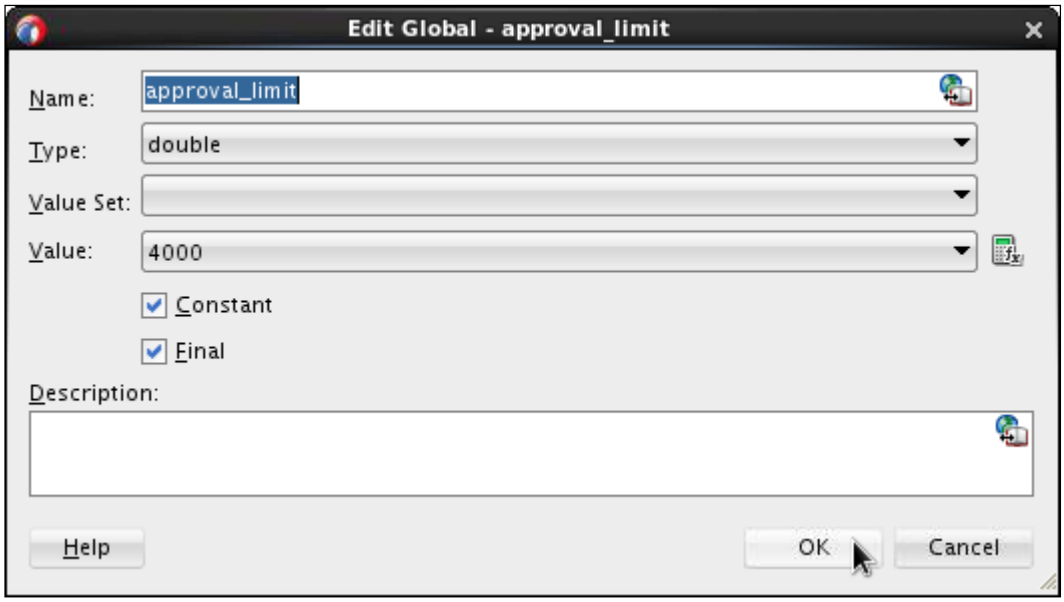
Creating a Global to Hold the Approval Limit

In this section, you create a global value to hold the approval limit value, which is used in the rule conditions that are created in the subsequent steps of this practice.

4. Create and initialize a global that is defined for the approval limit as a value of `4000`.
   a. Right-click the ApprovalRules component icon and select Edit.
   b. Click the Globals tab on the left side of the editor.
   c. Click the Create icon.
   d. Name the global `approval_limit`.
   e. Use the drop-down menu to set the type to `double` (*with a lowercase "d"*).
   f. Use Expression Builder to define a value of `4000`.
   g. Select the Constant check box and ensure that the Final check box is selected.
   h. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Renaming the Default Ruleset

In this section, you rename the default initial ruleset that is already created in the dictionary, before you create the rules in the ruleset.

5. Rename the `Ruleset1` ruleset.
   a. Click the `Ruleset1` tab on the left side of the editor.
   b. In the Name field, enter the text: `approvalruleset`, and press Enter.

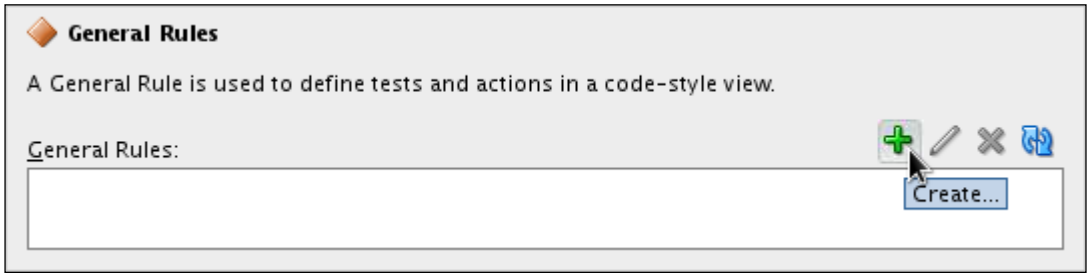
      Note: Renaming the ruleset is not strictly required, but applying a meaningful name makes future maintenance easier.

Creating the `abovelimit` Rule in the `approvalruleset` Ruleset

In this section, you create a rule in `approvalruleset`. The rule checks to see if the total price is greater than the approval limit and if so, it sets the required approval Boolean result to `true`.

Note: The structure that contains the result is called `approvalandiscount`, which contains a `discount` element. The `discount` element is not used in this course application.

6. Create a new rule.
   a. In the General Rules pane, click Create.



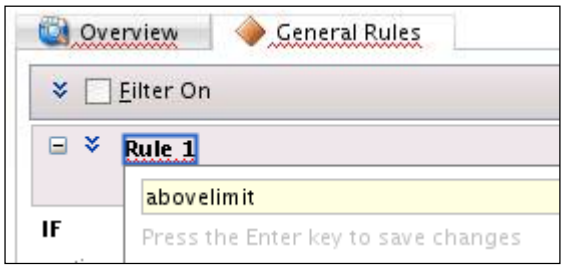
   A new rule named `Rule1` is created.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

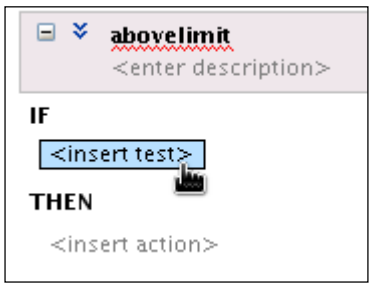
b. Click the new rule and rename it `abovelimit`.

Note: Remember to press Enter after entering the new name.



7. Create the test (or condition) in the IF section of the `abovelimit` rule.

a. Click the `<insert test>` entry.



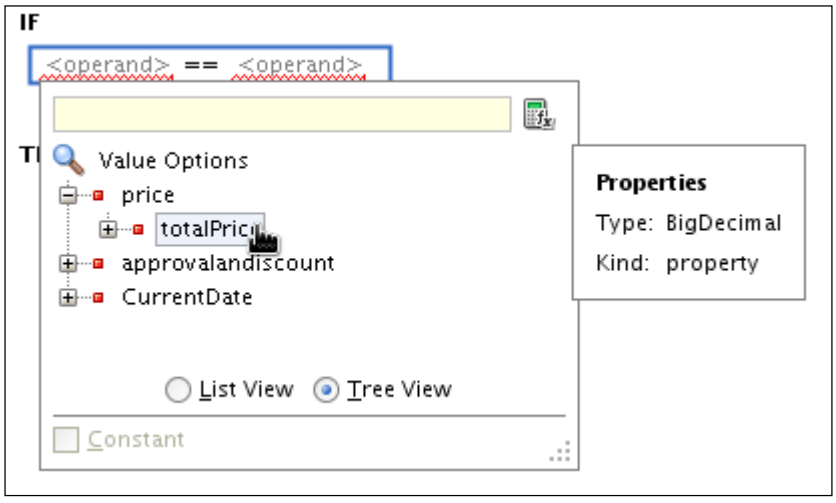
b. Select simple test.

Note: This creates a conditional structure with the format `<operand> == <operand>`. It also creates another `<insert test>` below it. Each part needs to be clicked to specify its value.

c. Set the left-side `<operand>`.

1) Click the `<operand>` text.

2) Select `price.totalPrice` from the drop-down menu.



d. Set the operator.

1) Click the "==" text.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

2) Select the greater-than (>) sign from the drop-down menu.

e. Set the right-side <operand>.

1) Click the <operand> text.

2) Select approval_limit from the drop-down menu.

Note: approval_limit is the global that you created and set to the value 4000.

8. Create the action in the THEN section.

a. Click the <insert action> text.

b. Select assert new from the drop-down menu.

Note: The assert new action creates a new fact object that needs to be defined and initialized to hold the desired values. In addition, another <insert action> is added below the one being created. This allows you to create multiple actions for a rule. In this case, you create one action.

c. Set the object created by the assert new action.

1) Click the <target> text.

2) Select the approvalandiscount option from the drop-down menu.

Note: The approvalandiscount structure (as defined in the XSD that is imported to define the output result structure for the Business Rule decision service) has child elements called *properties*. You need to set their values for the present condition.

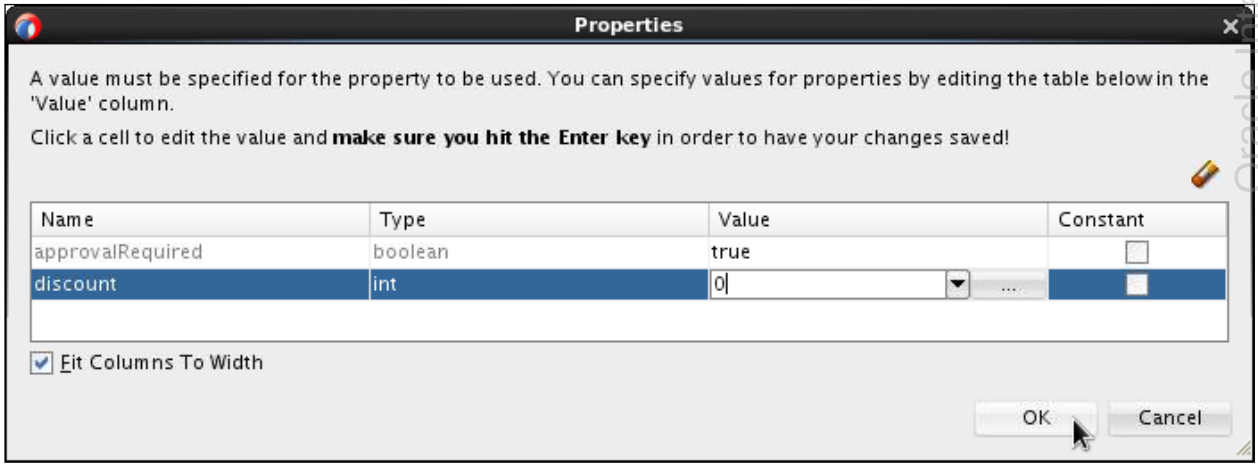
d. Set the approvalandiscount property values.

1) Click the <edit properties> text.

e. Set the approvalRequired property value to true (which can be selected from the drop-down menu in the Value column).

1) Enter the value 0 (zero) in the discount Value column.

2) Verify your work and click OK.



The Properties pane is closed.

f. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

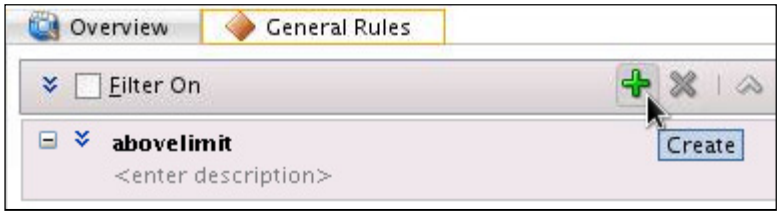


Creating the `withinlimit` Rule in the `approvalruleset` Ruleset

In this section, you create a `withinlimit` rule that tests the condition where the total price is less than or equal to the approval limit, in which case the `approvalRequired` property should be set to a value of `false`.

9. Create a new rule.

   a. In the General Rules pane of the `approvalruleset` tab, click Create.



   b. Click the new rule name and rename it `withinlimit`.

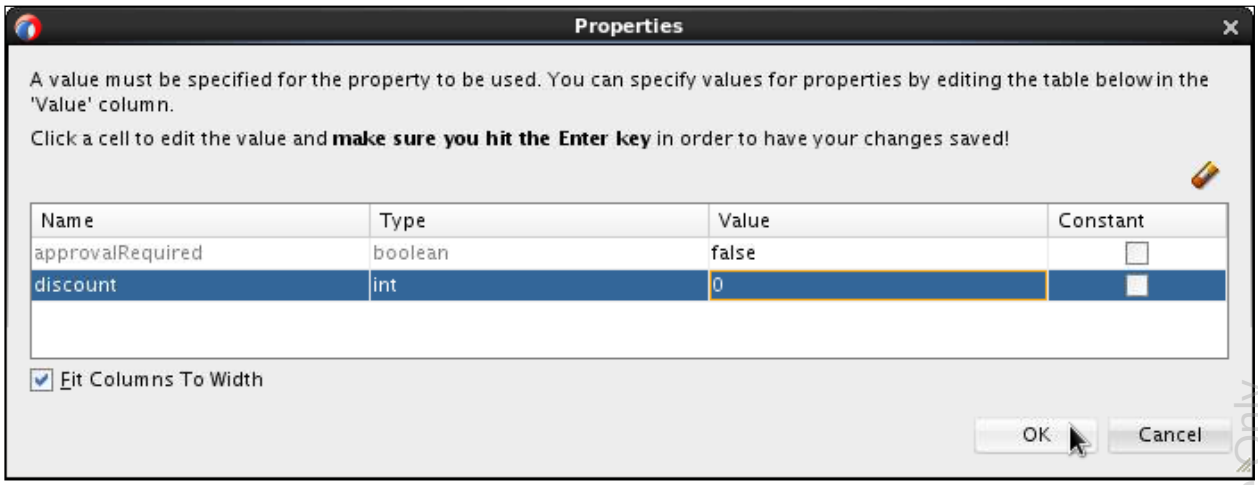
   c. Configure the test (condition) where the conditional expression is expressed as `price.totalPrice <= approval_limit`.



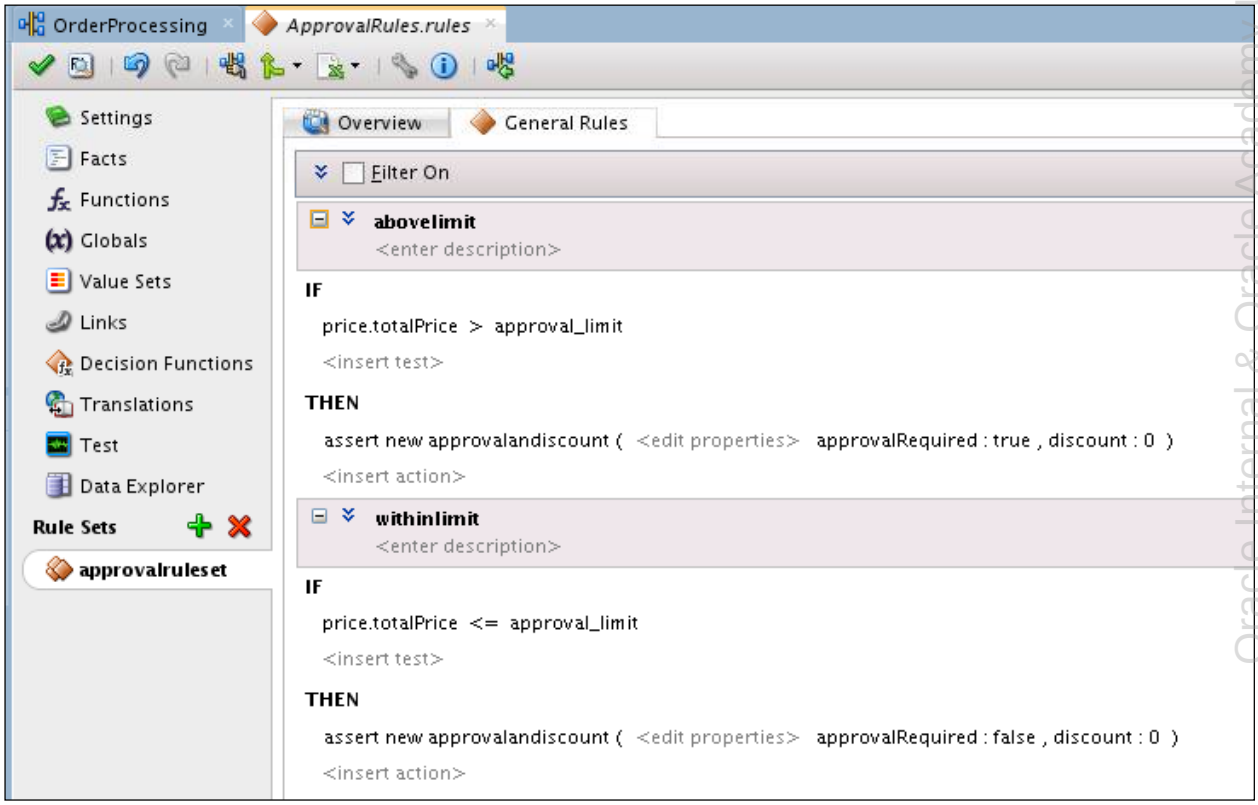
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

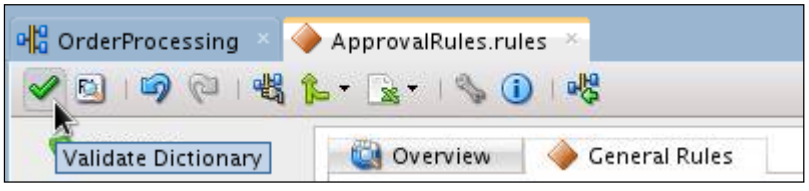
d. In the THEN section, click `<insert action>`, select the `assert new approvalandiscount` object (as you did for the `abovelimit` rule), and set the properties so that `approvalRequired` is `false` and discount is `0` (zero).



e. Verify your work.



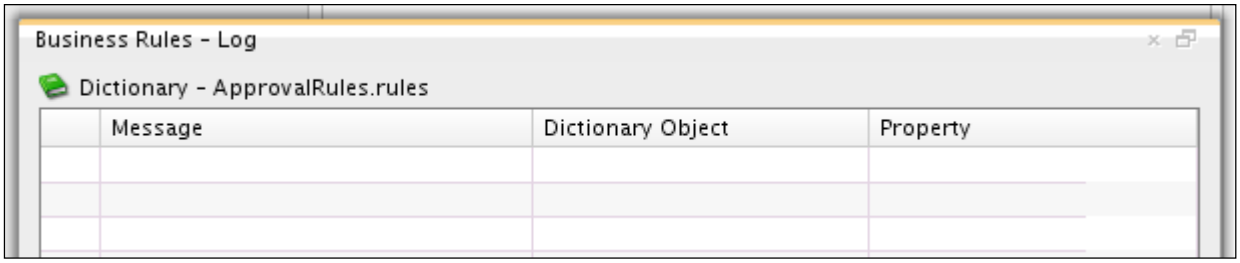
10. In the `ApprovalRules.rules` window, click the Validate icon on the toolbar to ensure that the rule dictionary contains valid definitions.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Business Rule Validation – Log opens near the bottom of the pane. A successful validation results in a blank log.



11. Save your work and close `ApprovalRules.rules`.

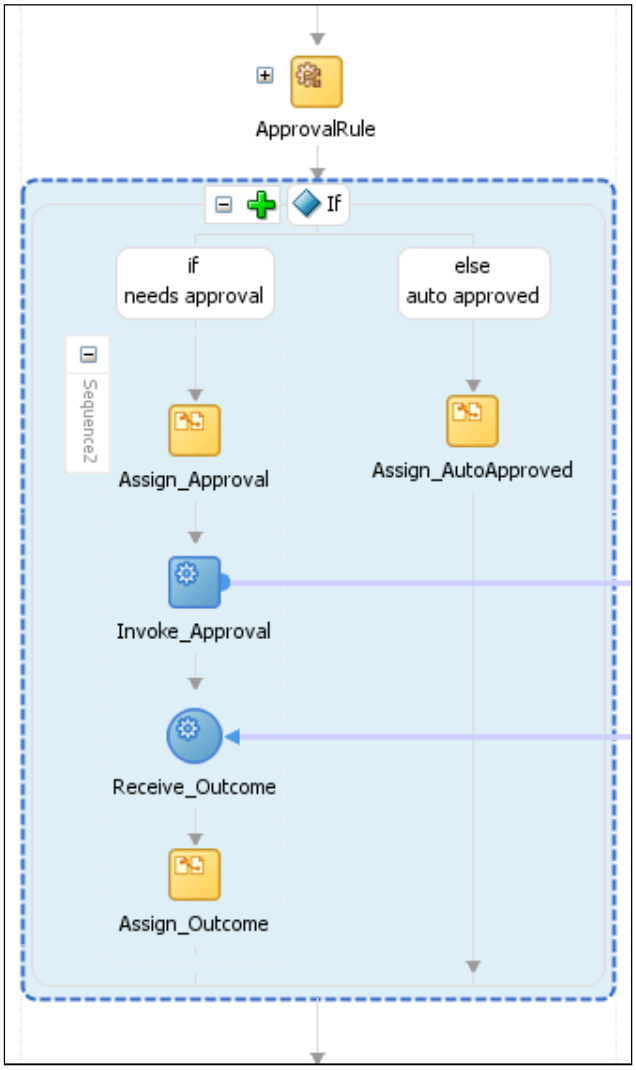
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 12-2: Implementing Business Rules in a BPEL Process

Overview

In this section, you create the BPEL process and configure a series of activities. You first add a business rule to determine whether or not the order data requires manual approval. You then add and configure activities so that those orders that require approval are passed to the Approval application that you completed in the previous practice. The outcome of that approval is then stored in a variable and passed to additional activities (not shown here) for additional processing.



Assumptions

This practice assumes that you have completed Practice 12-1 successfully.

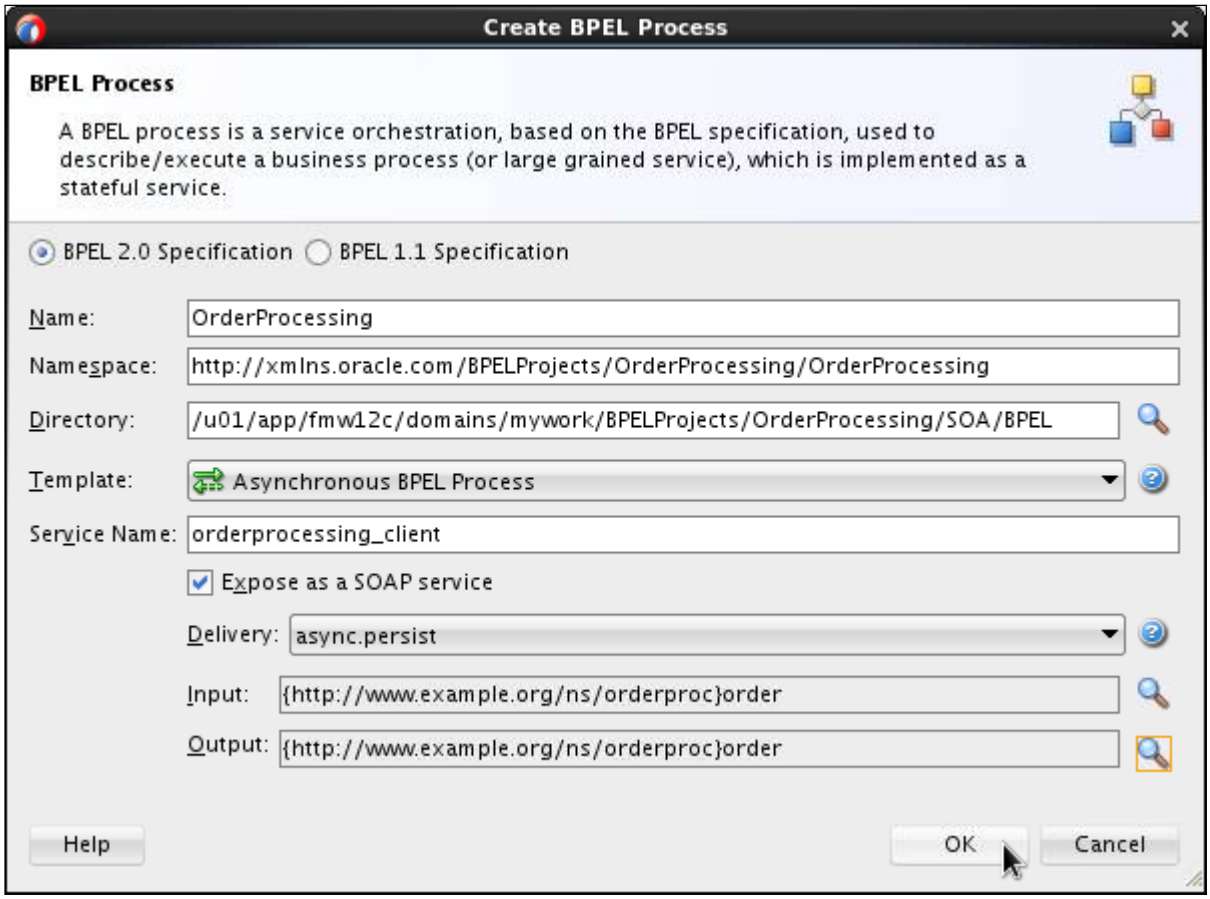
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Tasks

Creating and Configuring a BPEL Process

1. Create and configure a BPEL service component.
   a. Add a BPEL process to the Components swimlane in the `composite.xml` editor.
   b. Name the process `OrderProcessing`.
   c. Accept the default Asynchronous Template.
   d. To define the input, perform the following steps:
      1) Click the Browse icon to access the Type Chooser window. Then, click the Import Schema file icon to import the file `/home/oracle/labs/files/xsd/orderProcessing.xsd` into the project.
      2) Deselect the "Maintain original directory structure for imported files" check box.
      3) Select the `order` element.
      4) Click OK.
   e. To define the output, browse to the `order` element of `orderProcessing.xsd`.
   f. Verify your work and click OK.



   g. Save your work.

Oracle Internal & Oracle Academy Use Only

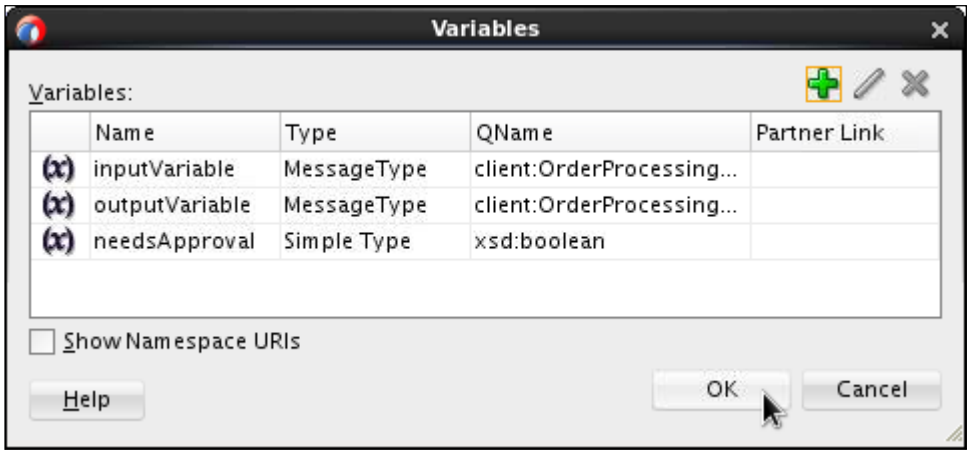
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

2. In the composite overview window, wire the BPEL process to ApprovalService.

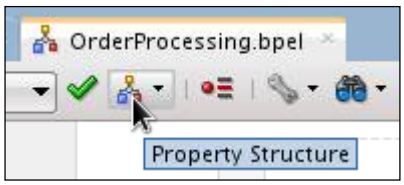
   ApprovalService is associated with the BPEL process as a partner link.

3. Right-click the BPEL component and select Edit.

4. Create a process-level Boolean variable named `needsApproval`.



   Note: The preceding screenshot was produced by clicking the Variables icon on the Process Scope toolbar. If you select the Property Structure > Variables option from the BPEL editor toolbar menu (see the following), the screen differs, but you still are able to create the desired variable.



5. Create and configure a Business Rules component.

   a. Add a Business Rule SOA Component to the process between the Receive and the callback activities.

   b. Rename the rule `ApprovalRule`.

   c. Right-click ApprovalRule and select Edit.

      The Business Rule settings window opens.

   d. Specify the dictionary by selecting ApprovalRules from the drop-down menu.

      Additional configuration options appear. The Service is set to ApprovalRulesService, and the Operation is set to “Execute function and reset the session.”

      Note: You may see the operation set to `COMBO_ITEM_PATTERN_FUNC` instead. This is because of a bug in JDeveloper.

   e. Assign Input Facts.

      1) Click the Create icon.

      2) The Decision Fact Map window opens.

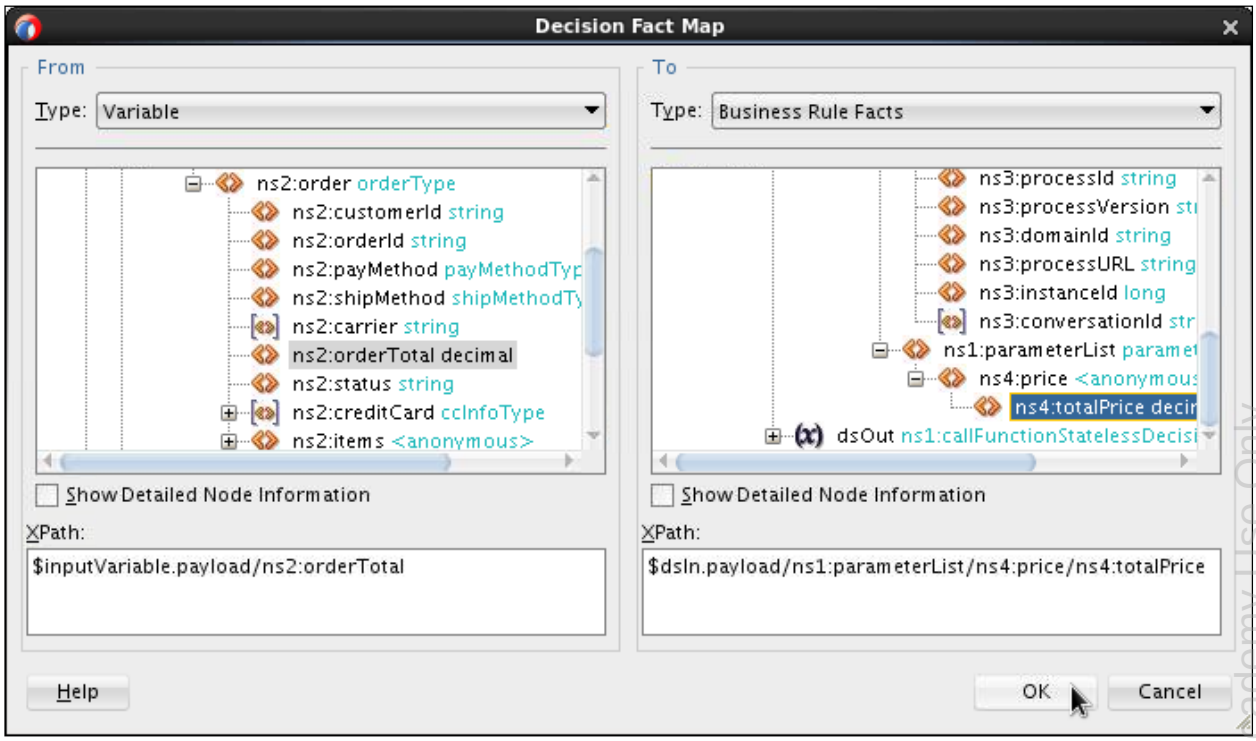
      3) In the left pane, navigate to the `orderTotal` inputVariable.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4) In the right pane, navigate to the `totalPrice` element of the `dSIn` variable.

5) Verify your work and click OK.



f. Assign Output Facts.

1) Click the Assign Output Facts tab.

2) Click the Create icon.

3) The Decision Fact Map opens.

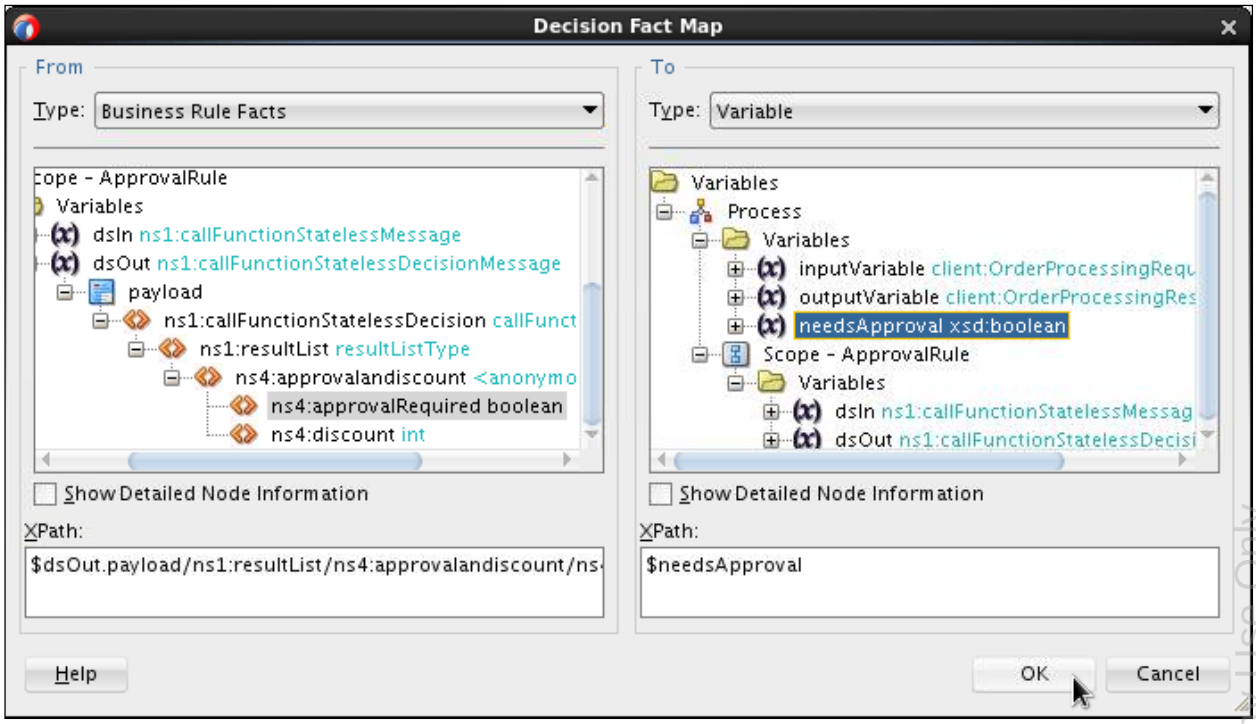
4) In the left pane, navigate to the `approvalRequired` element of the `dsOut` variable.

5) In the right pane, navigate to the process variable `needsApproval`.

6) Verify your work and click OK.

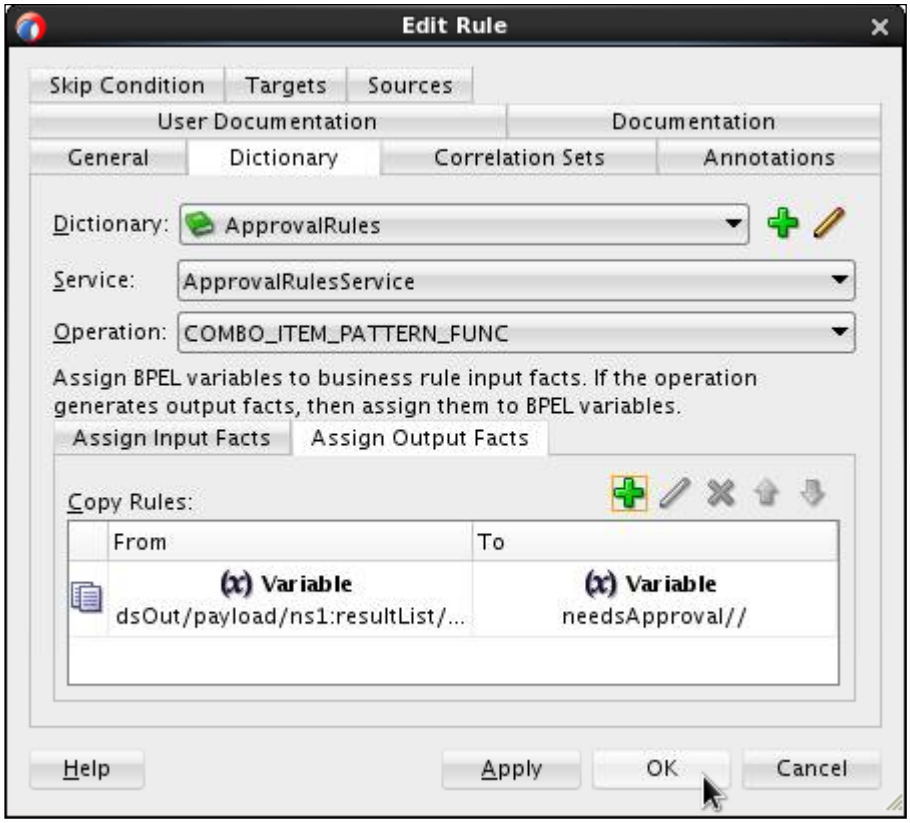
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



You are returned to the Business Rule dialog box.

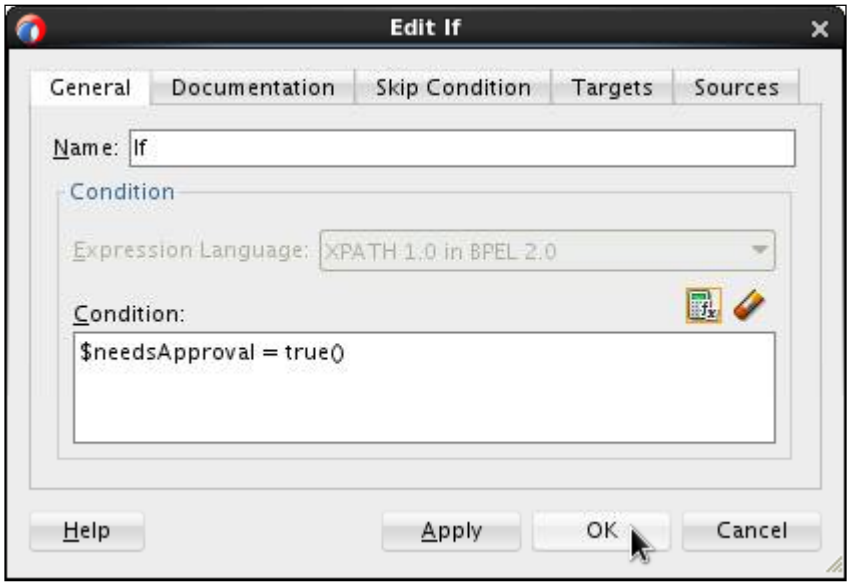
g. Verify your work and click OK.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

6. Create and configure an If activity.
   a. Add an If activity following the Business Rule activity.
   b. Set the conditional expression of the If to test whether `$needsApproval=true()`.
   c. Label the branches `'if needs approval'` and `'else auto approve'`.



7. Add an Assign activity to the <`else auto approve`> branch.
   a. Add an Assign activity to the <`else auto approve`> branch.
   b. Name the activity `Assign_AutoApproved`.
   c. Right-click `Assign_AutoApproved` and select Edit.
   d. Use Expression Builder to assign the value `'APPROVED'` to the status element of `inputVariable`.
   e. Verify and save your work.

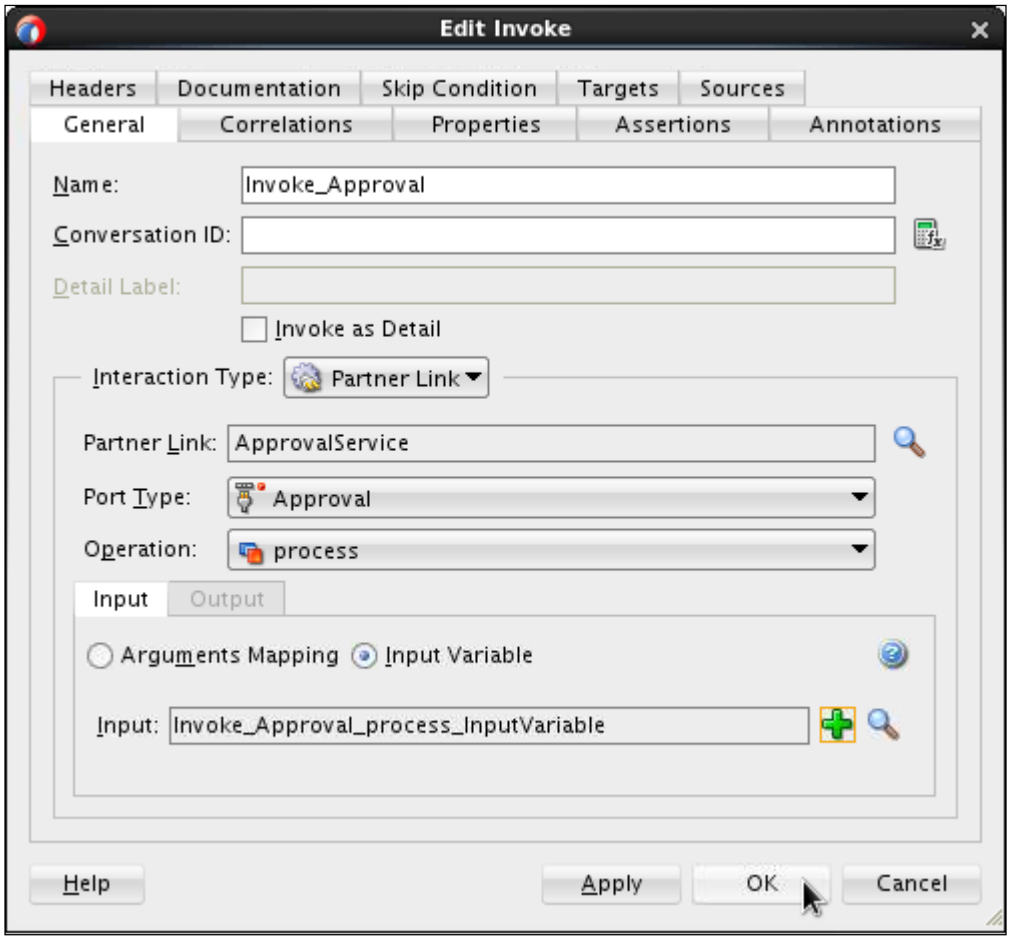


8. Create and configure an Invoke activity in the `<if>` branch to call the Approval application.
   a. Add an Invoke activity to the `<if>` branch.
   b. Wire the Invoke activity to the partner link `ApprovalService`.

      The Edit Invoke window is displayed.
   c. Name the Invoke `Invoke_Approval`.
   d. Create a new Input variable.
   e. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

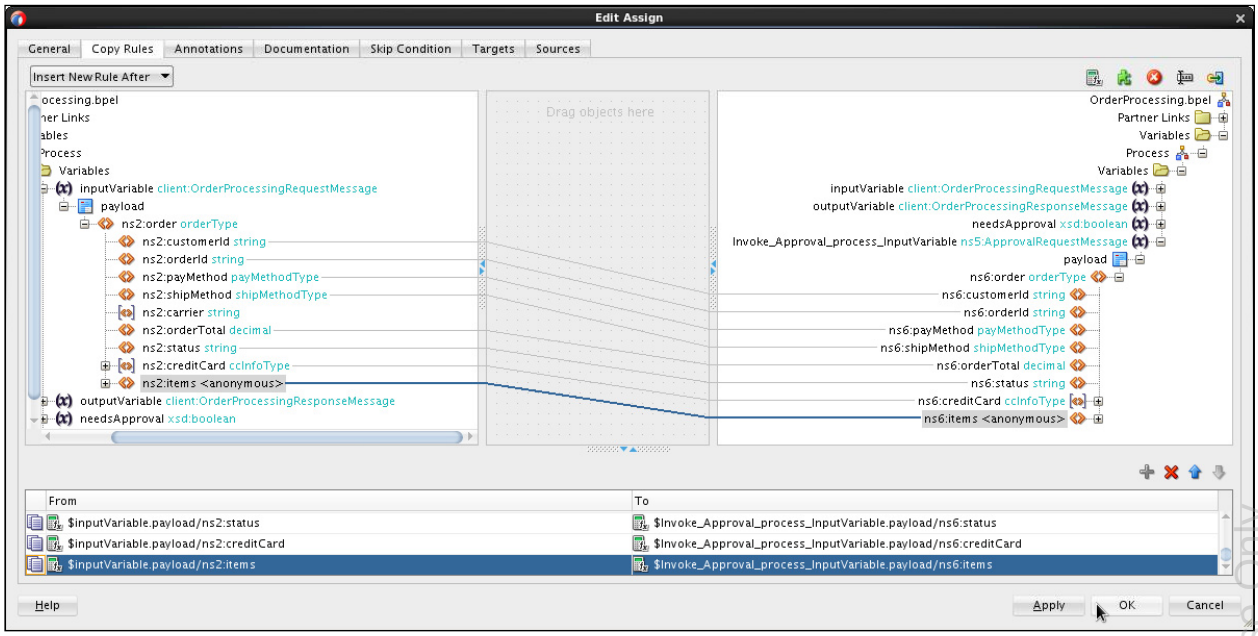
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



9. Create and configure an Assign activity.
   a. Add an Assign activity just above `Invoke_Approval`.
   b. Name the activity `Assign_Approval`.
   c. Right-click the new activity and select Edit.
   d. Map each individual element of `inputVariable` (except `carrier`) to its corresponding element in `Invoke_Approval_process_InputVariable`.
   e. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

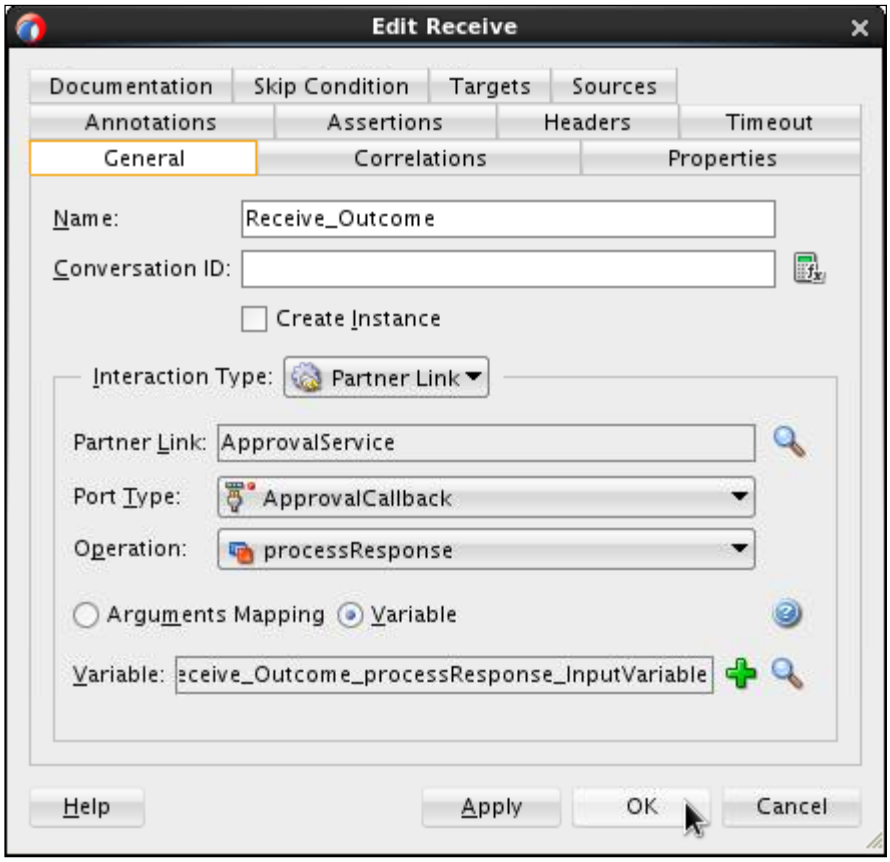
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



10. Create and configure a Receive activity.
    a. Add a Receive activity just below `Invoke_Approval`.
    b. Right-click the new activity and select Edit.
    c. Name the activity `Receive_Outcome`.
    d. Use the Browse button to choose the partner link `ApprovalService`.
    e. Auto-create the variable `Receive_Outcome_processResponse_InputVariable`.
    f. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



11. Create and configure an Assign activity to save the approval outcome to `inputVariable`.
    a. Add an Assign activity just below `Receive_Outcome`.
    b. Name the activity `Assign_Outcome`.
    c. Right-click the new activity and select Edit.
    d. Map the `Receive_Outcome` variable element `status` to the corresponding element in `inputVariable`.
    e. Verify and save your work.



12. Create and configure an Assign activity to populate the business process output variable, which is returned to the calling client.
    a. Add an Assign activity just above `callback_Client`.
    b. Name the activity `Assign_Output`.
    c. Right-click the new activity and select Edit.
    d. Map `inputVariable.payload` to `OutputVariable.payload`.
    e. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 12-3: Deploying and Testing the Application

Overview

In this practice, you deploy and test the `OrderProcessing` composite application.

Assumptions

This practice assumes the following:

- You have completed Practice 9 for the lesson titled “Implementing Human Workflow and Notifications” successfully, and that the `Approval` application is still deployed to the application server.
- You have completed all work in Practice 12-2 successfully.

Tasks

Deploying the OrderProcessing Composite Application

1. Deploy the `OrderProcessing` application to IntegratedWebLogicServer.

   Remember to select the “Overwrite any existing composites with the same revision ID” check box.

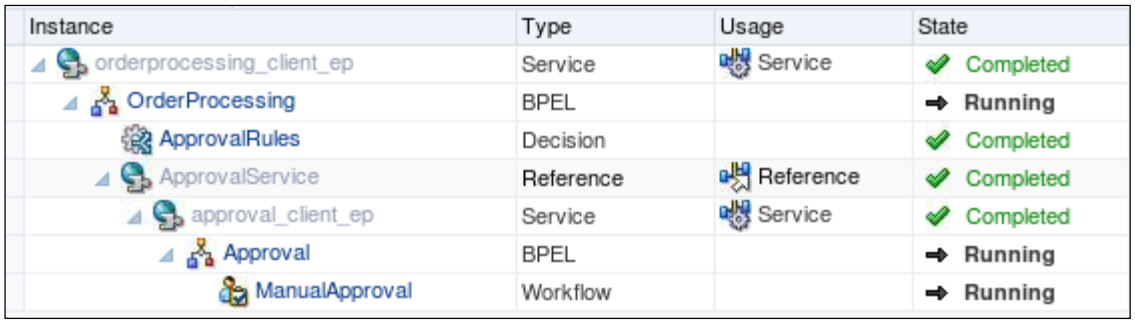
Testing the Composite Application

In this section, you submit an order whose total is greater than 4000 so that the `ApprovalRules` business rule returns a result that requires manual approval of the order. As in the previous practice, as employee `jcooper`, you check your email to read the notification about the assigned task, and then log in to the Worklist application to approve the order.

2. Open the Oracle Enterprise Manager page (http://localhost:7101/em).
   a. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “OrderProcessing [1.0]” link.
   b. On the “OrderProcessing [1.0]” home page, click Test.
   c. On the Test Web Service page, scroll down to the Input Arguments section to the Request tab and select XML view.
   d. Replace the XML data with the contents of the file `/home/oracle/labs/files/xml_in/businessRules_input.xml`.
   e. Click Test Web Service.
   f. On the Response tab, click the Launch Flow Trace link.
   g. Which entries have completed? Which are still running? Why?

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



3. Manually approve the order in the Worklist application:

   a. Access the Oracle BPM Worklist application at http://soa12c.example.com:7101/integration/worklistapp.

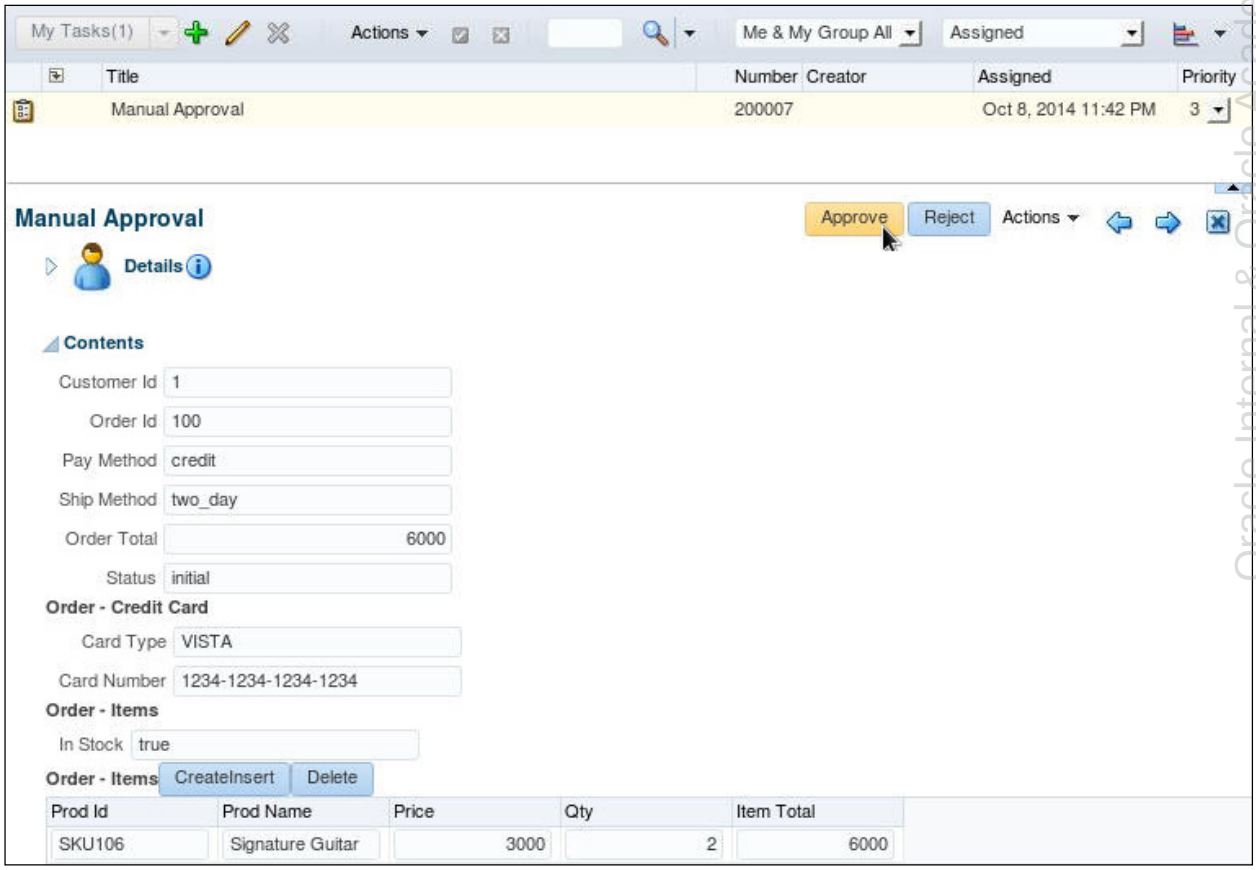
   b. Log in to the application as user `jcooper` with the password `welcome1`.

      On the Worklist home page, the Inbox is selected and the Manual Approval task is present in the top-right pane.

   c. Click the Manual Approval task entry.

      The order details are displayed in the lower-right pane.

   d. Click Approve.

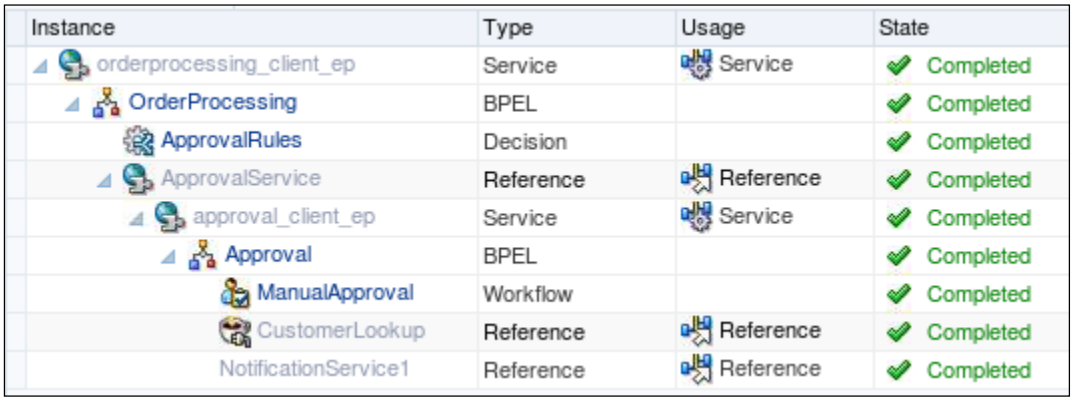


   e. Log out of the Worklist application and close the webpage.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4. Relaunch the Flow Trace and verify that the process has now completed.



Instance	Type	Usage	State
orderprocessing_client_ep	Service	Service	Completed
OrderProcessing	BPEL		Completed
ApprovalRules	Decision		Completed
ApprovalService	Reference	Reference	Completed
approval_client_ep	Service	Service	Completed
Approval	BPEL		Completed
ManualApproval	Workflow		Completed
CustomerLookup	Reference	Reference	Completed
NotificationService1	Reference	Reference	Completed

Additional Optional Tests

If you have time, you can submit some additional orders. Consider these scenarios:

5. Initiate a test with an order that requires manual approval and, at the manual approval stage, reject it.
6. In the Flow Trace, examine the callbackClient payload.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Payload for Activity: callbackClient

Find | Go to Line

```
<?xml version="1.0" encoding="UTF-8"?><outputVariable>
   <part xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      <order xmlns="http://www.example.org/ns/orderproc" xmlns

   <ns1:customerId>1</ns1:customerId>

   <ns1:orderId>100</ns1:orderId>

   <ns1:payMethod>credit</ns1:payMethod>

   <shipMethod>two_day</shipMethod>

   <ns1:carrier/>

   <ns1:orderTotal>6000</ns1:orderTotal>

   <status>rejected</status>

   <ns1:creditCard>

      <ns1:cardType>VISTA</ns1:cardType>

      <ns1:cardNumber>1234-1234-1234-1234</ns1:cardNumber>

   </ns1:creditCard>

   <ns1:items>

      <ns1:inStock>true</ns1:inStock>

      <ns1:item>

         <ns1:prodId>SKU106</ns1:prodId>

         <ns1:prodName>Signature Guitar</ns1:prodName>

         <ns1:price>3000</ns1:price>

         <ns1:qty>2</ns1:qty>

         <ns1:itemTotal>6000</ns1:itemTotal>

      </ns1:item>

   </ns1:items>

  </order>
   </part>
</outputVariable>

```

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 12-4: Adding a Shipping Rules Business Rule [Optional]

Overview

In this practice, you modify, redeploy, and test the `OrderProcessing` composite application. You add a second Business Rules component, which (if the order is approved) determines the shipping company for the order based on the delivery options included in the order data.

Assumptions

This practice assumes that you have completed Practice 12-3 successfully, and that the `OrderProcessing` application is still deployed to the application server.

Tasks

Creating and Configuring the `ShippingRules` Business Rules Component

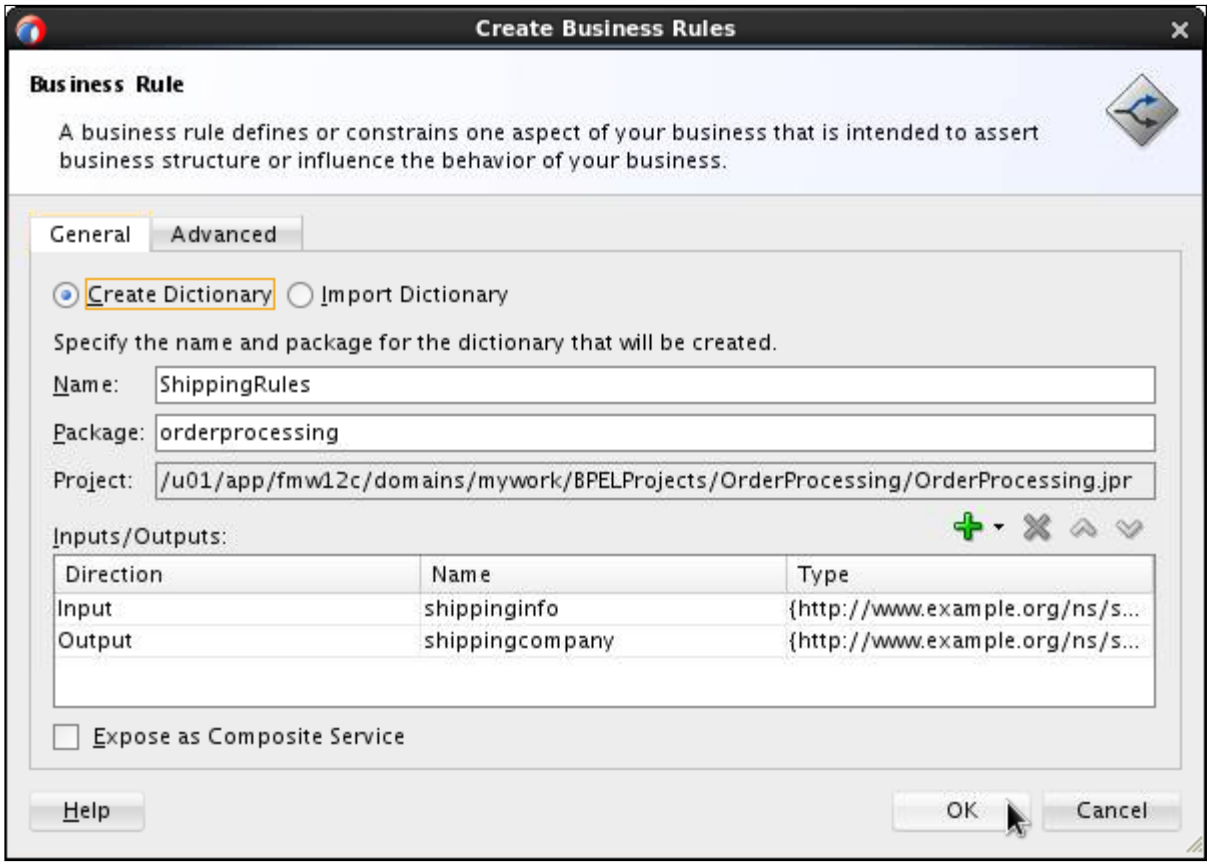
In this section, you create a second Business Rules component. This component uses a Decision Table to assign a shipping carrier based on the shipping method specified in the order detail. You configure the decision service interface by defining the service name and specifying the input and output fact structures, which are drawn from an XSD file that you import into the project.

1. Create and configure another Business Rules component in the assembly model.
   a. In `composite.xml`, add a Business Rules component to the Components swimlane.
   b. Name the component `ShippingRules`.
   c. Add input.
      1) Select Create > Input.
      2) Import `/home/oracle/labs/files/xsd/ordershipment.xsd`.

         Remember to deselect the “Maintain original directory structure for imported files” check box.
      3) Select the `shippinginfo` element.
   d. Add output.
      1) Select Create > Output.
      2) Navigate to the `shippingcompany` element of `ordershipment.xsd`.
   e. Define a name for the decision service.
      1) Click the Advanced tab.
      2) Name the service `ShippingRulesService` (*case-sensitive*).
   f. Return to the General tab. Verify and save your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Defining the `ShippingRules` Decision Table

In this section, you edit the `ShippingRules` Business Rules component and create the dictionary definitions, including the Decision Table and its related rules.

2. Right-click the ShippingRules component and select Edit.

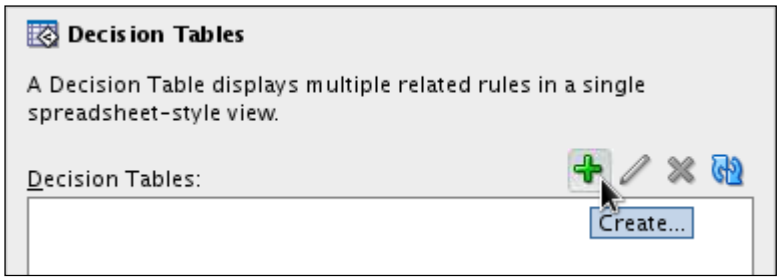
   The `ShippingRules.rules` window opens.

3. Rename the default ruleset `shippingruleset`.

Creating a Decision Table

4. Create and configure a Decision Table.

   a. Click the Create icon in the Decision Tables pane.



   b. Rename the `DecisionTable 1` rule `carrierrule`.

      Tip: Press Enter to save the new name.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Creating Rules in the Decision Table

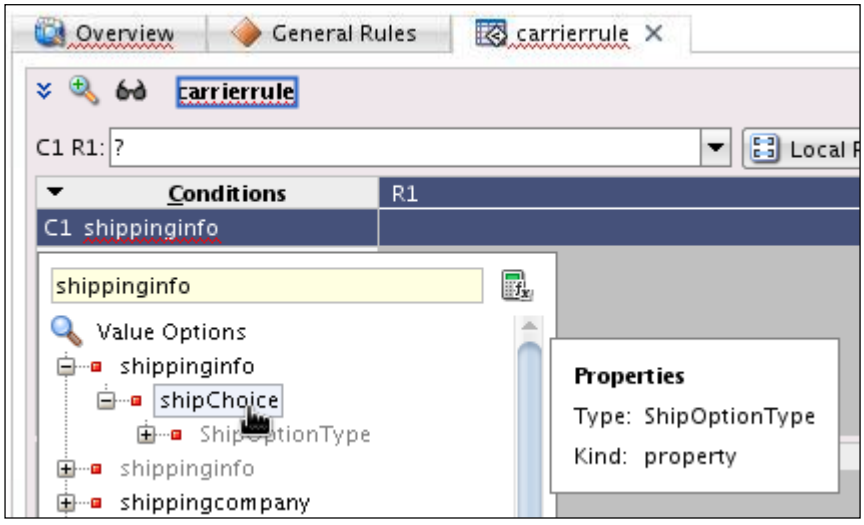
In this section, you define three rules in the Decision Table. The rules are created after you define the condition. After a rule comparison value is set for the condition, you can specify the action.

5. Create the Decision Table condition.

   a. In Decision Table Conditions, click the `<insert condition>` text.



   b. From the Conditions drop-down menu, double-click `shippinginfo` and click `shipChoice`.



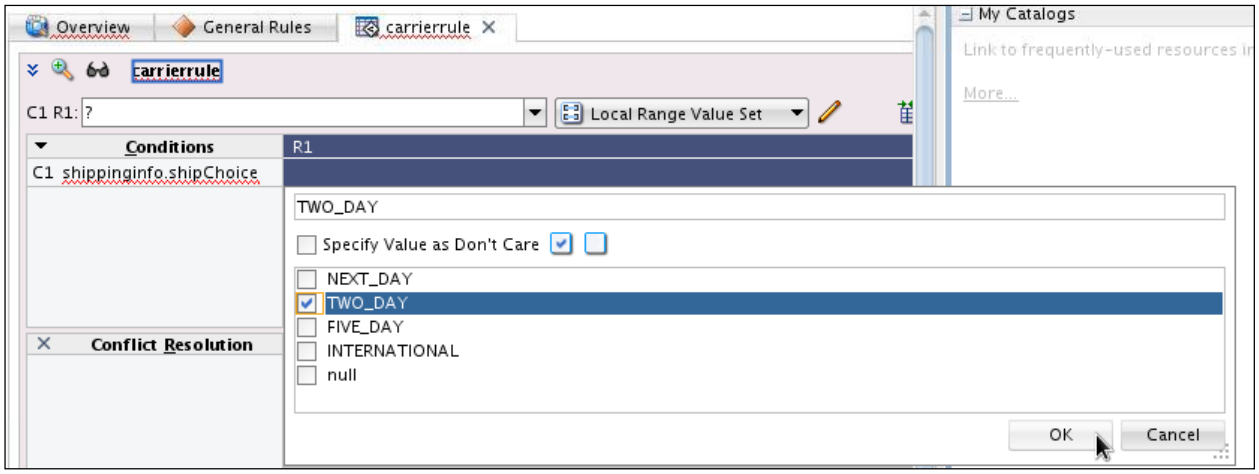
   Note: Adding and setting the condition creates the first rule template in the column titled R1.

6. To set the comparison value TWO_DAY shipping in the rule for R1, perform the following steps:

   a. In the R1 column and the C1 condition row, click the cell and select the `TWO_DAY` check box from the drop-down menu.

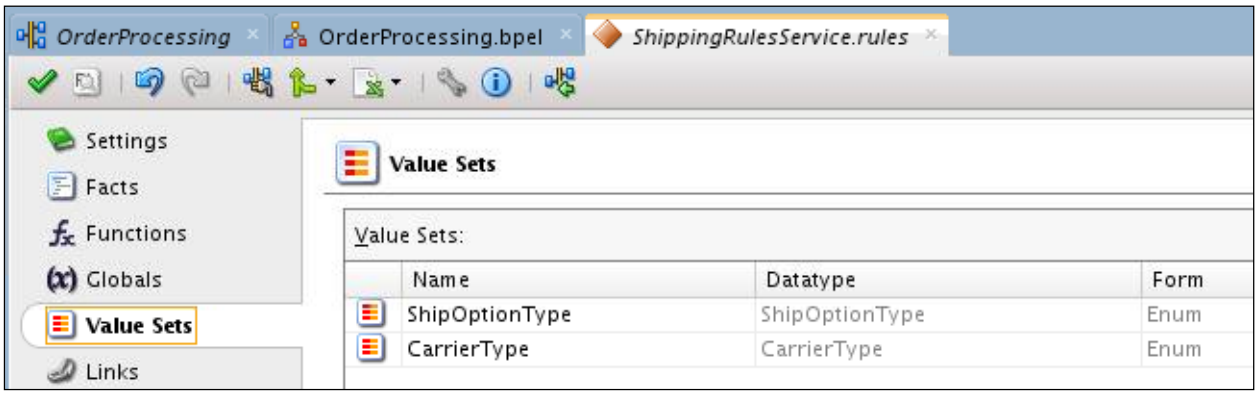
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Note: The value `TWO_DAY` is added to the selected cell.

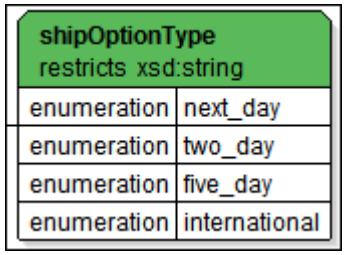
Have you considered where the list of values for the shipChoice element comes from? If you are curious, take a look at the Value Sets definitions in the `shippingRules.rules` dictionary, as in the following example:



Ask yourself, how did these value set definitions get created? Do their names offer any insight?

Apart from the fact that these value sets have an XML Fact type definition (refer to the Facts section in the `ShippingRules.rules` definition), these XML Facts were derived from XML elements, in `ordershipment.xsd`, which were used to define the Input and Output variables for the ShippingRules Business Rules component.

If you examine the `ordershipment.xsd` file, you can find the definitions for these types shown with the associated enumerations. For example, the `shipOptionType` definition in the XSD is as follows:



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

If you are curious, consider editing the corresponding ShipOptionType value set to view how its definition resembles the XSD element definition.

b. Create the action for the rule and set the action values.

1) In the first column under the Actions section, click the `<insert action>` text.

2) Double-click `assert new`.

   The Action Editor window opens.

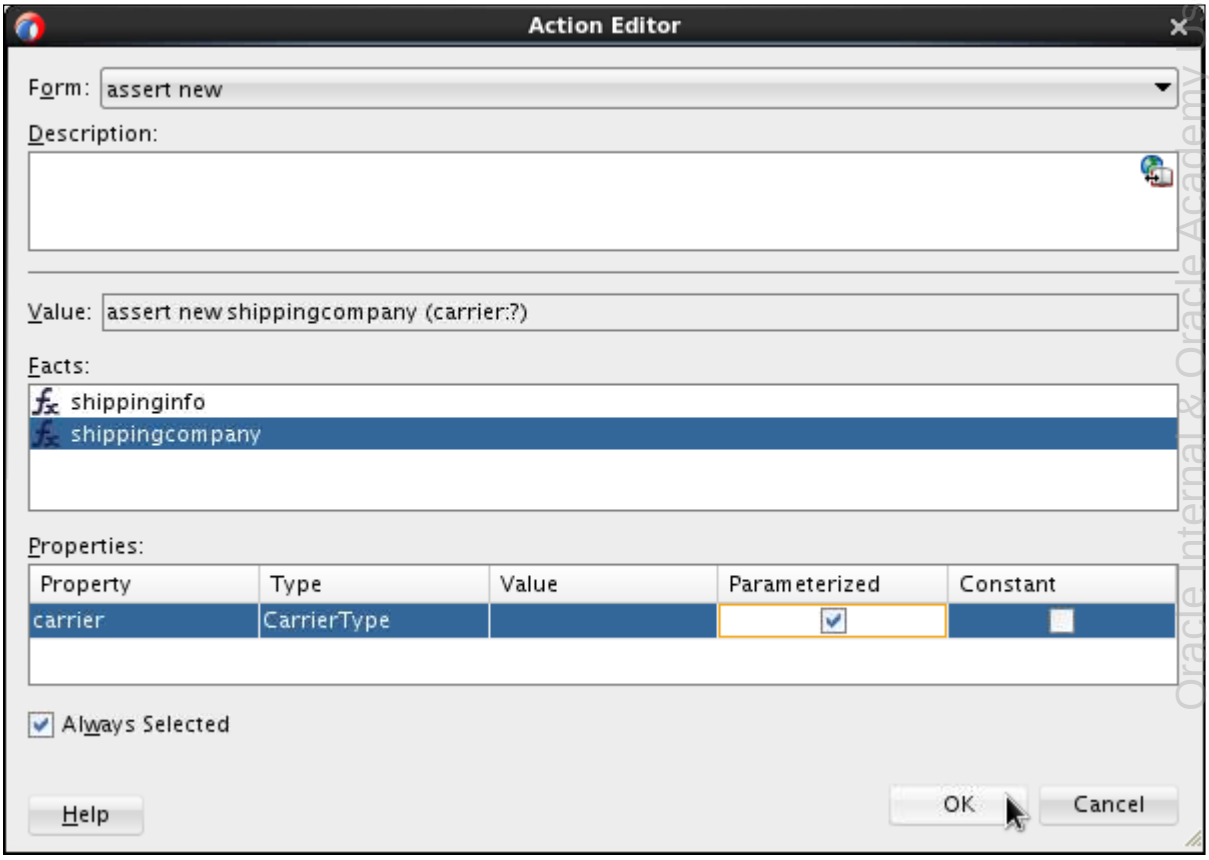
3) Select the `shippingcompany` entry in the Facts section.

4) In Properties for the `carrier` property, select Parameterized.

   Note: When you select Parameterized, a cell is added for each rule in the Decision Table, and the property can be set on a per-rule basis.

5) At the bottom of the window, select Always Selected.

6) Verify your work and click OK.

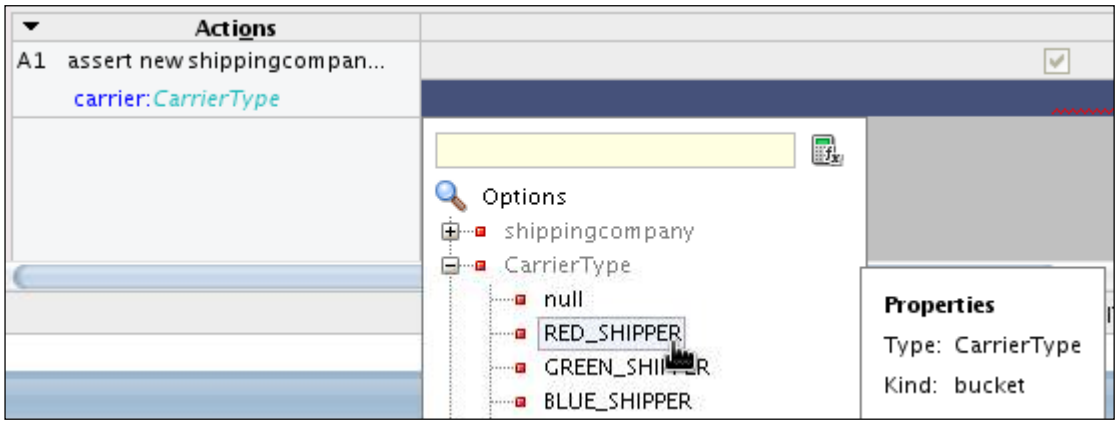


Note: The Value field is constructed from the selections you make. The Always Selected check box ensures that the action is always selected for each rule.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

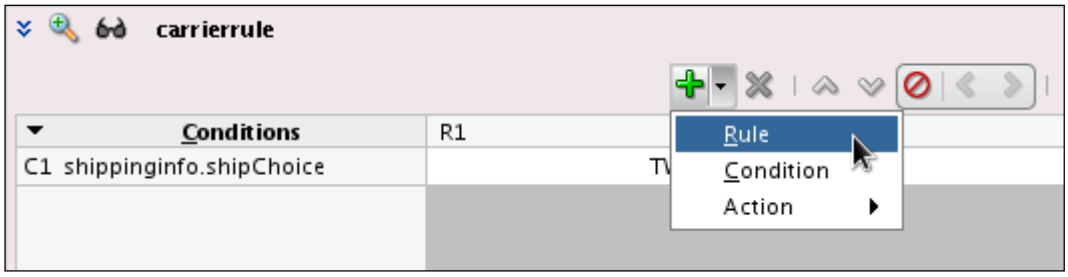
c. In the A1 row that is created for the `assert new shippingcompany` action, click the cell under the R1 column and select the CarrierType `RED_SHIPPER` value.



Note: The configuration that you have performed so far in the Decision Table effectively allows the business rule to apply the logic "If the shipping choice is `TWO_DAY`, then assign carrier `RED_SHIPPER`." The remaining steps configure similar logic for the other shipping choices and carriers defined in the value sets that you examined earlier.

7. Create the second rule in the Decision Table for the Green Shipper.

   a. On the toolbar at the top of the Decision Table, click the Create icon and select Rule.



   A new column is added to the table.

   b. To set the condition in the new column, click the cell in the C1 row and select the `FIVE_DAY` check box.

   c. In the A1 row for the new column, click the intersecting cell and select the CarrierType `GREEN_SHIPPER` from the drop-down menu.

8. To create the third rule in the Decision Table for the Blue Shipper, perform the following steps:

   a. On the toolbar at the top of the Decision Table, click the Create icon and select Rule.

   b. To set the condition comparison value in the new rule column, click the cell in the C1 row and select both the `NEXT_DAY` and `INTERNATIONAL` check boxes.

   c. To set the action values, click the A1 cell in the new column and select the CarrierType `BLUE_SHIPPER` from the drop-down menu.

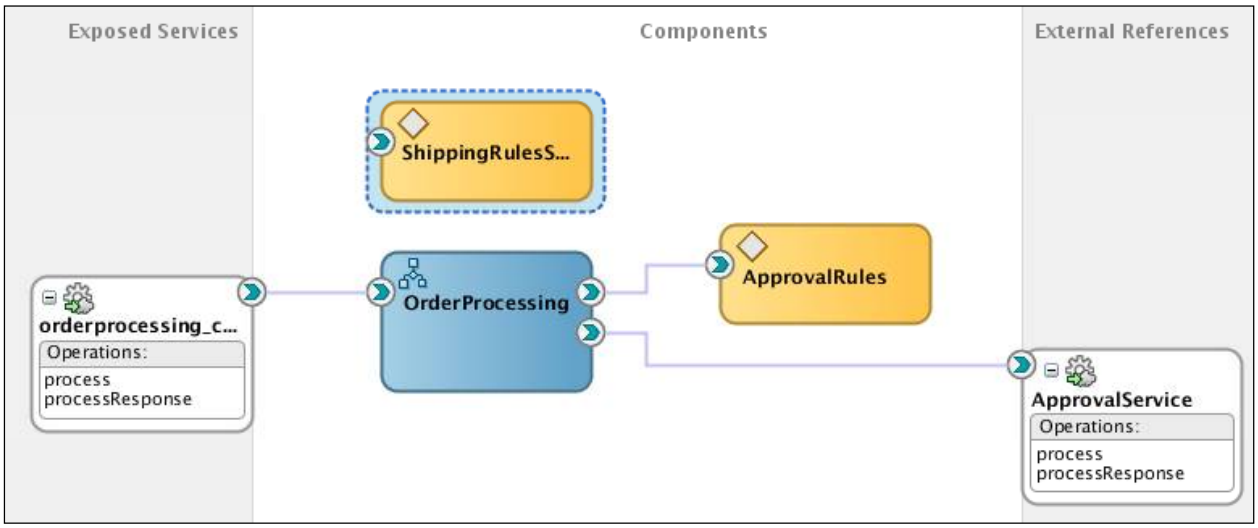
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

9. Verify your work.

Conditions	R1	R2	R3
C1 shippinginfo.shipChoice	TWO_DAY	FIVE_DAY	NEXT_DAY,INTERNATIONAL
Conflict Resolution			
Actions			
A1 assert new shippingcompany	☑	☑	☑
carrier:CarrierType	CarrierType.RED_SHIPPER	CarrierType.GREEN_SHIPPER	CarrierType.BLUE_SHIPPER

10. Save your work.
11. Close the `ShippingRules.rules` window.
12. Verify your assembly model.



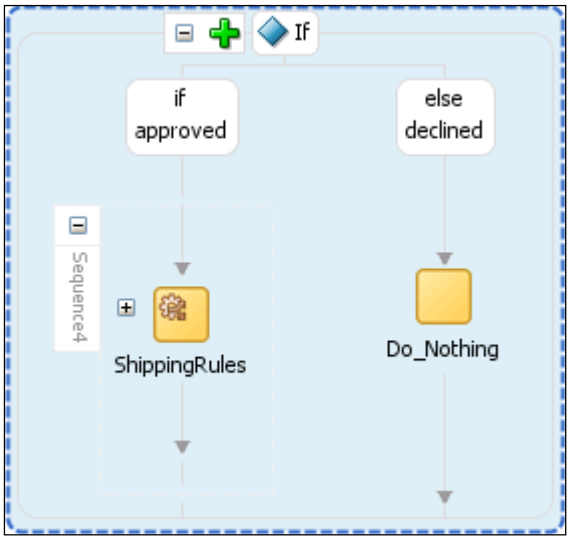
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 12-5: Modifying the BPEL Process [Optional]

Overview

In this section, you modify the BPEL process that was created earlier in this practice. You test whether or not the order was approved. If the order is approved, a business rule activity is employed to determine an appropriate carrier for the order. The order data is updated accordingly and returned to the calling client. The following image shows the portion of the business process that you are about to build:



Assumptions

This practice assumes that you have completed Practice 12-4 successfully, and that the `OrderProcessing` application is still deployed to the application server.

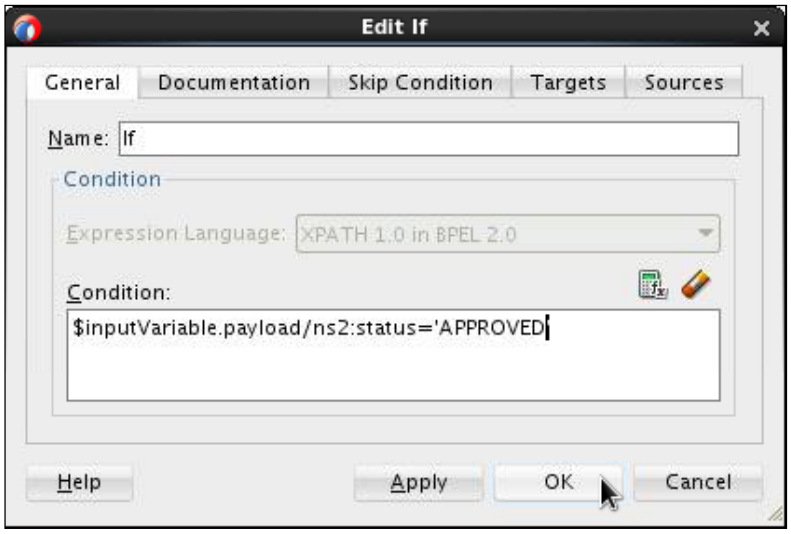
Tasks

Finishing Processing of Approved Orders

1. Open the BPEL Process OrderProcessing for editing.
2. Create and configure an If activity to test the approval status of the order.
   a. Collapse the If activity for clarity.
   b. Add a second If activity between the first If and the Assign_Output activities.
   c. Label the branches `'if APPROVED'` and `'else REJECTED'`.
   d. Set the conditional expression of the If to test whether `status=APPROVED`.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Configuring the <else REJECTED> Branch

3. Add an Empty activity to the <else REJECTED> branch. Name it Do_Nothing.

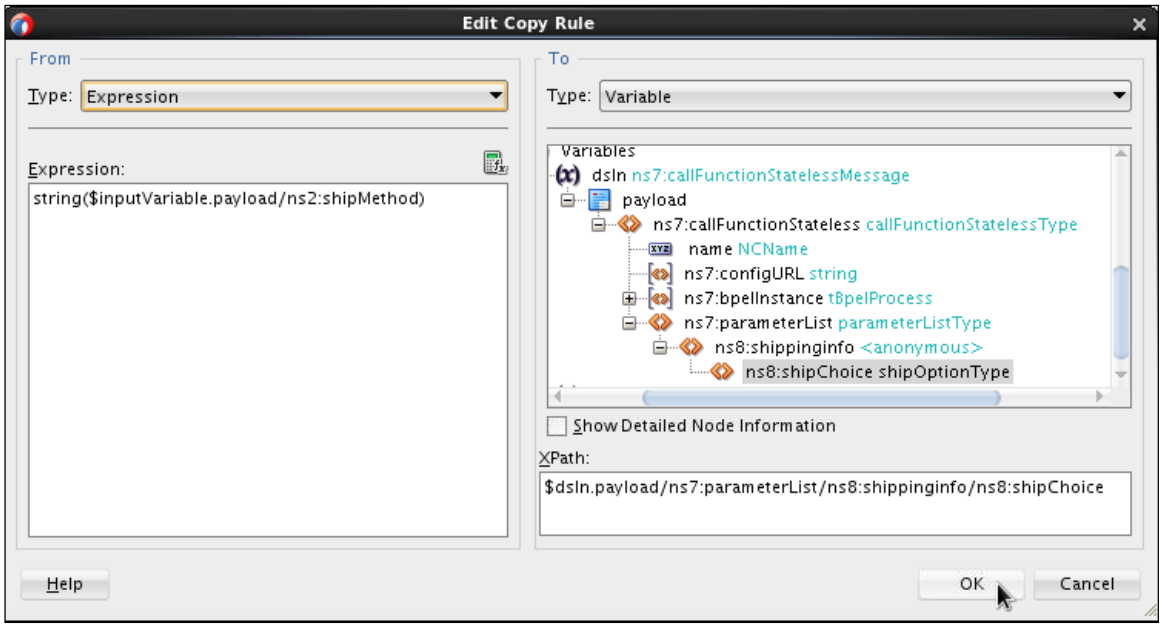
Configuring the <if APPROVED> Branch

4. Create and configure a business rule activity to choose the shipping company.
   a. Add a Business Rule activity to the <if APPROVED> branch.
   b. Right-click the new activity and select Edit.
   c. Name the rule ShippingRules.
   d. Specify the dictionary by selecting ShippingRules from the drop-down menu.
   e. Assign Input Facts.
      1) Click the Create icon.

         The Decision Fact Map window opens.
      2) In the left pane, select Expression from the Type drop-down menu.
      3) Use Expression Builder to define the expression `string($inputVariable.payload/ns4:shipMethod)`.
      4) In the right pane, select the shipChoice element of the dsIn variable.
      5) Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Note: Wrapping the `shipMethod` element in the `string` function as you did in the preceding step 3 is not strictly required. However, if you remove the `string` function, you get a compiler warning about incompatible types for the specified Assign Input Facts settings.

f. Assign Output Facts.

1) Click the Assign Output Facts tab.

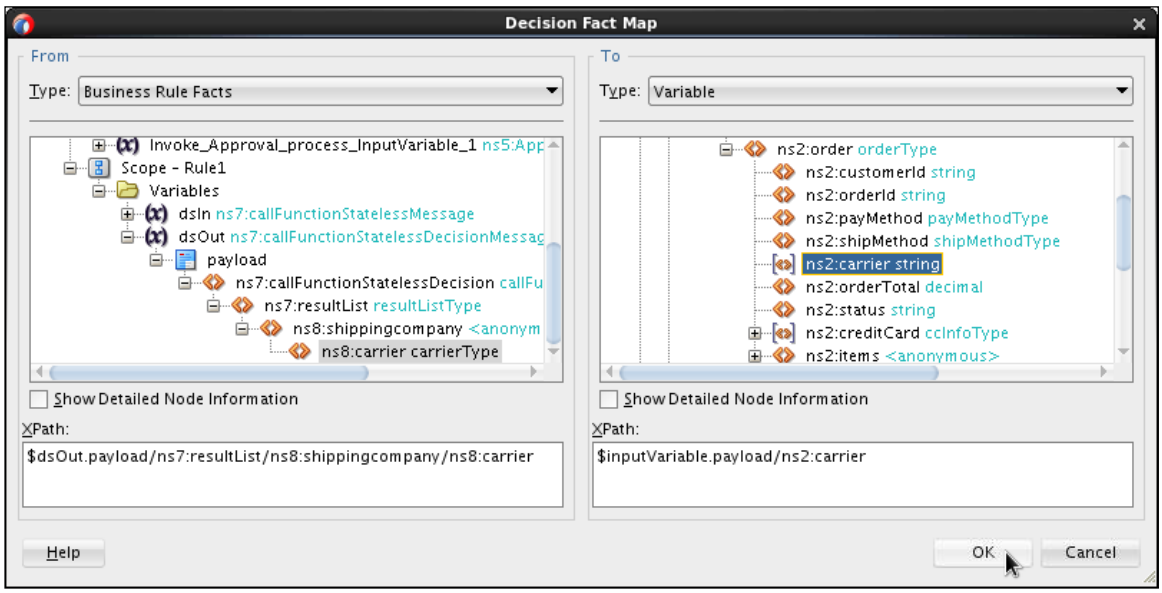
2) Click the Create icon.

   The Decision Fact Map window opens.

3) In the left pane, select the `carrier` element of the `dsOut` variable.

4) In the right pane, select the `carrier` element of `inputVariable`.

5) Verify your work and click OK.

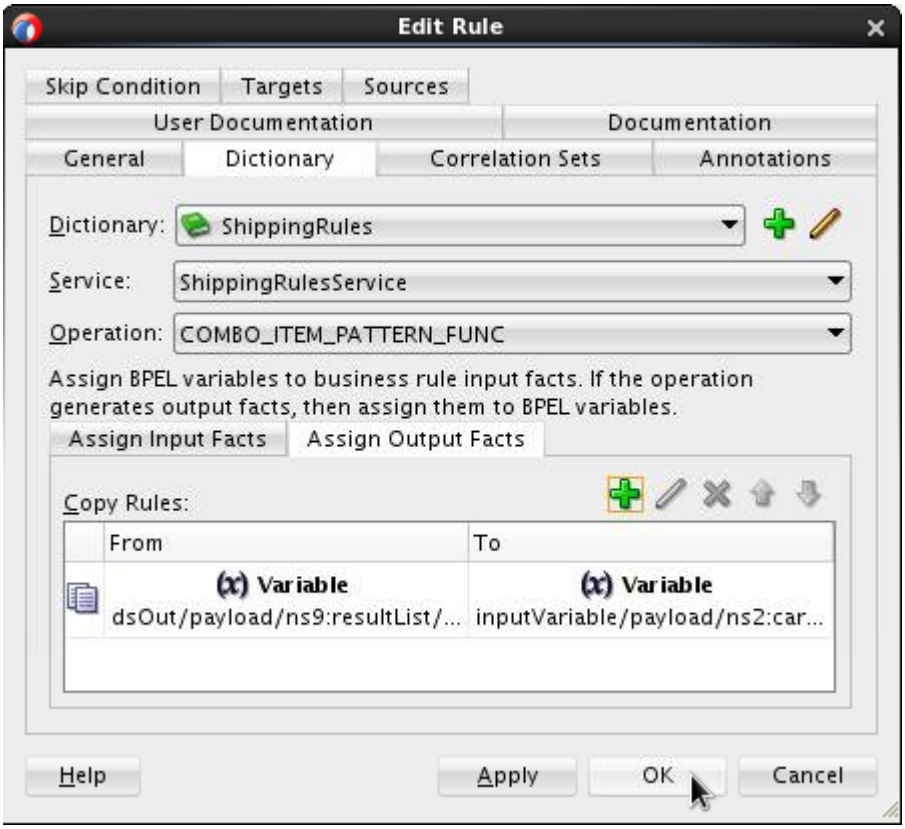


Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

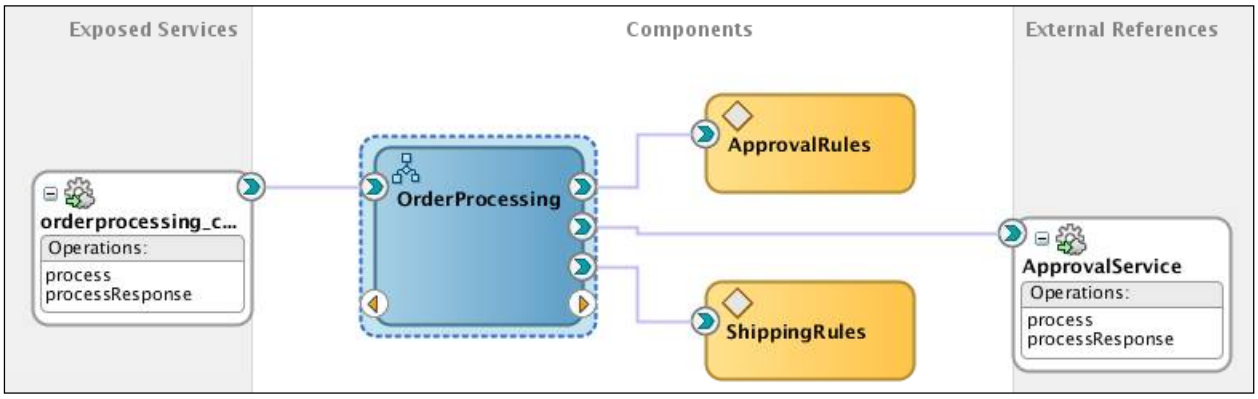
You are returned to the Edit Rule dialog box.

g. Verify your work and click OK.



5. Close `OrderProcessing.bpel`.

6. Verify that the components in your assembly model are correctly wired.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 12-6: Deploying and Testing the `OrderProcessing` Application [Optional]

Overview

In this practice, you redeploy and test the `OrderProcessing` composite application.

Assumptions

This practice assumes the following:

- You have completed Practice 9 for the lesson titled “Implementing Human Workflow and Notifications” successfully, and that the `Approval` application is still deployed to the application server.
- You have completed all work in Practice 12-5 successfully.

Tasks

Redeploying the Enrollment Composite Application

1. Redeploy the `OrderProcessing` application to IntegratedWebLogicServer.

Testing the Composite Application

In this section, you again submit an order whose total is greater than 4000 so that the `ApprovalRules` business rule returns a result that requires manual approval of the order. As in the previous practice, as employee `jcooper`, you check your email to read the notification about the assigned task, and then log in to the Worklist application to approve the order.

2. Open the Oracle Enterprise Manager page (http://localhost:7101/em).
   a. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “OrderProcessing [1.0]” link.
   b. On the “OrderProcessing [1.0]” home page, click Test.
   c. On the Test Web Service page, scroll down to the Input Arguments section to the Request tab and select XML view.
   d. Replace the XML data with the contents of the file `/home/oracle/labs/files/xml_in/businessRules_input.xml`.
   e. Click Test Web Service.
3. Manually approve the order in the Worklist application:
   a. Access the Oracle BPM Worklist application at http://localhost:7101/integration/worklistapp.
   b. Log in to the application as `jcooper` with the password `welcome1`.

      On the Worklist home page, the Inbox is selected and the Manual Approval task is present in the top-right pane.
   c. Click the Manual Approval task entry.

      The order details are displayed in the lower-right pane.
   d. Click Approve.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Ship Method value is `two_day`. Recall that this affects the shipping company that is used for the shipping request.

e. Log out of the Worklist application and close the webpage.

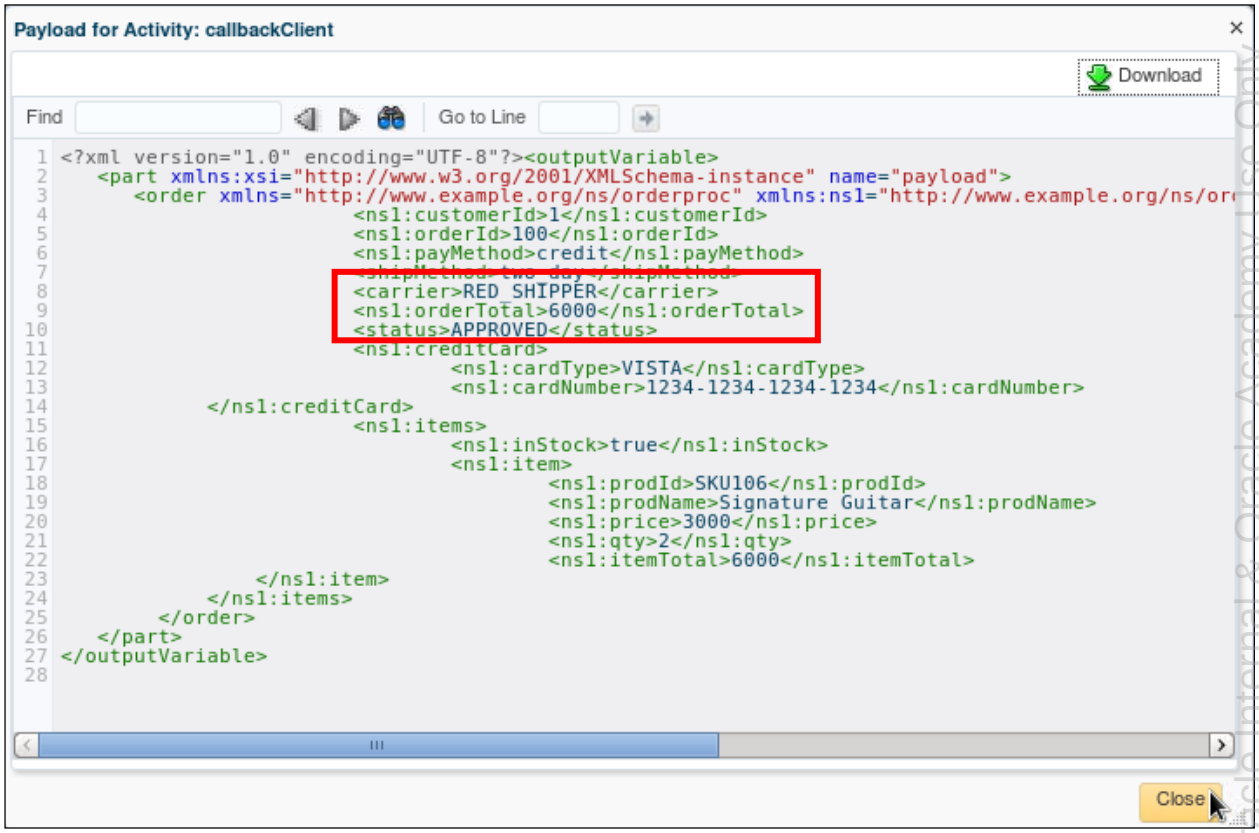
4. Return to the Flow Trace page and click the Refresh icon on the top-right corner of the page.

5. Scroll down the Trace tree and verify that the carrier selected is `RED_SHIPPER`.

   a. Click the BPEL process.

   b. Expand the payload for the callbackClient activity.

   c. Verify that the shipMethod is `two_day`, and the resulting carrier value is `RED_SHIPPER`.



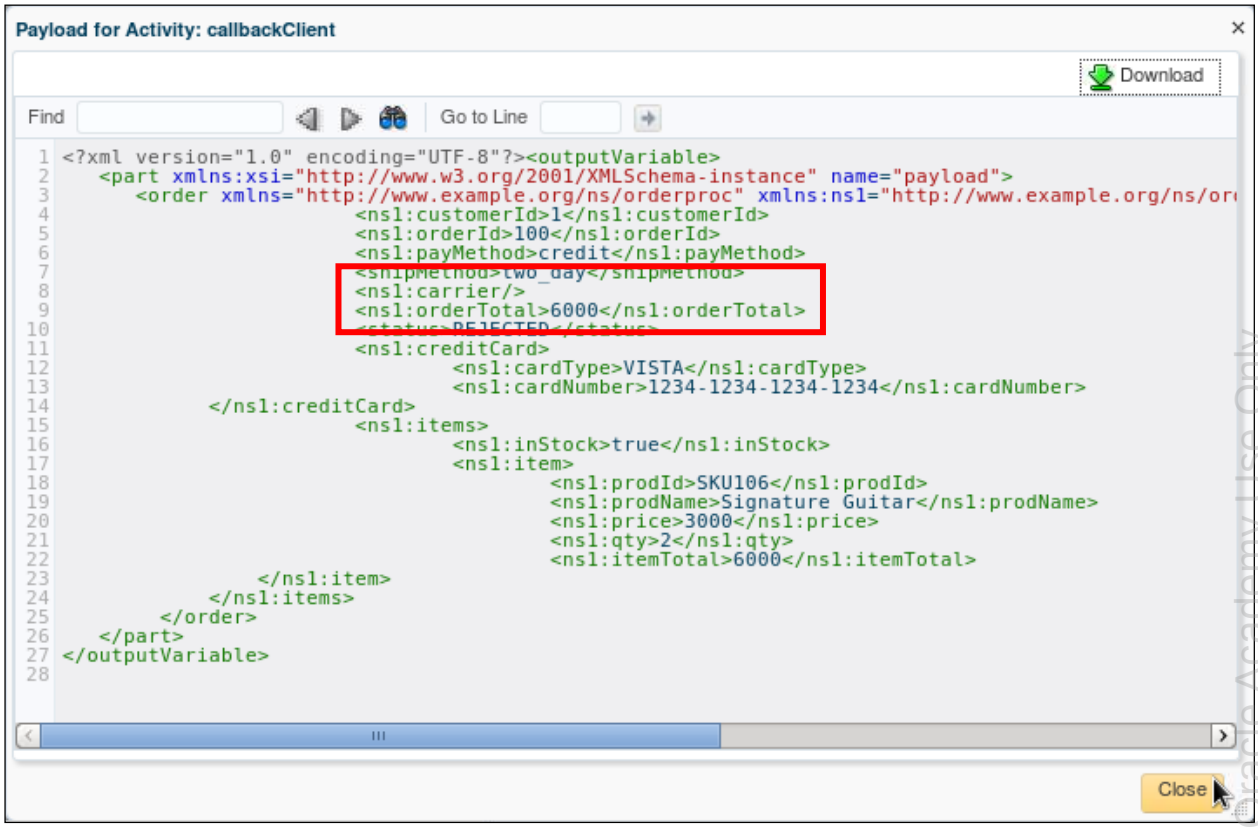
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Additional Optional Test

If you have time, you can submit some additional orders. Consider these scenarios:

6. Initiate a test with an order that requires manual approval and, at the manual approval stage, reject it.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 13: Designing and Managing Business Events

Chapter 13

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 13: Overview

Practices Overview

The goal of this practice is to create and implement event publication and subscription services.

The database provided for this course includes orders for processing and shipping products. Some of these orders have a `waiting` status, which indicates that they were placed in waiting until the products for that order are available in inventory.

In this scenario, you create two composite applications. The first accepts a product ID and the amount (quantity) of products to add to the database inventory. It then publishes an event that includes the product ID and the quantity added.

The second composite application includes a Mediator that subscribes to the published event. When the event is received, the Mediator invokes a BPEL process. This process retrieves details for waiting orders that include the newly restocked item among their list of items. If there is sufficient new inventory to fill the order for that item, it then updates the order status from "waiting" to "active."

You create both these composite applications from templates. These templates provide a starting point that includes all components, except those directly related to publishing and/or subscribing to events. You build the event-specific pieces from the beginning.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 13-1: Publishing an Event

Overview

In this practice, you create a new project from a template. Within this project, you define an event and publish it.

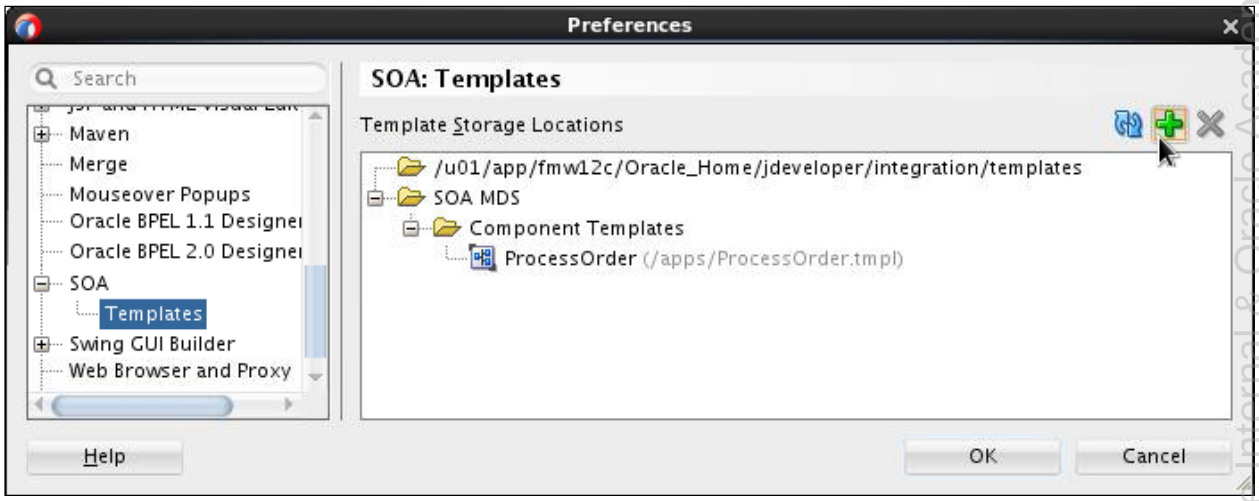
Assumptions

This practice makes no assumptions.

Configuring the Templates Directory in JDeveloper

In an earlier practice titled “Using a Component Template,” you imported the contents of a JAR file into the MDS Repository, and then used the template that was contained in that JAR file. Templates can also be stored and shared as individual files. You begin this practice by pointing JDeveloper to a templates directory, and then using one of the templates in that directory as the starting point for a new project.

1. Add a new Template directory to JDeveloper preferences.
   a. In the JDeveloper Application Navigator menu, select Tools > Preferences > SOA > Templates.

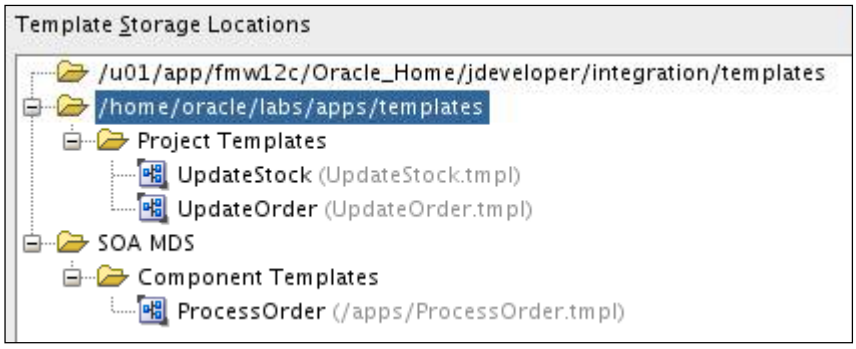


   b. Click the Add Storage Location icon.
   c. Navigate to `/home/oracle/labs/apps/templates`.
   d. Click Select.

      The new directory is added and the templates that it contains are listed.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



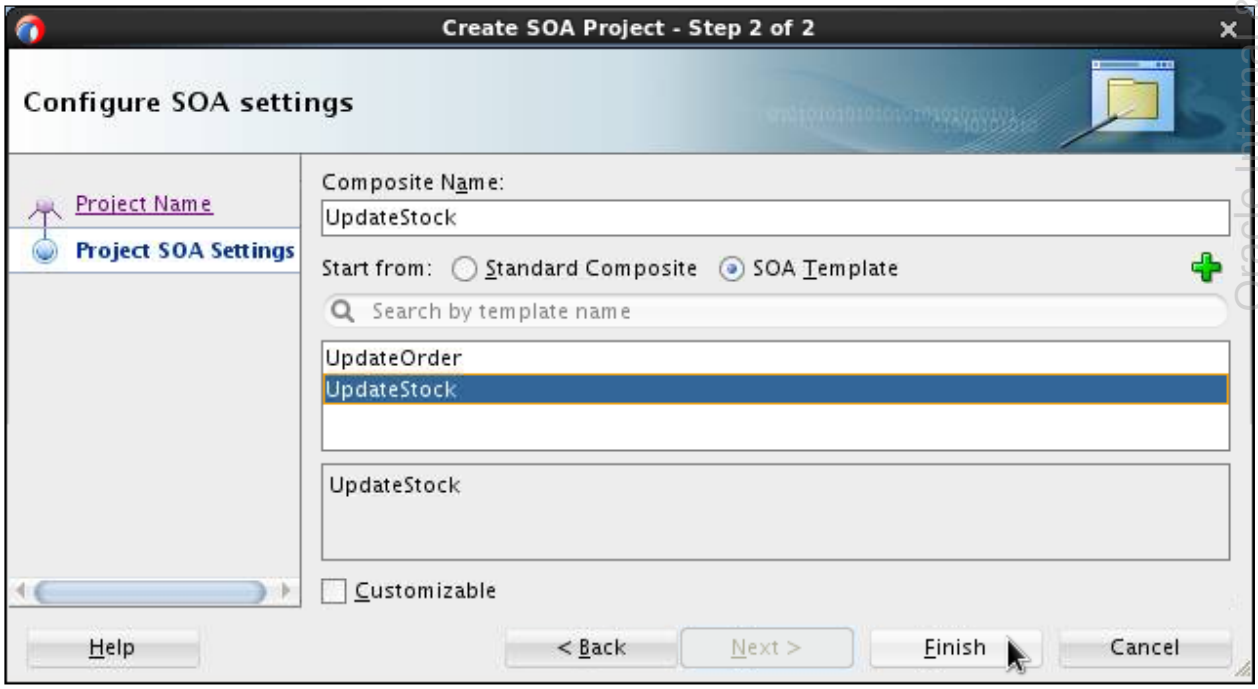
e. Click OK.

Creating a Project from a Template

2. Create a new SOA project from a template.
   a. In JDeveloper, verify that the BPELProjects application is selected.
   b. From the Application Navigator menu, select New > Project.

      The New Gallery appears.
   c. Select SOA Project and click OK.

      The Create SOA Project wizard appears.
   d. Name the project `UpdateStock`.
   e. Click Next.
   f. Select the Start from: SOA Template option.
   g. Select the UpdateStock template.
   h. Verify your choices and click Finish.

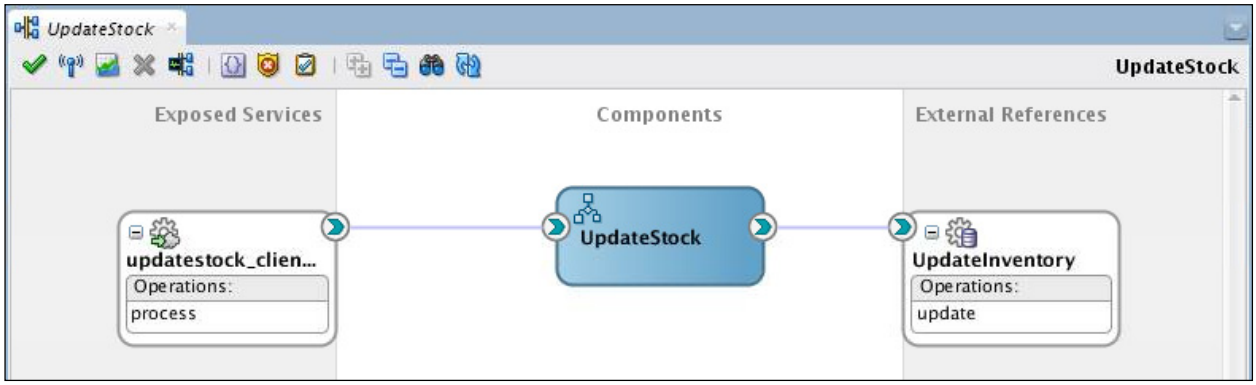


The composite overview window appears.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

i. Verify your project and save your work.



Creating an Event

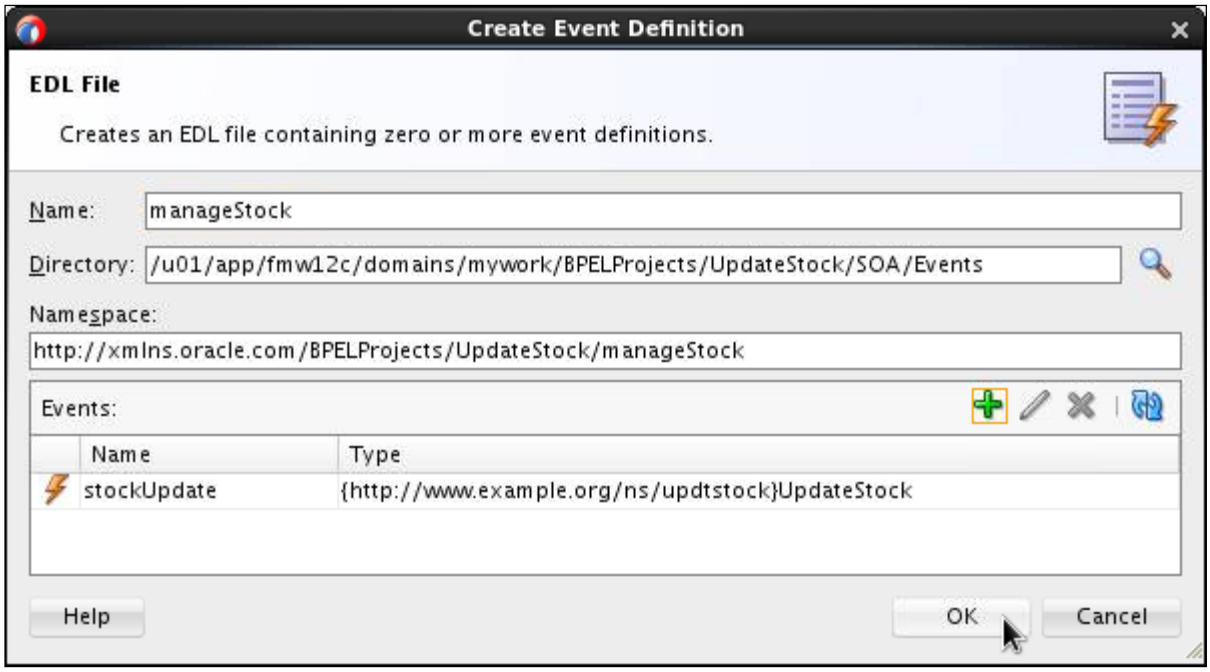
3. Create and configure an event.
   a. From the JDeveloper main menu, select File > New > Event Definition.

      The Create Event Definition window opens.
   b. Name the Event Definition `manageStock`.
   c. Add an event to the Event Definition file.
      1) Click the Create icon.
      2) Name the event `stockUpdate`.
      3) Use the Browse icon and Type Chooser to navigate to the `UpdateStock` element of the `updateStock.xsd` file (in the current project).

         This describes the format of the event.
      4) Click OK to close the Type Chooser.
   d. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



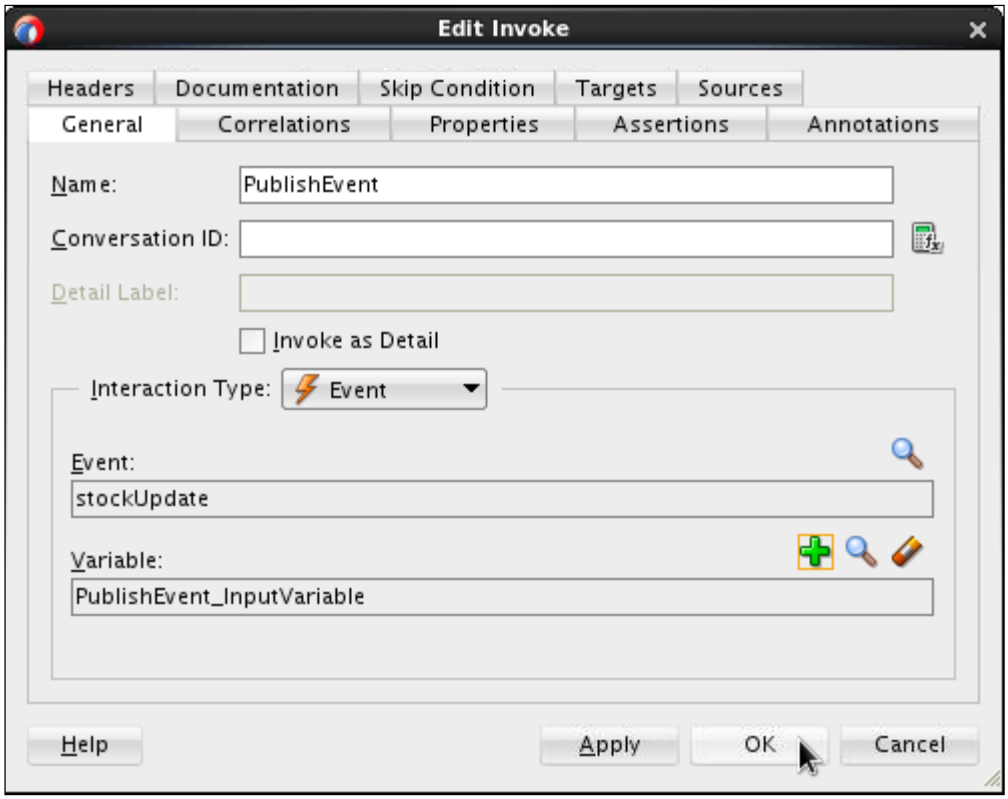
e. Close the `manageStock.edl` file.

Note: The `manageStock.edl` file can be reopened to create, modify, or delete event definitions.

4. Add an Invoke activity to publish an event from the BPEL process.
   a. Open the UpdateStock BPEL process.
   b. Add an Invoke activity to the process, just after `Invoke_UpdateInventory`.
   c. Right-click the Invoke activity and select Edit.
   d. Rename the Invoke activity `PublishEvent`.
   e. Select the Event interaction type.
   f. Specify the event to publish.
      1) Click the Browse button to display the Published Events window.
      2) Click the Publish New Event icon to display the Event Chooser.
      3) Select the `stockUpdate` event.
      4) Click OK to close the Event Chooser.
      5) Click OK to close the Published Events window.
   g. Create an Input variable.
   h. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

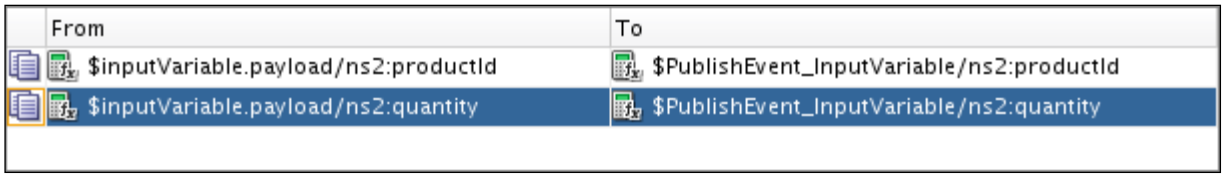
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



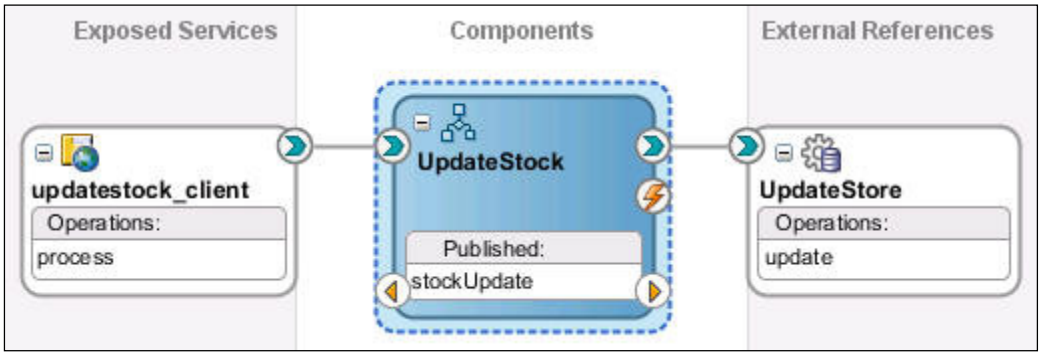
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

5. Create and configure an Assign activity to populate the event before publication.
   a. Add an Assign activity to the process just before `PublishEvent`.
   b. Name the new activity `Assign_Event`.
   c. Right-click the new activity and select Edit.
   d. Copy the Input variable `productId` to the `productId` element of the `PublishEvent` Input variable.
   e. Copy the Input variable `quantity` to the `quantity` element of the `PublishEvent` Input variable.
   f. Verify your work and click OK.



6. Save your work and close `UpdateStock.bpel`.
7. Verify your composite application.



8. Deploy the `UpdateStock` application to IntegratedWebLogicServer.

   Remember to select the “Overwrite any existing composites with the same revision ID” check box.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 13-2: Subscribing to an Event

Overview

In this practice, you create another new project from a template. Within this project, you add a Mediator to subscribe to the event. You then test publication and subscription.

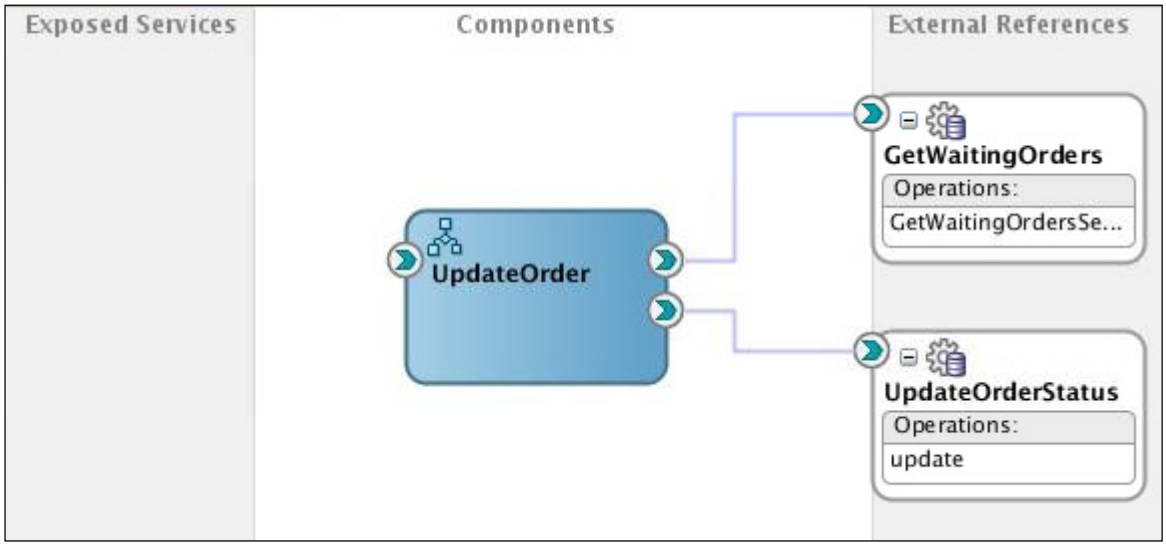
Assumptions

This practice assumes that you successfully completed Practice 13-1.

Tasks

Creating a Project from a Template

1. Create a new SOA project from a template.
   a. From the JDeveloper Application Navigator menu, select New > Project.
      The New Gallery appears.
   b. Select SOA Project and click OK.
      The Create SOA Project wizard appears.
   c. Name the project `UpdateOrder`.
   d. Click Next.
   e. Select the Start from: SOA Template option.
   f. Select the UpdateOrder template.
   g. Verify your choices and click Finish.
      The composite overview window is opened.
   h. Verify the composite and save your work.



Oracle Internal & Oracle Academy Use Only

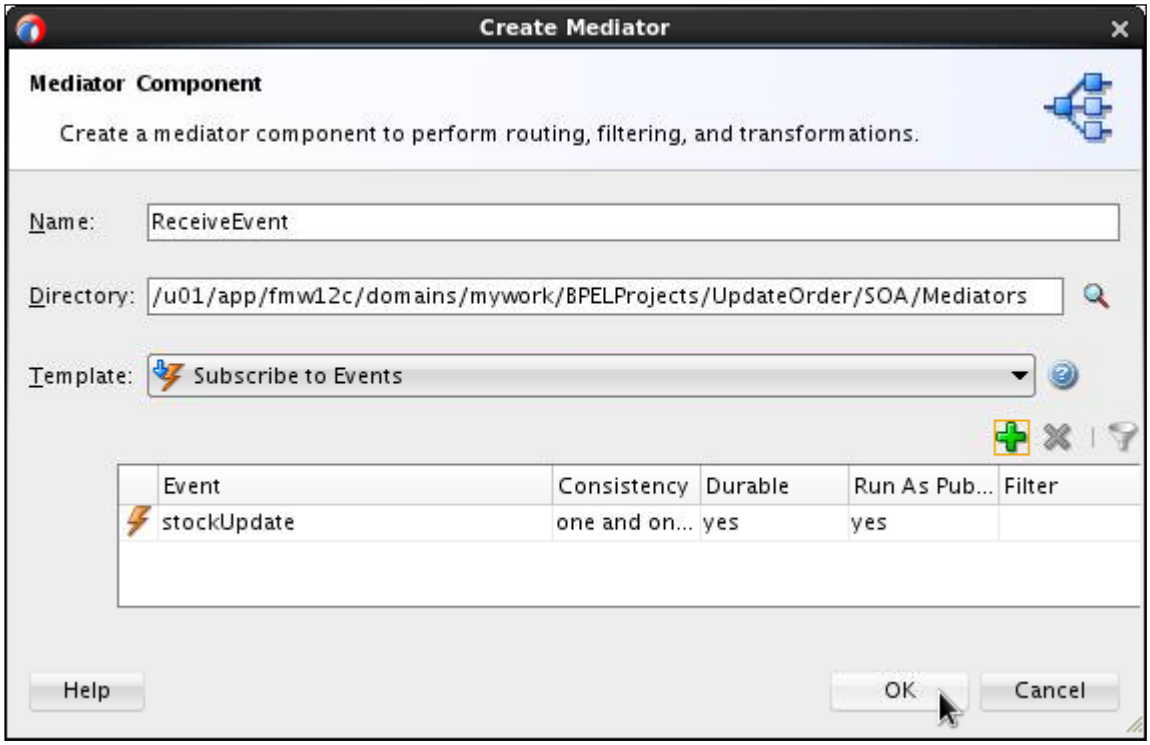
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Adding a Mediator

2. Create and configure a Mediator component.
   a. Add a Mediator to the Components swimlane in the `composite.xml` editor.
   b. Name the Mediator `ReceiveEvent`.
   c. Select the “Subscribe to Events” template.
   d. Click the Subscribe to a New Event icon.

      The Event Chooser opens.
   e. Highlight the `stockUpdate` event.
   f. Click OK.

      The Localize Files dialog box is displayed.
   g. Deselect the “Maintain original directory structure” check box and click OK.
   h. Verify your work and click OK.



3. In the `composite.xml` design window, wire the `ReceiveEvent` Mediator to the `UpdateOrder` BPEL process.
4. Create and configure the Mediator transformation.
   a. In the `composite.xml` design window, right-click the Mediator `ReceiveEvent` and select Edit.
   b. To the right of the Transform Using field, click the Mapper file icon.

      The Event Transformation Map window opens.
   c. Click the Create Mapping icon.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The Create Transformation Map window opens.

d. Click OK.

An empty mapping is opened.

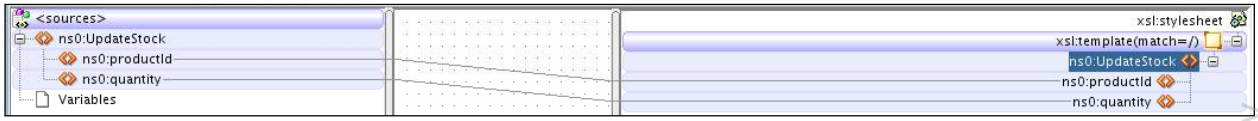
e. Click OK to close the Event Transformation Map window.

f. In the Mapper, map `updateStock` to `updateStock`.

The AutoMapper dialog box appears.

g. Click OK to accept all defaults.

h. Verify and save your work.



i. Close the mapping and the ReceiveEvent.mplan editors.

5. Deploy the UpdateOrder application to IntegratedWebLogicServer.

Remember to select the “Overwrite any existing composites with the same revision ID” check box.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 13-3: Testing Event Publication and Subscription

Overview

In this practice, you perform a test of the event publication and subscription logic that you created. You initiate a test of the `UpdateStock` composite application, which publishes a `StockUpdate` event. The `UpdateOrder` composite application receives the event as a subscriber to the event.

Assumptions

This practice assumes that you have completed practices 13-1 and 13-2 successfully.

Tasks

1. Create orders in the database for processing.
   a. In JDeveloper, open the file `/home/oracle/labs/scripts/SQL/JDBreset_order_status.sql`.
   b. Click the Run Script icon.
      The Select Connection dialog box appears.
   c. Click OK to accept the SOA connection.
      The STATUS column of the `BCA_ORDERS` table of the database is updated.
2. Repeat step 1 to run `JDBlist_waitingorders.sql`.
   Three orders with a status of “waiting” are listed.

Query Result | SQL | All Rows Fetched: 3 in 0.026 seconds

	ORD_ID	CUST_ID	PAY_METHOD	SHIP_METHOD	CARRIER	CARD_TYPE	CARD_NUMBER	TOTAL_PRICE	DATE_ORDERED	DATE_SHIPPED	STATUS
1	1001	1	credit	two_day	(null)	ACARD	5678-5678-5678-5678	6500.00	2014-06-27	(null)	waiting
2	1002	2	credit	next_day	(null)	VCARD	1234-1234-1234-1234	4190.00	2014-06-27	(null)	waiting
3	1003	1	credit	five_day	(null)	ACARD	5678-5678-5678-5678	12000.00	2014-06-27	(null)	waiting

   Each of these orders includes an order for one or more of the SKU106 item (a total of seven units). In the next steps, you update the inventory for this item.

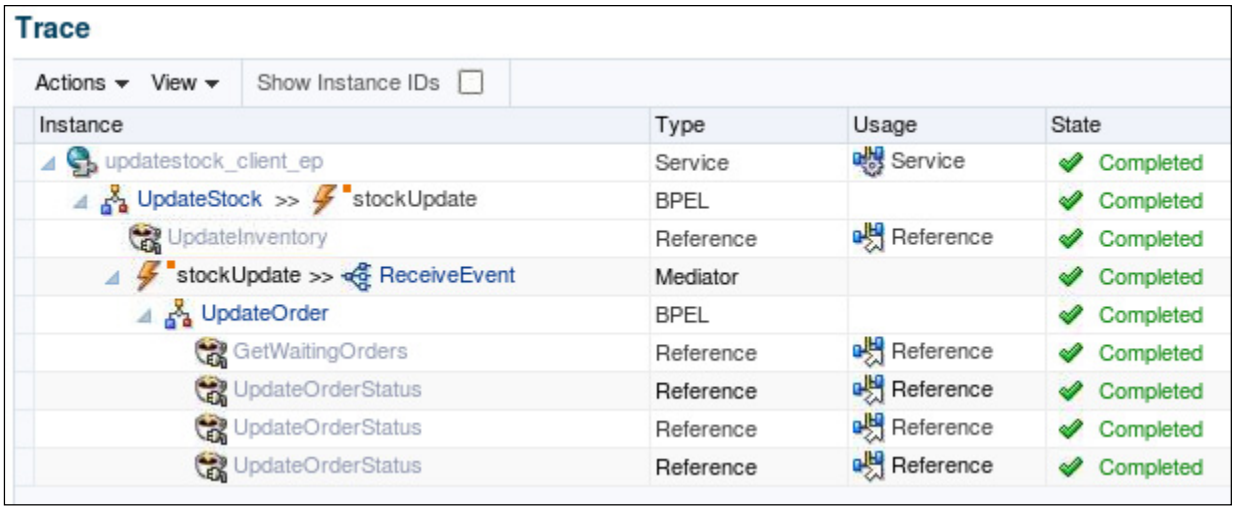
   Tip: Leave the two SQL files open in JDeveloper. You use them again in this practice.
3. In Enterprise Manager, navigate to the test page for the `UpdateStock` application.
   a. Open Enterprise Manager.
   b. Navigate to the `UpdateStock` application.
   c. On the `UpdateStock` home page, click Test.
      The test page opens.
4. Test the application.
   a. On the Request tab, in the tree view, enter the product ID `SKU106` and a quantity of `8`.
   b. Click Test Web Service.
      The Response tab displays the message “The web service invocation was successful.”

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

c. Click Launch Flow Trace.

The Flow Trace page reveals the current state of the test.



5. In JDeveloper, run `JDBlist_waitingorders.sql`. (See step 1 if needed.)

   No waiting orders are displayed.

	ORD_ID	CUST_ID	PAY_METHOD	SHIP_METHOD	CARRIER	CARD_TYPE	CARD_NUMBER	TOTAL_PRICE	DATE_ORDERED	DATE_SHIPPED	STATUS
1	1001	1	credit	two_day	(null)	ACARD	5678-5678-5678-5678	6500.00	2014-08-30	(null)	active
2	1002	2	credit	next_day	(null)	VCARD	1234-1234-1234-1234	4190.00	2014-08-30	(null)	active
3	1003	1	credit	five_day	(null)	ACARD	5678-5678-5678-5678	12000.00	2014-08-30	(null)	active

6. In JDeveloper, run `JDBreset_order_status.sql`.

   Orders are restored to a waiting state.

7. Test the application again, but supply a quantity of 3 this time. (See step 4 if needed.)

   Compare the Flow Trace to the preceding screenshot. Do they differ? How?

8. In JDeveloper, run `JDBlist_waitingorders.sql`. (See step 1 if needed.)
   - How many waiting orders are listed? Why?
   - **Hint:** You may need to drill down into the UpdateOrder BPEL process to answer this question.

9. In JDeveloper, close the open SQL scripts.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 14: Testing and Debugging

Chapter 14

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 14: Overview

Practices Overview

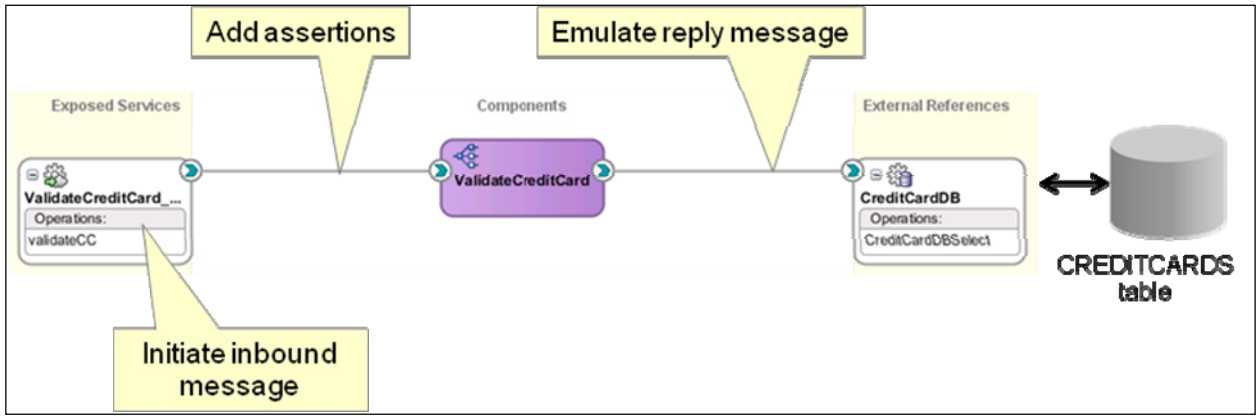
In this practice, you create and execute a test suite that contains two test cases for a composite application.

Recall that in an earlier practice, you built a composite application to validate a credit card purchase by checking the card number and the credit limit against the information in a database. Suppose that your company is preparing to accept private cards that begin with the four digits 9000. Since this modification, the routing rule has not been tested. The reason for the path not being tested is that the `BCA_CREDITCARDS` database table does not have credit cards that begin with the digits 9000. However, with the use of test cases, you have the ability to emulate the Database adapter response to return a valid or invalid status for credit cards that begin with 9000 as if the credit card existed in the database.

With the preceding scenario in mind, in this practice, you create a test suite for the CCValidate composite process and the following test cases:

- Test a credit card number starting with the digits 9000 and returning a `VALID` status.
- Test a credit card number starting with the digits 9000 and returning an `INVALID` status.

The following image illustrates these test case scenarios:



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 14-1: Creating a Test Suite for the CCValidate Composite

Overview

In this practice, your task is to create a test suite in the CCValidate composite application. When you create a test suite, you also create its first test case. The test case that you create:

- Emulates a request that contains a credit card number beginning with `'9000'` to test the message flow path through the Mediator component to the CreditCardDB service
- Emulates a `VALID` response being received as a reply from the CreditCardDB service. The response is emulated because the `BCA_CREDITCARDS` table in the database does not have any credit cards that begin with `'9000'`.

Assumptions

This practice assumes that you successfully created and deployed the CCValidate project in Practice 5 for the lesson titled “Using JMS and JDBC Adapters.”

Tasks

1. In the JDeveloper window, ensure that the Basics name is selected from the Application drop-down menu, and that the CCValidate project and its SOA Content folder are expanded.
2. Create the test suite with its first test case.
   a. In the Application Navigator, right-click CCValidate > SOA Content > testsuites and select Create Test Suite.
   b. Name the suite `testsuite_vcc`.
   c. Use the information in the following table to define the test:

Step	Pane	Action
1	Name	Name: `test_valid_9000` Click Next.
2	Service and Operation	Click Next.

3	Input Message	Click Generate Sample.
		Replace the generated data:
		`<CCNumber>: 9000-1234-1234-1234`
		`<amount>: 200`
		Click Next.

4	Output Message	Click Generate Sample. Replace the generated data: <status>: VALID Click Finish.

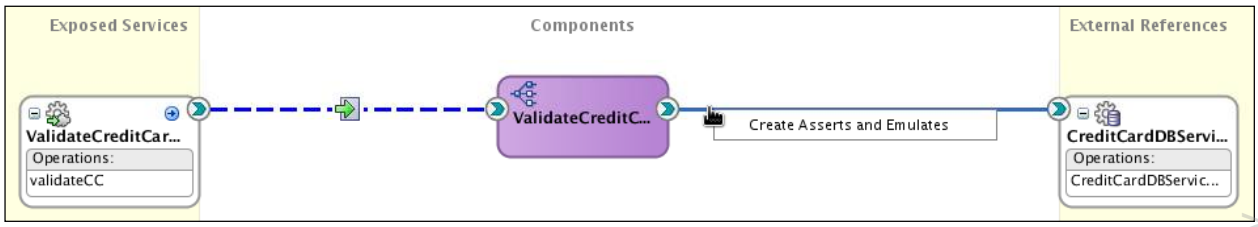
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

The `test_valid_9000.xml` file is opened in the test case editor window, which resembles the composite assembly model, except that the Exposed Services and External References columns are in a different color.

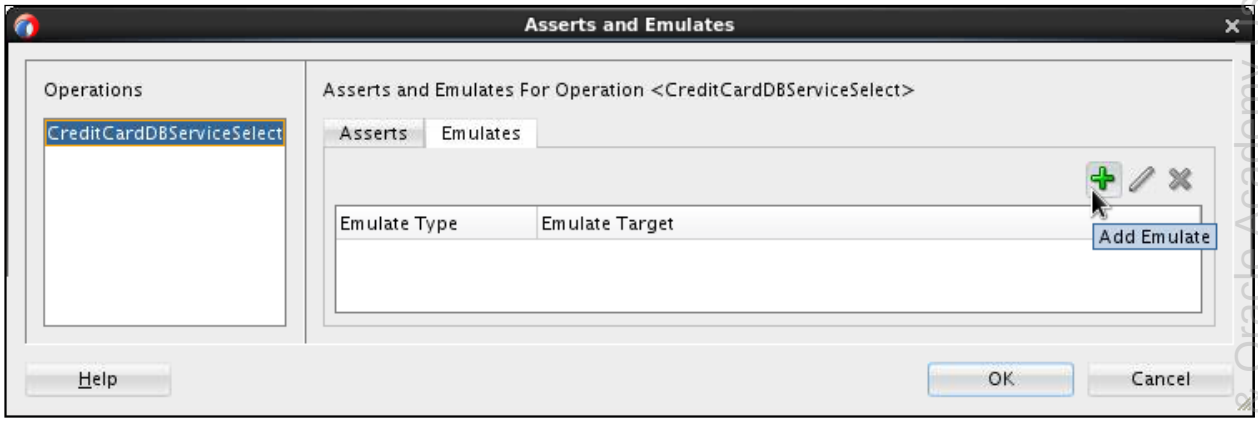
3. Create the emulated output of `VALID` from the CreditCardDB service for the `test_valid_9000` test case.

   a. Right-click the wire that connects the ValidateCreditCard Mediator component with the CreditCardDB service and select Create Asserts and Emulates.



   The Asserts and Emulates window opens.

   b. Click the Emulates tab and click the Add Emulate icon.



   The Create Emulate window opens. Observe that the Emulate Output option is selected.

   c. Generate and modify the message contents.

      1) Click Generate Sample.

         The Enter Value section is populated with a generated message. The initial generated XML sample contains three <`BcaCreditcards`> elements with their associated child elements.

      2) Delete the last two <`BcaCreditcards`> elements and the child elements they contain.

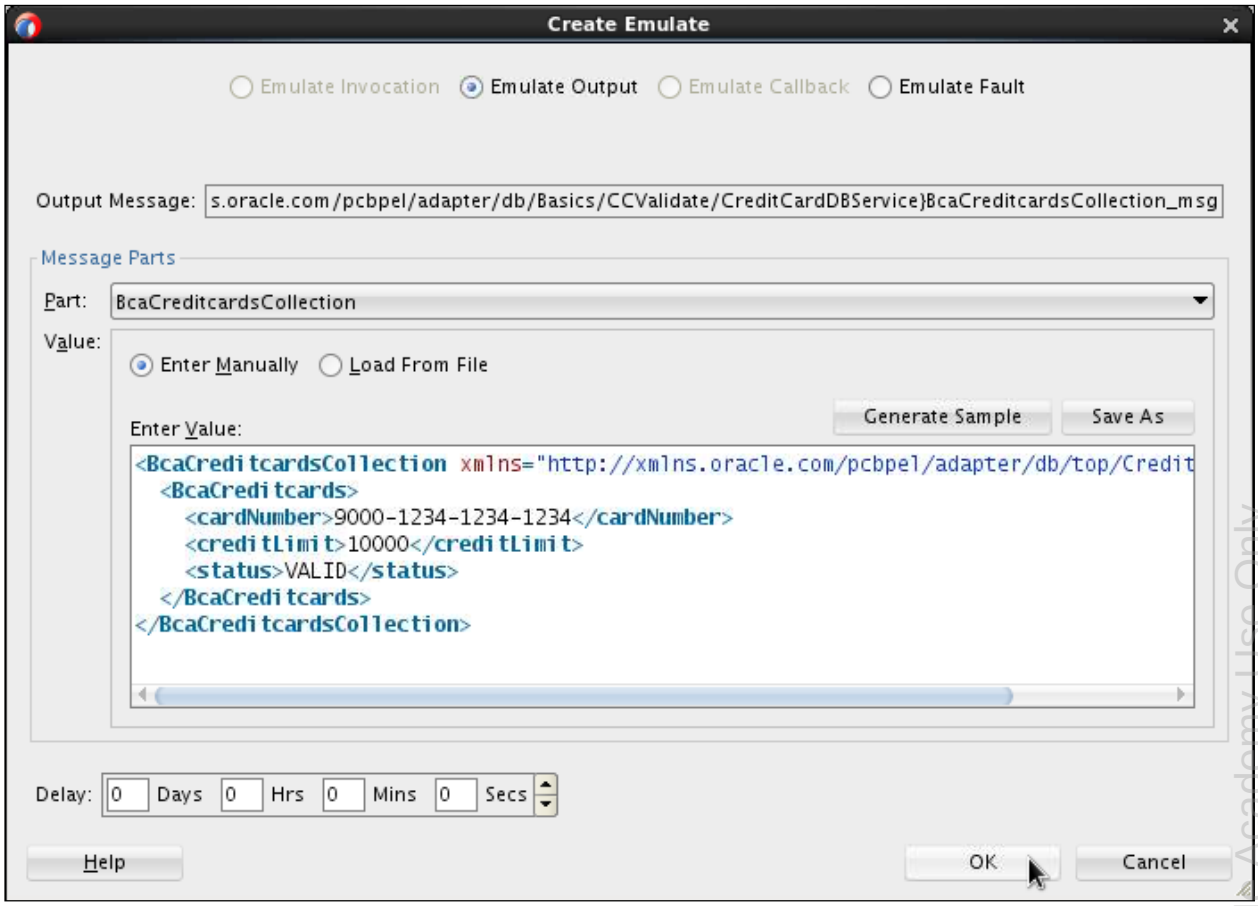
      3) In the remaining <`BcaCreditcards`> element and its child elements, replace the data as follows:

         - `<cardNumber>: 9000-1234-1234-1234`
         - `<creditLimit>: 10000`
         - `<status>: VALID`

      4) Verify your work and click OK to create the emulated message.

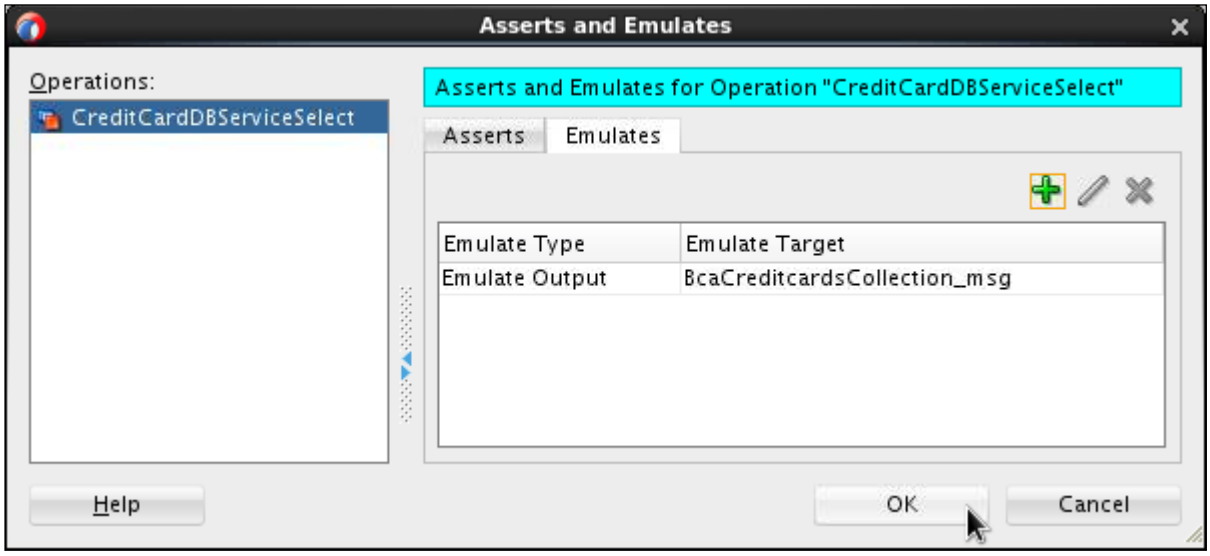
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



You are returned to the Asserts and Emulates window.

d. Confirm the addition of the emulated output (response) message and click OK.

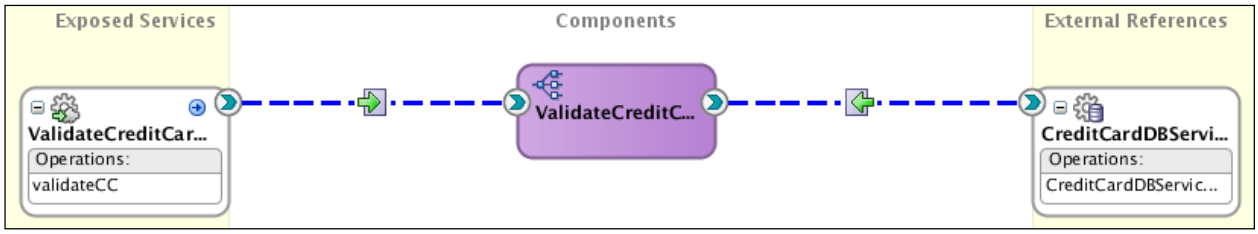


You are returned to the `test_valid_9000.xml` window.

e. Verify that the emulated output message is added with the presence of a grey box icon containing a green arrow that points to the left. In addition, the wire between the Mediator component and its target service is displayed as a dotted line.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



4. Save your work and close the `test_valid_9000.xml` window.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 14-2: Creating Test Cases in the CCValidate Test Suite

Overview

In this practice, you create two more test cases for the `testsuite_vcc` test suite that was created in Practice 11-1. In this test suite, the initiating message data is slightly different and the emulated output (response) from the CreditCardDB service is `INVALID`. The final test case uses an initiating message to test the message path flow through the application, and uses assertions to check whether the actual response that is returned to the initiator is an expected value of `VALID` or `INVALID`.

Assumptions

This practice assumes that you have completed Practice 11-1.

Tasks

1. Create the new test.
   a. In the JDeveloper Application Navigator, right-click the testsuites > testsuit_vcc > tests folder and select Create Test.
   b. Use the information in the following table to define the test:

Step	Pane	Action
1	Name	Name: `test_invalid_9000` Click Next.
2	Service and Operation	Click Next.

3	Input Message	Click Generate Sample.
		Replace the generated data:
		`<CCNumber>: 9000-1111-2222-3333`
		`<amount>: 2500`
		Click Next.

4	Output Message	Click Generate Sample. Replace the generated data: <status>: `INVALID` Click Finish.

2. Create the emulated output wire action for the `INVALID` response.
   a. Right-click the wire connecting the ValidateCreditCard Mediator component with the CreditCardDB service and select Create Asserts and Emulates.
   b. Click the Emulates tab and click the Add Emulate icon.
   c. Click Generate Sample.

Oracle Internal & Oracle Academy Use Only

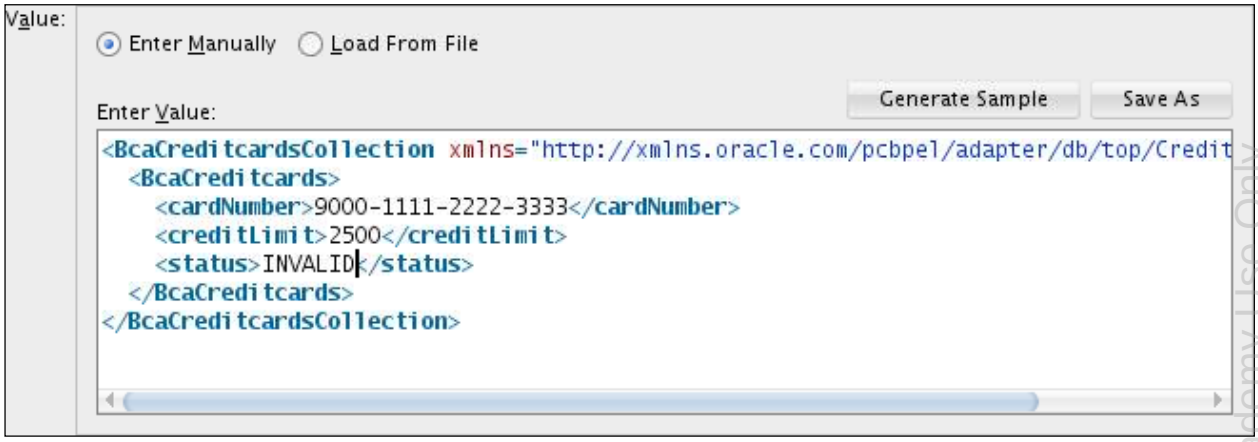
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

d. Delete the last two `<BcaCreditcards>` elements and the child elements they contain.

e. In the remaining `<BcaCreditcards>` element and its child elements, replace the data as follows:

- `<cardNumber>: 9000-1111-2222-3333`
- `<creditLimit>: 2500`
- `<status>: INVALID`

f. Verify your work and click OK.



You are returned to the Asserts and Emulates window.

g. Click OK.

h. Save your work and close the `test_invalid_9000.xml` window.

3. Create a third test case with assertions in the `testsuite_vcc` test suite.

a. Right-click the `tests` folder and select Create Test.

b. Use the information in the following table to define the test:

Step	Pane	Action
1	Name	Name: `test_response_9999` Click Next.
2	Service and Operation	Click Next.

3	Input Message	Click Generate Sample.
		Replace the generated data:
		`<CCNumber>: 9999-9999-9999-9999`
		`<amount>: 3500`
		Click Next.

4	Output Message	Click Generate Sample. Replace the generated data:

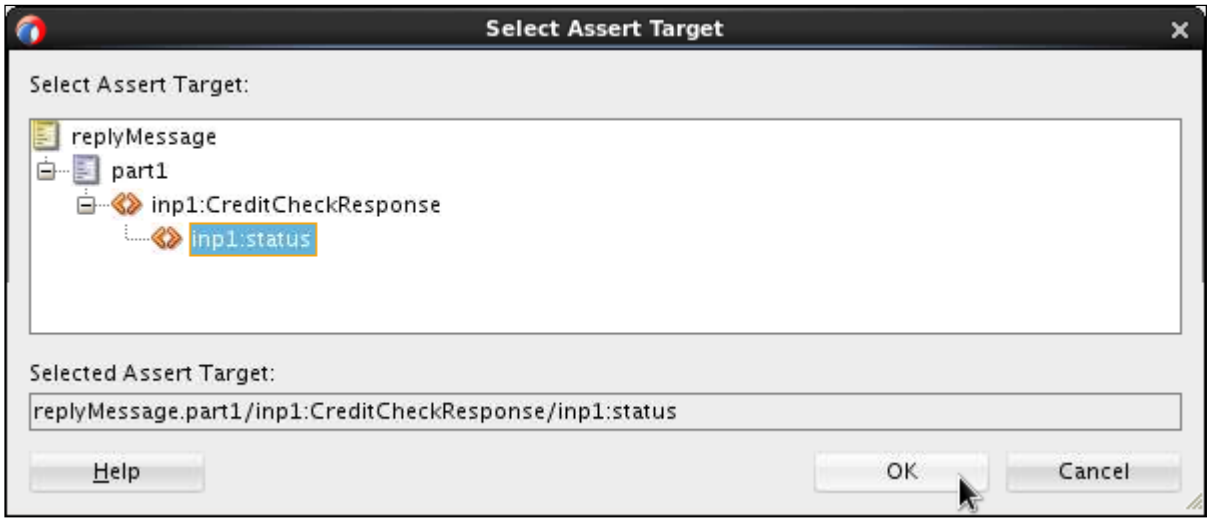
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

		<status>: VALID Click Finish.

Creating a Value-Based Assertion

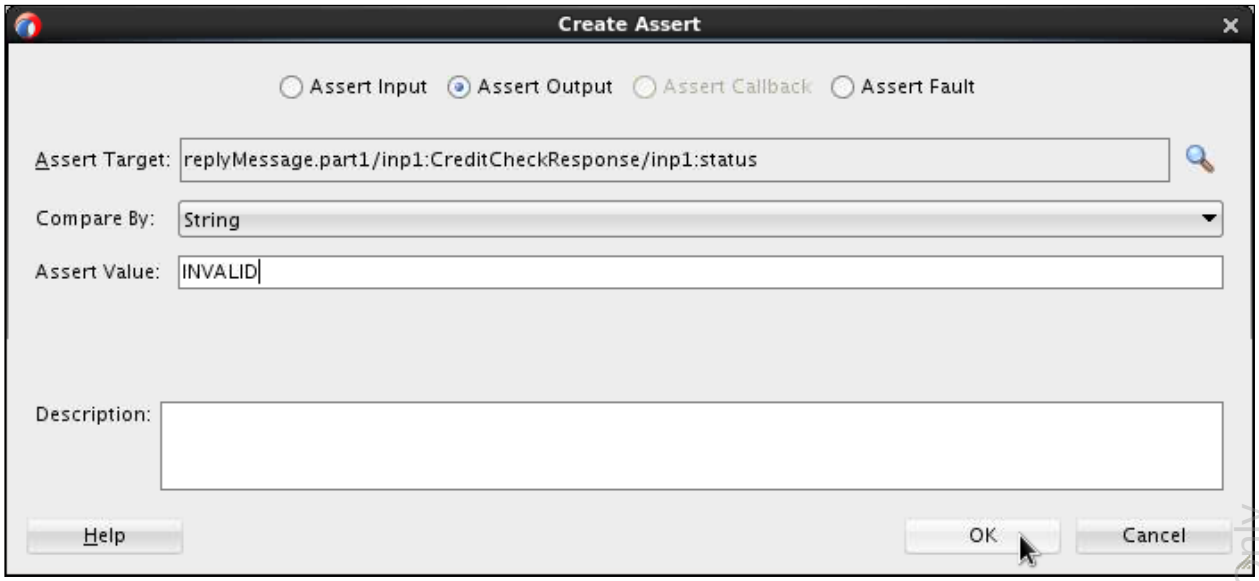
4. Create a value-based assertion for a possible `INVALID` response from the ValidateCreditCard Mediator component.
   a. Right-click the wire connecting the ValidateCreditCard_ep Exposed Services icon and the ValidateCreditCard Mediator and select Edit Asserts and Emulates.
   b. Click the Asserts tab.
   c. Click the Add Assert icon.
   d. Select the Assert Output option.
   e. Assert Target: Click Browse.
   f. Expand the message hierarchy, select `status`, and click OK.



   g. With the Assert Output option and Assert Target selected, set the following values:
      1) Compare By: string
      2) Assert Value: `INVALID`
   h. Verify your work and click OK.

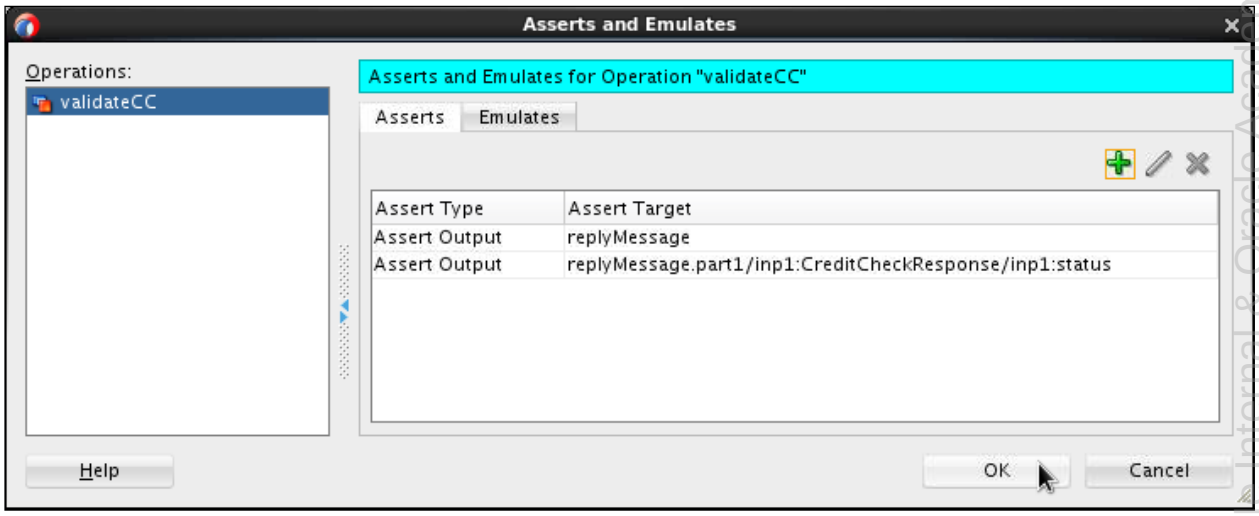
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

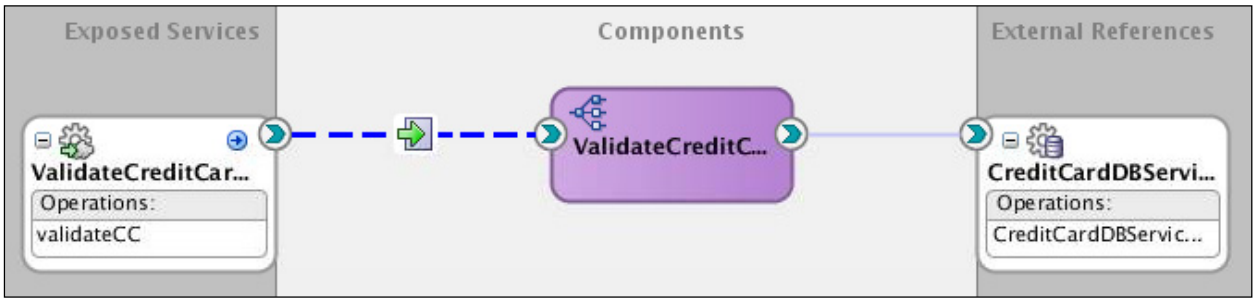


You are returned to the Asserts and Emulates window.

i. On the Asserts tabbed page, confirm that you have two assertions of the Assert Output type, and click OK.



5. On the `test_response_9999.xml` tabbed page, verify the creation of the wire action.



6. Save your work and close the `test_response_9999.xml` window.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 14-3: Deploying and Executing the CCValidate Test Suite

Overview

In this practice, you deploy the CCValidate composite application along with its test suite and test cases. When it is deployed, by using Oracle Enterprise Manager Fusion Middleware Control, you can initiate and examine the results of the test suite and test cases.

Assumptions

This practice assumes that you have successfully completed all work up to this point.

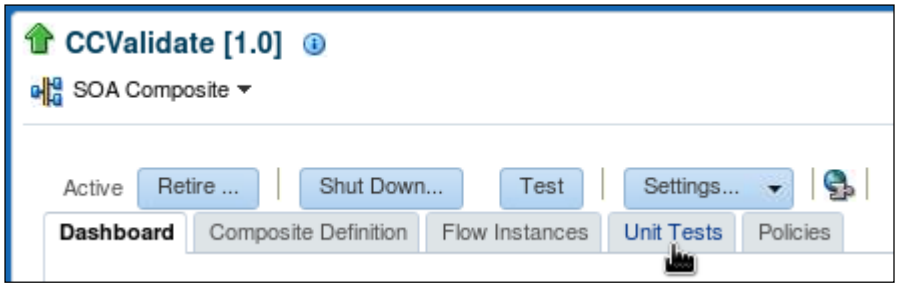
Tasks

Redeploying the Composite Application

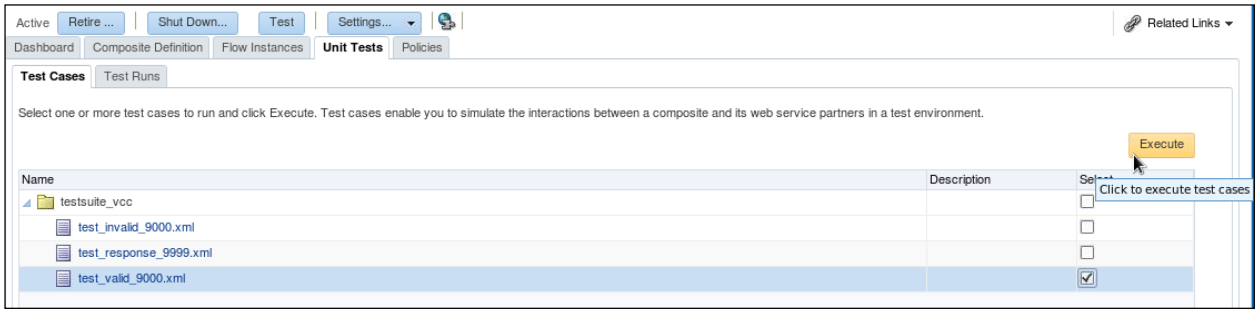
1. In JDeveloper, redeploy the CCValidate composite application to IntegratedWebLogicServer.

Accessing the Unit Test Window in Enterprise Manager

2. Open the Oracle Enterprise Manager page (http://localhost:7101/em).
   a. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “CCValidate [1.0]” link.
   b. On the “CCValidate [1.0]” page, click the Unit Tests tab.



3. Run the `test_valid_9000` test case.
   a. Expand the `testsuite_vcc` entry.
   b. Select the check box next to the row with the `test_valid_9000.xml` test case.
   c. Click Execute.

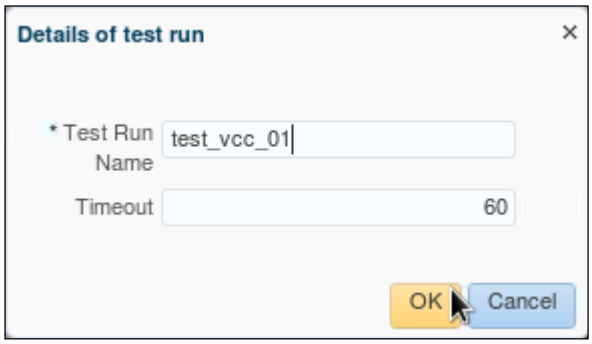


Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

d. In the “Details of test run” window, enter the Test Run Name value: `test_vcc_01`.

Note: It is a good idea to choose unique names for your test runs. The test run name that is selected can be used in the test runs search criteria to locate the results for that test run instance.



e. Click OK.

The web browser is refreshed and the Test Runs tabbed page is displayed.

f. Collapse the Search section to hide the search criteria entry fields.

g. In the “Results of Test Run: test_ccv_01” section, click Refresh Test Status.

Note: You may need to click Refresh Test Status several times before the test run results begin to appear on the page.

After a short wait and some clicks of the Refresh Test Status button, the test_vcc_01 test row appears in the Test Run Name column below the collapsed Search section. The `test_valid_9000.xml` entry is displayed. The Status column contains a green check mark with a `Passed` result.



Note: The `Passed` status indicates that the test was successful for the conditions that were created. In other words, the routing rule in the Mediator component functions as it was designed to work.

4. Run all the tests in the test suite.

a. Click the Test Cases tab to return to the Test Cases tabbed page.

b. Select the entry in testsuite_vcc row, and click Execute.

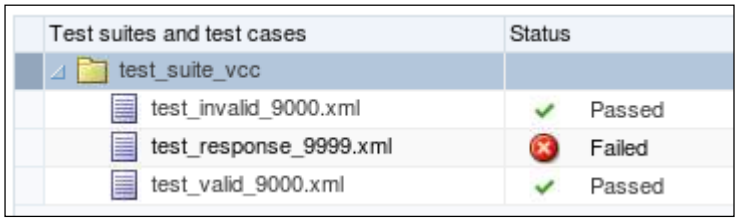
Note: Selecting the test suite entry causes all the test cases to be executed.

c. In the “Details of test run” window, enter the Test Run Name value: `test_vcc_all`.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

d. Wait for a short time (a couple of minutes) and click Refresh Test Status until the `test_vcc_all` test run entry appears.



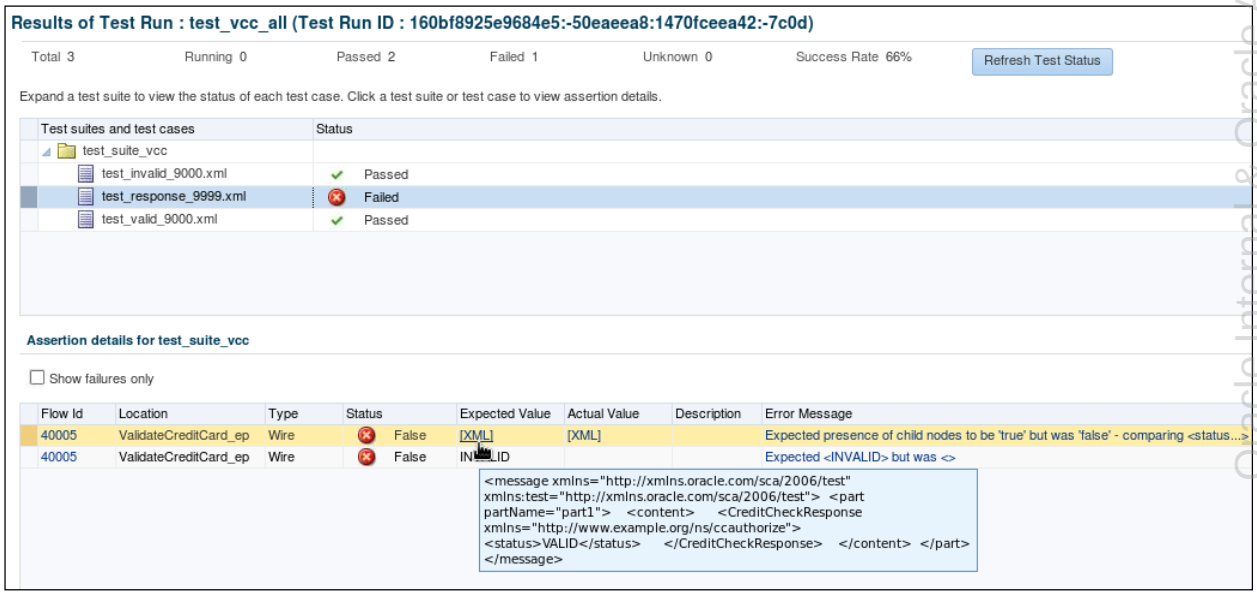
In this test run, all tests have a `Passed` status except the `test_response_9999.xml` test case.

e. Click the `test_response_9999.xml` row to display more information about the failure in the "Assertion details for testsuite_vcc" section.

Note: Why are the failures assertion failures? Recall that the `test-response_9999.xml` test case is the only test case that contains assertions. It appears that an assertion failed. This could indicate a process execution problem or data problem.

f. In the "Assertion details for testsuite_vcc" section, examine the cause for the XML-based assertion failure by clicking either of the [XML] links in the row entry that appears. Click the [XML] link in the Expected Value column.

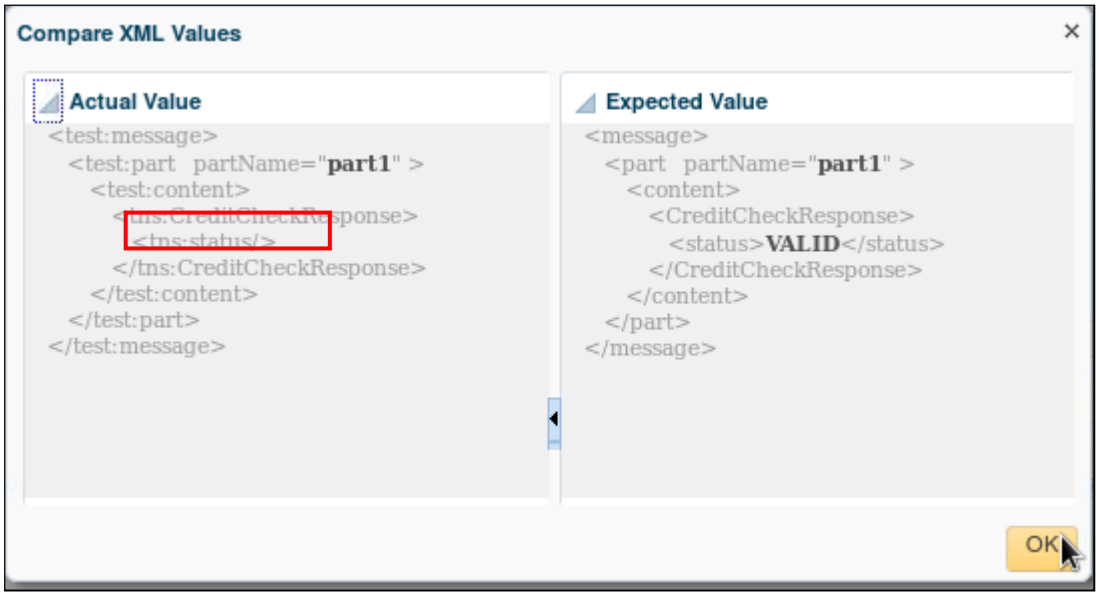
Note: The value-based assertion also failed because it expected the value `VALID` when the actual value return is an empty string. This is visible in the table row.



Oracle Internal & Oracle Academy Use Only

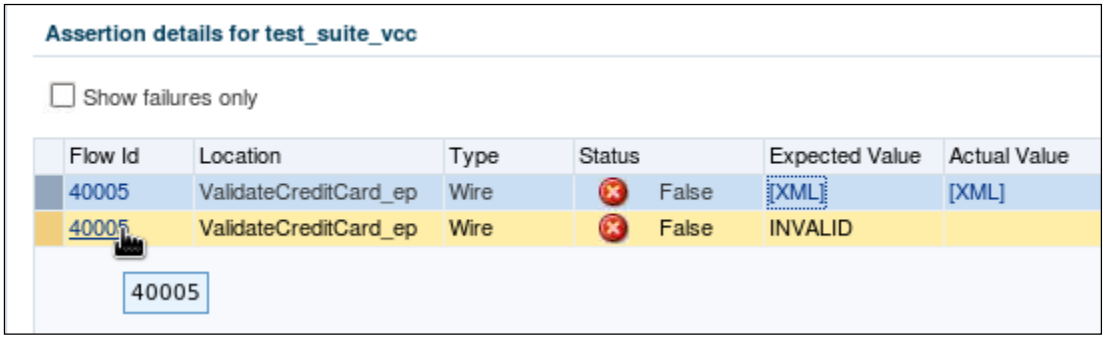
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

g. On the Compare XML Values page, for the XML-based assertion details, examine the actual XML value returned and compare it to the expected value. This enables you to identify the reason for the test failure and determine the next action to take, such as examining the process flow and logic to locate the cause of the problem. In this case, the assertion failed because the status element is an empty element when the value `VALID` was expected inside the status element.



Note: The actual cause of this problem is built in to the course application scenario. In this case, the cause is that the CreditCardDB service returns an empty status value if the credit card value does not exist in the database table. The choices of solution are varied and depend on the business requirements. You do not take action to repair this problem in the course practices.

5. In addition, for each test case, a composite instance is created and run as would be the case for an instance that is created from receiving an actual message (rather than an emulated test) on initiation. In the “Assertion details” table, you can click the instance ID link in the Flow Id column to view the Flow Trace for that composite application instance to examine the process audit trail and flow pages.



6. (Optional) Click the Composite Instance link to view the Flow Trace details.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 14-4: Debugging a Composite Application in JDeveloper

Overview

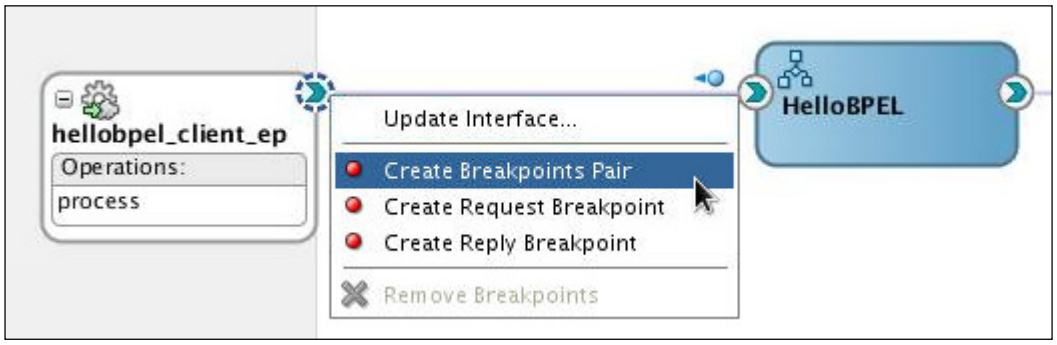
In this practice, you use the SOA debugger to step through a running application.

Assumptions

This practice assumes that you successfully created and deployed the project HelloBPEL in Practice 6 for the lesson titled “Introduction to BPEL.”

Tasks

1. In the JDeveloper window, ensure that the BPELProjects name is selected from the Application drop-down menu, and that the HelloBPEL project is expanded.
2. Open the HelloBPEL `composite.xml` file for editing.
3. Add breakpoint pairs to the interface and the BPEL process.
   a. Right-click the interface (blue arrow) icon and select Create Breakpoints Pair.

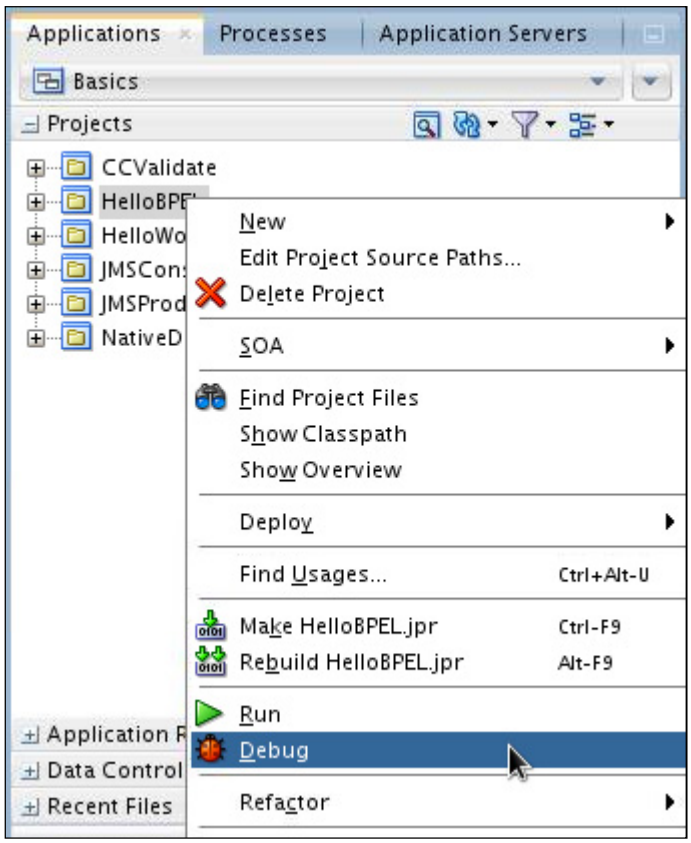


   b. Create a second breakpoints pair on the BPEL process.

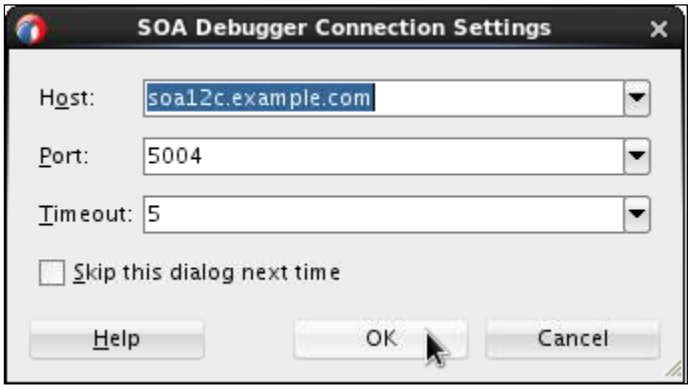
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

4. In the Application Navigator pane, right-click the HelloBPEL project and select Debug.

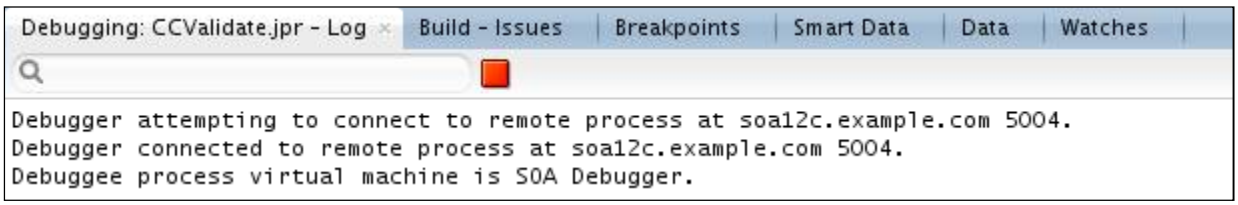


5. Click OK to accept the default connection settings.



The Deploy HelloBPEL dialog box is displayed.

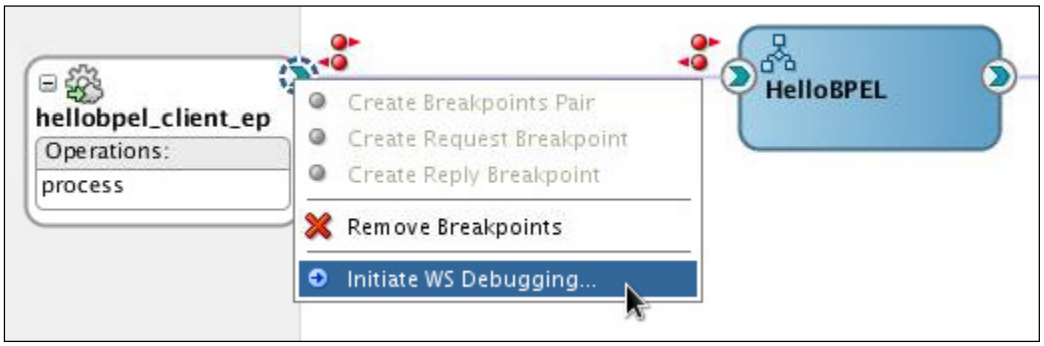
6. Verify that the debugger is successfully connected.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

7. Right-click the interface icon and select Initiate WS Debugging.

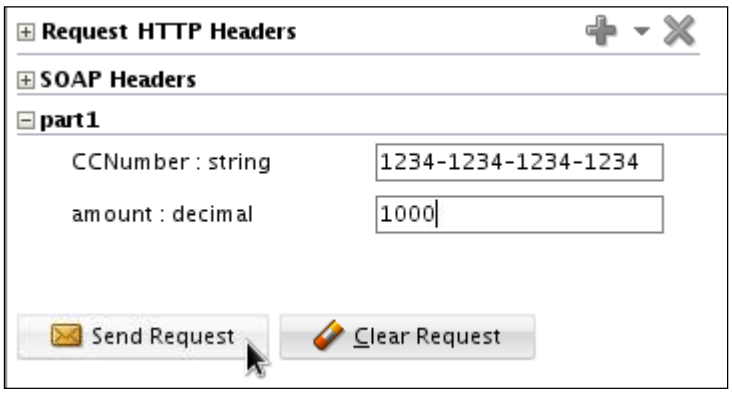


The HTTP Analyzer is opened.

8. Enter the following values:

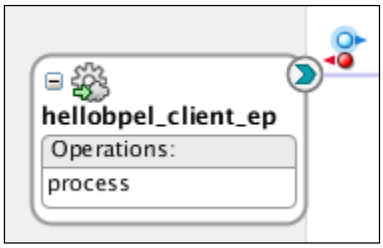
CCNumber: `1234-1234-1234-1234`

amount: `1000`

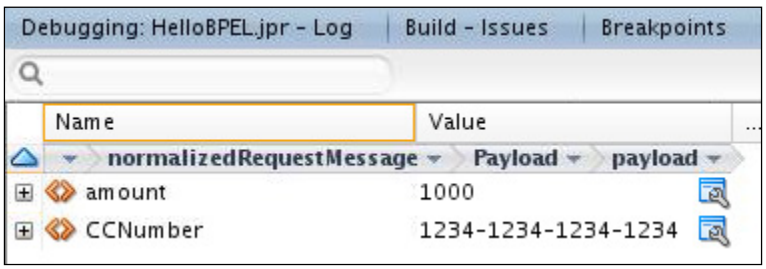


9. Click Send Request.

The debugger stops at the first breakpoint (which turns blue and starts pulsing).



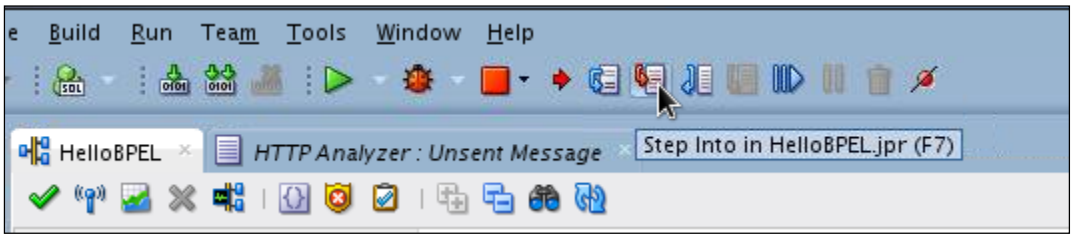
10. Click the Data tab and examine the incoming (normalizedRequestMessage payload) values.



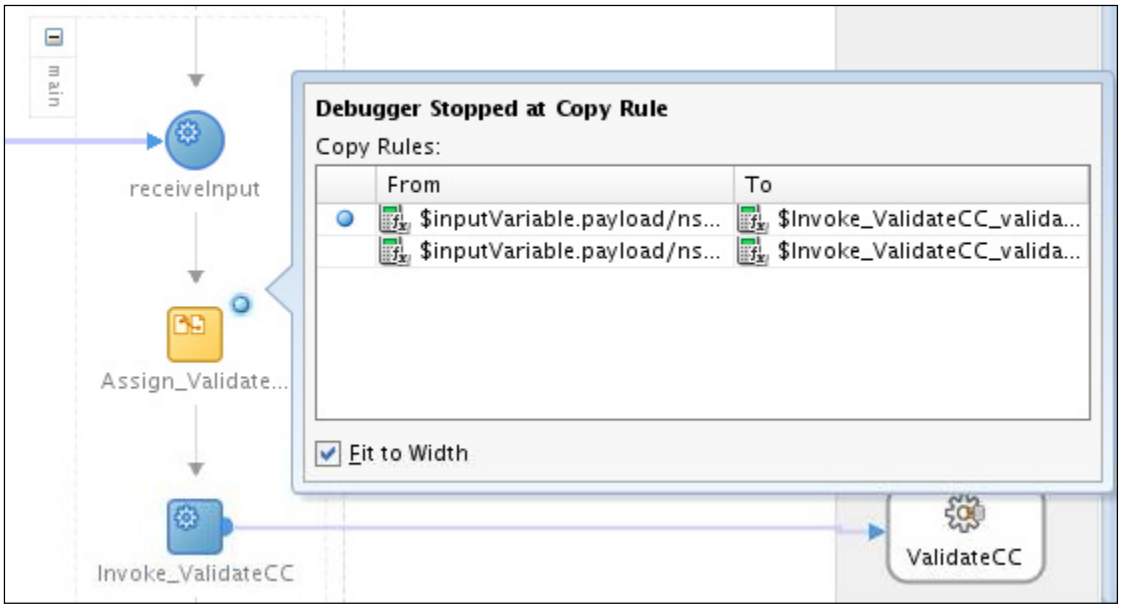
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

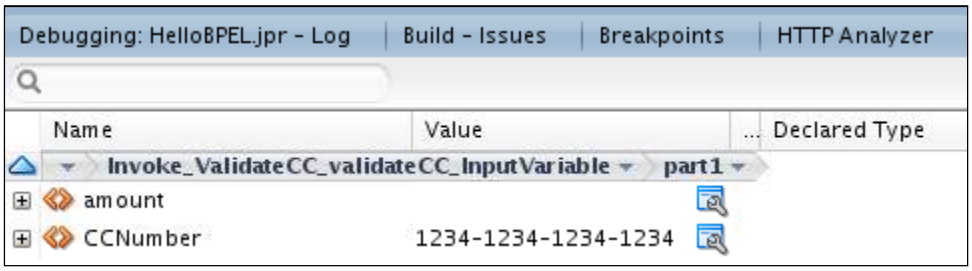
11. On the JDeveloper main menu, click Step Into (F7).



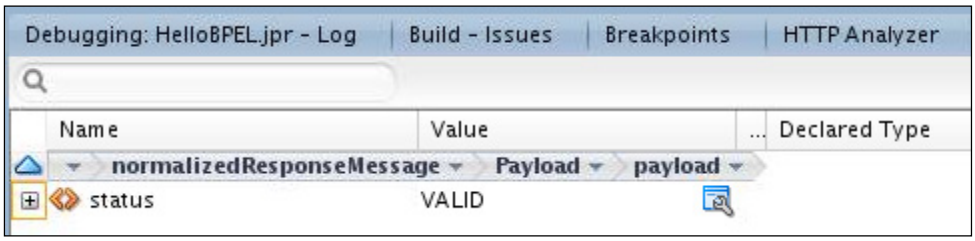
12. Repeat until the debugger arrives at the first Assign activity.



13. Press F7 again to observe the progress of the individual copy statements in the Assign activity. Expand the variables in the data window as needed.



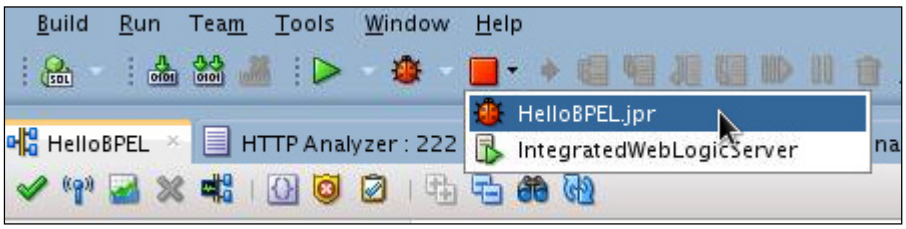
14. Continue to use the F7 and F8 keys to move forward through the BPEL process. After exiting the BPEL process and returning to the hellobpel_client_ep interface, pause to observe the value of the response message.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

15. On completion of the composite instance, on the main JDeveloper menu, click the Terminate icon, and select to terminate the debugger session.



Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practices for Lesson 15: Securing Composite Applications and Invoking Secured Services

Chapter 15

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

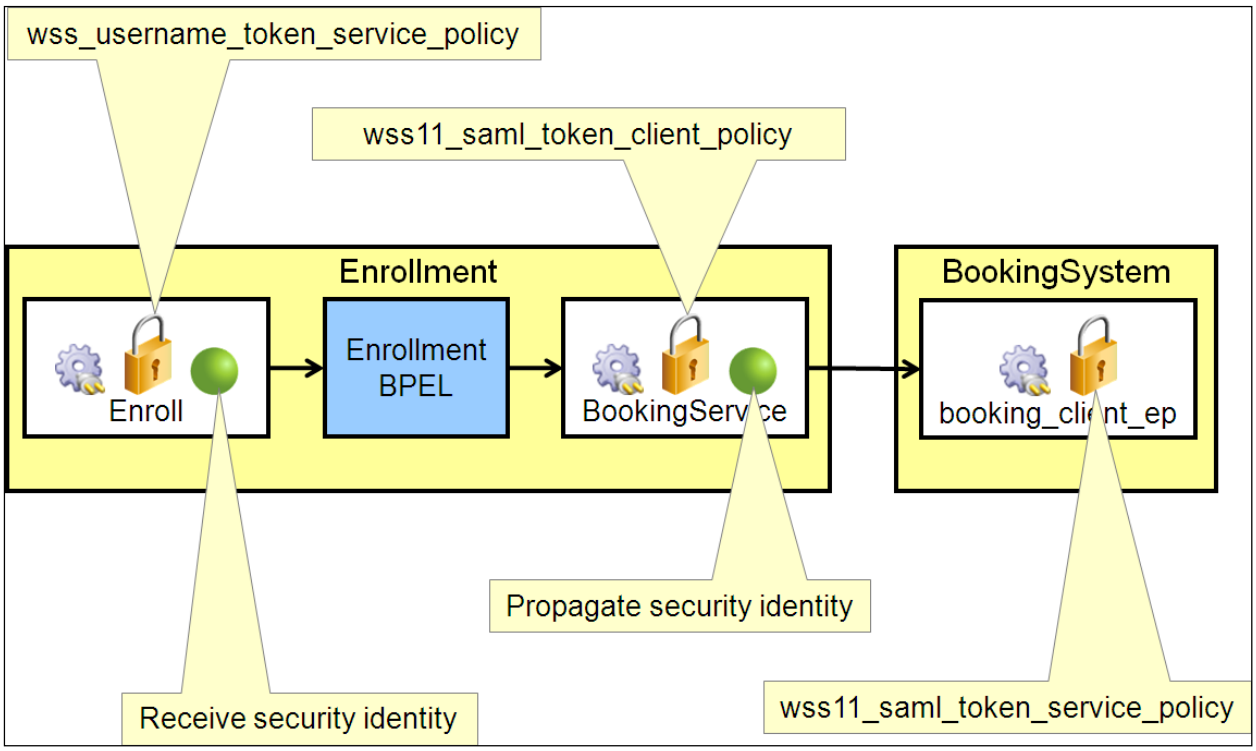
Practices for Lesson 15: Overview

Practices Overview

The goal of this practice is to apply security policies to web services endpoints. Applying security policies is done in two ways:

- Post-deployment, by using Oracle Enterprise Manager Fusion Middleware Control to attach a security policy to selected service endpoints
- Before deployment (at design time), by attaching policies to service endpoints in JDeveloper

You apply policies both at design time and post-deployment to the `Enroll` and `BookingSystem` applications that you created earlier. The following image illustrates the security policy attachments that are applied in this practice:



Your tasks in the practice are to:

- Secure the invocation of the `Enroll` composite application entry point (`Enroll`) by using the UsernameToken policy. This requires that you enter a valid set of credentials (username and password) to initiate the service.
- Secure the `BookingSystem` composite application entry point (`Booking_client_ep`) by using SAML service tokens, which requires the security identity to be propagated
- Propagate identity by using SAML client tokens on the ValidCCService, which is the external reference for the `Booking_client_ep` entry point

In the practice, you apply the policies to the deployed applications by using Oracle Enterprise Manager. After removing the policies in Oracle Enterprise Manager, you use JDeveloper to secure the `BookingSystem` service and ensure that it is invoked securely by using design-time

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

policy attachments. You then remove the design-time policies so that the remaining practices in the course do not require the additional step of providing WS-Security credentials for each test.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 15-1: Applying Security Policies Post Deployment

Overview

In this practice, you attach the WS-Security UsernameToken policy to the `Enroll` service entry point of the `Enroll` composite application. You test the service with and without supplying security credentials. You then attach SAML policies to secure the `BookingSystem` composite application entry point and its client to propagate the security identity that is acquired through the use of the UsernameToken policy. You test to see if execution is successful. Finally, you remove the UsernameToken policy and test to observe what happens if you try to propagate security credentials that have not been supplied.

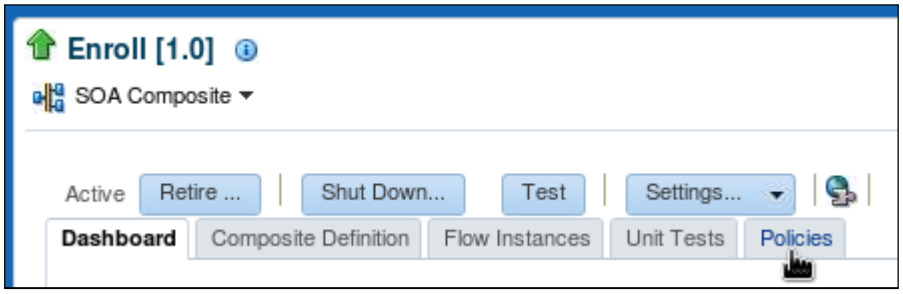
Assumptions

This practice assumes that you have completed the practice for lesson 8 titled “Handling Faults in Composite Applications” successfully and that the `Enroll` and `BookingSystem` applications are still deployed.

Tasks

Attaching the UsernameToken Policy to `Enroll`

1. Access the policy page for the `Enroll` composite application entry point called `Enroll`.
   a. Open the Oracle Enterprise Manager page (http://localhost:7101/em).
   b. In the Target Navigation pane, expand the SOA > soa-infra > default nodes in the tree. Click the “Enroll [1.0]” link.
   c. On the “Enroll [1.0]” page, click the Policies tab.



2. Attach `oracle/wss_username_token_service_policy` to the `Enroll` component:
   a. Select `Enroll` from the “Attach To/Detach From” drop-down menu.
   b. In the Search field above the Name column, enter `wss_username` and press Enter.

      Note: Using the search feature makes it easier to locate the desired policy.
   c. In the Available Policies section, select the row containing the Name `oracle/wss_username_token_service_policy`.
   d. Click Attach.
   e. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



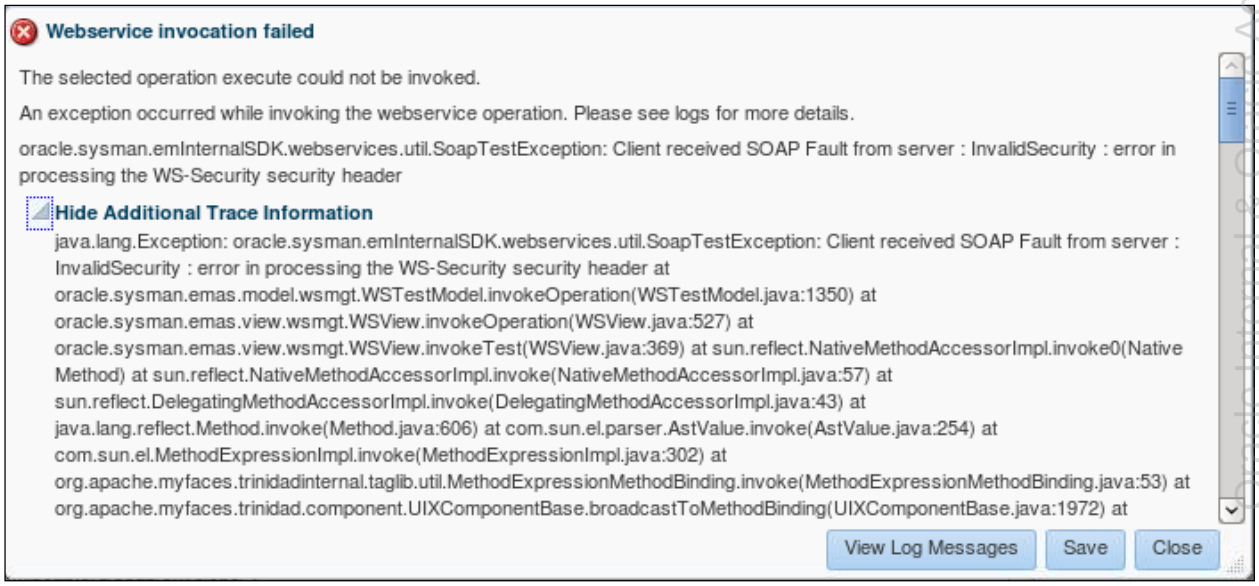
The `Enroll` application now requires a username and password.

Testing the WSS UsernameToken Security Policy

3. Perform a test of the attached security policy.
   a. On the "Enroll [1.0]" page, click Test.
   b. On the Test Web Service page, click the Request tab. In the Input Arguments section, use the Browse button to replace the supplied XML text with the contents of the file `/home/oracle/labs/files/xml_in/enrollment_input.xml`.
   c. Click Test Web Service.
   d. What is the result of this test?

      Answer: The "Webservice invocation failed" window is displayed.

   e. In the "Webservice invocation failed" window, expand Show Additional Trace Information. What does it tell you?



The text stating "`error in processing the WS-Security security header`" hints at the answer. You did not provide the WS-Security information (a valid username and password), which is sent in the request header.

   f. Close the "Webservice invocation failed" window.

4. Execute another test. This time, provide the WS-Security credentials for the WSS UsernameToken style of security.
   a. On the Test Web Service page, expand the Security section at the top of the Request tab area.

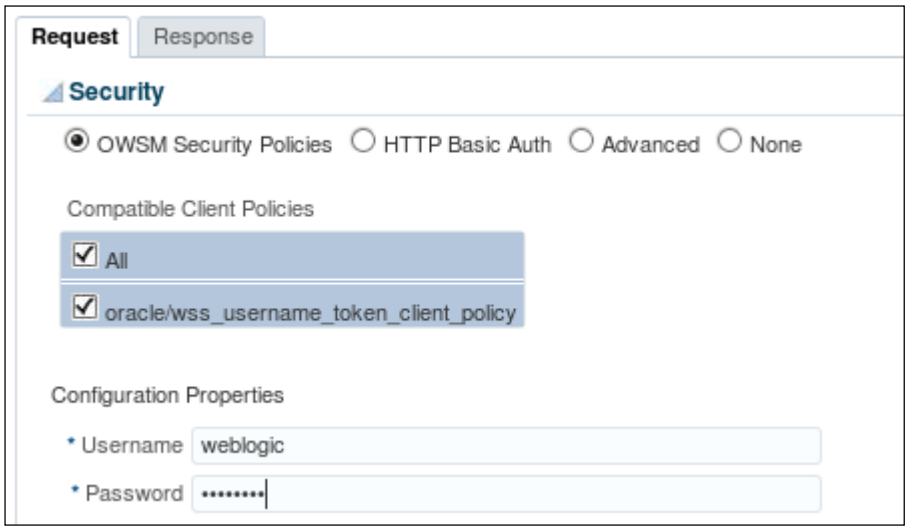
Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

b. Select the OWSM Security Policies option.

c. In the Compatible Client Policies section, select the oracle/wss_username_token_client_policy check box.

d. Enter `weblogic` in the Username field and `welcome1` in the Password field.



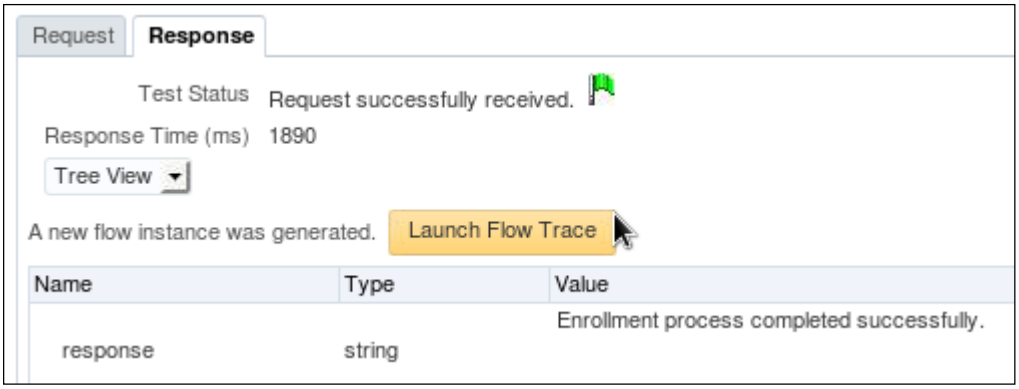
e. Click Test Web Service.

Note: The XML View text should have retained the data copied from the `/home/oracle/labs/files/xml_in/enrollment_input.xml` file for this test.

f. What happened this time?

1) Did you get an error message?

Answer: No error message is displayed. The `Enroll` composite instance is initiated successfully, as indicated by the appearance of the Launch Flow Trace button on the Response tabbed page.



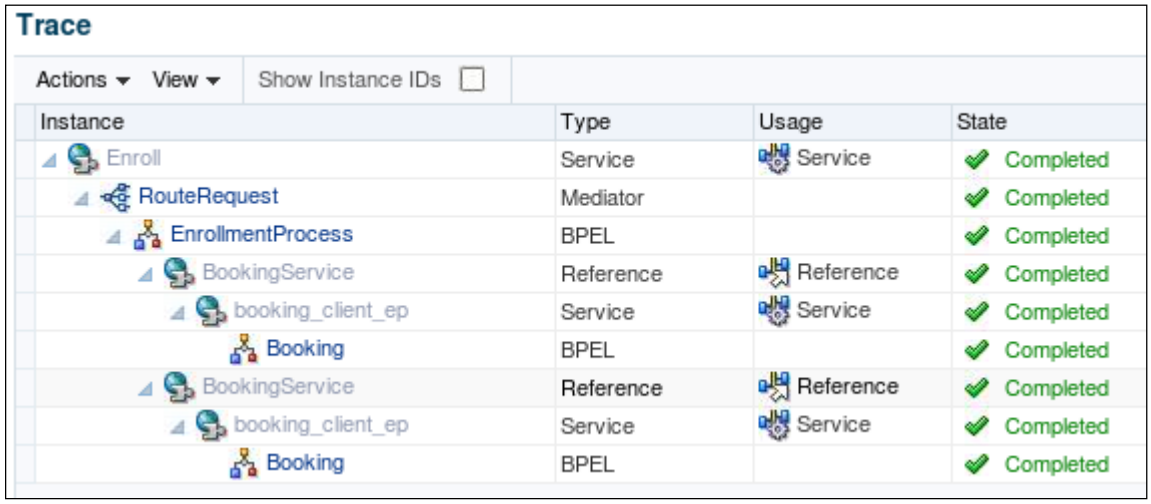
2) Did the process complete successfully based on the supplied data?

Answer: Yes

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

3) Click the Launch Flow Trace button. Confirm that the `BookingService` application, which has not been secured yet, was invoked successfully as well.



4) Close the Flow Trace window.

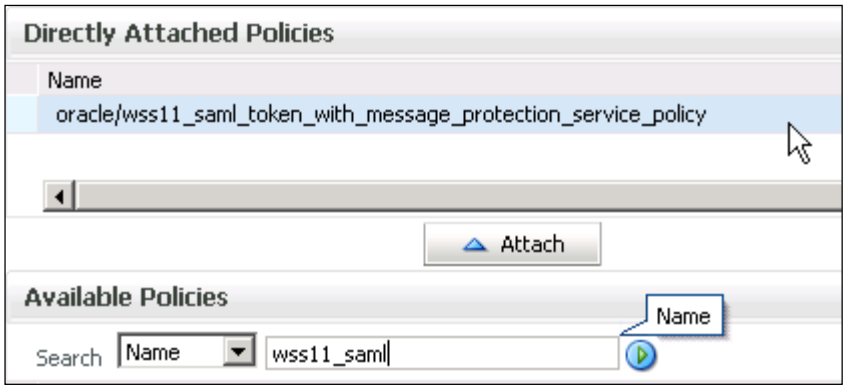
Attaching the SAML Service Policy to `Booking_client_ep`

In this section, you attach the SAML service security policy to the `Booking_client_ep` entry point of the `BookingSystem` composite application.

5. Access the policy page for the `BookingSystem` composite application entry point called `Booking_client_ep`.
   a. On the Oracle Enterprise Manager home page, in the Target Navigation pane, expand the SOA node and click the “BookingSystem [1.0]” link.
   b. Click the Policies tab.
6. Attach `oracle/wss11_saml_token_with_message_protection_service_policy` to `Booking_client_ep`.
   a. Click “Attach To/Detach From” and click Booking_client_ep.
   b. Enter `wss11_saml` in the Search field above the Name column and press Enter.
   c. In the Available Policies section, select the row containing the name `oracle/wss11_saml_token_with_message_protection_service_policy`.
   d. Click Attach.
   e. Verify your work and click OK.

Oracle Internal & Oracle Academy Use Only

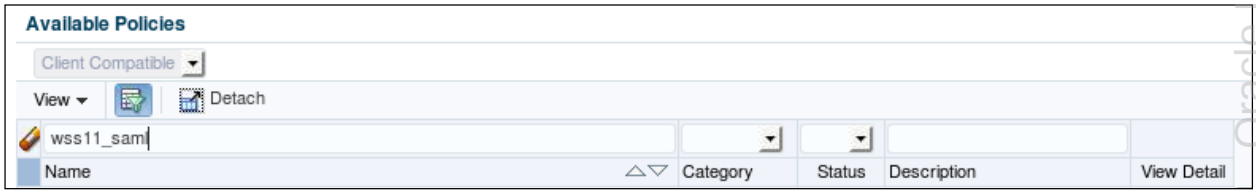
Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



Attaching the SAML Client Policy to `BookingService`

In this section, you configure the SAML client security policy on the `BookingService` external reference of the `Enroll` composite application. The SAML client security policy ensures that the security credentials, which are received through the WS-Security heading (from the attached UsernameToken security policy requirement), are propagated to the `Booking_client_ep` entry point. Remember that the `Booking_client_ep` entry point is accessed through the `BookingService` external reference, which is invoked by the `Enrollment` BPEL Process component.

7. Access the `Enroll` composite application policies page.
   a. On the Oracle Enterprise Manager home page, in the Target Navigation pane, click the "Enroll [1.0]" link.
   b. On the "Enroll [1.0]"page, click the Policies tab.
8. Attach `oracle/ wss11_saml_token_with_message_protection_client_policy` (the SAML client policy) to the `BookingService` external reference in the `Enroll` composite application.
   a. Click "Attach To/Detach From," and then click `BookingService`.
   b. In the Available Policies section, enter `wss11_saml` in the Name search field and press Enter.

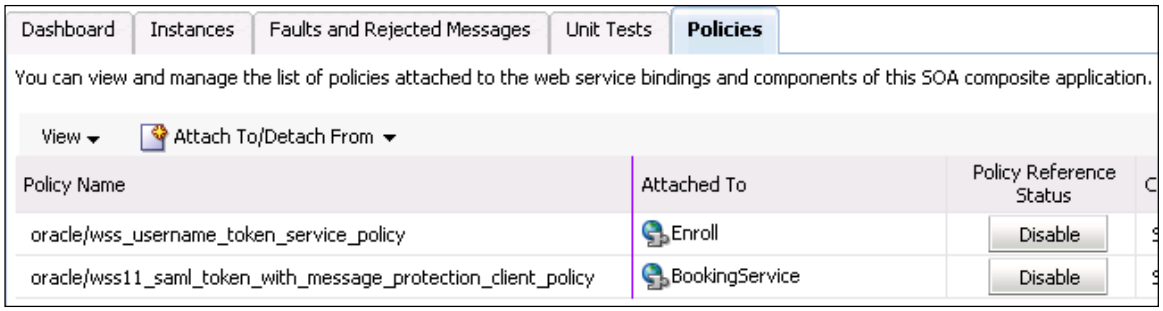


   c. In the Available Policies section, select the row containing the name `oracle/wss11_saml_token_with_message_protection_client_policy`.
   d. Click Attach.
   e. The policy is listed in the Directly Attached Policies section.
   f. Click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

g. Confirm that the two policies, `oracle/wss_username_token_service_policy` and the newly attached `oracle/wss11_saml_token_with_message_protection_client_policy`, are listed in the policy table.



Testing Security Propagation with the UsernameToken and SAML Policies

In this section, you initiate a test of the `Enroll` composite application and provide security credentials to test if the process works, indicating that the security credentials are successfully propagated. To verify this, you perform another test after disabling the UsernameToken security policy to observe what happens when the credentials are not enforced and therefore not propagated.

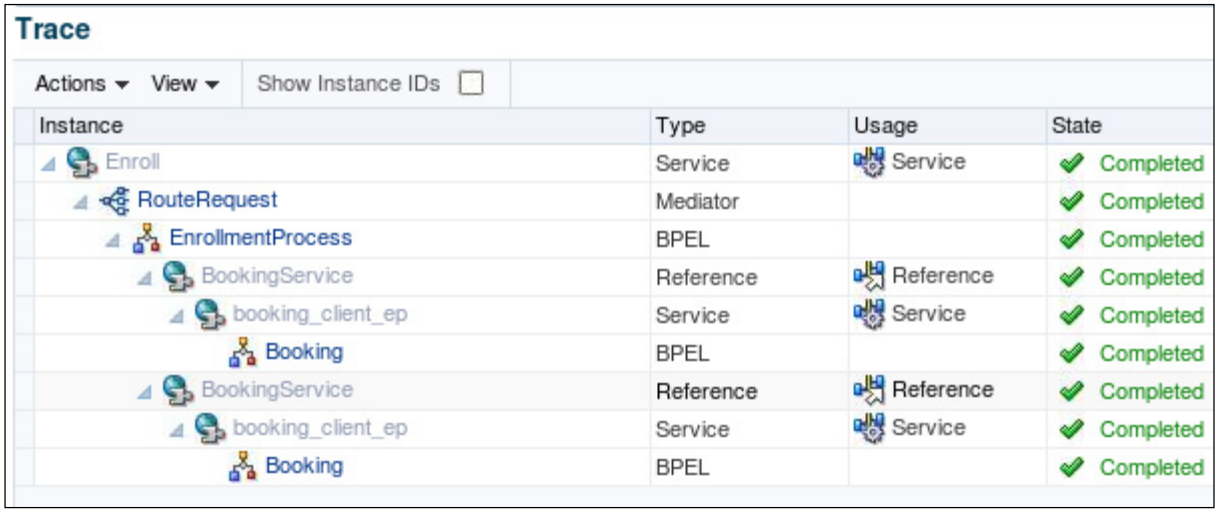
9. Initiate a security propagation test of the `Enroll` composite application through to the `BookingSystem` composite application.
   a. On the “Enroll [1.0]” home page, click Test.
   b. On the Test Web Service page, expand the Security section at the top of the Request tab area.
   c. Select the OWSM Security Policies option.
   d. Select the oracle/wss_username_token_client_policy check box.
   e. Enter `weblogic` in the Username field and `welcome1` in the Password field.
   f. On the “Enroll [1.0]” Web Service Test page, scroll down to the Input Arguments section on the Request tab. Select the XML View mode and replace the supplied XML text with the contents of the `/home/oracle/labs/files/xml_in/enrollment_input.xml` file.
   g. Click Test Web Service.

      The Response tabbed page is displayed.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

h. Click the Launch Message Flow Trace button. Confirm that all appropriate process components executed with a Completed status.



i. Close the Flow Trace window.

10. Disable the WSS UsernameToken policy in the `Enroll` composite application.

Note: This eliminates the need to provide security credentials when initiating the `Enroll` composite application. No security identity information can be propagated to the `Booking_client_ep` entry point of the `BookingSystem` composite application.

a. On the Oracle Enterprise Manager home page, in the Target Navigation pane, click the "Enroll [1.0]" link.

b. On the "Enroll [1.0]" home page, click the Policies tab.

c. On the "Enroll [1.0]" Policies tabbed page, from the list of attached policies, select the `oracle/wss_username_token_service_policy` row and click Disable.

d. In the Confirmation dialog box, click Yes.

e. On the "Enroll [1.0]" page, click Test.

f. In the Request tab section, scroll down to the Input Arguments section, select the XML View mode, and replace the supplied XML text with the contents of the `/home/oracle/labs/files/xml_in/enrollment_input.xml` file.

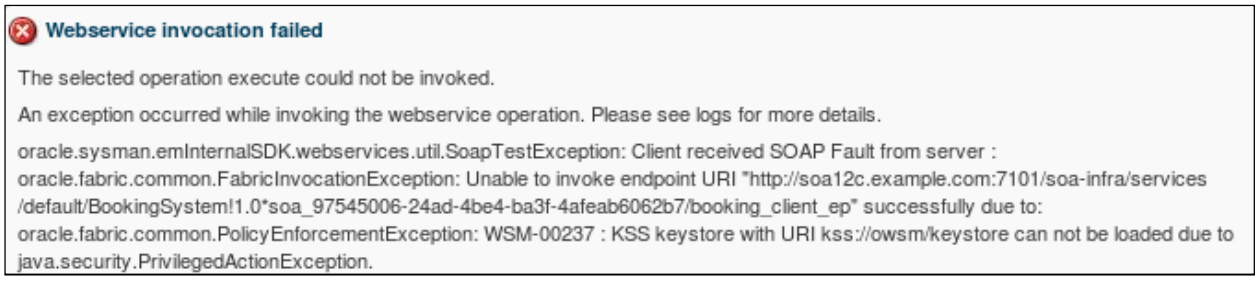
Note: Because you disabled the policy, there is no need to provide the WS UsernameToken credentials.

g. Click Test Web Service.

The web service invocation fails. An error message indicating that the endpoint cannot be accessed is displayed. This result is expected because no credentials were provided to the `Enroll` composite application. Therefore, there is no authenticated security identity that can be propagated to the `BookingSystem` service entry point, which still enforces the requirement to provide security information because of the attached SAML security policies.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



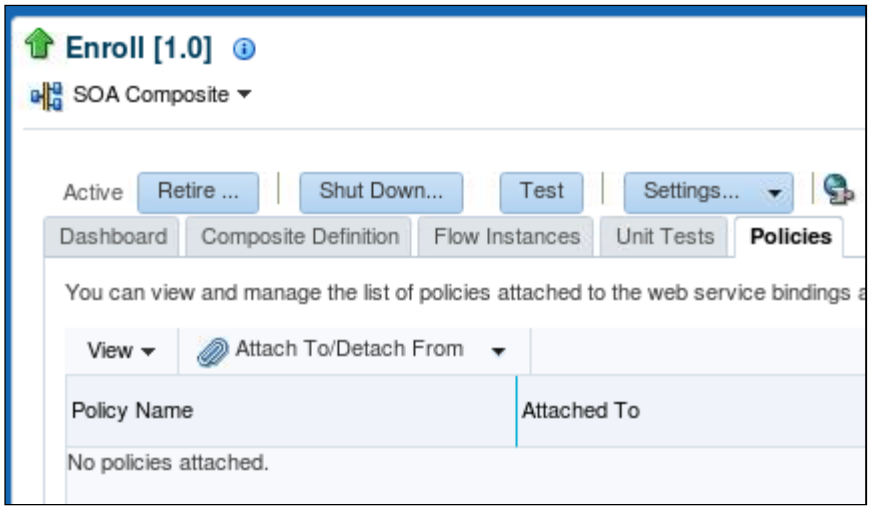
Detaching All Security Policies

In this section, you detach the security policies so that you can perform the same security attachment tasks at design time in the JDeveloper Composite Editor in the next practice.

11. Access the Policies tab of the `Enroll` composite application.
    a. In the Target Navigation pane, click the “Enroll [1.0]” link.
    b. On the “Enroll [1.0]” page, click the Policies tab.
12. Detach the security policies from the `Enroll` composite application.
    a. Click “Attach To/Detach From” > `BookingService`.
    b. In Attached Policies, select the `oracle/wss11_saml_token_with_message_protection_client_policy` entry.
    c. Click Detach.
    d. Verify that no policies are attached and click OK.
    e. Select “Attach To/Detach From” > Enroll.
    f. In Attached Policies, select the `oracle/wss_username_token_service_policy` entry.
    g. Click Detach.
    h. Verify that there are no policies attached and click OK.
    i. On the “Enroll [1.0]” Policies page, verify that all security policies have been detached from the composite application.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



13. Repeat the previous steps to access the Policies tab of the `BookingSystem` composite application and detach the policy `oracle/wss11_saml_token_with_message_protection_service_policy` from `Booking_client_ep`.
14. Verify your work.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Practice 15-2: Applying Security Policies at Design Time

Overview

In this practice, you effectively repeat the attachment of the policies used in the previous practice. However, this time you attach the policies to component endpoints at design time by using the Composite Editor. After deploying the secured composite applications, you initiate a test, and then disable the security and redeploy the applications so that the remaining course practices do not require security credentials to be supplied for every test that is subsequently performed.

Note: In JDeveloper, to attach policies to endpoints, right-click an exposed service, external references, or components in the Composite Editor and select Configure SOA WS Policies.

Assumptions

This practice assumes that you have completed Practice 12-1 titled “Creating and Configuring a Composite Application” successfully.

Tasks

1. In JDeveloper, open the `BookingSystem composite.xml` application file.

Attaching the SAML Service Policy to `Booking_client_ep`

2. Attach `oracle/wss11_saml_token_with_message_protection_service_policy` to the `Booking_client_ep` exposed service.
   a. Right-click the `Booking_client_ep` exposed service icon and select Configure SOA WS Policies.
   b. For the Security section, click the Add Security Policy icon.
   c. Scroll down and select the `oracle/wss11_saml_token_with_message_protection_service_policy` entry.
   d. Click OK.
   e. Confirm the addition of the `oracle/wss11_saml_token_with_message_protection_service_policy` entry to the Security section and click OK.

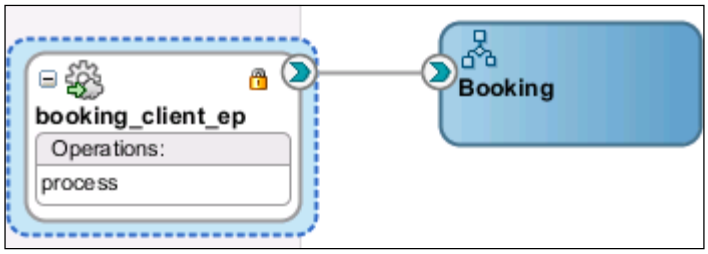


Note: The green check mark next to the policy indicates that the policy is enabled.

In the Exposed Services column, the `Booking_client_ep` icon contains a lock icon to indicate that it has been secured.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.



3. Save your work and close the `BookingSystem composite.xml` window.

Attaching the SAML Client Policy to `BookingService`

In this section, you modify the `Enroll` composite application. You attach the appropriate security policies to propagate the SAML security to the `BookingSystem` composite from the `BookingService` external reference.

4. In JDeveloper, open the `Enroll` project composite overview window.
5. Attach `oracle/wss11_saml_token_with_message_protection_client_policy` to the `BookingService` external reference.
   a. Right-click the `BookingService` external reference icon and select Configure SOA WS Policies.
   b. In the Security section, click the Add Security Policy icon.
   c. Scroll down and select the `oracle/wss11_saml_token_with_message_protection_client_policy` entry.
   d. Click OK.
   e. Confirm the addition of the `oracle/wss11_saml_token_with_message_protection_client_policy` entry to the Security section and click OK.

      The `BookingService` icon displays a lock icon to indicate that it has been secured.

Attaching the UsernameToken Policy to `Enroll`

In this section, you enforce the requirements for obtaining credentials for the `Enroll` composite by using the UsernameToken policy on its `Enroll` entry point. Credentials are required so that they can be propagated to the `BookingService`.

6. Attach `oracle/wss_username_token_service_policy` to the `Enroll` exposed service.
   a. Right-click the `Enroll` exposed service icon and select Configure SOA WS Policies.
   b. For the Security section, click the Add Security Policy icon.
   c. Scroll down and select the `oracle/wss_username_token_service_policy` entry.
   d. Click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

e. Confirm the addition of the `oracle/wss_username_token_service_policy` entry to the Security section and click OK.

   In the Exposed Services column, the `Enroll` icon contains a lock icon to indicate that it has been secured.

7. Save your work and close the `Enroll composite.xml` window.

Deploying `BookingSystem` and Deploying and Testing `Enroll`

8. Deploy the `BookingSystem` project with the security attachments to IntegratedWebLogicServer.
9. Deploy the `Enroll` project with the security attachments to IntegratedWebLogicServer.

Testing the Design-Time Security Settings in Oracle Enterprise Manager

10. Test if the UsernameToken security is enforced.
    a. On the Oracle Enterprise Manager home page, in the Target Navigation pane, click the "Enroll [1.0]" link.
    b. On the "Enroll [1.0]" page, click Test.

       Do not specify the WS UsernameToken credentials.
    c. Scroll down to the Input Arguments section on the Request tab and select the XML View mode.
    d. Replace the supplied XML text with the contents of the `/home/oracle/labs/files/xml_in/enrollment_input.xml` file.
    e. Click Test Web Service.

       After a short delay, the "Webservice invocation failed" window is displayed. This indicates that the invocation of the `Enroll` application failed because security credentials were not provided when they were required.



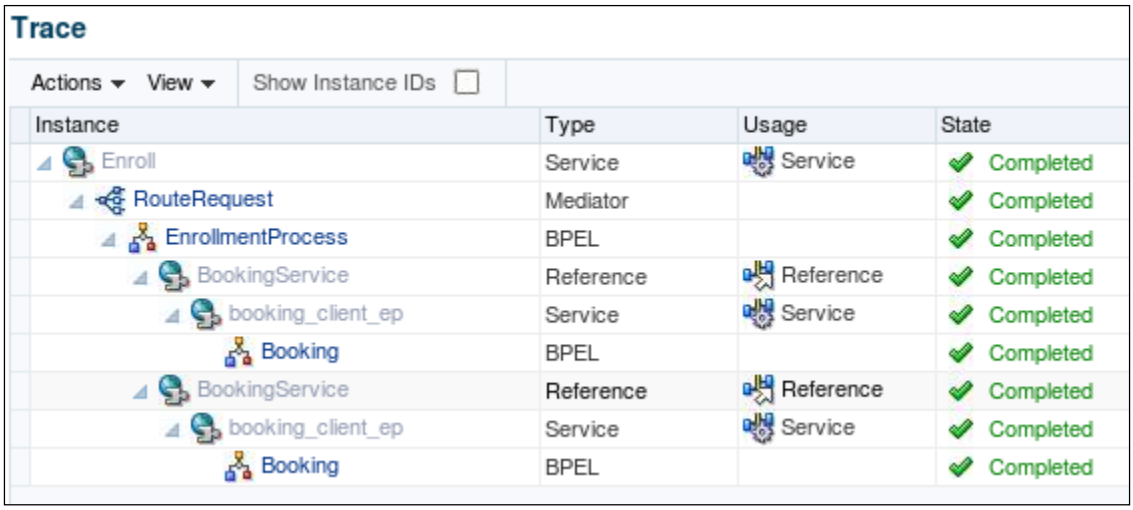
11. Repeat the test. This time, supply a valid username and password in the WSS UsernameToken settings.

    This time, the "Enroll [1.0]" Test Web Service > Response tabbed page is displayed with the Launch Message Flow Trace link, indicating that the service invocation is successful.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

12. On the “Enroll [1.0]” Test Web Service > Response tab, click the Launch Message Flow Trace link to verify that the process completed as expected, indicating that `BookingService` propagated the credentials to `Booking_client_ep` in the `BookingSystem` composite application.



13. Close the web browser window that contains the Flow Trace page.

Disabling Security Policies and Redeploying Applications

14. Disable the UsernameToken security policies so that they are not enforced by the `Enroll` composite application.
    a. In JDeveloper, open the Enroll composite overview window.
    b. Right-click the Enroll icon and select Configure SOA WS Policies.

       The Configure SOA WS Policies window opens.
    c. Click the Disable icon next to the `oracle/wss_username_token_service_policy` entry.
    d. Verify your work and click OK.



       This disables the security policy so that it is not enforced.
    e. In the `composite.xml` window, verify that the lock icon on the `Enroll` icon is displayed in an unlocked state.
15. Disable the SAML client security policy on the `BookingService` external reference of the Enroll composite application.
    a. Right-click the `BookingService` icon and select Configure SOA WS Policies.
    b. Click the Disable icon next to the `oracle/wss11_saml_token_with_message_protection_client_policy` entry.
    c. Click OK.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

d. In the `composite.xml` window, verify that the lock icon on the `BookingService` icon is displayed in an unlocked state.

16. Save your work and close the `composite.xml` window.
17. Disable the SAML service security policy on the `BookingSystem` composite application exposed service (entry point).
    a. Open the `BookingSystem` composite overview window.
    b. Right-click the `Booking_client_ep` icon and select Configure SOA WS Policies.
    c. Click the Disable icon next to the `oracle/wss11_saml_token_with_message_protection_service_policy` entry.
    d. Click OK.
    e. In the `composite.xml` window, verify that the lock icon on the `Booking_client_ep` icon is displayed in an unlocked state.
18. Save your work and close the `composite.xml` window.

Redeploying the Composite Applications with Security Policies Disabled

19. Deploy `BookingSystem` to IntegratedWebLogicServer.
20. Deploy `Enroll` to IntegratedWebLogicServer.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.

Oracle Internal & Oracle Academy Use Only

Copyright © 2014, Oracle and/or its affiliates. All rights reserved.